

ARC simulation and reconstruction

Generated by Doxygen 1.13.1

1 Namespace Index	1
1.1 Namespace List	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Namespace Documentation	9
5.1 de Namespace Reference	9
5.2 ParticleMass Namespace Reference	9
5.2.1 Detailed Description	9
5.2.2 Function Documentation	9
5.2.2.1 GetMass()	9
5.3 PhotonMapper Namespace Reference	10
5.3.1 Detailed Description	10
5.3.2 Function Documentation	10
5.3.2.1 IsScatteredInAerogel()	10
5.3.2.2 PhotonMirrorDistance()	10
5.3.2.3 TracePhoton()	10
5.3.2.4 TracePhotonToDetector()	11
5.3.2.5 TracePhotonToMirror()	11
5.4 PhotonReconstructor Namespace Reference	11
5.4.1 Function Documentation	11
5.4.1.1 GetReflectionPoint()	11
5.4.1.2 IsReconstructed()	12
5.4.1.3 ReconstructCosCherenkovAngle()	12
5.4.1.4 ReconstructPhoton()	12
5.4.1.5 SolveQuartic()	12
5.4.2 Variable Documentation	13
5.4.2.1 FailedReconstruction	13
5.5 ResolutionUtilities Namespace Reference	13
5.5.1 Function Documentation	13
5.5.1.1 CalculateResolution()	13
5.5.1.2 DoFit()	14
5.5.1.3 fcn()	14
5.5.1.4 PlotProjections()	14
5.6 Utilities Namespace Reference	14
5.6.1 Function Documentation	15
5.6.1.1 GenerateRandomBarrelTrack()	15

5.6.1.2 GenerateRandomBarrelTrackZRange()	15
5.6.1.3 GenerateRandomEndCapTrack()	15
5.6.1.4 GetCherenkovAngleDifference()	15
5.6.1.5 GetIndexRefraction()	16
5.6.1.6 GetMomentumMag()	16
5.6.1.7 GetPredictedCherenkovAngle()	16
5.6.1.8 Random()	17
5.6.1.9 TrackPhotons()	17
5.6.1.10 TrackSeed()	17
5.6.1.11 VectorFromSpherical()	17
6 Class Documentation	19
6.1 ARCVector Class Reference	19
6.1.1 Detailed Description	19
6.1.2 Constructor & Destructor Documentation	19
6.1.2.1 ARCVector() [1/4]	19
6.1.2.2 ARCVector() [2/4]	20
6.1.2.3 ARCVector() [3/4]	21
6.1.2.4 ARCVector() [4/4]	21
6.1.3 Member Function Documentation	21
6.1.3.1 AssignLocalCoordinates()	21
6.1.3.2 ConvertToGlobal()	22
6.1.3.3 GlobalVector()	22
6.1.3.4 IsLocal()	22
6.1.3.5 LocalVector()	22
6.1.3.6 MapPhi()	22
6.1.3.7 operator+=()	22
6.1.3.8 ReflectY()	23
6.1.3.9 ReflectZ()	23
6.1.3.10 SetGlobalVector()	23
6.1.3.11 SetX()	23
6.1.3.12 SetZ()	23
6.2 BarrelHelixFunctor Class Reference	24
6.2.1 Detailed Description	24
6.2.2 Constructor & Destructor Documentation	25
6.2.2.1 BarrelHelixFunctor() [1/2]	25
6.2.2.2 BarrelHelixFunctor() [2/2]	25
6.2.3 Member Function Documentation	25
6.2.3.1 operator()()	25
6.3 BarrelRadiatorArray Class Reference	26
6.3.1 Detailed Description	27
6.3.2 Constructor & Destructor Documentation	27

6.3.2.1 BarrelRadiatorArray() [1/2]	27
6.3.2.2 BarrelRadiatorArray() [2/2]	27
6.3.3 Member Function Documentation	27
6.3.3.1 DrawRadiatorArray()	27
6.3.3.2 FindRadiator()	28
6.3.3.3 IsAboveMainRow()	28
6.3.3.4 IsAboveUpperRow()	29
6.3.3.5 IsBelowMainRow()	29
6.4 BarrelRadiatorCell Class Reference	30
6.4.1 Detailed Description	32
6.4.2 Constructor & Destructor Documentation	32
6.4.2.1 BarrelRadiatorCell() [1/2]	32
6.4.2.2 BarrelRadiatorCell() [2/2]	32
6.4.3 Member Function Documentation	32
6.4.3.1 DrawRadiatorGeometry()	32
6.4.3.2 IsDetectorInsideCell()	33
6.4.3.3 IsEdgeCell()	33
6.4.3.4 IsInsideCell()	33
6.4.3.5 ReversePhiRotation()	33
6.4.4 Member Data Documentation	33
6.4.4.1 m_BarrelRadius	33
6.5 CherenkovFile Struct Reference	34
6.5.1 Detailed Description	34
6.5.2 Constructor & Destructor Documentation	35
6.5.2.1 CherenkovFile()	35
6.5.3 Member Function Documentation	35
6.5.3.1 Close()	35
6.5.3.2 Fill()	35
6.5.4 Member Data Documentation	35
6.5.4.1 CherenkovAngle_Reco	35
6.5.4.2 CherenkovAngle_Reco_TrueEmissionPoint	35
6.5.4.3 CherenkovAngle_True	35
6.5.4.4 CherenkovTree	36
6.5.4.5 CosTheta	36
6.5.4.6 Entrance_x	36
6.5.4.7 Entrance_y	36
6.5.4.8 Entrance_z	36
6.5.4.9 File	36
6.5.4.10 FinalRadiatorColumnNumber	36
6.5.4.11 FinalRadiatorRowNumber	37
6.5.4.12 HasMigrated	37
6.5.4.13 MaxPhotons	37

6.5.4.14 MirrorHit_x	37
6.5.4.15 MirrorHit_y	37
6.5.4.16 MirrorHit_z	37
6.5.4.17 Momentum	37
6.5.4.18 NumberGoodPhotons	38
6.5.4.19 NumberPhotons	38
6.5.4.20 ParticleLocation	38
6.5.4.21 Phi	38
6.5.4.22 PhotonEnergy	38
6.5.4.23 PhotonHit_x	38
6.5.4.24 PhotonHit_y	38
6.5.4.25 PhotonHit_z	38
6.5.4.26 PhotonStatus	39
6.5.4.27 RadiatorColumnNumber	39
6.5.4.28 RadiatorRowNumber	39
6.5.4.29 Significance	39
6.5.4.30 SinglePhotonResolution	39
6.5.4.31 TotalResolution	39
6.5.4.32 TrackNumber	39
6.6 de::IOptimizable::Constraints Struct Reference	40
6.6.1 Detailed Description	40
6.6.2 Constructor & Destructor Documentation	40
6.6.2.1 Constraints()	40
6.6.3 Member Function Documentation	40
6.6.3.1 Check()	40
6.6.4 Member Data Documentation	40
6.6.4.1 isConstrained	40
6.6.4.2 lower	41
6.6.4.3 upper	41
6.7 de::DifferentialEvolution Class Reference	41
6.7.1 Detailed Description	41
6.7.2 Constructor & Destructor Documentation	41
6.7.2.1 DifferentialEvolution()	41
6.7.3 Member Function Documentation	42
6.7.3.1 GetBestAgent()	42
6.7.3.2 GetBestCost()	42
6.7.3.3 GetPopulationWithCosts()	42
6.7.3.4 InitPopulation()	42
6.7.3.5 Optimize()	42
6.7.3.6 PrintPopulation()	43
6.7.3.7 SelectionAndCorssing()	43
6.8 EndCapHelixFunctor Class Reference	43

6.8.1 Detailed Description	44
6.8.2 Constructor & Destructor Documentation	44
6.8.2.1 EndCapHelixFunctor() [1/2]	44
6.8.2.2 EndCapHelixFunctor() [2/2]	44
6.8.3 Member Function Documentation	44
6.8.3.1 operator()	44
6.9 EndCapRadiatorArray Class Reference	45
6.9.1 Detailed Description	46
6.9.2 Constructor & Destructor Documentation	46
6.9.2.1 EndCapRadiatorArray() [1/2]	46
6.9.2.2 EndCapRadiatorArray() [2/2]	46
6.9.3 Member Function Documentation	47
6.9.3.1 DrawRadiatorArray()	47
6.9.3.2 FindRadiator()	47
6.9.3.3 IsBelowEvenRow()	47
6.9.3.4 IsBelowOddRow()	47
6.9.3.5 IsBelowThisRow()	48
6.10 EndCapRadiatorCell Class Reference	48
6.10.1 Detailed Description	50
6.10.2 Constructor & Destructor Documentation	50
6.10.2.1 EndCapRadiatorCell() [1/2]	50
6.10.2.2 EndCapRadiatorCell() [2/2]	51
6.10.3 Member Function Documentation	51
6.10.3.1 DrawRadiatorGeometry()	51
6.10.3.2 IsDetectorInsideCell()	51
6.10.3.3 IsEdgeCell()	51
6.10.3.4 IsInsideCell() [1/2]	51
6.10.3.5 IsInsideCell() [2/2]	52
6.10.4 Member Data Documentation	52
6.10.4.1 m_InvalidCells	52
6.10.4.2 m_ValidCells	52
6.11 EventDisplay Class Reference	52
6.11.1 Detailed Description	53
6.11.2 Constructor & Destructor Documentation	53
6.11.2.1 EventDisplay()	53
6.11.3 Member Function Documentation	53
6.11.3.1 AddObject() [1/2]	53
6.11.3.2 AddObject() [2/2]	53
6.11.3.3 DrawEventDisplay()	54
6.12 HalfRadiatorCell Class Reference	54
6.12.1 Detailed Description	56
6.12.2 Constructor & Destructor Documentation	57

6.12.2.1 HalfRadiatorCell() [1/2]	57
6.12.2.2 HalfRadiatorCell() [2/2]	57
6.12.3 Member Function Documentation	57
6.12.3.1 DrawRadiatorGeometry()	57
6.12.3.2 IsInsideCell()	57
6.13 HelixFunctor Class Reference	58
6.13.1 Detailed Description	58
6.13.2 Constructor & Destructor Documentation	58
6.13.2.1 HelixFunctor()	58
6.13.3 Member Function Documentation	58
6.13.3.1 operator()	58
6.14 HelixPath Class Reference	59
6.14.1 Detailed Description	59
6.14.2 Constructor & Destructor Documentation	59
6.14.2.1 HelixPath() [1/2]	59
6.14.2.2 HelixPath() [2/2]	59
6.14.3 Member Function Documentation	59
6.14.3.1 GetDirection()	59
6.14.3.2 GetPosition()	60
6.14.3.3 MapPhi()	60
6.14.3.4 ReflectY()	60
6.14.3.5 ReflectZ()	60
6.14.3.6 ResetOrigin()	60
6.14.3.7 SolvePathLength()	61
6.15 de::IOptimizable Class Reference	61
6.15.1 Detailed Description	62
6.15.2 Constructor & Destructor Documentation	62
6.15.2.1 ~IOptimizable()	62
6.15.3 Member Function Documentation	62
6.15.3.1 EvaluateCost()	62
6.15.3.2 GetConstraints()	62
6.15.3.3 NumberOfParameters()	62
6.16 MirrorHelixFunctor Class Reference	63
6.16.1 Detailed Description	63
6.16.2 Constructor & Destructor Documentation	64
6.16.2.1 MirrorHelixFunctor() [1/2]	64
6.16.2.2 MirrorHelixFunctor() [2/2]	64
6.16.3 Member Function Documentation	64
6.16.3.1 operator()	64
6.17 Particle Class Reference	65
6.17.1 Detailed Description	66
6.17.2 Constructor & Destructor Documentation	66

6.17.2.1 Particle()	66
6.17.2.2 ~Particle()	66
6.17.3 Member Function Documentation	66
6.17.3.1 ConvertBackToGlobalCoordinates()	66
6.17.3.2 ConvertToRadiatorCoordinates()	66
6.17.3.3 FindRadiator()	67
6.17.3.4 GetPosition()	67
6.17.3.5 GetRadiatorCell()	67
6.17.3.6 GetRadiatorColumnNumber()	67
6.17.3.7 GetRadiatorRowNumber()	67
6.17.3.8 IsAtRadiator()	67
6.17.3.9 MapPhi()	67
6.17.3.10 ReflectY()	68
6.17.3.11 ReflectZ()	68
6.17.4 Member Data Documentation	68
6.17.4.1 m_Position	68
6.17.4.2 m_RadiatorCell	68
6.18 ParticleTrack Class Reference	69
6.18.1 Detailed Description	70
6.18.2 Member Enumeration Documentation	70
6.18.2.1 Location	70
6.18.3 Constructor & Destructor Documentation	71
6.18.3.1 ParticleTrack()	71
6.18.3.2 ~ParticleTrack()	71
6.18.4 Member Function Documentation	71
6.18.4.1 Beta()	71
6.18.4.2 ConvertBackToGlobalCoordinates()	72
6.18.4.3 ConvertToRadiatorCoordinates()	72
6.18.4.4 DrawParticleTrack()	72
6.18.4.5 GeneratePhoton()	72
6.18.4.6 GeneratePhotonFromAerogel()	73
6.18.4.7 GeneratePhotonFromGas()	73
6.18.4.8 GeneratePhotonsFromAerogel()	73
6.18.4.9 GeneratePhotonsFromGas()	73
6.18.4.10 GetEntranceWindowPosition()	73
6.18.4.11 GetParticleLocation()	73
6.18.4.12 IsAtRadiator()	73
6.18.4.13 MapPhi()	73
6.18.4.14 ReflectY()	74
6.18.4.15 ReflectZ()	74
6.18.4.16 SetRadiator()	74
6.18.4.17 TrackThroughAerogel()	74

6.18.4.18 TrackThroughGasToMirror()	74
6.18.4.19 TrackThroughTracker()	74
6.18.4.20 TrackToNextCell()	75
6.19 Photon Class Reference	75
6.19.1 Detailed Description	77
6.19.2 Member Enumeration Documentation	77
6.19.2.1 Radiator	77
6.19.2.2 Status	78
6.19.3 Constructor & Destructor Documentation	78
6.19.3.1 Photon() [1/3]	78
6.19.3.2 ~Photon()	79
6.19.3.3 Photon() [2/3]	79
6.19.3.4 Photon() [3/3]	79
6.19.4 Member Function Documentation	79
6.19.4.1 AddAerogelTravelDistance()	79
6.19.4.2 ConvertBackToGlobalCoordinates()	80
6.19.4.3 ConvertToRadiatorCoordinates()	80
6.19.4.4 DrawPhotonPath()	80
6.19.4.5 GetAerogelTravelDistance()	80
6.19.4.6 GetCosCherenkovAngle()	80
6.19.4.7 GetDirection()	80
6.19.4.8 GetEmissionPoint()	81
6.19.4.9 GetEnergy()	81
6.19.4.10 GetMirrorHitPosition()	81
6.19.4.11 GetParticleDirection()	81
6.19.4.12 GetRadiator()	81
6.19.4.13 GetStatus()	81
6.19.4.14 GetWeight()	82
6.19.4.15 HasPhotonMigrated()	82
6.19.4.16 IsAtRadiator()	82
6.19.4.17 KickPhoton()	82
6.19.4.18 MapPhi()	82
6.19.4.19 PhotonHasMigrated()	83
6.19.4.20 PropagatePhoton()	83
6.19.4.21 PutPhotonToEmissionPoint()	83
6.19.4.22 ReflectY()	83
6.19.4.23 ReflectZ()	84
6.19.4.24 RegisterMirrorHitPosition()	84
6.19.4.25 UpdatePhotonStatus()	84
6.20 PhotonHit Struct Reference	84
6.20.1 Detailed Description	85
6.20.2 Member Data Documentation	85

6.20.2.1 m_CentreHitDistance	85
6.20.2.2 m_HitPosition	86
6.20.2.3 m_Photon	86
6.21 PhotonReconstructor::QuarticSolution Struct Reference	86
6.21.1 Detailed Description	86
6.21.2 Member Data Documentation	86
6.21.2.1 m_CosBeta	86
6.21.2.2 m_DegenerateSolution	87
6.21.2.3 m_SinBeta	87
6.22 RadiatorArray Class Reference	87
6.22.1 Detailed Description	88
6.22.2 Constructor & Destructor Documentation	88
6.22.2.1 RadiatorArray() [1/2]	88
6.22.2.2 RadiatorArray() [2/2]	88
6.22.2.3 ~RadiatorArray()	88
6.22.3 Member Function Documentation	88
6.22.3.1 DrawRadiatorArray()	88
6.22.3.2 FindRadiator()	89
6.22.3.3 GetRadiatorCell()	89
6.22.3.4 operator>()	89
6.22.4 Member Data Documentation	89
6.22.4.1 m_Cells	89
6.22.4.2 m_NumberMainRowCells	89
6.22.4.3 m_xHexDist	90
6.22.4.4 m_yHexDist	90
6.23 RadiatorCell Class Reference	90
6.23.1 Detailed Description	92
6.23.2 Constructor & Destructor Documentation	92
6.23.2.1 RadiatorCell() [1/2]	92
6.23.2.2 RadiatorCell() [2/2]	92
6.23.2.3 ~RadiatorCell()	93
6.23.3 Member Function Documentation	93
6.23.3.1 DrawRadiatorGeometry()	93
6.23.3.2 GetAerogelThickness()	93
6.23.3.3 GetCellNumber()	93
6.23.3.4 GetCoolingThickness()	93
6.23.3.5 GetDetector()	93
6.23.3.6 GetGlobalMirrorCentre()	94
6.23.3.7 GetHexagonSize()	94
6.23.3.8 GetMirrorCentre()	94
6.23.3.9 GetMirrorCurvature()	94
6.23.3.10 GetMirrorCurvatureCentreZ()	94

6.23.3.11 GetRadiatorPosition()	94
6.23.3.12 GetRadiatorRotation()	95
6.23.3.13 GetRadiatorThickness()	95
6.23.3.14 GetVesselThickness()	95
6.23.3.15 IsDetectorInsideCell()	95
6.23.3.16 IsEdgeCell()	95
6.23.3.17 IsInsideCell() [1/2]	95
6.23.3.18 IsInsideCell() [2/2]	96
6.23.3.19 ReversePhiRotation()	96
6.23.3.20 SetDetectorPosition()	96
6.23.3.21 SetDetectorTilt()	96
6.23.3.22 SetMirrorCurvature()	96
6.23.3.23 SetMirrorXPosition()	97
6.23.3.24 SetMirrorZPosition()	97
6.23.4 Member Data Documentation	97
6.23.4.1 m_AerogelThickness	97
6.23.4.2 m_CellNumber	97
6.23.4.3 m_CoolingThickness	97
6.23.4.4 m_DefaultMirrorCentre	98
6.23.4.5 m_Detector	98
6.23.4.6 m_HexagonSize	98
6.23.4.7 m_MirrorCentre	98
6.23.4.8 m_MirrorCurvature	98
6.23.4.9 m_Position	98
6.23.4.10 m_RadiatorThickness	99
6.23.4.11 m_Rotation	99
6.23.4.12 m_VesselThickness	99
6.24 ReconstructedPhoton Struct Reference	99
6.24.1 Detailed Description	100
6.24.2 Constructor & Destructor Documentation	101
6.24.2.1 ReconstructedPhoton()	101
6.24.3 Member Data Documentation	102
6.24.3.1 m_CosCherenkovAngle	102
6.24.3.2 m_CosCherenkovAngle_TrueEmissionPoint	102
6.24.3.3 m_Photon	102
6.25 ResolutionOptimizable Class Reference	103
6.25.1 Detailed Description	104
6.25.2 Constructor & Destructor Documentation	104
6.25.2.1 ResolutionOptimizable()	104
6.25.3 Member Function Documentation	104
6.25.3.1 EvaluateCost()	104
6.25.3.2 GetConstraints()	104

6.25.3.3 GetFixedParameters()	105
6.25.3.4 NumberOfParameters()	105
6.26 ResolutionUtilities::ResolutionStruct Struct Reference	105
6.26.1 Detailed Description	105
6.26.2 Member Data Documentation	105
6.26.2.1 CentreHitDistance	105
6.26.2.2 HitCorrectCell	106
6.26.2.3 HitTopWall	106
6.26.2.4 N	106
6.26.2.5 SumWeights	106
6.26.2.6 x	106
6.27 Utilities::ResolutionStruct Struct Reference	106
6.27.1 Detailed Description	107
6.27.2 Member Data Documentation	107
6.27.2.1 CentreHitDistance	107
6.27.2.2 HitCorrectCell	107
6.27.2.3 HitTopWall	107
6.27.2.4 N	107
6.27.2.5 SumWeights	108
6.27.2.6 x	108
6.28 Settings Class Reference	108
6.28.1 Detailed Description	108
6.28.2 Constructor & Destructor Documentation	109
6.28.2.1 Settings()	109
6.28.3 Member Function Documentation	109
6.28.3.1 AddSettings()	109
6.28.3.2 Exists()	109
6.28.3.3 GetBool()	109
6.28.3.4 GetDouble()	109
6.28.3.5 GetInt()	110
6.28.3.6 GetIntVector()	110
6.28.3.7 GetSizeT()	110
6.28.3.8 GetSizeTVector()	110
6.28.3.9 GetString()	110
6.29 SiPM Class Reference	111
6.29.1 Detailed Description	111
6.29.2 Constructor & Destructor Documentation	111
6.29.2.1 SiPM()	111
6.29.3 Member Function Documentation	111
6.29.3.1 AddPhotonHit()	111
6.29.3.2 DrawSiPM()	111
6.29.3.3 GetDetectorSizeX()	112

6.29.3.4 GetDetectorSizeY()	112
6.29.3.5 GetDetectorTilt()	112
6.29.3.6 GetDetectorXPosition()	112
6.29.3.7 GetDetectorZPosition()	112
6.29.3.8 GetPhotonHits()	113
6.29.3.9 IsDetectorHit()	113
6.29.3.10 PlotHits()	113
6.29.3.11 SetDetectorPosition()	113
6.29.3.12 SetDetectorTilt()	113
6.30 TrackingVolume Class Reference	114
6.30.1 Detailed Description	114
6.30.2 Constructor & Destructor Documentation	114
6.30.2.1 TrackingVolume()	114
6.30.3 Member Function Documentation	114
6.30.3.1 DrawARCGeometry()	114
6.30.3.2 GetFieldStrength()	114
6.30.3.3 GetRadius()	115
6.31 ZPlaneHelixFunctor Class Reference	115
6.31.1 Detailed Description	116
6.31.2 Constructor & Destructor Documentation	116
6.31.2.1 ZPlaneHelixFunctor() [1/2]	116
6.31.2.2 ZPlaneHelixFunctor() [2/2]	116
6.31.3 Member Function Documentation	116
6.31.3.1 operator()()	116
7 File Documentation	119
7.1 apps/OptimiseARC.cpp File Reference	119
7.1.1 Typedef Documentation	120
7.1.1.1 Tracks	120
7.1.2 Function Documentation	120
7.1.2.1 main()	120
7.2 OptimiseARC.cpp	120
7.3 apps/RunARC.cpp File Reference	122
7.3.1 Function Documentation	123
7.3.1.1 main()	123
7.4 RunARC.cpp	123
7.5 include/ARCVector.h File Reference	127
7.5.1 Typedef Documentation	127
7.5.1.1 Rotation3D	127
7.5.1.2 Vector	128
7.6 ARCVector.h	129
7.7 include/BarrelRadiatorArray.h File Reference	129

7.8 BarrelRadiatorArray.h	130
7.9 include/BarrelRadiatorCell.h File Reference	131
7.10 BarrelRadiatorCell.h	132
7.11 include/DifferentialEvolution.h File Reference	132
7.11.1 Detailed Description	133
7.12 DifferentialEvolution.h	133
7.13 include/EndCapRadiatorArray.h File Reference	137
7.14 EndCapRadiatorArray.h	138
7.15 include/EndCapRadiatorCell.h File Reference	139
7.16 EndCapRadiatorCell.h	140
7.17 include/EventDisplay.h File Reference	140
7.18 EventDisplay.h	141
7.19 include/HalfRadiatorCell.h File Reference	142
7.19.1 Macro Definition Documentation	142
7.19.1.1 HAIFRADIATORCELL	142
7.20 HalfRadiatorCell.h	143
7.21 include/HelixFunctor.h File Reference	143
7.22 HelixFunctor.h	144
7.23 include/HelixPath.h File Reference	144
7.23.1 Typedef Documentation	145
7.23.1.1 RotationZ	145
7.24 HelixPath.h	146
7.25 include/Particle.h File Reference	146
7.26 Particle.h	147
7.27 include/ParticleMass.h File Reference	147
7.28 ParticleMass.h	148
7.29 include/ParticleTrack.h File Reference	148
7.30 ParticleTrack.h	149
7.31 include/Photon.h File Reference	150
7.32 Photon.h	150
7.33 include/PhotonMapper.h File Reference	152
7.34 PhotonMapper.h	153
7.35 include/PhotonReconstructor.h File Reference	153
7.36 PhotonReconstructor.h	154
7.37 include/RadiatorArray.h File Reference	155
7.38 RadiatorArray.h	155
7.39 include/RadiatorCell.h File Reference	156
7.40 RadiatorCell.h	157
7.41 include/ReconstructedPhoton.h File Reference	158
7.42 ReconstructedPhoton.h	158
7.43 include/ResolutionOptimizable.h File Reference	159
7.43.1 Typedef Documentation	159

7.43.1.1 Tracks	159
7.44 ResolutionOptimizable.h	160
7.45 include/ResolutionUtilities.h File Reference	160
7.45.1 Typedef Documentation	161
7.45.1.1 Tracks	161
7.46 ResolutionUtilities.h	162
7.47 include/Settings.h File Reference	162
7.47.1 Typedef Documentation	163
7.47.1.1 ssMap	163
7.48 Settings.h	163
7.49 include/SiPM.h File Reference	164
7.50 SiPM.h	164
7.51 include/TrackingVolume.h File Reference	165
7.52 TrackingVolume.h	166
7.53 include/Utilities.h File Reference	166
7.54 Utilities.h	168
7.55 src/ARCVector.cpp File Reference	168
7.56 ARCVector.cpp	169
7.57 src/BarrelRadiatorArray.cpp File Reference	170
7.58 BarrelRadiatorArray.cpp	171
7.59 src/BarrelRadiatorCell.cpp File Reference	173
7.59.1 Typedef Documentation	174
7.59.1.1 RotationY	174
7.60 BarrelRadiatorCell.cpp	174
7.61 apps/CMakeLists.txt File Reference	176
7.62 src/CMakeLists.txt File Reference	176
7.63 src/EndCapRadiatorArray.cpp File Reference	176
7.64 EndCapRadiatorArray.cpp	177
7.65 src/EndCapRadiatorCell.cpp File Reference	179
7.66 EndCapRadiatorCell.cpp	180
7.67 src/EventDisplay.cpp File Reference	183
7.68 EventDisplay.cpp	183
7.69 src/HalfRadiatorCell.cpp File Reference	184
7.70 HalfRadiatorCell.cpp	185
7.71 src/HelixFunctor.cpp File Reference	186
7.72 HelixFunctor.cpp	187
7.73 src/HelixPath.cpp File Reference	188
7.73.1 Typedef Documentation	188
7.73.1.1 Functor1D	188
7.74 HelixPath.cpp	188
7.75 src/Particle.cpp File Reference	189
7.76 Particle.cpp	190

7.77 src/ParticleMass.cpp File Reference	191
7.78 ParticleMass.cpp	192
7.79 src/ParticleTrack.cpp File Reference	192
7.80 ParticleTrack.cpp	193
7.81 src/Photon.cpp File Reference	197
7.82 Photon.cpp	198
7.83 src/PhotonMapper.cpp File Reference	201
7.84 PhotonMapper.cpp	202
7.85 src/PhotonReconstructor.cpp File Reference	203
7.86 PhotonReconstructor.cpp	204
7.87 src/RadiatorArray.cpp File Reference	205
7.88 RadiatorArray.cpp	206
7.89 src/RadiatorCell.cpp File Reference	206
7.90 RadiatorCell.cpp	207
7.91 src/ReconstructedPhoton.cpp File Reference	209
7.92 ReconstructedPhoton.cpp	209
7.93 src/ResolutionOptimizable.cpp File Reference	209
7.93.1 Typedef Documentation	210
7.93.1.1 Constraints	210
7.94 ResolutionOptimizable.cpp	210
7.95 src/ResolutionUtilities.cpp File Reference	211
7.96 ResolutionUtilities.cpp	212
7.97 src/Settings.cpp File Reference	215
7.98 Settings.cpp	215
7.99 src/SiPM.cpp File Reference	217
7.100 SiPM.cpp	218
7.101 src/TrackingVolume.cpp File Reference	220
7.102 TrackingVolume.cpp	220
7.103 src/Utilities.cpp File Reference	221
7.104 Utilities.cpp	222
Index	227

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

de	9
ParticleMass	9
PhotonMapper	10
PhotonReconstructor	11
ResolutionUtilities	13
Utilities	14

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

ARCVector	19
CherenkovFile	34
de::IOptimizable::Constraints	40
de::DifferentialEvolution	41
EventDisplay	52
HelixFunctor	58
BarrelHelixFunctor	24
EndCapHelixFunctor	43
MirrorHelixFunctor	63
ZPlaneHelixFunctor	115
HelixPath	59
de::IOptimizable	61
ResolutionOptimizable	103
Particle	65
ParticleTrack	69
Photon	75
PhotonHit	84
PhotonReconstructor::QuarticSolution	86
RadiatorArray	87
BarrelRadiatorArray	26
EndCapRadiatorArray	45
RadiatorCell	90
BarrelRadiatorCell	30
HalfRadiatorCell	54
EndCapRadiatorCell	48
ReconstructedPhoton	99
ResolutionUtilities::ResolutionStruct	105
Utilities::ResolutionStruct	106
Settings	108
SiPM	111
TrackingVolume	114

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

ARCVector	19
BarrelHelixFunctor	24
BarrelRadiatorArray	26
BarrelRadiatorCell	30
CherenkovFile	34
de::IOptimizable::Constraints	40
de::DifferentialEvolution	41
EndCapHelixFunctor	43
EndCapRadiatorArray	45
EndCapRadiatorCell	48
EventDisplay	52
HalfRadiatorCell	54
HelixFunctor	58
HelixPath	59
de::IOptimizable	61
MirrorHelixFunctor	63
Particle	65
ParticleTrack	69
Photon	75
PhotonHit	84
PhotonReconstructor::QuarticSolution	86
RadiatorArray	87
RadiatorCell	90
ReconstructedPhoton	99
ResolutionOptimizable	103
ResolutionUtilities::ResolutionStruct	105
Utilities::ResolutionStruct	106
Settings	108
SiPM	111
TrackingVolume	114
ZPlaneHelixFunctor	115

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

apps/OptimiseARC.cpp	119
apps/RunARC.cpp	122
include/ARCVector.h	127
include/BarrelRadiatorArray.h	129
include/BarrelRadiatorCell.h	131
include/DifferentialEvolution.h	132
include/EndCapRadiatorArray.h	137
include/EndCapRadiatorCell.h	139
include/EventDisplay.h	140
include/HalfRadiatorCell.h	142
include/HelixFunctor.h	143
include/HelixPath.h	144
include/Particle.h	146
include/ParticleMass.h	147
include/ParticleTrack.h	148
include/Photon.h	150
include/PhotonMapper.h	152
include/PhotonReconstructor.h	153
include/RadiatorArray.h	155
include/RadiatorCell.h	156
include/ReconstructedPhoton.h	158
include/ResolutionOptimizable.h	159
include/ResolutionUtilities.h	160
include/Settings.h	162
include/SiPM.h	164
include/TrackingVolume.h	165
include/Utilities.h	166
src/ARCVector.cpp	168
src/BarrelRadiatorArray.cpp	170
src/BarrelRadiatorCell.cpp	173
src/EndCapRadiatorArray.cpp	176
src/EndCapRadiatorCell.cpp	179
src/EventDisplay.cpp	183
src/HalfRadiatorCell.cpp	184
src/HelixFunctor.cpp	186

src/ HelixPath.cpp	188
src/ Particle.cpp	189
src/ ParticleMass.cpp	191
src/ ParticleTrack.cpp	192
src/ Photon.cpp	197
src/ PhotonMapper.cpp	201
src/ PhotonReconstructor.cpp	203
src/ RadiatorArray.cpp	205
src/ RadiatorCell.cpp	206
src/ ReconstructedPhoton.cpp	209
src/ ResolutionOptimizable.cpp	209
src/ ResolutionUtilities.cpp	211
src/ Settings.cpp	215
src/ SiPM.cpp	217
src/ TrackingVolume.cpp	220
src/ Utilities.cpp	221

Chapter 5

Namespace Documentation

5.1 de Namespace Reference

Classes

- class [DifferentialEvolution](#)
- class [IOptimizable](#)

5.2 ParticleMass Namespace Reference

Functions

- double [GetMass](#) (int PID)

5.2.1 Detailed Description

[ParticleMass](#) is a namespace storing all the particle masses in a static map

5.2.2 Function Documentation

5.2.2.1 GetMass()

```
double ParticleMass::GetMass (  
    int PID)
```

Get the particle mass from its PDG code (antiparticles have the same mass) Throws `std::invalid_argument` if the code is not known

Definition at line 11 of file [ParticleMass.cpp](#).

5.3 PhotonMapper Namespace Reference

Functions

- double [PhotonMirrorDistance](#) (const [Photon](#) &photon)
- void [TracePhotonToMirror](#) ([Photon](#) &photon)
- [PhotonHit](#) [TracePhotonToDetector](#) ([Photon](#) &photon)
- std::unique_ptr< [PhotonHit](#) > [TracePhoton](#) ([Photon](#) &photon, const [RadiatorArray](#) &radiatorArray)
- bool [IsScatteredInAerogel](#) (const [Photon](#) &photon)

5.3.1 Detailed Description

[PhotonMapper](#) is a namespace containing the functions for mapping photons from their emission point to the detector plane

5.3.2 Function Documentation

5.3.2.1 IsScatteredInAerogel()

```
bool PhotonMapper::IsScatteredInAerogel (  
    const Photon & photon)
```

Check if photon has been scattered in the aerogel

Definition at line 112 of file [PhotonMapper.cpp](#).

5.3.2.2 PhotonMirrorDistance()

```
double PhotonMapper::PhotonMirrorDistance (  
    const Photon & photon)
```

Find the distance between the photon and the mirror

Definition at line 16 of file [PhotonMapper.cpp](#).

5.3.2.3 TracePhoton()

```
std::unique_ptr< PhotonHit > PhotonMapper::TracePhoton (  
    Photon & photon,  
    const RadiatorArray & radiatorArray)
```

Trace photon from emission point to detector and register detector hit

Definition at line 87 of file [PhotonMapper.cpp](#).

5.3.2.4 TracePhotonToDetector()

```
PhotonHit PhotonMapper::TracePhotonToDetector (
    Photon & photon)
```

Trace photon from mirror to detector plane and register detector hit

Definition at line 55 of file [PhotonMapper.cpp](#).

5.3.2.5 TracePhotonToMirror()

```
void PhotonMapper::TracePhotonToMirror (
    Photon & photon)
```

Trace photon from emission point to mirror

Definition at line 32 of file [PhotonMapper.cpp](#).

5.4 PhotonReconstructor Namespace Reference

Classes

- struct [QuarticSolution](#)

Functions

- double [ReconstructCosCherenkovAngle](#) (const [PhotonHit](#) &photonHit, bool TrueEmissionPoint)
- [ReconstructedPhoton](#) [ReconstructPhoton](#) (const [PhotonHit](#) &photonHit)
- bool [IsReconstructed](#) (double CosAngle)
- [QuarticSolution](#) [SolveQuartic](#) (double EmissionMirrorDist, double DetectionMirrorParaDist, double DetectionMirrorPerpDist)
- [Vector](#) [GetReflectionPoint](#) (const [Vector](#) &EmissionPoint, const [Vector](#) &DetectionPoint, double DetectionMirrorParaDist, double DetectionMirrorPerpDist, double EmissionMirrorDist, const [QuarticSolution](#) &quarticSolution, std::size_t SolutionNumber)

Variables

- constexpr double [FailedReconstruction](#) = -2.0

5.4.1 Function Documentation

5.4.1.1 GetReflectionPoint()

```
Vector PhotonReconstructor::GetReflectionPoint (
    const Vector & EmissionPoint,
    const Vector & DetectionPoint,
    double DetectionMirrorParaDist,
    double DetectionMirrorPerpDist,
    double EmissionMirrorDist,
    const QuarticSolution & quarticSolution,
    std::size_t SolutionNumber)
```

Helper function to calculate the reflection point after solving the quartic equation

Definition at line 115 of file [PhotonReconstructor.cpp](#).

5.4.1.2 IsReconstructed()

```
bool PhotonReconstructor::IsReconstructed (
    double CosAngle) [inline]
```

Check if a reconstructed cosine is a valid result rather than the failure sentinel

Definition at line 38 of file [PhotonReconstructor.h](#).

5.4.1.3 ReconstructCosCherenkovAngle()

```
double PhotonReconstructor::ReconstructCosCherenkovAngle (
    const PhotonHit & photonHit,
    bool TrueEmissionPoint)
```

Reconstruct cosine of Cherenkov angle of photon from the photon hit and charged track

Parameters

<i>photonHit</i>	The photon hit in the detector
<i>TrueEmissionPoint</i>	Set to true to use the true emission point

Definition at line 17 of file [PhotonReconstructor.cpp](#).

5.4.1.4 ReconstructPhoton()

```
ReconstructedPhoton PhotonReconstructor::ReconstructPhoton (
    const PhotonHit & photonHit)
```

Reconstruct photon from the photon hit, charged track and mirror geometry

Parameters

<i>photonHit</i>	The photon hit in the detector
<i>radiatorCell</i>	The radiator cell

Definition at line 66 of file [PhotonReconstructor.cpp](#).

5.4.1.5 SolveQuartic()

```
QuarticSolution PhotonReconstructor::SolveQuartic (
    double EmissionMirrorDist,
    double DetectionMirrorParaDist,
    double DetectionMirrorPerpDist)
```

Solve quartic equation to determine sine of angle between emission and reflection point, relative to the mirror centre

Parameters

<i>EmissionMirrorDist</i>	Distance between the emission point and mirror centre
<i>DetectionMirrorParaDist</i>	Parallel component of distance between detection point and mirror centre
<i>DetectionMirrorPerpDist</i>	Perpendicular component of distance between detection point and mirror centre

Definition at line 75 of file [PhotonReconstructor.cpp](#).

5.4.2 Variable Documentation

5.4.2.1 FailedReconstruction

```
double PhotonReconstructor::FailedReconstruction = -2.0 [constexpr]
```

Cosine value returned when the reconstruction fails

Definition at line 34 of file [PhotonReconstructor.h](#).

5.5 ResolutionUtilities Namespace Reference

Classes

- struct [ResolutionStruct](#)

Functions

- double [CalculateResolution](#) (const [Tracks](#) &Particles, const [RadiatorCell](#) *radiatorCell, const [RadiatorArray](#) &radiatorArray, std::size_t Seed, bool IncludeCentrePenalty)
- double [fcn](#) (double MirrorCurvature, double MirrorXPosition, double MirrorZPosition, double DetectorPosition, double DetectorTilt, [RadiatorCell](#) &radiatorCell, const [RadiatorArray](#) &radiatorArray, const [Tracks](#) &Particles, std::size_t Seed, bool IncludeCentrePenalty)
- void [PlotProjections](#) ([RadiatorCell](#) &radiatorCell, const [RadiatorArray](#) &radiatorArray, const [Tracks](#) &Particles)
- void [DoFit](#) ([RadiatorCell](#) &radiatorCell, const [RadiatorArray](#) &radiatorArray, const [Tracks](#) &Particles, std::size_t Column, std::size_t Row)

5.5.1 Function Documentation

5.5.1.1 CalculateResolution()

```
double ResolutionUtilities::CalculateResolution (
    const Tracks & Particles,
    const RadiatorCell * radiatorCell,
    const RadiatorArray & radiatorArray,
    std::size_t Seed,
    bool IncludeCentrePenalty)
```

Helper function where the (parallel) calculation takes place

Definition at line 45 of file [ResolutionUtilities.cpp](#).

5.5.1.2 DoFit()

```
void ResolutionUtilities::DoFit (
    RadiatorCell & radiatorCell,
    const RadiatorArray & radiatorArray,
    const Tracks & Particles,
    std::size_t Column,
    std::size_t Row)
```

Do the actual fit and save the results

Definition at line 226 of file [ResolutionUtilities.cpp](#).

5.5.1.3 fcn()

```
double ResolutionUtilities::fcn (
    double MirrorCurvature,
    double MirrorXPosition,
    double MirrorZPosition,
    double DetectorPosition,
    double DetectorTilt,
    RadiatorCell & radiatorCell,
    const RadiatorArray & radiatorArray,
    const Tracks & Particles,
    std::size_t Seed,
    bool IncludeCentrePenalty)
```

The "cost" function for minimisation

Definition at line 101 of file [ResolutionUtilities.cpp](#).

5.5.1.4 PlotProjections()

```
void ResolutionUtilities::PlotProjections (
    RadiatorCell & radiatorCell,
    const RadiatorArray & radiatorArray,
    const Tracks & Particles)
```

Load fit results and plot the fit projections

Definition at line 127 of file [ResolutionUtilities.cpp](#).

5.6 Utilities Namespace Reference

Classes

- struct [ResolutionStruct](#)

Functions

- [Vector VectorFromSpherical](#) (double R, double CosTheta, double Phi)
- [Vector GenerateRandomBarrelTrack](#) (double &CosTheta, double &Phi)
- [Vector GenerateRandomBarrelTrackZRange](#) (double z_min, double z_max)
- [Vector GenerateRandomEndCapTrack](#) ()
- double [GetMomentumMag](#) ()
- TRandom3 & [Random](#) ()
- ULong_t [TrackSeed](#) (std::size_t Seed, std::size_t TrackNumber)
- [ResolutionStruct TrackPhotons](#) ([ParticleTrack](#) particleTrack, const [RadiatorCell](#) &radiatorCell, const [RadiatorArray](#) &radiatorArray)
- double [GetIndexRefraction](#) ([Photon::Radiator](#) Radiator, bool ChromaticDispersion, double Energy=0.0)
- double [GetPredictedCherenkovAngle](#) (double Momentum, int ID, [Photon::Radiator](#) Radiator)
- double [GetCherenkovAngleDifference](#) (double Momentum, int ID1, int ID2, [Photon::Radiator](#) Radiator)

5.6.1 Function Documentation

5.6.1.1 GenerateRandomBarrelTrack()

```
Vector Utilities::GenerateRandomBarrelTrack (
    double & CosTheta,
    double & Phi)
```

Generate a random barrel track which is uniform in z and phi

Definition at line 44 of file [Utilities.cpp](#).

5.6.1.2 GenerateRandomBarrelTrackZRange()

```
Vector Utilities::GenerateRandomBarrelTrackZRange (
    double z_min,
    double z_max)
```

Generate a random barrel track which is uniform in z and phi

Definition at line 25 of file [Utilities.cpp](#).

5.6.1.3 GenerateRandomEndCapTrack()

```
Vector Utilities::GenerateRandomEndCapTrack ()
```

Generate a random end cap track which is uniform in x and y

Definition at line 64 of file [Utilities.cpp](#).

5.6.1.4 GetCherenkovAngleDifference()

```
double Utilities::GetCherenkovAngleDifference (
    double Momentum,
    int ID1,
    int ID2,
    Photon::Radiator Radiator)
```

Get the difference in Cherenkov angle between two mass hypotheses

Parameters

<i>Momentum</i>	The particle momentum
<i>ID1</i>	The first particle hypothesis
<i>ID2</i>	The second particle hypothesis
<i>Radiator</i>	Gas or aerogel

Definition at line 234 of file [Utilities.cpp](#).

5.6.1.5 GetIndexRefraction()

```
double Utilities::GetIndexRefraction (
    Photon::Radiator Radiator,
    bool ChromaticDispersion,
    double Energy = 0.0)
```

Get index of fraction

Parameters

<i>Radiator</i>	Gas or aerogel
<i>Energy</i>	The photon energy

Definition at line 183 of file [Utilities.cpp](#).

5.6.1.6 GetMomentumMag()

```
double Utilities::GetMomentumMag ()
```

Get the magnitude of the particle momentum

Definition at line 89 of file [Utilities.cpp](#).

5.6.1.7 GetPredictedCherenkovAngle()

```
double Utilities::GetPredictedCherenkovAngle (
    double Momentum,
    int ID,
    Photon::Radiator Radiator)
```

Get the predicted Cherenkov angle

Parameters

<i>Momentum</i>	The particle momentum
<i>ID</i>	The particle ID
<i>Radiator</i>	Gas or aerogel

Definition at line 224 of file [Utilities.cpp](#).

5.6.1.8 Random()

```
TRandom3 & Utilities::Random ()
```

Thread-local random number generator used for photon generation and detection. Each thread owns one generator so that tracks can be processed in parallel.

Definition at line 243 of file [Utilities.cpp](#).

5.6.1.9 TrackPhotons()

```
ResolutionStruct Utilities::TrackPhotons (  
    ParticleTrack particleTrack,  
    const RadiatorCell & radiatorCell,  
    const RadiatorArray & radiatorArray)
```

Track photons through radiator cells and return the information about resolutions.

Definition at line 103 of file [Utilities.cpp](#).

5.6.1.10 TrackSeed()

```
ULong_t Utilities::TrackSeed (  
    std::size_t Seed,  
    std::size_t TrackNumber)
```

Seed of the photon generator for one track, mixed from the run seed and the track number so that each track gets an independent sequence that does not depend on the thread it runs on.

Definition at line 248 of file [Utilities.cpp](#).

5.6.1.11 VectorFromSpherical()

```
Vector Utilities::VectorFromSpherical (  
    double R,  
    double CosTheta,  
    double Phi)
```

Create a vector from spherical coordinates.

Definition at line 18 of file [Utilities.cpp](#).

Chapter 6

Class Documentation

6.1 ARCVector Class Reference

```
#include <ARCVector.h>
```

Public Member Functions

- [ARCVector](#) ()=default
- [ARCVector](#) (const [Vector](#) &Vec)
- [ARCVector](#) (const [Vector](#) &Vec, const [Vector](#) &RadiatorPosition, const [Rotation3D](#) &RadiatorRotation)
- [ARCVector](#) (const [ARCVector](#) &Vec)=default
- void [AssignLocalCoordinates](#) (const [Vector](#) &Position, const [Rotation3D](#) &Rotation)
- bool [IsLocal](#) () const
- void [ConvertToGlobal](#) ()
- const [Vector](#) & [LocalVector](#) () const
- [Vector](#) [GlobalVector](#) () const
- [ARCVector](#) & [operator+=](#) (const [Vector](#) &Vec)
- void [MapPhi](#) (double DeltaPhi)
- void [ReflectZ](#) ()
- void [ReflectY](#) ()
- void [SetX](#) (double x)
- void [SetZ](#) (double z)
- void [SetGlobalVector](#) (const [Vector](#) &Vec)

6.1.1 Detailed Description

Definition at line 22 of file [ARCVector.h](#).

6.1.2 Constructor & Destructor Documentation

6.1.2.1 ARCVector() [1/4]

```
ARCVector::ARCVector () [default]
```

Constructor that initialises the vector at the origin in global coordinates

6.1.2.2 **ARCVector()** [2/4]

```
ARCVector::ARCVector (  
    const Vector & Vec) [explicit]
```

Constructor that initialises the vector in the global coordinate system

Parameters

<i>Vec</i>	The global vector
------------	-------------------

Definition at line 14 of file [ARCVector.cpp](#).

6.1.2.3 ARCVector() [3/4]

```
ARCVector::ARCVector (
    const Vector & Vec,
    const Vector & RadiatorPosition,
    const Rotation3D & RadiatorRotation)
```

Constructor that initialises the vector directly in the local coordinates

Parameters

<i>Vec</i>	The local vector
<i>RadiatorPosition</i>	The position of the origin of the local coordinates
<i>RadiatorRotation</i>	The rotation from global to local coordinates

Definition at line 17 of file [ARCVector.cpp](#).

6.1.2.4 ARCVector() [4/4]

```
ARCVector::ARCVector (
    const ARCVector & Vec) [default]
```

Copy constructor is default

6.1.3 Member Function Documentation

6.1.3.1 AssignLocalCoordinates()

```
void ARCVector::AssignLocalCoordinates (
    const Vector & Position,
    const Rotation3D & Rotation)
```

Assign a local coordinate system

Parameters

<i>Position</i>	The position of the local coordinate system
<i>Rotation</i>	The rotation from the global to local coordinate system

Definition at line 24 of file [ARCVector.cpp](#).

6.1.3.2 ConvertToGlobal()

```
void ARCVector::ConvertToGlobal ()
```

Delete local coordinates and turn this vector global again

Definition at line 37 of file [ARCVector.cpp](#).

6.1.3.3 GlobalVector()

```
Vector ARCVector::GlobalVector () const
```

Get the global vector

Definition at line 50 of file [ARCVector.cpp](#).

6.1.3.4 IsLocal()

```
bool ARCVector::IsLocal () const
```

Returns true if the local coordinate system is defined

Definition at line 33 of file [ARCVector.cpp](#).

6.1.3.5 LocalVector()

```
const Vector & ARCVector::LocalVector () const
```

Get the local vector

Definition at line 42 of file [ARCVector.cpp](#).

6.1.3.6 MapPhi()

```
void ARCVector::MapPhi (  
    double DeltaPhi)
```

Rotate vector in the azimuthal direction

Definition at line 65 of file [ARCVector.cpp](#).

6.1.3.7 operator+=()

```
ARCVector & ARCVector::operator+= (  
    const Vector & Vec)
```

Operator for adding a vector

Definition at line 60 of file [ARCVector.cpp](#).

6.1.3.8 ReflectY()

```
void ARCVector::ReflectY ()
```

Reflect vector along y direction

Definition at line 86 of file [ARCVector.cpp](#).

6.1.3.9 ReflectZ()

```
void ARCVector::ReflectZ ()
```

Reflect vector along z direction

Definition at line 76 of file [ARCVector.cpp](#).

6.1.3.10 SetGlobalVector()

```
void ARCVector::SetGlobalVector (
    const Vector & Vect)
```

Set global vector

Definition at line 104 of file [ARCVector.cpp](#).

6.1.3.11 SetX()

```
void ARCVector::SetX (
    double x)
```

Set the x-component, only use this to set the mirror position!

Definition at line 96 of file [ARCVector.cpp](#).

6.1.3.12 SetZ()

```
void ARCVector::SetZ (
    double z)
```

Set the z-component, only use this to set the mirror position!

Definition at line 100 of file [ARCVector.cpp](#).

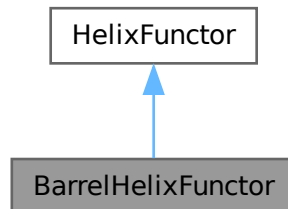
The documentation for this class was generated from the following files:

- [include/ARCVector.h](#)
- [src/ARCVector.cpp](#)

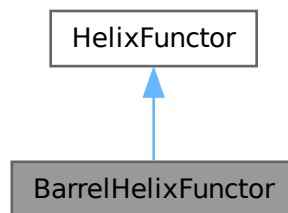
6.2 BarrelHelixFunctor Class Reference

```
#include <HelixFunctor.h>
```

Inheritance diagram for BarrelHelixFunctor:



Collaboration diagram for BarrelHelixFunctor:



Public Member Functions

- [BarrelHelixFunctor](#) (double Radius)
- [BarrelHelixFunctor](#) (const [BarrelHelixFunctor](#) &Functor)=delete
- virtual double [operator\(\)](#) (const [Vector](#) &Vect) const override

Public Member Functions inherited from [HelixFunctor](#)

- [HelixFunctor](#) ()=default

6.2.1 Detailed Description

[BarrelHelixFunctor](#) is a functor for tracking the particle to a certain barrel radius

Definition at line 30 of file [HelixFunctor.h](#).

6.2.2 Constructor & Destructor Documentation

6.2.2.1 BarrelHelixFunctor() [1/2]

```
BarrelHelixFunctor::BarrelHelixFunctor (  
    double Radius)
```

Constructor that sets up the radius that we want to track particle to

Parameters

<i>Radius</i>	The radius we're tracking the particle to
---------------	---

Definition at line 6 of file [HelixFunctor.cpp](#).

6.2.2.2 BarrelHelixFunctor() [2/2]

```
BarrelHelixFunctor::BarrelHelixFunctor (  
    const BarrelHelixFunctor & Functor) [delete]
```

We do not need copy constructor

6.2.3 Member Function Documentation

6.2.3.1 operator>()

```
double BarrelHelixFunctor::operator() (  
    const Vector & Vect) const [override], [virtual]
```

Virtual () operator

Parameters

<i>Vect</i>	Vector position
-------------	---------------------------------

Implements [HelixFunctor](#).

Definition at line 9 of file [HelixFunctor.cpp](#).

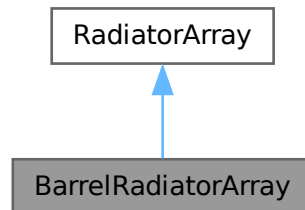
The documentation for this class was generated from the following files:

- include/[HelixFunctor.h](#)
- src/[HelixFunctor.cpp](#)

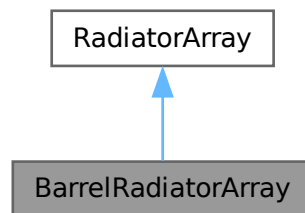
6.3 BarrelRadiatorArray Class Reference

```
#include <BarrelRadiatorArray.h>
```

Inheritance diagram for BarrelRadiatorArray:



Collaboration diagram for BarrelRadiatorArray:



Public Member Functions

- [BarrelRadiatorArray](#) ()
- [BarrelRadiatorArray](#) (const [BarrelRadiatorArray](#) &radiatorCell)=delete
- `std::vector< std::pair< std::unique_ptr< TObject >, std::string > >` [DrawRadiatorArray](#) () const override
- virtual const [RadiatorCell](#) * [FindRadiator](#) ([Particle](#) &particle) const override
- bool [IsBelowMainRow](#) (double x, double y) const
- bool [IsAboveUpperRow](#) (double x, double y, double Radius) const
- bool [IsAboveMainRow](#) (double x, double y) const

Public Member Functions inherited from [RadiatorArray](#)

- [RadiatorArray](#) ()
- [RadiatorArray](#) (const [RadiatorArray](#) &radiatorArray)=delete
- virtual [~RadiatorArray](#) ()=default
- [RadiatorCell](#) * [GetRadiatorCell](#) (std::size_t i, std::size_t j)
- const [RadiatorCell](#) * [operator\(\)](#) (std::size_t i, std::size_t j) const

Additional Inherited Members

Protected Attributes inherited from [RadiatorArray](#)

- `std::vector< std::unique_ptr< RadiatorCell > > m_Cells`
- `const std::size_t m_NumberMainRowCells`
- `const double m_xHexDist`
- `const double m_yHexDist`

6.3.1 Detailed Description

Definition at line 22 of file [BarrelRadiatorArray.h](#).

6.3.2 Constructor & Destructor Documentation

6.3.2.1 [BarrelRadiatorArray\(\)](#) [1/2]

```
BarrelRadiatorArray::BarrelRadiatorArray ()
```

Constructor that sets up the geometry

Definition at line 15 of file [BarrelRadiatorArray.cpp](#).

6.3.2.2 [BarrelRadiatorArray\(\)](#) [2/2]

```
BarrelRadiatorArray::BarrelRadiatorArray (
    const BarrelRadiatorArray & radiatorCell) [delete]
```

Delete copy constructor

6.3.3 Member Function Documentation

6.3.3.1 [DrawRadiatorArray\(\)](#)

```
std::vector< std::pair< std::unique_ptr< TObject >, std::string > > BarrelRadiatorArray::↵
DrawRadiatorArray () const [override], [virtual]
```

Draw all the radiator cells

Implements [RadiatorArray](#).

Definition at line 39 of file [BarrelRadiatorArray.cpp](#).

6.3.3.2 FindRadiator()

```
const RadiatorCell * BarrelRadiatorArray::FindRadiator (  
    Particle & particle) const [override], [virtual]
```

Check which radiator the particle goes through It will map the track momentum and position if the track hits an equivalent radiator cell

Implements [RadiatorArray](#).

Definition at line 52 of file [BarrelRadiatorArray.cpp](#).

6.3.3.3 IsAboveMainRow()

```
bool BarrelRadiatorArray::IsAboveMainRow (  
    double x,  
    double y) const
```

Check if particle track hits above the main row

Parameters

<i>x</i>	Position along the z direction of the barrel, or x in local coordinates
<i>y</i>	Position along the curved direction, or y in location coordinates

Definition at line 132 of file [BarrelRadiatorArray.cpp](#).

6.3.3.4 IsAboveUpperRow()

```
bool BarrelRadiatorArray::IsAboveUpperRow (  
    double x,  
    double y,  
    double Radius) const
```

Check if particle track hits above the upper row

Parameters

<i>x</i>	Position along the z direction of the barrel, or x in local coordinates
<i>y</i>	Position along the curved direction, or y in location coordinates
<i>Radius</i>	The radial coordinate of the particle in global coordinates

Definition at line 126 of file [BarrelRadiatorArray.cpp](#).

6.3.3.5 IsBelowMainRow()

```
bool BarrelRadiatorArray::IsBelowMainRow (  
    double x,  
    double y) const
```

Check if particle track hits below the main row

Parameters

<i>x</i>	Position along the z direction of the barrel, or x in local coordinates
<i>y</i>	Position along the curved direction, or y in location coordinates

Definition at line 112 of file [BarrelRadiatorArray.cpp](#).

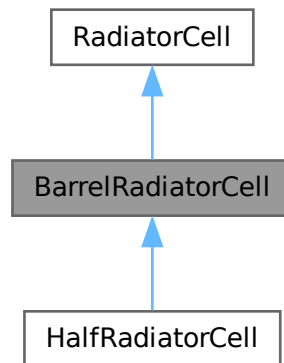
The documentation for this class was generated from the following files:

- [include/BarrelRadiatorArray.h](#)
- [src/BarrelRadiatorArray.cpp](#)

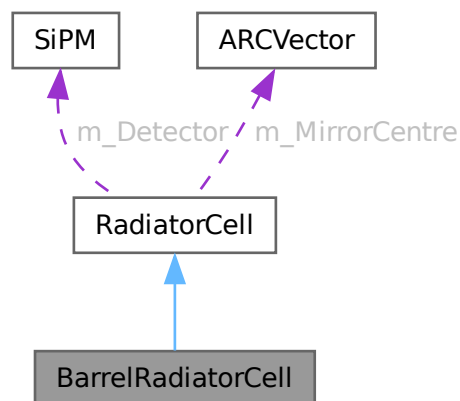
6.4 BarrelRadiatorCell Class Reference

```
#include <BarrelRadiatorCell.h>
```

Inheritance diagram for BarrelRadiatorCell:



Collaboration diagram for BarrelRadiatorCell:



Public Member Functions

- [BarrelRadiatorCell](#) (std::size_t CellColumnNumber, std::size_t CellRowNumber, double HexagonSize)
- [BarrelRadiatorCell](#) (const [BarrelRadiatorCell](#) &radiatorCell)=delete
- virtual bool [IsInsideCell](#) (const [ARCVector](#) &Position) const override
- virtual std::vector< std::pair< std::unique_ptr< TObject >, std::string > > [DrawRadiatorGeometry](#) () const override
- virtual [RotationZ ReversePhiRotation](#) () const override
- virtual bool [IsDetectorInsideCell](#) () const override
- virtual bool [IsEdgeCell](#) () const override

Public Member Functions inherited from RadiatorCell

- [RadiatorCell](#) (std::size_t CellColumnNumber, std::size_t CellRowNumber, double HexagonSize, const [Vector](#) &Position, const [Rotation3D](#) &Rotation, const std::string &Prefix="")
- [RadiatorCell](#) (const [RadiatorCell](#) &radiatorCell)=delete
- virtual [~RadiatorCell](#) ()=default
- double [GetRadiatorThickness](#) () const
- double [GetVesselThickness](#) () const
- double [GetCoolingThickness](#) () const
- double [GetAerogelThickness](#) () const
- const [Vector](#) & [GetMirrorCentre](#) () const
- [Vector](#) [GetGlobalMirrorCentre](#) () const
- double [GetMirrorCurvature](#) () const
- const [Vector](#) & [GetRadiatorPosition](#) () const
- const [Rotation3D](#) & [GetRadiatorRotation](#) () const
- bool [IsInsideCell](#) (const [Photon](#) &photon) const
- double [GetHexagonSize](#) () const
- std::pair< std::size_t, std::size_t > [GetCellNumber](#) () const
- const [SiPM](#) & [GetDetector](#) () const
- void [SetMirrorCurvature](#) (double Curvature)
- void [SetMirrorXPosition](#) (double x)
- void [SetMirrorZPosition](#) (double z)
- void [SetDetectorPosition](#) (double x)
- void [SetDetectorTilt](#) (double Angle)

Protected Attributes

- const double [m_BarrelRadius](#)

Protected Attributes inherited from RadiatorCell

- const [Vector](#) [m_Position](#)
- const [Rotation3D](#) [m_Rotation](#)
- const double [m_RadiatorThickness](#)
- const double [m_VesselThickness](#)
- const double [m_CoolingThickness](#)
- const double [m_AerogelThickness](#)
- const double [m_HexagonSize](#)
- const std::pair< std::size_t, std::size_t > [m_CellNumber](#)
- [SiPM](#) [m_Detector](#)
- double [m_MirrorCurvature](#)
- const [Vector](#) [m_DefaultMirrorCentre](#)
- [ARCVector](#) [m_MirrorCentre](#)

Additional Inherited Members

Protected Member Functions inherited from RadiatorCell

- double [GetMirrorCurvatureCentreZ](#) () const

6.4.1 Detailed Description

[BarrelRadiatorCell](#) inherits from [RadiatorCell](#) and contains the position of the radiator

Definition at line 17 of file [BarrelRadiatorCell.h](#).

6.4.2 Constructor & Destructor Documentation

6.4.2.1 BarrelRadiatorCell() [1/2]

```
BarrelRadiatorCell::BarrelRadiatorCell (
    std::size_t CellColumnNumber,
    std::size_t CellRowNumber,
    double HexagonSize)
```

Constructor that sets up the geometry and a position of the barrel cell

Parameters

<i>CellColumnNumber</i>	The column number of this cell
<i>CellRowNumber</i>	The row number of this cell
<i>HexagonSize</i>	The length between two opposite hexagon edges (not points)

Definition at line 20 of file [BarrelRadiatorCell.cpp](#).

6.4.2.2 BarrelRadiatorCell() [2/2]

```
BarrelRadiatorCell::BarrelRadiatorCell (
    const BarrelRadiatorCell & radiatorCell) [delete]
```

Delete copy constructor

6.4.3 Member Function Documentation

6.4.3.1 DrawRadiatorGeometry()

```
std::vector< std::pair< std::unique_ptr< TObject >, std::string > > BarrelRadiatorCell::↵
DrawRadiatorGeometry () const [override], [virtual]
```

Draw radiator geometry

Implements [RadiatorCell](#).

Reimplemented in [HalfRadiatorCell](#).

Definition at line 47 of file [BarrelRadiatorCell.cpp](#).

6.4.3.2 IsDetectorInsideCell()

```
bool BarrelRadiatorCell::IsDetectorInsideCell () const [override], [virtual]
```

Function that checks if the detector is contained inside the radiator cell

Implements [RadiatorCell](#).

Definition at line 154 of file [BarrelRadiatorCell.cpp](#).

6.4.3.3 IsEdgeCell()

```
bool BarrelRadiatorCell::IsEdgeCell () const [override], [virtual]
```

Checks if the cell is at the edge (near the end cap)

Implements [RadiatorCell](#).

Definition at line 210 of file [BarrelRadiatorCell.cpp](#).

6.4.3.4 IsInsideCell()

```
bool BarrelRadiatorCell::IsInsideCell (  
    const ARCVector & Position) const [override], [virtual]
```

Function that checks if the position is inside the radiator

Implements [RadiatorCell](#).

Reimplemented in [HalfRadiatorCell](#).

Definition at line 32 of file [BarrelRadiatorCell.cpp](#).

6.4.3.5 ReversePhiRotation()

```
Rotation2 BarrelRadiatorCell::ReversePhiRotation () const [override], [virtual]
```

Get the reverse rotation in phi, for plotting purposes

Reimplemented from [RadiatorCell](#).

Definition at line 145 of file [BarrelRadiatorCell.cpp](#).

6.4.4 Member Data Documentation

6.4.4.1 m_BarrelRadius

```
const double BarrelRadiatorCell::m_BarrelRadius [protected]
```

Radius of the barrel where the entrance window is

Definition at line 57 of file [BarrelRadiatorCell.h](#).

The documentation for this class was generated from the following files:

- include/[BarrelRadiatorCell.h](#)
- src/[BarrelRadiatorCell.cpp](#)

6.5 CherenkovFile Struct Reference

Public Member Functions

- [CherenkovFile](#) (const std::string &Filename)
- void [Fill](#) ()
- void [Close](#) ()

Public Attributes

- TFile [File](#)
- TTree [CherenkovTree](#)
- double [CherenkovAngle_Reco_TrueEmissionPoint](#) [MaxPhotons]
- double [CherenkovAngle_Reco](#) [MaxPhotons]
- double [CherenkovAngle_True](#) [MaxPhotons]
- double [PhotonEnergy](#) [MaxPhotons]
- double [Entrance_x](#)
- double [Entrance_y](#)
- double [Entrance_z](#)
- double [MirrorHit_x](#)
- double [MirrorHit_y](#)
- double [MirrorHit_z](#)
- double [PhotonHit_x](#) [MaxPhotons]
- double [PhotonHit_y](#) [MaxPhotons]
- double [PhotonHit_z](#) [MaxPhotons]
- double [Momentum](#)
- double [CosTheta](#)
- double [Phi](#)
- int [NumberPhotons](#) = 0
- double [NumberGoodPhotons](#) = 0.0
- int [HasMigrated](#) [MaxPhotons]
- std::size_t [RadiatorRowNumber](#)
- std::size_t [RadiatorColumnNumber](#)
- std::size_t [TrackNumber](#)
- std::size_t [FinalRadiatorRowNumber](#)
- std::size_t [FinalRadiatorColumnNumber](#)
- [ParticleTrack::Location](#) [ParticleLocation](#)
- [Photon::Status](#) [PhotonStatus](#) [MaxPhotons]
- double [SinglePhotonResolution](#)
- double [TotalResolution](#)
- double [Significance](#)

Static Public Attributes

- static constexpr int [MaxPhotons](#) = 2000

6.5.1 Detailed Description

Definition at line 38 of file [RunARC.cpp](#).

6.5.2 Constructor & Destructor Documentation

6.5.2.1 CherenkovFile()

```
CherenkovFile::CherenkovFile (  
    const std::string & Filename) [inline]
```

Constructor that prepares the file and tree

Definition at line 68 of file [RunARC.cpp](#).

6.5.3 Member Function Documentation

6.5.3.1 Close()

```
void CherenkovFile::Close () [inline]
```

Save and close

Definition at line 136 of file [RunARC.cpp](#).

6.5.3.2 Fill()

```
void CherenkovFile::Fill () [inline]
```

Fill the TTree

Definition at line 130 of file [RunARC.cpp](#).

6.5.4 Member Data Documentation

6.5.4.1 CherenkovAngle_Reco

```
double CherenkovFile::CherenkovAngle_Reco[MaxPhotons]
```

Definition at line 51 of file [RunARC.cpp](#).

6.5.4.2 CherenkovAngle_Reco_TrueEmissionPoint

```
double CherenkovFile::CherenkovAngle_Reco_TrueEmissionPoint [MaxPhotons]
```

Definition at line 51 of file [RunARC.cpp](#).

6.5.4.3 CherenkovAngle_True

```
double CherenkovFile::CherenkovAngle_True[MaxPhotons]
```

Definition at line 52 of file [RunARC.cpp](#).

6.5.4.4 CherenkovTree

```
TTree CherenkovFile::CherenkovTree
```

The TTree with all information

Definition at line 46 of file [RunARC.cpp](#).

6.5.4.5 CosTheta

```
double CherenkovFile::CosTheta
```

Definition at line 56 of file [RunARC.cpp](#).

6.5.4.6 Entrance_x

```
double CherenkovFile::Entrance_x
```

Definition at line 53 of file [RunARC.cpp](#).

6.5.4.7 Entrance_y

```
double CherenkovFile::Entrance_y
```

Definition at line 53 of file [RunARC.cpp](#).

6.5.4.8 Entrance_z

```
double CherenkovFile::Entrance_z
```

Definition at line 53 of file [RunARC.cpp](#).

6.5.4.9 File

```
TFile CherenkovFile::File
```

The file

Definition at line 42 of file [RunARC.cpp](#).

6.5.4.10 FinalRadiatorColumnNumber

```
std::size_t CherenkovFile::FinalRadiatorColumnNumber
```

Definition at line 61 of file [RunARC.cpp](#).

6.5.4.11 FinalRadiatorRowNumber

```
std::size_t CherenkovFile::FinalRadiatorRowNumber
```

Definition at line 61 of file [RunARC.cpp](#).

6.5.4.12 HasMigrated

```
int CherenkovFile::HasMigrated[MaxPhotons]
```

Definition at line 59 of file [RunARC.cpp](#).

6.5.4.13 MaxPhotons

```
int CherenkovFile::MaxPhotons = 2000 [static], [constexpr]
```

All the variables that are saved

Definition at line 50 of file [RunARC.cpp](#).

6.5.4.14 MirrorHit_x

```
double CherenkovFile::MirrorHit_x
```

Definition at line 54 of file [RunARC.cpp](#).

6.5.4.15 MirrorHit_y

```
double CherenkovFile::MirrorHit_y
```

Definition at line 54 of file [RunARC.cpp](#).

6.5.4.16 MirrorHit_z

```
double CherenkovFile::MirrorHit_z
```

Definition at line 54 of file [RunARC.cpp](#).

6.5.4.17 Momentum

```
double CherenkovFile::Momentum
```

Definition at line 56 of file [RunARC.cpp](#).

6.5.4.18 NumberGoodPhotons

```
double CherenkovFile::NumberGoodPhotons = 0.0
```

Definition at line 58 of file [RunARC.cpp](#).

6.5.4.19 NumberPhotons

```
int CherenkovFile::NumberPhotons = 0
```

Definition at line 57 of file [RunARC.cpp](#).

6.5.4.20 ParticleLocation

```
ParticleTrack::Location CherenkovFile::ParticleLocation
```

Definition at line 62 of file [RunARC.cpp](#).

6.5.4.21 Phi

```
double CherenkovFile::Phi
```

Definition at line 56 of file [RunARC.cpp](#).

6.5.4.22 PhotonEnergy

```
double CherenkovFile::PhotonEnergy[MaxPhotons]
```

Definition at line 52 of file [RunARC.cpp](#).

6.5.4.23 PhotonHit_x

```
double CherenkovFile::PhotonHit_x[MaxPhotons]
```

Definition at line 55 of file [RunARC.cpp](#).

6.5.4.24 PhotonHit_y

```
double CherenkovFile::PhotonHit_y[MaxPhotons]
```

Definition at line 55 of file [RunARC.cpp](#).

6.5.4.25 PhotonHit_z

```
double CherenkovFile::PhotonHit_z[MaxPhotons]
```

Definition at line 55 of file [RunARC.cpp](#).

6.5.4.26 PhotonStatus

`Photon::Status CherenkovFile::PhotonStatus[MaxPhotons]`

Definition at line 63 of file [RunARC.cpp](#).

6.5.4.27 RadiatorColumnNumber

`std::size_t CherenkovFile::RadiatorColumnNumber`

Definition at line 60 of file [RunARC.cpp](#).

6.5.4.28 RadiatorRowNumber

`std::size_t CherenkovFile::RadiatorRowNumber`

Definition at line 60 of file [RunARC.cpp](#).

6.5.4.29 Significance

`double CherenkovFile::Significance`

Definition at line 64 of file [RunARC.cpp](#).

6.5.4.30 SinglePhotonResolution

`double CherenkovFile::SinglePhotonResolution`

Definition at line 64 of file [RunARC.cpp](#).

6.5.4.31 TotalResolution

`double CherenkovFile::TotalResolution`

Definition at line 64 of file [RunARC.cpp](#).

6.5.4.32 TrackNumber

`std::size_t CherenkovFile::TrackNumber`

Definition at line 60 of file [RunARC.cpp](#).

The documentation for this struct was generated from the following file:

- [apps/RunARC.cpp](#)

6.6 de::IOptimizable::Constraints Struct Reference

```
#include <DifferentialEvolution.h>
```

Public Member Functions

- [Constraints](#) (double [lower](#)=0.0, double [upper](#)=1.0, bool [isConstrained](#)=false)
- bool [Check](#) (double candidate)

Public Attributes

- double [lower](#)
- double [upper](#)
- bool [isConstrained](#)

6.6.1 Detailed Description

Definition at line 24 of file [DifferentialEvolution.h](#).

6.6.2 Constructor & Destructor Documentation

6.6.2.1 Constraints()

```
de::IOptimizable::Constraints::Constraints (
    double lower = 0.0,
    double upper = 1.0,
    bool isConstrained = false) [inline]
```

Definition at line 26 of file [DifferentialEvolution.h](#).

6.6.3 Member Function Documentation

6.6.3.1 Check()

```
bool de::IOptimizable::Constraints::Check (
    double candidate) [inline]
```

Definition at line 34 of file [DifferentialEvolution.h](#).

6.6.4 Member Data Documentation

6.6.4.1 isConstrained

```
bool de::IOptimizable::Constraints::isConstrained
```

Definition at line 55 of file [DifferentialEvolution.h](#).

6.6.4.2 lower

```
double de::IOptimizable::Constraints::lower
```

Definition at line 53 of file [DifferentialEvolution.h](#).

6.6.4.3 upper

```
double de::IOptimizable::Constraints::upper
```

Definition at line 54 of file [DifferentialEvolution.h](#).

The documentation for this struct was generated from the following file:

- [include/DifferentialEvolution.h](#)

6.7 de::DifferentialEvolution Class Reference

```
#include <DifferentialEvolution.h>
```

Public Member Functions

- [DifferentialEvolution](#) (const [IOptimizable](#) &costFunction, std::size_t populationSize, std::size_t randomSeed=123, bool shouldCheckConstraints=true, std::function< void(const [DifferentialEvolution](#) &)> callback=nullptr, std::function< bool(const [DifferentialEvolution](#) &)> terminationCondition=nullptr)
- void [InitPopulation](#) ()
- void [SelectionAndCorssing](#) ()
- std::vector< double > [GetBestAgent](#) () const
- double [GetBestCost](#) () const
- std::vector< std::pair< std::vector< double >, double > > [GetPopulationWithCosts](#) () const
- void [PrintPopulation](#) () const
- void [Optimize](#) (int iterations, bool verbose=true)

6.7.1 Detailed Description

Definition at line 64 of file [DifferentialEvolution.h](#).

6.7.2 Constructor & Destructor Documentation

6.7.2.1 DifferentialEvolution()

```
de::DifferentialEvolution::DifferentialEvolution (
    const IOptimizable & costFunction,
    std::size_t populationSize,
    std::size_t randomSeed = 123,
    bool shouldCheckConstraints = true,
    std::function< void(const DifferentialEvolution &)> callback = nullptr,
    std::function< bool(const DifferentialEvolution &)> terminationCondition = nullptr)
[inline]
```

Construct Differential Evolution optimizer

Parameters

<i>costFunction</i>	Cost function to minimize
<i>populationSize</i>	Number of agents in each optimization step
<i>randomSeed</i>	Set random seed to a fix value to have repeatable (non stochastic) experiments
<i>shouldCheckConstraints</i>	Should constraints be checked on for each new candidate. This check may be turned off to increase performance if the cost function is defined and has no local minimum outside of the constraints.
<i>callback</i>	Optional callback to be called after each optimization iteration has finished. Optimization iteration is defined as processing of single population with SelectionAndCrossing method.

Definition at line 79 of file [DifferentialEvolution.h](#).

6.7.3 Member Function Documentation

6.7.3.1 GetBestAgent()

```
std::vector< double > de::DifferentialEvolution::GetBestAgent () const [inline]
```

Definition at line 233 of file [DifferentialEvolution.h](#).

6.7.3.2 GetBestCost()

```
double de::DifferentialEvolution::GetBestCost () const [inline]
```

Definition at line 238 of file [DifferentialEvolution.h](#).

6.7.3.3 GetPopulationWithCosts()

```
std::vector< std::pair< std::vector< double >, double > > de::DifferentialEvolution::GetPopulationWithCosts () const [inline]
```

Definition at line 243 of file [DifferentialEvolution.h](#).

6.7.3.4 InitPopulation()

```
void de::DifferentialEvolution::InitPopulation () [inline]
```

Definition at line 111 of file [DifferentialEvolution.h](#).

6.7.3.5 Optimize()

```
void de::DifferentialEvolution::Optimize (
    int iterations,
    bool verbose = true) [inline]
```

Definition at line 266 of file [DifferentialEvolution.h](#).

6.7.3.6 PrintPopulation()

```
void de::DifferentialEvolution::PrintPopulation () const [inline]
```

Definition at line 254 of file [DifferentialEvolution.h](#).

6.7.3.7 SelectionAndCorssing()

```
void de::DifferentialEvolution::SelectionAndCorssing () [inline]
```

Definition at line 146 of file [DifferentialEvolution.h](#).

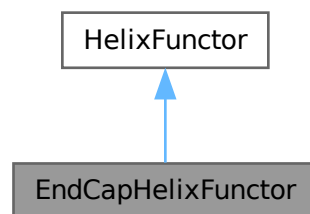
The documentation for this class was generated from the following file:

- include/[DifferentialEvolution.h](#)

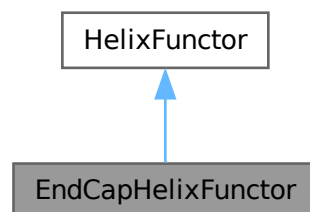
6.8 EndCapHelixFunctor Class Reference

```
#include <HelixFunctor.h>
```

Inheritance diagram for EndCapHelixFunctor:



Collaboration diagram for EndCapHelixFunctor:



Public Member Functions

- [EndCapHelixFunctor](#) (double z)
- [EndCapHelixFunctor](#) (const [BarrelHelixFunctor](#) &Functor)=delete
- virtual double [operator\(\)](#) (const [Vector](#) &Vect) const override

Public Member Functions inherited from [HelixFunctor](#)

- [HelixFunctor](#) ()=default

6.8.1 Detailed Description

[EndCapHelixFunctor](#) is a functor for tracking the particle to the end cap

Definition at line 56 of file [HelixFunctor.h](#).

6.8.2 Constructor & Destructor Documentation

6.8.2.1 [EndCapHelixFunctor\(\)](#) [1/2]

```
EndCapHelixFunctor::EndCapHelixFunctor (
    double z)
```

Constructor that sets up the end cap distance we want to track particle to

Parameters

z	The z distance we're tracking the particle to
---	---

Definition at line 13 of file [HelixFunctor.cpp](#).

6.8.2.2 [EndCapHelixFunctor\(\)](#) [2/2]

```
EndCapHelixFunctor::EndCapHelixFunctor (
    const BarrelHelixFunctor & Functor) [delete]
```

We do not need copy constructor

6.8.3 Member Function Documentation

6.8.3.1 [operator\(\)](#)()

```
double EndCapHelixFunctor::operator() (
    const Vector & Vect) const [override], [virtual]
```

Virtual () operator

Parameters

<i>Vect</i>	Vector position
-------------	---------------------------------

Implements [HelixFunctor](#).

Definition at line 16 of file [HelixFunctor.cpp](#).

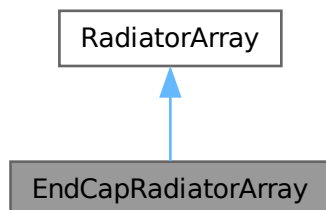
The documentation for this class was generated from the following files:

- include/[HelixFunctor.h](#)
- src/[HelixFunctor.cpp](#)

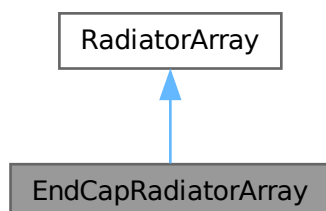
6.9 EndCapRadiatorArray Class Reference

```
#include <EndCapRadiatorArray.h>
```

Inheritance diagram for EndCapRadiatorArray:



Collaboration diagram for EndCapRadiatorArray:



Public Member Functions

- [EndCapRadiatorArray](#) ()
- [EndCapRadiatorArray](#) (const [EndCapRadiatorArray](#) &radiatorCell)=delete
- std::vector< std::pair< std::unique_ptr< TObject >, std::string > > [DrawRadiatorArray](#) () const override
- virtual const [RadiatorCell](#) * [FindRadiator](#) ([Particle](#) &particle) const override
- bool [IsBelowThisRow](#) (std::size_t Row, double x, double y) const
- bool [IsBelowOddRow](#) (double x, double y) const
- bool [IsBelowEvenRow](#) (double x, double y) const

Public Member Functions inherited from [RadiatorArray](#)

- [RadiatorArray](#) ()
- [RadiatorArray](#) (const [RadiatorArray](#) &radiatorArray)=delete
- virtual [~RadiatorArray](#) ()=default
- [RadiatorCell](#) * [GetRadiatorCell](#) (std::size_t i, std::size_t j)
- const [RadiatorCell](#) * [operator\(\)](#) (std::size_t i, std::size_t j) const

Additional Inherited Members

Protected Attributes inherited from [RadiatorArray](#)

- std::vector< std::unique_ptr< [RadiatorCell](#) > > [m_Cells](#)
- const std::size_t [m_NumberMainRowCells](#)
- const double [m_xHexDist](#)
- const double [m_yHexDist](#)

6.9.1 Detailed Description

Definition at line 22 of file [EndCapRadiatorArray.h](#).

6.9.2 Constructor & Destructor Documentation

6.9.2.1 EndCapRadiatorArray() [1/2]

```
EndCapRadiatorArray::EndCapRadiatorArray ()
```

Constructor that sets up the geometry

Definition at line 16 of file [EndCapRadiatorArray.cpp](#).

6.9.2.2 EndCapRadiatorArray() [2/2]

```
EndCapRadiatorArray::EndCapRadiatorArray (
    const EndCapRadiatorArray & radiatorCell) [delete]
```

Delete copy constructor

6.9.3 Member Function Documentation

6.9.3.1 DrawRadiatorArray()

```
std::vector< std::pair< std::unique_ptr< TObject >, std::string > > EndCapRadiatorArray::↵
DrawRadiatorArray () const [override], [virtual]
```

Draw all the radiator cells

Implements [RadiatorArray](#).

Definition at line 32 of file [EndCapRadiatorArray.cpp](#).

6.9.3.2 FindRadiator()

```
const RadiatorCell * EndCapRadiatorArray::FindRadiator (
    Particle & particle) const [override], [virtual]
```

Check which radiator the particle goes through It will map the track momentum and position if the track hits an equivalent radiator cell

Implements [RadiatorArray](#).

Definition at line 45 of file [EndCapRadiatorArray.cpp](#).

6.9.3.3 IsBelowEvenRow()

```
bool EndCapRadiatorArray::IsBelowEvenRow (
    double x,
    double y) const
```

Check if particle track hits below a row with even row number It is assumed the y coordinate has been shifted so it's inside the main row

Parameters

<i>x</i>	Position along the z direction of the barrel, or x in local coordinates
<i>y</i>	Position along the curved direction, or y in location coordinates

Definition at line 138 of file [EndCapRadiatorArray.cpp](#).

6.9.3.4 IsBelowOddRow()

```
bool EndCapRadiatorArray::IsBelowOddRow (
    double x,
    double y) const
```

Check if particle track hits below a row with odd row number It is assumed the y coordinate has been shifted so it's inside the main row

Parameters

<i>x</i>	Position along the z direction of the barrel, or x in local coordinates
<i>y</i>	Position along the curved direction, or y in location coordinates

Definition at line 124 of file [EndCapRadiatorArray.cpp](#).

6.9.3.5 IsBelowThisRow()

```
bool EndCapRadiatorArray::IsBelowThisRow (
    std::size_t Row,
    double x,
    double y) const
```

Check if particle track hits below a particular row

Parameters

<i>Row</i>	The row number
<i>x</i>	Position along the z direction of the barrel, or x in local coordinates
<i>y</i>	Position along the curved direction, or y in location coordinates

Definition at line 114 of file [EndCapRadiatorArray.cpp](#).

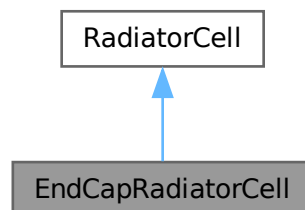
The documentation for this class was generated from the following files:

- [include/EndCapRadiatorArray.h](#)
- [src/EndCapRadiatorArray.cpp](#)

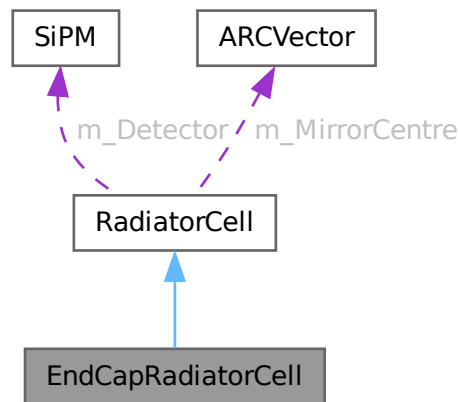
6.10 EndCapRadiatorCell Class Reference

```
#include <EndCapRadiatorCell.h>
```

Inheritance diagram for EndCapRadiatorCell:



Collaboration diagram for EndCapRadiatorCell:



Public Member Functions

- [EndCapRadiatorCell](#) (std::size_t CellColumnNumber, std::size_t CellRowNumber, double HexagonSize)
- [EndCapRadiatorCell](#) (const [EndCapRadiatorCell](#) &radiatorCell)=delete
- virtual bool [IsInsideCell](#) (const [ARCVector](#) &Position) const override
- bool [IsInsideCell](#) (const [Vector](#) &Position) const
- virtual std::vector< std::pair< std::unique_ptr< TObject >, std::string > > [DrawRadiatorGeometry](#) () const override
- virtual bool [IsDetectorInsideCell](#) () const override
- virtual bool [IsEdgeCell](#) () const override

Public Member Functions inherited from [RadiatorCell](#)

- [RadiatorCell](#) (std::size_t CellColumnNumber, std::size_t CellRowNumber, double HexagonSize, const [Vector](#) &Position, const [Rotation3D](#) &Rotation, const std::string &Prefix="")
- [RadiatorCell](#) (const [RadiatorCell](#) &radiatorCell)=delete
- virtual [~RadiatorCell](#) ()=default
- double [GetRadiatorThickness](#) () const
- double [GetVesselThickness](#) () const
- double [GetCoolingThickness](#) () const
- double [GetAerogelThickness](#) () const
- const [Vector](#) & [GetMirrorCentre](#) () const
- [Vector](#) [GetGlobalMirrorCentre](#) () const
- double [GetMirrorCurvature](#) () const
- const [Vector](#) & [GetRadiatorPosition](#) () const
- const [Rotation3D](#) & [GetRadiatorRotation](#) () const
- bool [IsInsideCell](#) (const [Photon](#) &photon) const
- double [GetHexagonSize](#) () const
- std::pair< std::size_t, std::size_t > [GetCellNumber](#) () const
- const [SiPM](#) & [GetDetector](#) () const
- void [SetMirrorCurvature](#) (double Curvature)

- void [SetMirrorXPosition](#) (double x)
- void [SetMirrorZPosition](#) (double z)
- void [SetDetectorPosition](#) (double x)
- void [SetDetectorTilt](#) (double Angle)
- virtual [RotationZ ReversePhiRotation](#) () const

Static Public Attributes

- static constexpr std::array< std::pair< std::size_t, std::size_t >, 23 > [m_ValidCells](#)
- static constexpr std::array< std::pair< std::size_t, std::size_t >, 1 > [m_NotValidCells](#)

Additional Inherited Members

Protected Member Functions inherited from [RadiatorCell](#)

- double [GetMirrorCurvatureCentreZ](#) () const

Protected Attributes inherited from [RadiatorCell](#)

- const [Vector](#) [m_Position](#)
- const [Rotation3D](#) [m_Rotation](#)
- const double [m_RadiatorThickness](#)
- const double [m_VesselThickness](#)
- const double [m_CoolingThickness](#)
- const double [m_AerogelThickness](#)
- const double [m_HexagonSize](#)
- const std::pair< std::size_t, std::size_t > [m_CellNumber](#)
- [SiPM](#) [m_Detector](#)
- double [m_MirrorCurvature](#)
- const [Vector](#) [m_DefaultMirrorCentre](#)
- [ARCVector](#) [m_MirrorCentre](#)

6.10.1 Detailed Description

[EndCapRadiatorCell](#) inherits from [RadiatorCell](#) It is not curved and its position is in the end caps

Definition at line 18 of file [EndCapRadiatorCell.h](#).

6.10.2 Constructor & Destructor Documentation

6.10.2.1 [EndCapRadiatorCell\(\)](#) [1/2]

```
EndCapRadiatorCell::EndCapRadiatorCell (
    std::size_t CellColumnNumber,
    std::size_t CellRowNumber,
    double HexagonSize)
```

Default constructor that calls parent constructor

Definition at line 13 of file [EndCapRadiatorCell.cpp](#).

6.10.2.2 EndCapRadiatorCell() [2/2]

```
EndCapRadiatorCell::EndCapRadiatorCell (  
    const EndCapRadiatorCell & radiatorCell) [delete]
```

Delete copy constructor

6.10.3 Member Function Documentation

6.10.3.1 DrawRadiatorGeometry()

```
std::vector< std::pair< std::unique_ptr< TObject >, std::string > > EndCapRadiatorCell::↵  
DrawRadiatorGeometry () const [override], [virtual]
```

Draw radiator geometry

Implements [RadiatorCell](#).

Definition at line 49 of file [EndCapRadiatorCell.cpp](#).

6.10.3.2 IsDetectorInsideCell()

```
bool EndCapRadiatorCell::IsDetectorInsideCell () const [override], [virtual]
```

Function that checks if the detector is contained inside the radiator cell

Implements [RadiatorCell](#).

Definition at line 147 of file [EndCapRadiatorCell.cpp](#).

6.10.3.3 IsEdgeCell()

```
bool EndCapRadiatorCell::IsEdgeCell () const [override], [virtual]
```

Checks if the cell is at the edge (near the end cap)

Implements [RadiatorCell](#).

Definition at line 260 of file [EndCapRadiatorCell.cpp](#).

6.10.3.4 IsInsideCell() [1/2]

```
bool EndCapRadiatorCell::IsInsideCell (  
    const ARCVector & Position) const [override], [virtual]
```

Function that checks if the position is inside the radiator

Implements [RadiatorCell](#).

Definition at line 26 of file [EndCapRadiatorCell.cpp](#).

6.10.3.5 IsInsideCell() [2/2]

```
bool EndCapRadiatorCell::IsInsideCell (
    const Vector & Position) const
```

Function that checks if the position is inside the radiator

Definition at line 40 of file [EndCapRadiatorCell.cpp](#).

6.10.4 Member Data Documentation

6.10.4.1 m_NotValidCells

```
std::array<std::pair<std::size_t, std::size_t>, 1> EndCapRadiatorCell::m_NotValidCells [static],
[constexpr]
```

Initial value:

```
{{
    {7, 5}
}}
```

All the cell numbers that are invalid, but next to valid cells for the end cap

Definition at line 66 of file [EndCapRadiatorCell.h](#).

6.10.4.2 m_ValidCells

```
std::array<std::pair<std::size_t, std::size_t>, 23> EndCapRadiatorCell::m_ValidCells [static],
[constexpr]
```

Initial value:

```
{{
    {1, 1}, {2, 1}, {3, 1}, {4, 1}, {5, 1}, {6, 1}, {7, 1},
    {2, 2}, {3, 2}, {4, 2}, {5, 2}, {6, 2}, {7, 2}, {8, 2},
    {3, 3}, {4, 3}, {5, 3}, {6, 3}, {7, 3},
    {5, 4}, {6, 4}, {7, 4},
    {6, 5}
}}
```

All the cell numbers that are valid for the end cap

Definition at line 55 of file [EndCapRadiatorCell.h](#).

The documentation for this class was generated from the following files:

- [include/EndCapRadiatorCell.h](#)
- [src/EndCapRadiatorCell.cpp](#)

6.11 EventDisplay Class Reference

```
#include <EventDisplay.h>
```

Public Member Functions

- [EventDisplay](#) ()=default
- void [DrawEventDisplay](#) ()
- void [AddObject](#) (std::unique_ptr< TObject > Object, const std::string &Option="")
- void [AddObject](#) (std::vector< std::pair< std::unique_ptr< TObject >, std::string > > Objects)

6.11.1 Detailed Description

[EventDisplay](#) is a class for storing and drawing all the objects that are in the event

Definition at line 16 of file [EventDisplay.h](#).

6.11.2 Constructor & Destructor Documentation

6.11.2.1 EventDisplay()

```
EventDisplay::EventDisplay () [default]
```

Default constructor

6.11.3 Member Function Documentation

6.11.3.1 AddObject() [1/2]

```
void EventDisplay::AddObject (
    std::unique_ptr< TObject > Object,
    const std::string & Option = "")
```

Add object to event display

Definition at line 96 of file [EventDisplay.cpp](#).

6.11.3.2 AddObject() [2/2]

```
void EventDisplay::AddObject (
    std::vector< std::pair< std::unique_ptr< TObject >, std::string > > Objects)
```

Add objects to event display

Definition at line 102 of file [EventDisplay.cpp](#).

6.11.3.3 DrawEventDisplay()

```
void EventDisplay::DrawEventDisplay ()
```

Draw the event display

Definition at line 12 of file [EventDisplay.cpp](#).

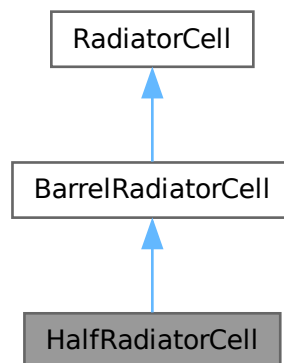
The documentation for this class was generated from the following files:

- [include/EventDisplay.h](#)
- [src/EventDisplay.cpp](#)

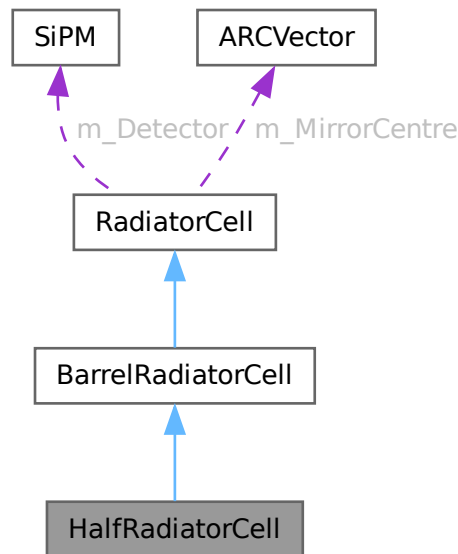
6.12 HalfRadiatorCell Class Reference

```
#include <HalfRadiatorCell.h>
```

Inheritance diagram for HalfRadiatorCell:



Collaboration diagram for HalfRadiatorCell:



Public Member Functions

- [HalfRadiatorCell](#) (std::size_t CellColumnNumber, std::size_t CellRowNumber, double HexagonSize)
- [HalfRadiatorCell](#) (const [HalfRadiatorCell](#) &radiatorCell)=delete
- virtual bool [IsInsideCell](#) (const [ARCVector](#) &Position) const override
- virtual std::vector< std::pair< std::unique_ptr< TObject >, std::string > > [DrawRadiatorGeometry](#) () const override

Public Member Functions inherited from [BarrelRadiatorCell](#)

- [BarrelRadiatorCell](#) (std::size_t CellColumnNumber, std::size_t CellRowNumber, double HexagonSize)
- [BarrelRadiatorCell](#) (const [BarrelRadiatorCell](#) &radiatorCell)=delete
- virtual [RotationZ ReversePhiRotation](#) () const override
- virtual bool [IsDetectorInsideCell](#) () const override
- virtual bool [IsEdgeCell](#) () const override

Public Member Functions inherited from [RadiatorCell](#)

- [RadiatorCell](#) (std::size_t CellColumnNumber, std::size_t CellRowNumber, double HexagonSize, const [Vector](#) &Position, const [Rotation3D](#) &Rotation, const std::string &Prefix="")
- [RadiatorCell](#) (const [RadiatorCell](#) &radiatorCell)=delete
- virtual [~RadiatorCell](#) ()=default
- double [GetRadiatorThickness](#) () const
- double [GetVesselThickness](#) () const
- double [GetCoolingThickness](#) () const
- double [GetAerogelThickness](#) () const

- const [Vector](#) & [GetMirrorCentre](#) () const
- [Vector](#) [GetGlobalMirrorCentre](#) () const
- double [GetMirrorCurvature](#) () const
- const [Vector](#) & [GetRadiatorPosition](#) () const
- const [Rotation3D](#) & [GetRadiatorRotation](#) () const
- bool [IsInsideCell](#) (const [Photon](#) &photon) const
- double [GetHexagonSize](#) () const
- std::pair< std::size_t, std::size_t > [GetCellNumber](#) () const
- const [SiPM](#) & [GetDetector](#) () const
- void [SetMirrorCurvature](#) (double Curvature)
- void [SetMirrorXPosition](#) (double x)
- void [SetMirrorZPosition](#) (double z)
- void [SetDetectorPosition](#) (double x)
- void [SetDetectorTilt](#) (double Angle)

Additional Inherited Members

Protected Member Functions inherited from [RadiatorCell](#)

- double [GetMirrorCurvatureCentreZ](#) () const

Protected Attributes inherited from [BarrelRadiatorCell](#)

- const double [m_BarrelRadius](#)

Protected Attributes inherited from [RadiatorCell](#)

- const [Vector](#) [m_Position](#)
- const [Rotation3D](#) [m_Rotation](#)
- const double [m_RadiatorThickness](#)
- const double [m_VesselThickness](#)
- const double [m_CoolingThickness](#)
- const double [m_AerogelThickness](#)
- const double [m_HexagonSize](#)
- const std::pair< std::size_t, std::size_t > [m_CellNumber](#)
- [SiPM](#) [m_Detector](#)
- double [m_MirrorCurvature](#)
- const [Vector](#) [m_DefaultMirrorCentre](#)
- [ARCVector](#) [m_MirrorCentre](#)

6.12.1 Detailed Description

Definition at line 11 of file [HalfRadiatorCell.h](#).

6.12.2 Constructor & Destructor Documentation

6.12.2.1 HalfRadiatorCell() [1/2]

```
HalfRadiatorCell::HalfRadiatorCell (
    std::size_t CellColumnNumber,
    std::size_t CellRowNumber,
    double HexagonSize)
```

Default constructor that simply calls the parent constructor

Definition at line 11 of file [HalfRadiatorCell.cpp](#).

6.12.2.2 HalfRadiatorCell() [2/2]

```
HalfRadiatorCell::HalfRadiatorCell (
    const HalfRadiatorCell & radiatorCell) [delete]
```

Delete copy constructor

6.12.3 Member Function Documentation

6.12.3.1 DrawRadiatorGeometry()

```
std::vector< std::pair< std::unique_ptr< TObject >, std::string > > HalfRadiatorCell::Draw↵
RadiatorGeometry () const [override], [virtual]
```

Draw radiator geometry

Reimplemented from [BarrelRadiatorCell](#).

Definition at line 26 of file [HalfRadiatorCell.cpp](#).

6.12.3.2 IsInsideCell()

```
bool HalfRadiatorCell::IsInsideCell (
    const ARCVector & Position) const [override], [virtual]
```

Function that checks if the position is inside the radiator

Reimplemented from [BarrelRadiatorCell](#).

Definition at line 17 of file [HalfRadiatorCell.cpp](#).

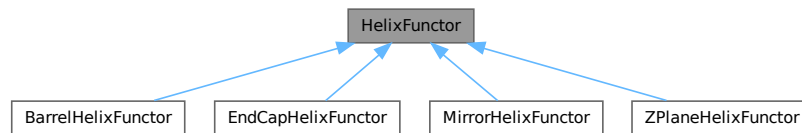
The documentation for this class was generated from the following files:

- include/[HalfRadiatorCell.h](#)
- src/[HalfRadiatorCell.cpp](#)

6.13 HelixFunctor Class Reference

```
#include <HelixFunctor.h>
```

Inheritance diagram for HelixFunctor:



Public Member Functions

- [HelixFunctor](#) ()=default
- virtual double [operator\(\)](#) (const [Vector](#) &Vect) const =0

6.13.1 Detailed Description

Definition at line 14 of file [HelixFunctor.h](#).

6.13.2 Constructor & Destructor Documentation

6.13.2.1 HelixFunctor()

```
HelixFunctor::HelixFunctor () [default]
```

Constructor is default

6.13.3 Member Function Documentation

6.13.3.1 operator()()

```
virtual double HelixFunctor::operator() (
    const Vector & Vect) const [pure virtual]
```

Virtual () operator

Parameters

<i>Vect</i>	Vector position
-------------	---------------------------------

Implemented in [BarrelHelixFunctor](#), [EndCapHelixFunctor](#), [MirrorHelixFunctor](#), and [ZPlaneHelixFunctor](#).

The documentation for this class was generated from the following file:

- include/[HelixFunctor.h](#)

6.14 HelixPath Class Reference

```
#include <HelixPath.h>
```

Public Member Functions

- [HelixPath](#) (const [Vector](#) &Momentum, int Q, double B)
- [HelixPath](#) (const [HelixPath](#) &Helix)=default
- [Vector GetPosition](#) (double s) const
- [Vector GetDirection](#) (double s) const
- double [SolvePathLength](#) (const [HelixFuncor](#) &Funcor, double Min, double Max) const
- void [ResetOrigin](#) (double NewOrigin_s)
- void [MapPhi](#) (double DeltaPhi)
- void [ReflectZ](#) ()
- void [ReflectY](#) ()

6.14.1 Detailed Description

Definition at line 17 of file [HelixPath.h](#).

6.14.2 Constructor & Destructor Documentation

6.14.2.1 HelixPath() [1/2]

```
HelixPath::HelixPath (
    const Vector & Momentum,
    int Q,
    double B)
```

Constructor that takes in the particle charge, momentum and magnetic field

Parameters

<i>Momentum</i>	Initial momentum vector of charged particle
<i>Q</i>	Particle charge, +1 or -1
<i>B</i>	Magnetic field

Definition at line 12 of file [HelixPath.cpp](#).

6.14.2.2 HelixPath() [2/2]

```
HelixPath::HelixPath (
    const HelixPath & Helix) [default]
```

We don't need copy constructor

6.14.3 Member Function Documentation

6.14.3.1 GetDirection()

```
Vector HelixPath::GetDirection (
    double s) const
```

Get the particle direction unit vector from the path length

Parameters

<i>s</i>	Path length coordinate
----------	------------------------

Definition at line 35 of file [HelixPath.cpp](#).

6.14.3.2 GetPosition()

```
Vector HelixPath::GetPosition (
    double s) const
```

Get the particle position from the path length

Parameters

<i>s</i>	Path length coordinate
----------	------------------------

Definition at line 21 of file [HelixPath.cpp](#).

6.14.3.3 MapPhi()

```
void HelixPath::MapPhi (
    double DeltaPhi)
```

Rotate around the phi direction to map particle to a valid radiator cell Warning! Need to call ResetOrigin first!

Parameters

<i>DeltaPhi</i>	Angle that is rotated in phi
-----------------	------------------------------

Definition at line 67 of file [HelixPath.cpp](#).

6.14.3.4 ReflectY()

```
void HelixPath::ReflectY ()
```

Reflect everything in the y-direction Warning! Need to call ResetOrigin first!

Definition at line 78 of file [HelixPath.cpp](#).

6.14.3.5 ReflectZ()

```
void HelixPath::ReflectZ ()
```

Reflect everything in the z-direction

Definition at line 73 of file [HelixPath.cpp](#).

6.14.3.6 ResetOrigin()

```
void HelixPath::ResetOrigin (
    double NewOrigin_s)
```

Function that resets the origin

Parameters

<i>NewOrigin</i> _↔ _s	The new path length origin
-------------------------------------	----------------------------

Definition at line 61 of file [HelixPath.cpp](#).

6.14.3.7 SolvePathLength()

```
double HelixPath::SolvePathLength (
    const HelixFunctor & Functor,
    double Min,
    double Max) const
```

Find the path length that solves the equation $f(x) = 0$

Parameters

<i>Functor</i>	Function that we want to find root of
----------------	---------------------------------------

Definition at line 49 of file [HelixPath.cpp](#).

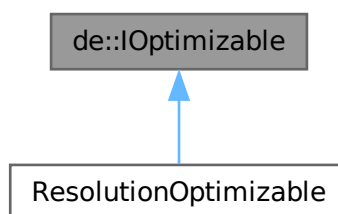
The documentation for this class was generated from the following files:

- include/[HelixPath.h](#)
- src/[HelixPath.cpp](#)

6.15 de::IOptimizable Class Reference

```
#include <DifferentialEvolution.h>
```

Inheritance diagram for de::IOptimizable:



Classes

- struct [Constraints](#)

Public Member Functions

- virtual double [EvaluateCost](#) (std::vector< double > inputs) const =0
- virtual std::size_t [NumberOfParameters](#) () const =0
- virtual std::vector< [Constraints](#) > [GetConstraints](#) () const =0
- virtual [~IOptimizable](#) ()

6.15.1 Detailed Description

Definition at line 21 of file [DifferentialEvolution.h](#).

6.15.2 Constructor & Destructor Documentation

6.15.2.1 ~IOptimizable()

```
virtual de::IOptimizable::~~IOptimizable () [inline], [virtual]
```

Definition at line 61 of file [DifferentialEvolution.h](#).

6.15.3 Member Function Documentation

6.15.3.1 EvaluateCost()

```
virtual double de::IOptimizable::EvaluateCost (  
    std::vector< double > inputs) const [pure virtual]
```

Implemented in [ResolutionOptimizable](#).

6.15.3.2 GetConstraints()

```
virtual std::vector< Constraints > de::IOptimizable::GetConstraints () const [pure virtual]
```

Implemented in [ResolutionOptimizable](#).

6.15.3.3 NumberOfParameters()

```
virtual std::size_t de::IOptimizable::NumberOfParameters () const [pure virtual]
```

Implemented in [ResolutionOptimizable](#).

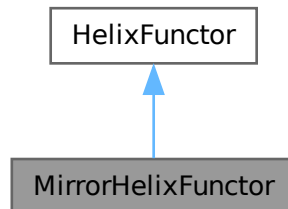
The documentation for this class was generated from the following file:

- include/[DifferentialEvolution.h](#)

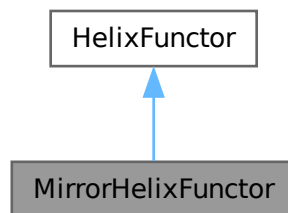
6.16 MirrorHelixFunctor Class Reference

```
#include <HelixFunctor.h>
```

Inheritance diagram for MirrorHelixFunctor:



Collaboration diagram for MirrorHelixFunctor:



Public Member Functions

- [MirrorHelixFunctor](#) (const [Vector](#) &MirrorPosition, double Curvature)
- [MirrorHelixFunctor](#) (const [MirrorHelixFunctor](#) &Functor)=delete
- virtual double [operator\(\)](#) (const [Vector](#) &Vect) const override

Public Member Functions inherited from [HelixFunctor](#)

- [HelixFunctor](#) ()=default

6.16.1 Detailed Description

[MirrorHelixFunctor](#) is a functor for tracking the particle to the spherical mirror

Definition at line 127 of file [HelixFunctor.h](#).

6.16.2 Constructor & Destructor Documentation

6.16.2.1 MirrorHelixFunctor() [1/2]

```
MirrorHelixFunctor::MirrorHelixFunctor (
    const Vector & MirrorPosition,
    double Curvature)
```

Constructor that sets up the mirror position and curvature

Parameters

<i>MirrorPosition</i>	Position of mirror in global coordinates
<i>Curvature</i>	Mirror curvature

Definition at line 41 of file [HelixFunctor.cpp](#).

6.16.2.2 MirrorHelixFunctor() [2/2]

```
MirrorHelixFunctor::MirrorHelixFunctor (
    const MirrorHelixFunctor & Functor) [delete]
```

We do not need copy constructor

6.16.3 Member Function Documentation

6.16.3.1 operator()

```
double MirrorHelixFunctor::operator() (
    const Vector & Vect) const [override], [virtual]
```

Virtual () operator

Parameters

<i>Vect</i>	Vector position
-------------	---------------------------------

Implements [HelixFunctor](#).

Definition at line 47 of file [HelixFunctor.cpp](#).

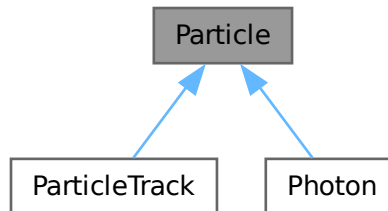
The documentation for this class was generated from the following files:

- [include/HelixFunctor.h](#)
- [src/HelixFunctor.cpp](#)

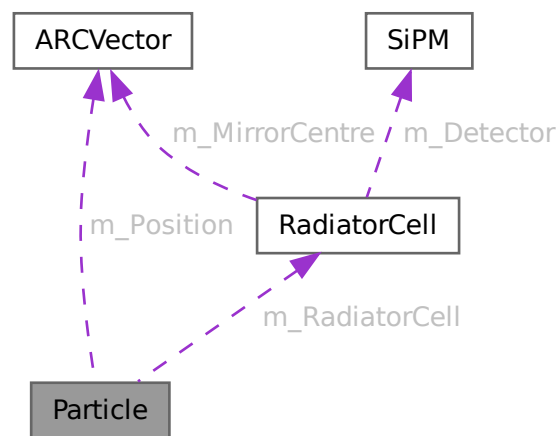
6.17 Particle Class Reference

```
#include <Particle.h>
```

Inheritance diagram for Particle:



Collaboration diagram for Particle:



Public Member Functions

- [Particle](#) (const [Vector](#) &Position, const [RadiatorCell](#) *radiatorCell=nullptr)
- [~Particle](#) ()=default
- bool [FindRadiator](#) (const [RadiatorArray](#) &radiatorArray)
- virtual void [ConvertToRadiatorCoordinates](#) ()
- virtual void [ConvertBackToGlobalCoordinates](#) ()
- const [ARCVector](#) & [GetPosition](#) () const
- virtual void [MapPhi](#) (double DeltaPhi)
- virtual void [ReflectZ](#) ()

- virtual void [ReflectY](#) ()
- const [RadiatorCell](#) * [GetRadiatorCell](#) () const
- virtual bool [IsAtRadiator](#) () const =0
- std::size_t [GetRadiatorColumnNumber](#) () const
- std::size_t [GetRadiatorRowNumber](#) () const

Protected Attributes

- [ARCVector](#) [m_Position](#)
- const [RadiatorCell](#) * [m_RadiatorCell](#)

6.17.1 Detailed Description

Definition at line 18 of file [Particle.h](#).

6.17.2 Constructor & Destructor Documentation

6.17.2.1 Particle()

```
Particle::Particle (
    const Vector & Position,
    const RadiatorCell * radiatorCell = nullptr)
```

Constructor that stores the position and radiator cell

Parameters

<i>Position</i>	The particle position
<i>radiatorCell</i>	The radiator cell the particle belong to

Definition at line 12 of file [Particle.cpp](#).

6.17.2.2 ~Particle()

```
Particle::~~Particle () [default]
```

Need virtual constructor since it's a virtual class

6.17.3 Member Function Documentation

6.17.3.1 ConvertBackToGlobalCoordinates()

```
void Particle::ConvertBackToGlobalCoordinates () [virtual]
```

Convert back to global coordinates

Reimplemented in [ParticleTrack](#), and [Photon](#).

Definition at line 47 of file [Particle.cpp](#).

6.17.3.2 ConvertToRadiatorCoordinates()

```
void Particle::ConvertToRadiatorCoordinates () [virtual]
```

Convert to local radiator coordinates

Parameters

<i>radiatorCell</i>	The radiator cell that we want the local coordinates of
---------------------	---

Reimplemented in [ParticleTrack](#), and [Photon](#).

Definition at line 36 of file [Particle.cpp](#).

6.17.3.3 FindRadiator()

```
bool Particle::FindRadiator (
    const RadiatorArray & radiatorArray)
```

Find the correct radiator that the particle goes through

Definition at line 26 of file [Particle.cpp](#).

6.17.3.4 GetPosition()

```
const ARCVector & Particle::GetPosition () const
```

Get particle position

Definition at line 18 of file [Particle.cpp](#).

6.17.3.5 GetRadiatorCell()

```
const RadiatorCell * Particle::GetRadiatorCell () const
```

Get a pointer to the radiator cell

Definition at line 22 of file [Particle.cpp](#).

6.17.3.6 GetRadiatorColumnNumber()

```
std::size_t Particle::GetRadiatorColumnNumber () const
```

Get the column number of the radiator

Definition at line 66 of file [Particle.cpp](#).

6.17.3.7 GetRadiatorRowNumber()

```
std::size_t Particle::GetRadiatorRowNumber () const
```

Get the row number of the radiator

Definition at line 74 of file [Particle.cpp](#).

6.17.3.8 IsAtRadiator()

```
virtual bool Particle::IsAtRadiator () const [pure virtual]
```

Check if particle is at the radiator so that we can search for the radiator cell

Implemented in [ParticleTrack](#), and [Photon](#).

6.17.3.9 MapPhi()

```
void Particle::MapPhi (
    double DeltaPhi) [virtual]
```

Rotate around the phi direction to map particle to a valid radiator cell

Parameters

<i>DeltaPhi</i>	Angle that is rotated in phi
-----------------	------------------------------

Reimplemented in [ParticleTrack](#), and [Photon](#).

Definition at line 54 of file [Particle.cpp](#).

6.17.3.10 ReflectY()

```
void Particle::ReflectY () [virtual]
```

Reflect everything in the y-direction

Reimplemented in [ParticleTrack](#), and [Photon](#).

Definition at line 62 of file [Particle.cpp](#).

6.17.3.11 ReflectZ()

```
void Particle::ReflectZ () [virtual]
```

Reflect everything in the z-direction since radiator cells are symmetric

Reimplemented in [ParticleTrack](#), and [Photon](#).

Definition at line 58 of file [Particle.cpp](#).

6.17.4 Member Data Documentation

6.17.4.1 m_Position

```
ARCVector Particle::m_Position [protected]
```

[Particle](#) position, in m

Definition at line 81 of file [Particle.h](#).

6.17.4.2 m_RadiatorCell

```
const RadiatorCell* Particle::m_RadiatorCell [protected]
```

Pointer to the radiator cell that track enters

Definition at line 85 of file [Particle.h](#).

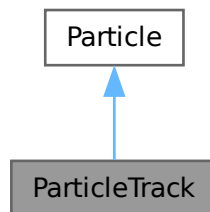
The documentation for this class was generated from the following files:

- include/[Particle.h](#)
- src/[Particle.cpp](#)

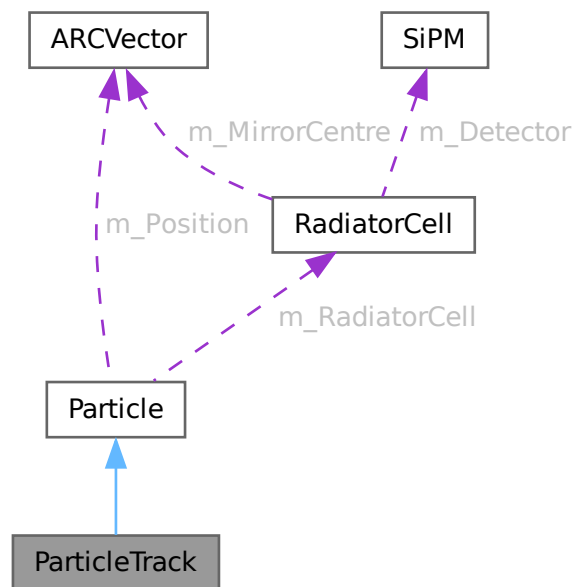
6.18 ParticleTrack Class Reference

```
#include <ParticleTrack.h>
```

Inheritance diagram for ParticleTrack:



Collaboration diagram for ParticleTrack:



Public Types

- enum class [Location](#) {
[TrackerVolume](#) , [EntranceWindow](#) , [MissedEntranceWindow](#) , [Radiator](#) ,
[Mirror](#) , [MissedMirror](#) }

Public Member Functions

- [ParticleTrack](#) (int ParticleID, const [Vector](#) &Momentum, std::size_t TrackNumber, double BField)
- [~ParticleTrack](#) ()=default
- bool [TrackThroughTracker](#) (const [TrackingVolume](#) &InnerTracker)
- void [SetRadiator](#) (const [RadiatorCell](#) *radiatorCell)
- virtual void [ConvertToRadiatorCoordinates](#) () override
- virtual void [ConvertBackToGlobalCoordinates](#) () override
- bool [TrackThroughAerogel](#) ()
- void [TrackThroughGasToMirror](#) ()
- bool [TrackToNextCell](#) (const [RadiatorArray](#) &radiatorArray)
- [Photon](#) [GeneratePhotonFromAerogel](#) () const
- [Photon](#) [GeneratePhotonFromGas](#) () const
- std::vector< [Photon](#) > [GeneratePhotonsFromAerogel](#) () const
- std::vector< [Photon](#) > [GeneratePhotonsFromGas](#) () const
- [Photon](#) [GeneratePhoton](#) (double Entry, double Exit, [Photon::Radiator](#) [Radiator](#)) const
- double [Beta](#) () const
- std::unique_ptr< [TLine](#) > [DrawParticleTrack](#) () const
- [Location](#) [GetParticleLocation](#) () const
- [Vector](#) [GetEntranceWindowPosition](#) () const
- virtual void [MapPhi](#) (double DeltaPhi) override
- virtual void [ReflectZ](#) () override
- virtual void [ReflectY](#) () override
- virtual bool [IsAtRadiator](#) () const override

Public Member Functions inherited from [Particle](#)

- [Particle](#) (const [Vector](#) &Position, const [RadiatorCell](#) *radiatorCell=nullptr)
- [~Particle](#) ()=default
- bool [FindRadiator](#) (const [RadiatorArray](#) &radiatorArray)
- const [ARCVector](#) & [GetPosition](#) () const
- const [RadiatorCell](#) * [GetRadiatorCell](#) () const
- std::size_t [GetRadiatorColumnNumber](#) () const
- std::size_t [GetRadiatorRowNumber](#) () const

Additional Inherited Members

Protected Attributes inherited from [Particle](#)

- [ARCVector](#) [m_Position](#)
- const [RadiatorCell](#) * [m_RadiatorCell](#)

6.18.1 Detailed Description

Definition at line 22 of file [ParticleTrack.h](#).

6.18.2 Member Enumeration Documentation

6.18.2.1 Location

```
enum class ParticleTrack::Location [strong]
```

Enum class with the location of the particle TrackerVolume: Inside the tracker volume, where the IP is Entrance↔ Window: [Particle](#) is at the entrance window to the radiator cell MissedEntranceWindow: [Particle](#) didn't hit this radiator entrance window MissedRadiator: [Particle](#) did not enter any radiator cells Radiator: Inside the radiatorCell Mirror: The particle has reached the mirror MissedMirror: The particle missed the mirror

Enumerator

TrackerVolume	
EntranceWindow	
MissedEntranceWindow	
Radiator	
Mirror	
MissedMirror	

Definition at line 49 of file [ParticleTrack.h](#).

6.18.3 Constructor & Destructor Documentation

6.18.3.1 ParticleTrack()

```
ParticleTrack::ParticleTrack (
    int ParticleID,
    const Vector & Momentum,
    std::size_t TrackNumber,
    double BField)
```

Construct a charged particle with momentum and ID at the interaction point

Parameters

<i>ParticleID</i>	PDG particle ID convention
<i>Momentum</i>	Particle momentum, in GeV
<i>Position</i>	Particle initial position, by default it's the origin
<i>BField</i>	Magnetic field that particle passes through

Definition at line 21 of file [ParticleTrack.cpp](#).

6.18.3.2 ~ParticleTrack()

```
ParticleTrack::~ParticleTrack () [default]
```

Need virtual constructor since it's a virtual class

6.18.4 Member Function Documentation

6.18.4.1 Beta()

```
double ParticleTrack::Beta () const
```

Get the particle speed, in units of c

Definition at line 302 of file [ParticleTrack.cpp](#).

6.18.4.2 ConvertBackToGlobalCoordinates()

```
void ParticleTrack::ConvertBackToGlobalCoordinates () [override], [virtual]
```

Convert back to global coordinates

Reimplemented from [Particle](#).

Definition at line 99 of file [ParticleTrack.cpp](#).

6.18.4.3 ConvertToRadiatorCoordinates()

```
void ParticleTrack::ConvertToRadiatorCoordinates () [override], [virtual]
```

Convert to local radiator coordinates

Reimplemented from [Particle](#).

Definition at line 81 of file [ParticleTrack.cpp](#).

6.18.4.4 DrawParticleTrack()

```
std::unique_ptr< TLine > ParticleTrack::DrawParticleTrack () const
```

Draw particle track and the photons

Definition at line 342 of file [ParticleTrack.cpp](#).

6.18.4.5 GeneratePhoton()

```
Photon ParticleTrack::GeneratePhoton (
    double Entry,
    double Exit,
    Photon::Radiator Radiator) const
```

Generate Cherenkov photon

Parameters

<i>Entry_s</i>	Path length at entry of radiator
<i>Exit_s</i>	Path length at exit of radiator
<i>Radiator</i>	The medium the photon was emitted in, to determine the index of refraction

Definition at line 247 of file [ParticleTrack.cpp](#).

6.18.4.6 GeneratePhotonFromAerogel()

`Photon ParticleTrack::GeneratePhotonFromAerogel () const`

Generate Cherenkov photon from aerogel

Definition at line 186 of file [ParticleTrack.cpp](#).

6.18.4.7 GeneratePhotonFromGas()

`Photon ParticleTrack::GeneratePhotonFromGas () const`

Generate Cherenkov photon from gas

Definition at line 197 of file [ParticleTrack.cpp](#).

6.18.4.8 GeneratePhotonsFromAerogel()

`std::vector< Photon > ParticleTrack::GeneratePhotonsFromAerogel () const`

Generate Cherenkov photons from aerogel according to Frank-Tamm relation

Definition at line 208 of file [ParticleTrack.cpp](#).

6.18.4.9 GeneratePhotonsFromGas()

`std::vector< Photon > ParticleTrack::GeneratePhotonsFromGas () const`

Generate Cherenkov photons from gas according to Frank-Tamm relation

Definition at line 227 of file [ParticleTrack.cpp](#).

6.18.4.10 GetEntranceWindowPosition()

`Vector ParticleTrack::GetEntranceWindowPosition () const`

Get the position of the entrance window

Definition at line 362 of file [ParticleTrack.cpp](#).

6.18.4.11 GetParticleLocation()

`ParticleTrack::Location ParticleTrack::GetParticleLocation () const`

Get particle location

Definition at line 358 of file [ParticleTrack.cpp](#).

6.18.4.12 IsAtRadiator()

`bool ParticleTrack::IsAtRadiator () const [override], [virtual]`

Check if particle is at the radiator so that we can search for the radiator cell

Implements [Particle](#).

Definition at line 366 of file [ParticleTrack.cpp](#).

6.18.4.13 MapPhi()

`void ParticleTrack::MapPhi (
double DeltaPhi) [override], [virtual]`

Rotate around the phi direction to map particle to a valid radiator cell

Parameters

<i>DeltaPhi</i>	Angle that is rotated in phi
-----------------	------------------------------

Reimplemented from [Particle](#).

Definition at line 319 of file [ParticleTrack.cpp](#).

6.18.4.14 ReflectY()

```
void ParticleTrack::ReflectY () [override], [virtual]
```

Reflect everything in the y-direction

Reimplemented from [Particle](#).

Definition at line 334 of file [ParticleTrack.cpp](#).

6.18.4.15 ReflectZ()

```
void ParticleTrack::ReflectZ () [override], [virtual]
```

Reflect everything in the z-direction since radiator cells are symmetric

Reimplemented from [Particle](#).

Definition at line 327 of file [ParticleTrack.cpp](#).

6.18.4.16 SetRadiator()

```
void ParticleTrack::SetRadiator (
    const RadiatorCell * radiatorCell)
```

Set radiator

Definition at line 76 of file [ParticleTrack.cpp](#).

6.18.4.17 TrackThroughAerogel()

```
bool ParticleTrack::TrackThroughAerogel ()
```

Track particle through radiator cell

Returns

Returns true if tracking through the magnetic field was successful

Definition at line 107 of file [ParticleTrack.cpp](#).

6.18.4.18 TrackThroughGasToMirror()

```
void ParticleTrack::TrackThroughGasToMirror ()
```

Helper function to track particle through gas until it hits the correct mirror

Definition at line 147 of file [ParticleTrack.cpp](#).

6.18.4.19 TrackThroughTracker()

```
bool ParticleTrack::TrackThroughTracker (
    const TrackingVolume & InnerTracker)
```

Track particle through inner tracker with magnetic field

Parameters

<i>InnerTracker</i>	The tracking volume
---------------------	---------------------

Returns

Returns true if tracking through the magnetic field was successful

Definition at line 40 of file [ParticleTrack.cpp](#).

6.18.4.20 TrackToNextCell()

```
bool ParticleTrack::TrackToNextCell (  
    const RadiatorArray & radiatorArray)
```

If particle misses the mirror in this cell, call this function to migrate to next cell

Definition at line 172 of file [ParticleTrack.cpp](#).

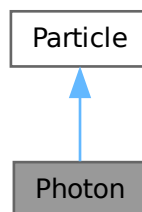
The documentation for this class was generated from the following files:

- [include/ParticleTrack.h](#)
- [src/ParticleTrack.cpp](#)

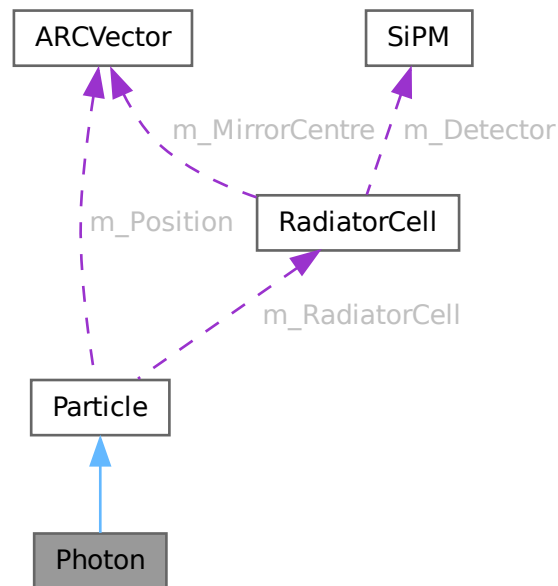
6.19 Photon Class Reference

```
#include <Photon.h>
```

Inheritance diagram for Photon:



Collaboration diagram for Photon:



Public Types

- enum class [Radiator](#) { [Aerogel](#) , [Gas](#) }
- enum class [Status](#) {
[Emitted](#) , [MirrorHit](#) , [MirrorMiss](#) , [EfficiencyMiss](#) ,
[DetectorHit](#) , [DetectorMiss](#) , [AerogelScattered](#) , [WallMiss](#) ,
[Backwards](#) , [OutsideCell](#) , [OutsideMirrorRadius](#) }

Public Member Functions

- [Photon](#) (const [Vector](#) &Position, const [Vector](#) &AssumedPosition, const [Vector](#) &Direction, const [Vector](#) &ParticleDirection, const [Vector](#) &AssumedParticleDirection, double Energy, double CosCherenkovAngle, [Radiator](#) radiator, const [RadiatorCell](#) *radiatorCell, bool IsTrackDrawn, double Weight)
- [~Photon](#) ()=default
- [Photon](#) (const [Photon](#) &photon)=delete
- [Photon](#) ([Photon](#) &&photon)=default
- std::vector< std::pair< std::unique_ptr< TObject >, std::string > > [DrawPhotonPath](#) () const
- void [PropagatePhoton](#) (const [Vector](#) &Displacement)
- const [Vector](#) & [GetDirection](#) () const
- void [KickPhoton](#) (const [Vector](#) &Kick)
- [Status](#) [GetStatus](#) () const
- const [Vector](#) & [GetEmissionPoint](#) (bool TrueEmissionPoint) const
- void [UpdatePhotonStatus](#) ([Status](#) status)
- double [GetEnergy](#) () const
- void [AddAerogelTravelDistance](#) (double Distance)
- double [GetAerogelTravelDistance](#) () const

- const [ARCVector](#) * [GetMirrorHitPosition](#) () const
- void [RegisterMirrorHitPosition](#) (const [Vector](#) &MirrorHitPosition)
- [Radiator](#) [GetRadiator](#) () const
- double [GetCosCherenkovAngle](#) () const
- virtual void [ConvertToRadiatorCoordinates](#) () override
- virtual void [ConvertBackToGlobalCoordinates](#) () override
- virtual void [MapPhi](#) (double DeltaPhi) override
- virtual void [ReflectZ](#) () override
- virtual void [ReflectY](#) () override
- virtual bool [IsAtRadiator](#) () const override
- void [PutPhotonToEmissionPoint](#) ()
- void [PhotonHasMigrated](#) ()
- bool [HasPhotonMigrated](#) () const
- const [Vector](#) & [GetParticleDirection](#) (bool TrueEmissionPoint) const
- double [GetWeight](#) () const

Public Member Functions inherited from [Particle](#)

- [Particle](#) (const [Vector](#) &Position, const [RadiatorCell](#) *radiatorCell=nullptr)
- [~Particle](#) ()=default
- bool [FindRadiator](#) (const [RadiatorArray](#) &radiatorArray)
- const [ARCVector](#) & [GetPosition](#) () const
- const [RadiatorCell](#) * [GetRadiatorCell](#) () const
- std::size_t [GetRadiatorColumnNumber](#) () const
- std::size_t [GetRadiatorRowNumber](#) () const

Additional Inherited Members

Protected Attributes inherited from [Particle](#)

- [ARCVector](#) [m_Position](#)
- const [RadiatorCell](#) * [m_RadiatorCell](#)

6.19.1 Detailed Description

Definition at line 23 of file [Photon.h](#).

6.19.2 Member Enumeration Documentation

6.19.2.1 Radiator

```
enum class Photon::Radiator [strong]
```

Enum class classifying which radiator the photon was emitted from

Enumerator

Aerogel	
Gas	

Definition at line 28 of file [Photon.h](#).

6.19.2.2 Status

```
enum class Photon::Status [strong]
```

Enum class classifying whether the photon has been tracked or if it missed the mirror
 Emitted: [Photon](#) just got emitted inside radiator
 MirrorHit: [Photon](#) is at the mirror
 MirrorMiss: [Photon](#) missed the mirror and ended up outside radiator
 EfficiencyMiss: [Photon](#) lost in mirror reflectivity, detector dead space or PDE
 DetectorHit: [Photon](#) hit the detector
 DetectorMiss: [Photon](#) missed the detector
 AerogelScattered: Scattered by the aerogel
 WallMiss: The photon hit the vessel wall on top, so an area without a mirror
 Backwards: The photon must travel backwards to hit mirror (impossible!)

Enumerator

Emitted	
MirrorHit	
MirrorMiss	
EfficiencyMiss	
DetectorHit	
DetectorMiss	
AerogelScattered	
WallMiss	
Backwards	
OutsideCell	
OutsideMirrorRadius	

Definition at line 41 of file [Photon.h](#).

6.19.3 Constructor & Destructor Documentation

6.19.3.1 Photon() [1/3]

```
Photon::Photon (
    const Vector & Position,
    const Vector & AssumedPosition,
    const Vector & Direction,
    const Vector & ParticleDirection,
    const Vector & AssumedParticleDirection,
    double Energy,
    double CosCherenkovAngle,
    Radiator radiator,
    const RadiatorCell * radiatorCell,
    bool IsTrackDrawn,
    double Weight)
```

Construct a photon with position, direction and energy Input vectors in constructor are in global coordinates All vectors are stored in the local coordinates internally

Parameters

<i>Position</i>	Position vector
<i>AssumedPosition</i>	The midpoint of the track in the radiator
<i>Direction</i>	Direction vector
<i>ParticleDirection</i>	Unit vector in the direction of the particle

Parameters

<i>AssumedParticleDirection</i>	Unit vector in the assumed particle direction
<i>Energy</i>	Photon energy
<i>CosCherenkovAngle</i>	Cosine of Cherenkov angle
<i>radiator</i>	Aerogel or gas radiator where this photon was emitted
<i>radiatorCell</i>	The radiator cell where this photon was emitted
<i>IsTrackDrawn</i>	True if the track that emitted this photon is drawn
<i>Weight</i>	The weight of this photon, inverse of the photon multiplier

Definition at line 18 of file [Photon.cpp](#).

6.19.3.2 ~Photon()

```
Photon::~Photon () [default]
```

Need virtual constructor since it's a virtual class

6.19.3.3 Photon() [2/3]

```
Photon::Photon (
    const Photon & photon) [delete]
```

We don't need copy constructor

6.19.3.4 Photon() [3/3]

```
Photon::Photon (
    Photon && photon) [default]
```

The move constructor is default

6.19.4 Member Function Documentation

6.19.4.1 AddAerogelTravelDistance()

```
void Photon::AddAerogelTravelDistance (
    double Distance)
```

Add travel distance through aerogel

Definition at line 140 of file [Photon.cpp](#).

6.19.4.2 ConvertBackToGlobalCoordinates()

```
void Photon::ConvertBackToGlobalCoordinates () [override], [virtual]
```

Convert back to global coordinates

Reimplemented from [Particle](#).

Definition at line 186 of file [Photon.cpp](#).

6.19.4.3 ConvertToRadiatorCoordinates()

```
void Photon::ConvertToRadiatorCoordinates () [override], [virtual]
```

Convert to local radiator coordinates

Reimplemented from [Particle](#).

Definition at line 171 of file [Photon.cpp](#).

6.19.4.4 DrawPhotonPath()

```
std::vector< std::pair< std::unique_ptr< TObject >, std::string > > Photon::DrawPhotonPath  
( ) const
```

Draw photon path

Definition at line 56 of file [Photon.cpp](#).

6.19.4.5 GetAerogelTravelDistance()

```
double Photon::GetAerogelTravelDistance () const
```

Get travel distance through aerogel

Definition at line 147 of file [Photon.cpp](#).

6.19.4.6 GetCosCherenkovAngle()

```
double Photon::GetCosCherenkovAngle () const
```

Get the cosine of the Cherenkov angle

Definition at line 167 of file [Photon.cpp](#).

6.19.4.7 GetDirection()

```
const Vector & Photon::GetDirection () const
```

Get the photon direction

Definition at line 109 of file [Photon.cpp](#).

6.19.4.8 GetEmissionPoint()

```
const Vector & Photon::GetEmissionPoint (
    bool TrueEmissionPoint) const
```

Get the emission point

Definition at line 121 of file [Photon.cpp](#).

6.19.4.9 GetEnergy()

```
double Photon::GetEnergy () const
```

Get the photon energy

Definition at line 136 of file [Photon.cpp](#).

6.19.4.10 GetMirrorHitPosition()

```
const ARCVector * Photon::GetMirrorHitPosition () const
```

Get mirror hit position, or nullptr if the photon didn't hit the mirror

Definition at line 151 of file [Photon.cpp](#).

6.19.4.11 GetParticleDirection()

```
const Vector & Photon::GetParticleDirection (
    bool TrueEmissionPoint) const
```

Get the direction of the particle that emitted this photon

Definition at line 241 of file [Photon.cpp](#).

6.19.4.12 GetRadiator()

```
Photon::Radiator Photon::GetRadiator () const
```

Get the radiator that the photon was generated in

Definition at line 163 of file [Photon.cpp](#).

6.19.4.13 GetStatus()

```
Photon::Status Photon::GetStatus () const
```

Get the photon status

Definition at line 117 of file [Photon.cpp](#).

6.19.4.14 GetWeight()

```
double Photon::GetWeight () const
```

Get the weight of this photon (inverse of the photon multiplier)

Definition at line 249 of file [Photon.cpp](#).

6.19.4.15 HasPhotonMigrated()

```
bool Photon::HasPhotonMigrated () const
```

Get flag that is true if photon migrates

Definition at line 237 of file [Photon.cpp](#).

6.19.4.16 IsAtRadiator()

```
bool Photon::IsAtRadiator () const [override], [virtual]
```

Check if particle is at the radiator so that we can search for the radiator cell

Implements [Particle](#).

Definition at line 225 of file [Photon.cpp](#).

6.19.4.17 KickPhoton()

```
void Photon::KickPhoton (
    const Vector & Kick)
```

Change the photon direction

Parameters

<i>Kick</i>	The kick given to the photon to change its direction
-------------	--

Definition at line 113 of file [Photon.cpp](#).

6.19.4.18 MapPhi()

```
void Photon::MapPhi (
    double DeltaPhi) [override], [virtual]
```

Rotate around the phi direction to map particle to a valid radiator cell

Parameters

<i>DeltaPhi</i>	Angle that is rotated in phi
-----------------	------------------------------

Reimplemented from [Particle](#).

Definition at line 197 of file [Photon.cpp](#).

6.19.4.19 PhotonHasMigrated()

```
void Photon::PhotonHasMigrated ()
```

Call this function is photon has migrated to a different cell

Definition at line 233 of file [Photon.cpp](#).

6.19.4.20 PropagatePhoton()

```
void Photon::PropagatePhoton (  
    const Vector & Displacement)
```

Propagate the photon

Parameters

<i>Displacement</i>	The displace that the photon is propagated
---------------------	--

Definition at line 105 of file [Photon.cpp](#).

6.19.4.21 PutPhotonToEmissionPoint()

```
void Photon::PutPhotonToEmissionPoint ()
```

Set photon position back to the emission point

Definition at line 229 of file [Photon.cpp](#).

6.19.4.22 ReflectY()

```
void Photon::ReflectY () [override], [virtual]
```

Reflect everything in the y-direction

Reimplemented from [Particle](#).

Definition at line 216 of file [Photon.cpp](#).

6.19.4.23 ReflectZ()

```
void Photon::ReflectZ () [override], [virtual]
```

Reflect everything in the z-direction since radiator cells are symmetric

Reimplemented from [Particle](#).

Definition at line 206 of file [Photon.cpp](#).

6.19.4.24 RegisterMirrorHitPosition()

```
void Photon::RegisterMirrorHitPosition (  
    const Vector & MirrorHitPosition)
```

Register mirror hit position

Definition at line 155 of file [Photon.cpp](#).

6.19.4.25 UpdatePhotonStatus()

```
void Photon::UpdatePhotonStatus (  
    Photon::Status status)
```

Update photon status

Definition at line 129 of file [Photon.cpp](#).

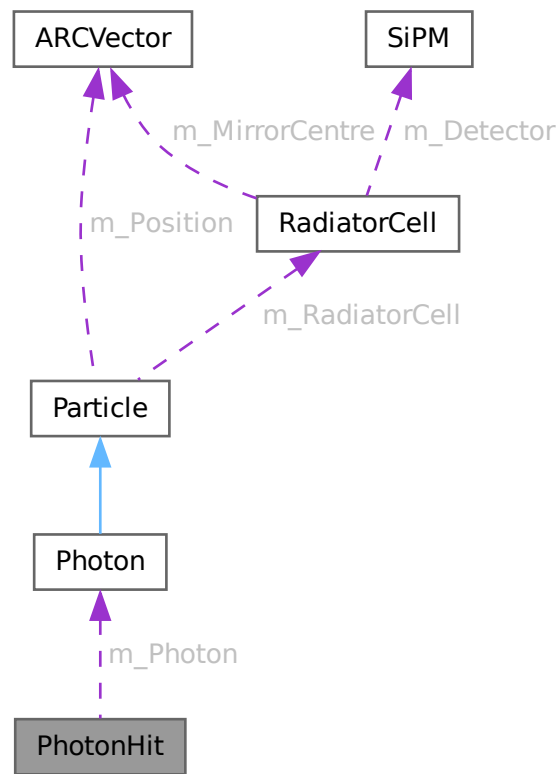
The documentation for this class was generated from the following files:

- include/[Photon.h](#)
- src/[Photon.cpp](#)

6.20 PhotonHit Struct Reference

```
#include <SiPM.h>
```

Collaboration diagram for PhotonHit:



Public Attributes

- const [Vector](#) m_HitPosition
- const [Photon](#) * m_Photon
- const [Vector](#) m_CentreHitDistance

6.20.1 Detailed Description

Definition at line 22 of file [SiPM.h](#).

6.20.2 Member Data Documentation

6.20.2.1 m_CentreHitDistance

const [Vector](#) PhotonHit::m_CentreHitDistance

[Vector](#) between photon hit and centre of detector

Definition at line 34 of file [SiPM.h](#).

6.20.2.2 m_HitPosition

```
const Vector PhotonHit::m_HitPosition
```

Detector hit coordinates

Definition at line 26 of file [SiPM.h](#).

6.20.2.3 m_Photon

```
const Photon* PhotonHit::m_Photon
```

Pointer to the photon that caused this hit

Definition at line 30 of file [SiPM.h](#).

The documentation for this struct was generated from the following file:

- [include/SiPM.h](#)

6.21 PhotonReconstructor::QuarticSolution Struct Reference

```
#include <PhotonReconstructor.h>
```

Public Attributes

- `std::array< double, 2 > m\_SinBeta {}`
- `std::array< double, 2 > m\_CosBeta {}`
- `bool m\_DegenerateSolution = false`

6.21.1 Detailed Description

Struct with solutions to the quartic equation

Definition at line 44 of file [PhotonReconstructor.h](#).

6.21.2 Member Data Documentation

6.21.2.1 m_CosBeta

```
std::array<double, 2> PhotonReconstructor::QuarticSolution::m_CosBeta {}
```

Definition at line 49 of file [PhotonReconstructor.h](#).

6.21.2.2 m_DegenerateSolution

```
bool PhotonReconstructor::QuarticSolution::m_DegenerateSolution = false
```

Flag that is true if the number of real solutions is not 2

Definition at line 53 of file [PhotonReconstructor.h](#).

6.21.2.3 m_SinBeta

```
std::array<double, 2> PhotonReconstructor::QuarticSolution::m_SinBeta {}
```

Sine and cosine of angle between emission and reflection point, relative to the mirror centre

Definition at line 48 of file [PhotonReconstructor.h](#).

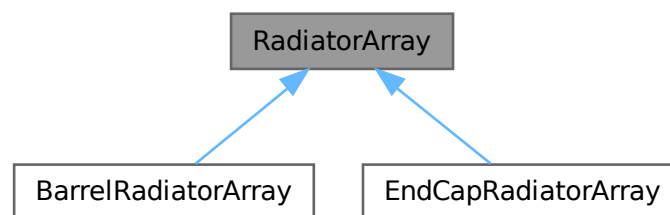
The documentation for this struct was generated from the following file:

- include/[PhotonReconstructor.h](#)

6.22 RadiatorArray Class Reference

```
#include <RadiatorArray.h>
```

Inheritance diagram for RadiatorArray:



Public Member Functions

- [RadiatorArray](#) ()
- [RadiatorArray](#) (const [RadiatorArray](#) &radiatorArray)=delete
- virtual [~RadiatorArray](#) ()=default
- [RadiatorCell](#) * [GetRadiatorCell](#) (std::size_t i, std::size_t j)
- const [RadiatorCell](#) * [operator\(\)](#) (std::size_t i, std::size_t j) const
- virtual const [RadiatorCell](#) * [FindRadiator](#) ([Particle](#) &particle) const =0
- virtual std::vector< std::pair< std::unique_ptr< TObject >, std::string > > [DrawRadiatorArray](#) () const =0

Protected Attributes

- `std::vector< std::unique_ptr< RadiatorCell > > m_Cells`
- `const std::size_t m_NumberMainRowCells`
- `const double m_xHexDist`
- `const double m_yHexDist`

6.22.1 Detailed Description

Definition at line 15 of file [RadiatorArray.h](#).

6.22.2 Constructor & Destructor Documentation

6.22.2.1 RadiatorArray() [1/2]

```
RadiatorArray::RadiatorArray ()
```

Default constructor

Definition at line 6 of file [RadiatorArray.cpp](#).

6.22.2.2 RadiatorArray() [2/2]

```
RadiatorArray::RadiatorArray (
    const RadiatorArray & radiatorArray) [delete]
```

Delete copy constructor

6.22.2.3 ~RadiatorArray()

```
virtual RadiatorArray::~~RadiatorArray () [virtual], [default]
```

Need virtual destructor because of polymorphism

6.22.3 Member Function Documentation

6.22.3.1 DrawRadiatorArray()

```
virtual std::vector< std::pair< std::unique_ptr< TObject >, std::string > > RadiatorArray↔
::DrawRadiatorArray () const [pure virtual]
```

Draw all the radiator cells

Implemented in [BarrelRadiatorArray](#), and [EndCapRadiatorArray](#).

6.22.3.2 FindRadiator()

```
virtual const RadiatorCell * RadiatorArray::FindRadiator (
    Particle & particle) const [pure virtual]
```

Check which radiator the particle goes through It will map the track momentum and position if the track hits an equivalent radiator cell

Implemented in [BarrelRadiatorArray](#), and [EndCapRadiatorArray](#).

6.22.3.3 GetRadiatorCell()

```
RadiatorCell * RadiatorArray::GetRadiatorCell (
    std::size_t i,
    std::size_t j)
```

Function to get a non-constant [RadiatorCell](#) object

Definition at line 13 of file [RadiatorArray.cpp](#).

6.22.3.4 operator()()

```
const RadiatorCell * RadiatorArray::operator() (
    std::size_t i,
    std::size_t j) const
```

Operator overload to get the individual radiator cells

Definition at line 22 of file [RadiatorArray.cpp](#).

6.22.4 Member Data Documentation

6.22.4.1 m_Cells

```
std::vector<std::unique_ptr<RadiatorCell> > RadiatorArray::m_Cells [protected]
```

[Vector](#) containing all the radiator cells

Definition at line 51 of file [RadiatorArray.h](#).

6.22.4.2 m_NumberMainRowCells

```
const std::size_t RadiatorArray::m_NumberMainRowCells [protected]
```

Number of cells in theta direction along the main row

Definition at line 55 of file [RadiatorArray.h](#).

6.22.4.3 m_xHexDist

```
const double RadiatorArray::m_xHexDist [protected]
```

The horizontal distance between two hexagons

Definition at line 59 of file [RadiatorArray.h](#).

6.22.4.4 m_yHexDist

```
const double RadiatorArray::m_yHexDist [protected]
```

The vertical distance between two hexagons

Definition at line 63 of file [RadiatorArray.h](#).

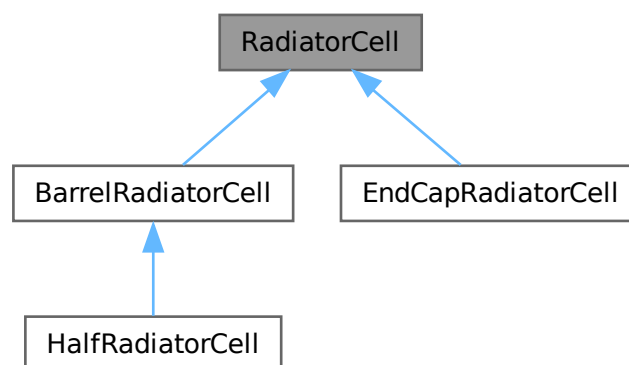
The documentation for this class was generated from the following files:

- [include/RadiatorArray.h](#)
- [src/RadiatorArray.cpp](#)

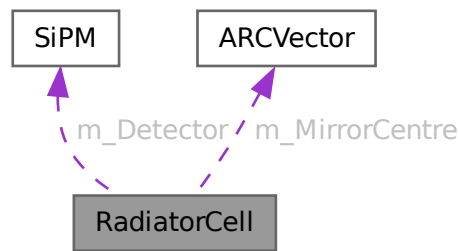
6.23 RadiatorCell Class Reference

```
#include <RadiatorCell.h>
```

Inheritance diagram for RadiatorCell:



Collaboration diagram for RadiatorCell:



Public Member Functions

- [RadiatorCell](#) (std::size_t CellColumnNumber, std::size_t CellRowNumber, double HexagonSize, const [Vector](#) &Position, const [Rotation3D](#) &Rotation, const std::string &Prefix="")
- [RadiatorCell](#) (const [RadiatorCell](#) &radiatorCell)=delete
- virtual [~RadiatorCell](#) ()=default
- double [GetRadiatorThickness](#) () const
- double [GetVesselThickness](#) () const
- double [GetCoolingThickness](#) () const
- double [GetAerogelThickness](#) () const
- const [Vector](#) & [GetMirrorCentre](#) () const
- [Vector](#) [GetGlobalMirrorCentre](#) () const
- double [GetMirrorCurvature](#) () const
- const [Vector](#) & [GetRadiatorPosition](#) () const
- const [Rotation3D](#) & [GetRadiatorRotation](#) () const
- virtual bool [IsInsideCell](#) (const [ARCVector](#) &Position) const =0
- bool [IsInsideCell](#) (const [Photon](#) &photon) const
- virtual std::vector< std::pair< std::unique_ptr< TObject >, std::string > > [DrawRadiatorGeometry](#) () const =0
- double [GetHexagonSize](#) () const
- std::pair< std::size_t, std::size_t > [GetCellNumber](#) () const
- const [SiPM](#) & [GetDetector](#) () const
- void [SetMirrorCurvature](#) (double Curvature)
- void [SetMirrorXPosition](#) (double x)
- void [SetMirrorZPosition](#) (double z)
- void [SetDetectorPosition](#) (double x)
- void [SetDetectorTilt](#) (double Angle)
- virtual bool [IsDetectorInsideCell](#) () const =0
- virtual bool [IsEdgeCell](#) () const =0
- virtual [RotationZ](#) [ReversePhiRotation](#) () const

Protected Member Functions

- double [GetMirrorCurvatureCentreZ](#) () const

Protected Attributes

- const [Vector](#) `m_Position`
- const [Rotation3D](#) `m_Rotation`
- const double `m_RadiatorThickness`
- const double `m_VesselThickness`
- const double `m_CoolingThickness`
- const double `m_AerogelThickness`
- const double `m_HexagonSize`
- const std::pair< std::size_t, std::size_t > `m_CellNumber`
- [SiPM](#) `m_Detector`
- double `m_MirrorCurvature`
- const [Vector](#) `m_DefaultMirrorCentre`
- [ARCVector](#) `m_MirrorCentre`

6.23.1 Detailed Description

Definition at line 28 of file [RadiatorCell.h](#).

6.23.2 Constructor & Destructor Documentation

6.23.2.1 RadiatorCell() [1/2]

```
RadiatorCell::RadiatorCell (
    std::size_t CellColumnNumber,
    std::size_t CellRowNumber,
    double HexagonSize,
    const Vector & Position,
    const Rotation3D & Rotation,
    const std::string & Prefix = "")
```

Constructor that sets up the geometry with a default cell position

Parameters

<i>CellColumnNumber</i>	The column number of this cell
<i>CellRowNumber</i>	The row number of this cell
<i>HexagonSize</i>	The length between two opposite hexagon edges (not points)
<i>Position</i>	The position of the radiator cell
<i>Rotation</i>	The rotation from global to local radiator coordinates
<i>Prefix</i>	Prefix in the names of cells in options file

Definition at line 14 of file [RadiatorCell.cpp](#).

6.23.2.2 RadiatorCell() [2/2]

```
RadiatorCell::RadiatorCell (
    const RadiatorCell & radiatorCell) [delete]
```

Delete copy constructor

6.23.2.3 ~RadiatorCell()

```
virtual RadiatorCell::~~RadiatorCell () [virtual], [default]
```

Need virtual destructor because of polymorphism

6.23.3 Member Function Documentation

6.23.3.1 DrawRadiatorGeometry()

```
virtual std::vector< std::pair< std::unique_ptr< TObject >, std::string > > RadiatorCell::↵  
DrawRadiatorGeometry () const [pure virtual]
```

Draw radiator geometry

Implemented in [BarrelRadiatorCell](#), [EndCapRadiatorCell](#), and [HalfRadiatorCell](#).

6.23.3.2 GetAerogelThickness()

```
double RadiatorCell::GetAerogelThickness () const
```

Get thickness of aerogel

Definition at line 70 of file [RadiatorCell.cpp](#).

6.23.3.3 GetCellNumber()

```
std::pair< std::size_t, std::size_t > RadiatorCell::GetCellNumber () const
```

Get cell number

Definition at line 109 of file [RadiatorCell.cpp](#).

6.23.3.4 GetCoolingThickness()

```
double RadiatorCell::GetCoolingThickness () const
```

Get thickness of cooling plate

Definition at line 66 of file [RadiatorCell.cpp](#).

6.23.3.5 GetDetector()

```
const SiPM & RadiatorCell::GetDetector () const
```

Get the detector

Definition at line 113 of file [RadiatorCell.cpp](#).

6.23.3.6 GetGlobalMirrorCentre()

```
Vector RadiatorCell::GetGlobalMirrorCentre () const
```

Get the mirror centre of curvature in global coordinates

Definition at line 78 of file [RadiatorCell.cpp](#).

6.23.3.7 GetHexagonSize()

```
double RadiatorCell::GetHexagonSize () const
```

Get length between two edges of a hexagon cell

Definition at line 105 of file [RadiatorCell.cpp](#).

6.23.3.8 GetMirrorCentre()

```
const Vector & RadiatorCell::GetMirrorCentre () const
```

Get the mirror centre of curvature in local coordinates

Definition at line 74 of file [RadiatorCell.cpp](#).

6.23.3.9 GetMirrorCurvature()

```
double RadiatorCell::GetMirrorCurvature () const
```

Get the mirror curvature

Definition at line 89 of file [RadiatorCell.cpp](#).

6.23.3.10 GetMirrorCurvatureCentreZ()

```
double RadiatorCell::GetMirrorCurvatureCentreZ () const [protected]
```

Get the centre of curvature of the mirror z coordinate in local coordinates

Definition at line 82 of file [RadiatorCell.cpp](#).

6.23.3.11 GetRadiatorPosition()

```
const Vector & RadiatorCell::GetRadiatorPosition () const
```

Get the radiator cell position in the global coordinates

Definition at line 97 of file [RadiatorCell.cpp](#).

6.23.3.12 GetRadiatorRotation()

```
const Rotation3D & RadiatorCell::GetRadiatorRotation () const
```

Get the radiator cell rotation from global to local coordinates

Definition at line 101 of file [RadiatorCell.cpp](#).

6.23.3.13 GetRadiatorThickness()

```
double RadiatorCell::GetRadiatorThickness () const
```

Get total radiator cell thickness

Definition at line 58 of file [RadiatorCell.cpp](#).

6.23.3.14 GetVesselThickness()

```
double RadiatorCell::GetVesselThickness () const
```

Get thickness of vessel

Definition at line 62 of file [RadiatorCell.cpp](#).

6.23.3.15 IsDetectorInsideCell()

```
virtual bool RadiatorCell::IsDetectorInsideCell () const [pure virtual]
```

Function that checks if the detector is contained inside the radiator cell

Implemented in [BarrelRadiatorCell](#), and [EndCapRadiatorCell](#).

6.23.3.16 IsEdgeCell()

```
virtual bool RadiatorCell::IsEdgeCell () const [pure virtual]
```

Checks if the cell is at the edge (near the end cap)

Implemented in [BarrelRadiatorCell](#), and [EndCapRadiatorCell](#).

6.23.3.17 IsInsideCell() [1/2]

```
virtual bool RadiatorCell::IsInsideCell (  
    const ARCVector & Position) const [pure virtual]
```

Function that checks if the position is inside the radiator

Implemented in [BarrelRadiatorCell](#), [EndCapRadiatorCell](#), and [HalfRadiatorCell](#).

6.23.3.18 IsInsideCell() [2/2]

```
bool RadiatorCell::IsInsideCell (
    const Photon & photon) const
```

Function that checks if the photon is inside the radiator

Definition at line 93 of file [RadiatorCell.cpp](#).

6.23.3.19 ReversePhiRotation()

```
RotationZ RadiatorCell::ReversePhiRotation () const [virtual]
```

Get the reverse rotation in phi, for plotting purposes

Reimplemented in [BarrelRadiatorCell](#).

Definition at line 137 of file [RadiatorCell.cpp](#).

6.23.3.20 SetDetectorPosition()

```
void RadiatorCell::SetDetectorPosition (
    double x)
```

Set the detector X position

Definition at line 129 of file [RadiatorCell.cpp](#).

6.23.3.21 SetDetectorTilt()

```
void RadiatorCell::SetDetectorTilt (
    double Angle)
```

Set the detector tilt, in radians

Definition at line 133 of file [RadiatorCell.cpp](#).

6.23.3.22 SetMirrorCurvature()

```
void RadiatorCell::SetMirrorCurvature (
    double Curvature)
```

Set the mirror curvature

Parameters

<i>Curvature</i>	New mirror curvature
------------------	----------------------

Definition at line 117 of file [RadiatorCell.cpp](#).

6.23.3.23 SetMirrorXPosition()

```
void RadiatorCell::SetMirrorXPosition (
    double x)
```

Set the mirror x position, relative to the default position

Definition at line 121 of file [RadiatorCell.cpp](#).

6.23.3.24 SetMirrorZPosition()

```
void RadiatorCell::SetMirrorZPosition (
    double z)
```

Set the mirror z position, relative to the default position

Definition at line 125 of file [RadiatorCell.cpp](#).

6.23.4 Member Data Documentation

6.23.4.1 m_AerogelThickness

```
const double RadiatorCell::m_AerogelThickness [protected]
```

Thickness of the aerogel

Definition at line 175 of file [RadiatorCell.h](#).

6.23.4.2 m_CellNumber

```
const std::pair<std::size_t, std::size_t> RadiatorCell::m_CellNumber [protected]
```

Radiator cell row number (first) and column number (second) For a single central cell, it is assigned number (0, 0) For an array of radiator cells, numbering starts from (0, 1) from the middle of the main row

Definition at line 186 of file [RadiatorCell.h](#).

6.23.4.3 m_CoolingThickness

```
const double RadiatorCell::m_CoolingThickness [protected]
```

Thickness of cooling plate

Definition at line 171 of file [RadiatorCell.h](#).

6.23.4.4 m_DefaultMirrorCentre

```
const Vector RadiatorCell::m_DefaultMirrorCentre [protected]
```

Default centre of curvature of the mirror

Definition at line 198 of file [RadiatorCell.h](#).

6.23.4.5 m_Detector

```
SiPM RadiatorCell::m_Detector [protected]
```

The [SiPM](#) in the radiator cell

Definition at line 190 of file [RadiatorCell.h](#).

6.23.4.6 m_HexagonSize

```
const double RadiatorCell::m_HexagonSize [protected]
```

The length between two edges of a hexagon

Definition at line 179 of file [RadiatorCell.h](#).

6.23.4.7 m_MirrorCentre

```
ARCVector RadiatorCell::m_MirrorCentre [protected]
```

Centre of curvature of the mirror

Definition at line 202 of file [RadiatorCell.h](#).

6.23.4.8 m_MirrorCurvature

```
double RadiatorCell::m_MirrorCurvature [protected]
```

Radius of curvature of the spherical mirror

Definition at line 194 of file [RadiatorCell.h](#).

6.23.4.9 m_Position

```
const Vector RadiatorCell::m_Position [protected]
```

The radiator cell position

Definition at line 151 of file [RadiatorCell.h](#).

6.23.4.10 m_RadiatorThickness

```
const double RadiatorCell::m_RadiatorThickness [protected]
```

The full thickness of the radiator cell

Definition at line 163 of file [RadiatorCell.h](#).

6.23.4.11 m_Rotation

```
const Rotation3D RadiatorCell::m_Rotation [protected]
```

The rotation from global coordinates to the local radiator coordinates

Definition at line 155 of file [RadiatorCell.h](#).

6.23.4.12 m_VesselThickness

```
const double RadiatorCell::m_VesselThickness [protected]
```

Thickness of the vessel

Definition at line 167 of file [RadiatorCell.h](#).

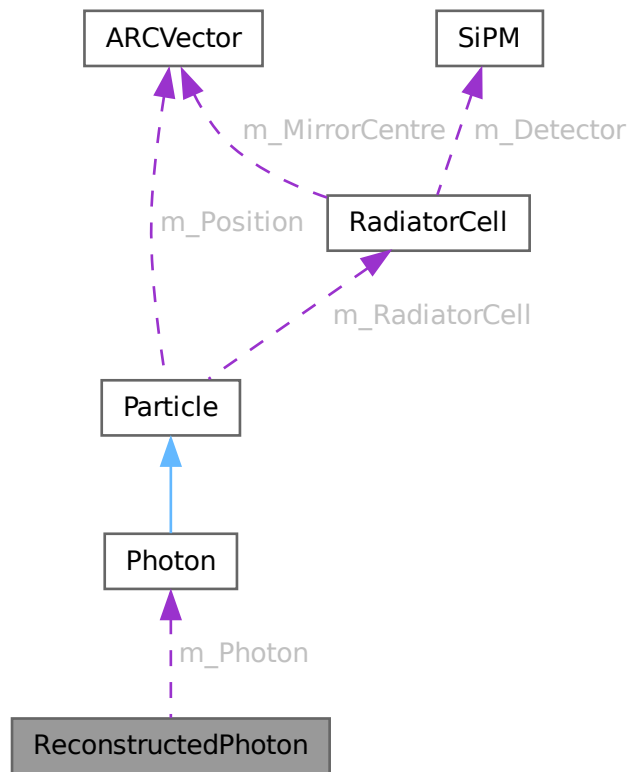
The documentation for this class was generated from the following files:

- [include/RadiatorCell.h](#)
- [src/RadiatorCell.cpp](#)

6.24 ReconstructedPhoton Struct Reference

```
#include <ReconstructedPhoton.h>
```

Collaboration diagram for ReconstructedPhoton:



Public Member Functions

- [ReconstructedPhoton](#) (const [Photon](#) &photon)

Public Attributes

- const [Photon](#) * [m_Photon](#)
- double [m_CosCherenkovAngle_TrueEmissionPoint](#)
- double [m_CosCherenkovAngle](#)

6.24.1 Detailed Description

[ReconstructedPhoton](#) contains the reconstructed Cherenkov angle from a photon hit

Definition at line 11 of file [ReconstructedPhoton.h](#).

6.24.2 Constructor & Destructor Documentation

6.24.2.1 ReconstructedPhoton()

```
ReconstructedPhoton::ReconstructedPhoton (  
    const Photon & photon)
```

Trivial constructor

Parameters

<i>photon</i>	The original photon
---------------	---------------------

Definition at line 5 of file [ReconstructedPhoton.cpp](#).

6.24.3 Member Data Documentation

6.24.3.1 m_CosCherenkovAngle

```
double ReconstructedPhoton::m_CosCherenkovAngle
```

The reconstructed Cherenkov angle, assming emission in the middle

Definition at line 28 of file [ReconstructedPhoton.h](#).

6.24.3.2 m_CosCherenkovAngle_TrueEmissionPoint

```
double ReconstructedPhoton::m_CosCherenkovAngle_TrueEmissionPoint
```

The reconstructed Cherenkov angle, using the true emission point

Definition at line 24 of file [ReconstructedPhoton.h](#).

6.24.3.3 m_Photon

```
const Photon* ReconstructedPhoton::m_Photon
```

The photon that is being reconstructed

Definition at line 20 of file [ReconstructedPhoton.h](#).

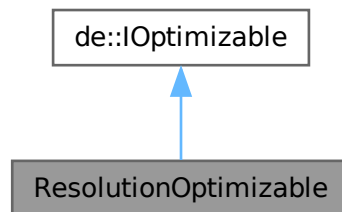
The documentation for this struct was generated from the following files:

- [include/ReconstructedPhoton.h](#)
- [src/ReconstructedPhoton.cpp](#)

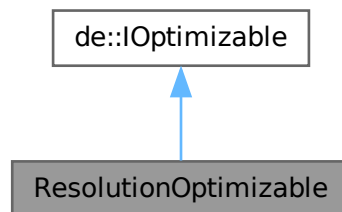
6.25 ResolutionOptimizable Class Reference

```
#include <ResolutionOptimizable.h>
```

Inheritance diagram for ResolutionOptimizable:



Collaboration diagram for ResolutionOptimizable:



Public Member Functions

- [ResolutionOptimizable](#) ([RadiatorCell](#) &radiatorCell, const [RadiatorArray](#) &radiatorArray, const [Tracks](#) &Particles)
- double [EvaluateCost](#) (std::vector< double > x) const override
- std::size_t [NumberOfParameters](#) () const override
- std::vector< [Constraints](#) > [GetConstraints](#) () const override
- std::unordered_map< std::size_t, double > [GetFixedParameters](#) () const

Public Member Functions inherited from [de::IOptimizable](#)

- virtual [~IOptimizable](#) ()

6.25.1 Detailed Description

Definition at line 20 of file [ResolutionOptimizable.h](#).

6.25.2 Constructor & Destructor Documentation

6.25.2.1 ResolutionOptimizable()

```
ResolutionOptimizable::ResolutionOptimizable (
    RadiatorCell & radiatorCell,
    const RadiatorArray & radiatorArray,
    const Tracks & Particles)
```

Constructor that saves the radiator configuration, tracks and photons

Parameters

<i>radiatorCell</i>	The radiator configuration
<i>radiatorArray</i>	The full radiator array
<i>ParticlesPhotons</i>	The generated particles and photons

Definition at line 14 of file [ResolutionOptimizable.cpp](#).

6.25.3 Member Function Documentation

6.25.3.1 EvaluateCost()

```
double ResolutionOptimizable::EvaluateCost (
    std::vector< double > x) const [override], [virtual]
```

Function that evaluates the resolution The free parameters are: Mirror curvature Shift in x position Shift in z position

Parameters

x	Free parameters
---	-----------------

Implements [de::IOptimizable](#).

Definition at line 35 of file [ResolutionOptimizable.cpp](#).

6.25.3.2 GetConstraints()

```
std::vector< Constraints > ResolutionOptimizable::GetConstraints () const [override], [virtual]
```

Set up the parameter space

Implements [de::IOptimizable](#).

Definition at line 58 of file [ResolutionOptimizable.cpp](#).

6.25.3.3 GetFixedParameters()

```
std::unordered_map< std::size_t, double > ResolutionOptimizable::GetFixedParameters () const
```

Get the fixed parameters

Definition at line 63 of file [ResolutionOptimizable.cpp](#).

6.25.3.4 NumberOfParameters()

```
std::size_t ResolutionOptimizable::NumberOfParameters () const [override], [virtual]
```

Find the number of free parameters

Implements [de::IOptimizable](#).

Definition at line 54 of file [ResolutionOptimizable.cpp](#).

The documentation for this class was generated from the following files:

- include/[ResolutionOptimizable.h](#)
- src/[ResolutionOptimizable.cpp](#)

6.26 ResolutionUtilities::ResolutionStruct Struct Reference

```
#include <Utilities.h>
```

Public Attributes

- double [x](#) = 0.0
- int [N](#) = 0
- bool [HitTopWall](#) = false
- bool [HitCorrectCell](#) = false
- [Vector](#) [CentreHitDistance](#) {0.0, 0.0, 0.0}
- double [SumWeights](#) = 0.0

6.26.1 Detailed Description

Struct containing sum of Cherenkov angle, sum of square of Cherenkov angle and total number of photons

6.26.2 Member Data Documentation

6.26.2.1 CentreHitDistance

```
Vector Utilities::ResolutionStruct::CentreHitDistance {0.0, 0.0, 0.0}
```

The average photon hit position

Definition at line 64 of file [Utilities.h](#).

6.26.2.2 HitCorrectCell

```
bool Utilities::ResolutionStruct::HitCorrectCell = false
```

Set to true if the particle hits the correct cell

Definition at line 60 of file [Utilities.h](#).

6.26.2.3 HitTopWall

```
bool Utilities::ResolutionStruct::HitTopWall = false
```

Set to true if particle hits the top wall of the radiator cell

Definition at line 56 of file [Utilities.h](#).

6.26.2.4 N

```
int Utilities::ResolutionStruct::N = 0
```

The number of photons

Definition at line 52 of file [Utilities.h](#).

6.26.2.5 SumWeights

```
double Utilities::ResolutionStruct::SumWeights = 0.0
```

Sum of the photon weights, the physical number of photons when the photon yield is scaled by the photon multiplier (the weight is its inverse)

Definition at line 69 of file [Utilities.h](#).

6.26.2.6 x

```
double Utilities::ResolutionStruct::x = 0.0
```

The Cherenkov angle

Definition at line 48 of file [Utilities.h](#).

The documentation for this struct was generated from the following file:

- [include/Utilities.h](#)

6.27 Utilities::ResolutionStruct Struct Reference

```
#include <Utilities.h>
```

Public Attributes

- double `x` = 0.0
- int `N` = 0
- bool `HitTopWall` = false
- bool `HitCorrectCell` = false
- `Vector` `CentreHitDistance` {0.0, 0.0, 0.0}
- double `SumWeights` = 0.0

6.27.1 Detailed Description

Struct containing sum of Cherenkov angle, sum of square of Cherenkov angle and total number of photons

Definition at line 44 of file [Utilities.h](#).

6.27.2 Member Data Documentation

6.27.2.1 CentreHitDistance

```
Vector Utilities::ResolutionStruct::CentreHitDistance {0.0, 0.0, 0.0}
```

The average photon hit position

Definition at line 64 of file [Utilities.h](#).

6.27.2.2 HitCorrectCell

```
bool Utilities::ResolutionStruct::HitCorrectCell = false
```

Set to true if the particle hits the correct cell

Definition at line 60 of file [Utilities.h](#).

6.27.2.3 HitTopWall

```
bool Utilities::ResolutionStruct::HitTopWall = false
```

Set to true if particle hits the top wall of the radiator cell

Definition at line 56 of file [Utilities.h](#).

6.27.2.4 N

```
int Utilities::ResolutionStruct::N = 0
```

The number of photons

Definition at line 52 of file [Utilities.h](#).

6.27.2.5 SumWeights

```
double Utilities::ResolutionStruct::SumWeights = 0.0
```

Sum of the photon weights, the physical number of photons when the photon yield is scaled by the photon multiplier (the weight is its inverse)

Definition at line 69 of file [Utilities.h](#).

6.27.2.6 x

```
double Utilities::ResolutionStruct::x = 0.0
```

The Cherenkov angle

Definition at line 48 of file [Utilities.h](#).

The documentation for this struct was generated from the following file:

- [include/Utilities.h](#)

6.28 Settings Class Reference

```
#include <Settings.h>
```

Public Member Functions

- [Settings](#) ()=delete

Static Public Member Functions

- static void [AddSettings](#) (const std::string &Name, const std::string &Filename)
- static std::string [GetString](#) (const std::string &Setting)
- static double [GetDouble](#) (const std::string &Setting)
- static int [GetInt](#) (const std::string &Setting)
- static std::size_t [GetSizeT](#) (const std::string &Setting)
- static bool [GetBool](#) (const std::string &Setting)
- static std::vector< int > [GetIntVector](#) (const std::string &Setting)
- static std::vector< std::size_t > [GetSizeTVector](#) (const std::string &Setting)
- static bool [Exists](#) (const std::string &Setting)

6.28.1 Detailed Description

Definition at line 17 of file [Settings.h](#).

6.28.2 Constructor & Destructor Documentation

6.28.2.1 Settings()

```
Settings::Settings () [delete]
```

Delete constructor since class is "static"

6.28.3 Member Function Documentation

6.28.3.1 AddSettings()

```
void Settings::AddSettings (
    const std::string & Name,
    const std::string & Filename) [static]
```

Add settings

Parameters

<i>Name</i>	Name of these settings
<i>Filename</i>	Filename with settings

Definition at line 16 of file [Settings.cpp](#).

6.28.3.2 Exists()

```
bool Settings::Exists (
    const std::string & Setting) [static]
```

Check if setting exists

Definition at line 105 of file [Settings.cpp](#).

6.28.3.3 GetBool()

```
bool Settings::GetBool (
    const std::string & Setting) [static]
```

Get bool setting (if string is "true" this evaluates to true)

Definition at line 77 of file [Settings.cpp](#).

6.28.3.4 GetDouble()

```
double Settings::GetDouble (
    const std::string & Setting) [static]
```

Get double setting

Definition at line 60 of file [Settings.cpp](#).

6.28.3.5 GetInt()

```
int Settings::GetInt (
    const std::string & Setting) [static]
```

Get integer setting

Definition at line 64 of file [Settings.cpp](#).

6.28.3.6 GetIntVector()

```
std::vector< int > Settings::GetIntVector (
    const std::string & Setting) [static]
```

Get vector of int, comma separated in options file

Definition at line 81 of file [Settings.cpp](#).

6.28.3.7 GetSizeT()

```
std::size_t Settings::GetSizeT (
    const std::string & Setting) [static]
```

Get unsigned integer setting

Definition at line 68 of file [Settings.cpp](#).

6.28.3.8 GetSizeTVector()

```
std::vector< std::size_t > Settings::GetSizeTVector (
    const std::string & Setting) [static]
```

Get vector of size_t, comma separated in options file

Definition at line 93 of file [Settings.cpp](#).

6.28.3.9 GetString()

```
std::string Settings::GetString (
    const std::string & Setting) [static]
```

Get string setting

Definition at line 40 of file [Settings.cpp](#).

The documentation for this class was generated from the following files:

- [include/Settings.h](#)
- [src/Settings.cpp](#)

6.29 SiPM Class Reference

```
#include <SiPM.h>
```

Public Member Functions

- [SiPM](#) ()
- [PhotonHit AddPhotonHit](#) ([Photon](#) &photon) const
- void [PlotHits](#) (const std::string &Filename, const std::vector< [PhotonHit](#) > &photonHits) const
- const std::vector< [PhotonHit](#) > & [GetPhotonHits](#) () const
- std::unique_ptr< TObject > [DrawSiPM](#) (const [Vector](#) &RadiatorPosition) const
- bool [IsDetectorHit](#) (const [Photon](#) &photon) const
- void [SetDetectorPosition](#) (double x)
- void [SetDetectorTilt](#) (double Angle)
- double [GetDetectorXPosition](#) () const
- double [GetDetectorZPosition](#) () const
- double [GetDetectorTilt](#) () const
- double [GetDetectorSizeX](#) () const
- double [GetDetectorSizeY](#) () const

6.29.1 Detailed Description

Definition at line 37 of file [SiPM.h](#).

6.29.2 Constructor & Destructor Documentation

6.29.2.1 SiPM()

```
SiPM::SiPM ()
```

Constructor setting up the detector coordinates and detector size

Definition at line 23 of file [SiPM.cpp](#).

6.29.3 Member Function Documentation

6.29.3.1 AddPhotonHit()

```
PhotonHit SiPM::AddPhotonHit (
    Photon & photon) const
```

Register a photon hit and return it

Definition at line 35 of file [SiPM.cpp](#).

6.29.3.2 DrawSiPM()

```
std::unique_ptr< TObject > SiPM::DrawSiPM (
    const Vector & RadiatorPosition) const
```

Draw [SiPM](#)

Parameters

<i>RadiatorPosition</i>	The position of the radiator cell
-------------------------	-----------------------------------

Definition at line 115 of file [SiPM.cpp](#).

6.29.3.3 GetDetectorSizeX()

```
double SiPM::GetDetectorSizeX () const
```

Get the detector X size

Definition at line 201 of file [SiPM.cpp](#).

6.29.3.4 GetDetectorSizeY()

```
double SiPM::GetDetectorSizeY () const
```

Get the detector Y size

Definition at line 205 of file [SiPM.cpp](#).

6.29.3.5 GetDetectorTilt()

```
double SiPM::GetDetectorTilt () const
```

Get the detector tilt, in radiations

Definition at line 197 of file [SiPM.cpp](#).

6.29.3.6 GetDetectorXPosition()

```
double SiPM::GetDetectorXPosition () const
```

Get the detector X position

Definition at line 189 of file [SiPM.cpp](#).

6.29.3.7 GetDetectorZPosition()

```
double SiPM::GetDetectorZPosition () const
```

Get the detector Z position

Definition at line 193 of file [SiPM.cpp](#).

6.29.3.8 GetPhotonHits()

```
const std::vector< PhotonHit > & SiPM::GetPhotonHits () const
```

Get photon hits in [SiPM](#)

6.29.3.9 IsDetectorHit()

```
bool SiPM::IsDetectorHit (
    const Photon & photon) const
```

Check if photon hit the detector

Definition at line 146 of file [SiPM.cpp](#).

6.29.3.10 PlotHits()

```
void SiPM::PlotHits (
    const std::string & Filename,
    const std::vector< PhotonHit > & photonHits) const
```

Plot photon hits

Definition at line 67 of file [SiPM.cpp](#).

6.29.3.11 SetDetectorPosition()

```
void SiPM::SetDetectorPosition (
    double x)
```

Set the detector X position

Definition at line 167 of file [SiPM.cpp](#).

6.29.3.12 SetDetectorTilt()

```
void SiPM::SetDetectorTilt (
    double Angle)
```

Set the detector tilt, in radians

Definition at line 171 of file [SiPM.cpp](#).

The documentation for this class was generated from the following files:

- [include/SiPM.h](#)
- [src/SiPM.cpp](#)

6.30 TrackingVolume Class Reference

```
#include <TrackingVolume.h>
```

Public Member Functions

- [TrackingVolume](#) ()
- double [GetRadius](#) () const
- double [GetFieldStrength](#) () const
- std::vector< std::pair< std::unique_ptr< TObject >, std::string > > [DrawARCGeometry](#) () const

6.30.1 Detailed Description

The [TrackingVolume](#) is the inner detector that contains the magnetic field and the interaction point

Definition at line 15 of file [TrackingVolume.h](#).

6.30.2 Constructor & Destructor Documentation

6.30.2.1 TrackingVolume()

```
TrackingVolume::TrackingVolume ()
```

Constructor that sets up the geometry of the tracking volume

Definition at line 12 of file [TrackingVolume.cpp](#).

6.30.3 Member Function Documentation

6.30.3.1 DrawARCGeometry()

```
std::vector< std::pair< std::unique_ptr< TObject >, std::string > > TrackingVolume::DrawARCGeometry () const
```

Draw ARC geometry

Definition at line 28 of file [TrackingVolume.cpp](#).

6.30.3.2 GetFieldStrength()

```
double TrackingVolume::GetFieldStrength () const
```

Get magnetic field strength

Definition at line 23 of file [TrackingVolume.cpp](#).

6.30.3.3 GetRadius()

```
double TrackingVolume::GetRadius () const
```

Get inner tracking radius

Definition at line 19 of file [TrackingVolume.cpp](#).

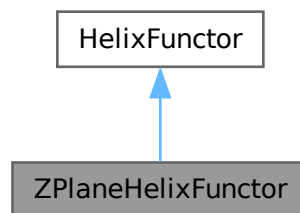
The documentation for this class was generated from the following files:

- [include/TrackingVolume.h](#)
- [src/TrackingVolume.cpp](#)

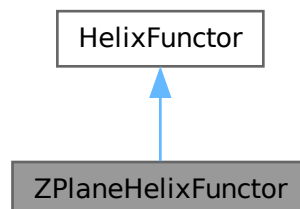
6.31 ZPlaneHelixFunctor Class Reference

```
#include <HelixFunctor.h>
```

Inheritance diagram for ZPlaneHelixFunctor:



Collaboration diagram for ZPlaneHelixFunctor:



Public Member Functions

- [ZPlaneHelixFunctor](#) (double z, const [Vector](#) &CellPosition, bool Barrel)
- [ZPlaneHelixFunctor](#) (const [ZPlaneHelixFunctor](#) &Functor)=delete
- virtual double [operator\(\)](#) (const [Vector](#) &Vect) const override

Public Member Functions inherited from [HelixFunctor](#)

- [HelixFunctor](#) ()=default

6.31.1 Detailed Description

[ZPlaneHelixFunctor](#) is a functor for tracking the particle through various the different layers of a barrel cell

Definition at line 84 of file [HelixFunctor.h](#).

6.31.2 Constructor & Destructor Documentation

6.31.2.1 ZPlaneHelixFunctor() [1/2]

```
ZPlaneHelixFunctor::ZPlaneHelixFunctor (
    double z,
    const Vector & CellPosition,
    bool Barrel)
```

Constructor that takes in the position of the cell and the z-coordinate of the local coordinate that we want to track particle to

Parameters

<i>z</i>	The z-coordinate
<i>CellPosition</i>	The position of the radiator cell
<i>Barrel</i>	Flag that is true for barrel cells, false for end cap cells

Definition at line 20 of file [HelixFunctor.cpp](#).

6.31.2.2 ZPlaneHelixFunctor() [2/2]

```
ZPlaneHelixFunctor::ZPlaneHelixFunctor (
    const ZPlaneHelixFunctor & Functor) [delete]
```

We do not need copy constructor

6.31.3 Member Function Documentation

6.31.3.1 operator()

```
double ZPlaneHelixFunctor::operator() (
    const Vector & Vect) const [override], [virtual]
```

Virtual () operator

Parameters

<i>Vect</i>	Vector position
-------------	---------------------------------

Implements [HelixFunctor](#).

Definition at line 28 of file [HelixFunctor.cpp](#).

The documentation for this class was generated from the following files:

- include/[HelixFunctor.h](#)
- src/[HelixFunctor.cpp](#)

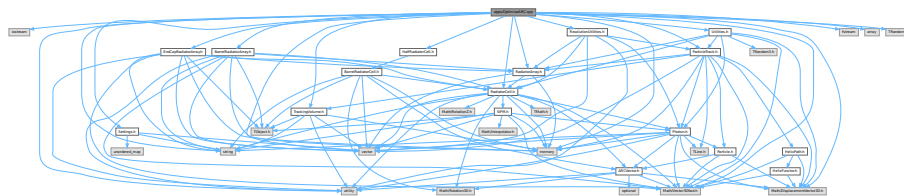
Chapter 7

File Documentation

7.1 apps/OptimiseARC.cpp File Reference

```
#include <iostream>
#include <string>
#include <utility>
#include <vector>
#include <fstream>
#include <array>
#include <memory>
#include "TRandom.h"
#include "Math/Vector3Dfwd.h"
#include "Math/DisplacementVector3D.h"
#include "Settings.h"
#include "RadiatorArray.h"
#include "BarrelRadiatorArray.h"
#include "EndCapRadiatorArray.h"
#include "RadiatorCell.h"
#include "HalfRadiatorCell.h"
#include "TrackingVolume.h"
#include "ParticleTrack.h"
#include "ResolutionUtilities.h"
#include "Utilities.h"
```

Include dependency graph for OptimiseARC.cpp:



Typedefs

- using [Tracks](#) = std::vector<[ParticleTrack](#)>

Functions

- `int main (int argc, char *argv[])`

7.1.1 Typedef Documentation

7.1.1.1 Tracks

using `Tracks` = `std::vector<ParticleTrack>`

Definition at line 30 of file [OptimiseARC.cpp](#).

7.1.2 Function Documentation

7.1.2.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 32 of file [OptimiseARC.cpp](#).

7.2 OptimiseARC.cpp

[Go to the documentation of this file.](#)

```
00001 // Martin Duy Tat 1st May 2022
00007
00008 #include<iostream>
00009 #include<string>
00010 #include<utility>
00011 #include<vector>
00012 #include<fstream>
00013 #include<array>
00014 #include<memory>
00015 #include"TRandom.h"
00016 #include"Math/Vector3Dfwd.h"
00017 #include"Math/DisplacementVector3D.h"
00018 #include"Settings.h"
00019 #include"RadiatorArray.h"
00020 #include"BarrelRadiatorArray.h"
00021 #include"EndCapRadiatorArray.h"
00022 #include"RadiatorCell.h"
00023 #include"HalfRadiatorCell.h"
00024 #include"TrackingVolume.h"
00025 #include"ParticleTrack.h"
00026 #include"ResolutionUtilities.h"
00027 #include"Utilities.h"
00028
00029 using Vector = ROOT::Math::XYZVector;
00030 using Tracks = std::vector<ParticleTrack>;
00031
00032 int main(int argc, char *argv[]) {
00033     if(argc%2 != 1 || argc < 3) {
00034         return 0;
00035     }
00036     std::cout << "Welcome to the ARC optimiser\n";
00037     for(int i = 3; i < argc; i += 2) {
00038         const std::string SettingsName = argv[i];
00039         const std::string SettingsFilename = argv[i + 1];
00040         Settings::AddSettings(SettingsName, SettingsFilename);
00041         std::cout << "Added " << SettingsName << " settings from ";
00042         std::cout << SettingsFilename << "\n";
00043     }
00044 }
```

```

00044     const std::string RunMode(argv[1]);
00045     const std::size_t Seed = Settings::GetSizeT("General/Seed");
00046     gRandom->SetSeed(Seed);
00047     Utilities::Random().SetSeed(Seed);
00048     std::cout << "Generating tracks...\n";
00049     const int ParticleID = Settings::GetInt("Particle/ID");
00050     const TrackingVolume InnerTracker;
00051     const std::size_t Column =
00052         static_cast<std::size_t>(std::stoi(std::string(argv[1])));
00053     const std::size_t Row =
00054         static_cast<std::size_t>(std::stoi(std::string(argv[2])));
00055     std::cout << "Tracks ready\n";
00056     std::unique_ptr<RadiatorArray> radiatorArray;
00057     const std::string BarrelOrEndcap =
00058         Settings::GetString("General/BarrelOrEndcap");
00059     if(BarrelOrEndcap == "Barrel") {
00060         radiatorArray = std::make_unique<BarrelRadiatorArray>();
00061     } else if(BarrelOrEndcap == "EndCap") {
00062         radiatorArray = std::make_unique<EndCapRadiatorArray>();
00063     } else {
00064         return 0;
00065     }
00066     RadiatorCell *radiatorCell = radiatorArray->GetRadiatorCell(Column, Row);
00067     if(!radiatorCell) {
00068         if(BarrelOrEndcap == "Barrel") {
00069             std::cout << "Barrel ";
00070         } else {
00071             std::cout << "End cap ";
00072         }
00073         std::cout << "radiator cell (" << Column << ", " << Row << ")";
00074         std::cout << " doesn't exist\n";
00075         return 0;
00076     }
00077     const double z_min = Settings::GetDouble("Particle/z_min");
00078     const double z_max = Settings::GetDouble("Particle/z_max");
00079     Tracks Particles;
00080     const std::size_t TotalNumberTracks =
00081         Settings::GetSizeT("General/NumberTracks");
00082     Particles.reserve(TotalNumberTracks);
00083     std::size_t NumberTracks = 0;
00084     while(NumberTracks < TotalNumberTracks) {
00085         auto GetMomentum = [=] () {
00086             if(BarrelOrEndcap == "Barrel") {
00087                 return Utilities::GenerateRandomBarrelTrackZRange(z_min, z_max);
00088             } else if(BarrelOrEndcap == "EndCap") {
00089                 return Utilities::GenerateRandomEndCapTrack();
00090             } else {
00091                 return Vector(0.0, 0.0, 0.0);
00092             }
00093         };
00094         const Vector Momentum = GetMomentum();
00095         ParticleTrack particleTrack(ParticleID, Momentum,
00096             NumberTracks, InnerTracker.GetFieldStrength());
00097         if(!particleTrack.TrackThroughTracker(InnerTracker)) {
00098             // Track didn't reach the radiator array, generate another one
00099             continue;
00100         }
00101         Particles.push_back(particleTrack);
00102         NumberTracks++;
00103     }
00104     if(Settings::GetBool("Optimisation/SinglePoints")) {
00105         while(true) {
00106             std::cout << "Input parameters:\n";
00107             std::array<double, 5> x;
00108             for(double &xx : x) {
00109                 std::cin >> xx;
00110             }
00111             double Value = ResolutionUtilities::fcx(x[0], x[1], x[2], x[3], x[4],
00112                 *radiatorCell, *radiatorArray,
00113                 Particles, Seed, false);
00114             std::cout << "fcx = " << Value << "\n";
00115             std::cout << "Calculate new value?(y/n)\n";
00116             std::string Answer;
00117             std::cin >> Answer;
00118             if(Answer != "y") {
00119                 return 0;
00120             }
00121         }
00122     }
00123     if(Settings::GetBool("Optimisation/DoFit")) {
00124         std::cout << "Differential evolution ready, sending off agents...\n";
00125         ResolutionUtilities::DoFit(*radiatorCell, *radiatorArray,
00126             Particles, Column, Row);
00127         std::cout << "ARC is optimised!\n";
00128     }
00129     if(Settings::GetBool("Optimisation/PlotProjections")) {
00130         std::cout << "Plotting...\n";

```

```

00131     ResolutionUtilities::PlotProjections(*radiatorCell,
00132                                         *radiatorArray,
00133                                         Particles);
00134     std::cout << "Resolution projections plotted\n";
00135 }
00136 return 0;
00137 }

```

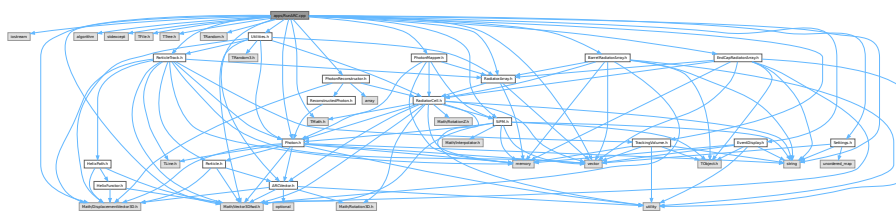
7.3 apps/RunARC.cpp File Reference

```

#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
#include <stdexcept>
#include "TFile.h"
#include "TTree.h"
#include "TMath.h"
#include "TRandom.h"
#include "Math/Vector3Dfwd.h"
#include "Math/DisplacementVector3D.h"
#include "ParticleTrack.h"
#include "TrackingVolume.h"
#include "Photon.h"
#include "PhotonMapper.h"
#include "PhotonReconstructor.h"
#include "RadiatorArray.h"
#include "Settings.h"
#include "EventDisplay.h"
#include "SiPM.h"
#include "BarrelRadiatorArray.h"
#include "EndCapRadiatorArray.h"
#include "Utilities.h"
#include "ARCVector.h"

```

Include dependency graph for RunARC.cpp:



Classes

- struct [CherenkovFile](#)

Functions

- int [main](#) (int argc, char *argv[])

7.3.1 Function Documentation

7.3.1.1 main()

```
int main (
    int argc,
    char * argv[])
```

Definition at line 142 of file [RunARC.cpp](#).

7.4 RunARC.cpp

[Go to the documentation of this file.](#)

```
00001 // Martin Duy Tat 1st May 2022
00010
00011 #include<iostream>
00012 #include<string>
00013 #include<vector>
00014 #include<algorithm>
00015 #include<stdexcept>
00016 #include"TFile.h"
00017 #include"TTree.h"
00018 #include"TMATH.h"
00019 #include"TRandom.h"
00020 #include"Math/Vector3Dfwd.h"
00021 #include"Math/DisplacementVector3D.h"
00022 #include"ParticleTrack.h"
00023 #include"TrackingVolume.h"
00024 #include"Photon.h"
00025 #include"PhotonMapper.h"
00026 #include"PhotonReconstructor.h"
00027 #include"RadiatorArray.h"
00028 #include"Settings.h"
00029 #include"EventDisplay.h"
00030 #include"SiPM.h"
00031 #include"BarrelRadiatorArray.h"
00032 #include"EndCapRadiatorArray.h"
00033 #include"Utilities.h"
00034 #include"ARCVector.h"
00035
00036 using Vector = ROOT::Math::XYZVector;
00037
00038 struct CherenkovFile {
00042     TFile File;
00046     TTree CherenkovTree;
00050     static constexpr int MaxPhotons = 2000;
00051     double CherenkovAngle_Reco_TrueEmissionPoint[MaxPhotons], CherenkovAngle_Reco[MaxPhotons],
00052           CherenkovAngle_True[MaxPhotons], PhotonEnergy[MaxPhotons];
00053     double Entrance_x, Entrance_y, Entrance_z;
00054     double MirrorHit_x, MirrorHit_y, MirrorHit_z;
00055     double PhotonHit_x[MaxPhotons], PhotonHit_y[MaxPhotons], PhotonHit_z[MaxPhotons];
00056     double Momentum, CosTheta, Phi;
00057     int NumberPhotons = 0;
00058     double NumberGoodPhotons = 0.0;
00059     int HasMigrated[MaxPhotons];
00060     std::size_t RadiatorRowNumber, RadiatorColumnNumber, TrackNumber;
00061     std::size_t FinalRadiatorRowNumber, FinalRadiatorColumnNumber;
00062     ParticleTrack::Location ParticleLocation;
00063     Photon::Status PhotonStatus[MaxPhotons];
00064     double SinglePhotonResolution, TotalResolution, Significance;
00068     CherenkovFile(const std::string &Filename):
00069         File(Filename.c_str(), "RECREATE"),
00070         CherenkovTree("CherenkovTree", "") {
00071         CherenkovTree.Branch("NumberPhotons", &NumberPhotons);
00072         CherenkovTree.Branch("NumberGoodPhotons", &NumberGoodPhotons);
00073         CherenkovTree.Branch("CherenkovAngle_Reco_TrueEmissionPoint",
00074                               &CherenkovAngle_Reco_TrueEmissionPoint,
00075                               "CherenkovAngle_Reco_TrueEmissionPoint[NumberPhotons]/D");
00076         CherenkovTree.Branch("CherenkovAngle_Reco",
00077                               &CherenkovAngle_Reco,
00078                               "CherenkovAngle_Reco[NumberPhotons]/D");
00079         CherenkovTree.Branch("CherenkovAngle_True",
00080                               &CherenkovAngle_True,
00081                               "CherenkovAngle_True[NumberPhotons]/D");
00082         CherenkovTree.Branch("PhotonEnergy",
```

```

00083         &PhotonEnergy,
00084         "PhotonEnergy[NumberPhotons]/D");
00085     CherenkovTree.Branch("FinalRadiatorColumnNumber",
00086         &FinalRadiatorColumnNumber,
00087         "FinalRadiatorColumnNumber/1");
00088     CherenkovTree.Branch("FinalRadiatorRowNumber",
00089         &FinalRadiatorRowNumber,
00090         "FinalRadiatorRowNumber/1");
00091     CherenkovTree.Branch("RadiatorRowNumber",
00092         &RadiatorRowNumber,
00093         "RadiatorRowNumber/1");
00094     CherenkovTree.Branch("RadiatorColumnNumber",
00095         &RadiatorColumnNumber,
00096         "RadiatorColumnNumber/1");
00097     CherenkovTree.Branch("TrackNumber",
00098         &TrackNumber,
00099         "TrackNumber/1");
00100     CherenkovTree.Branch("HasMigrated",
00101         &HasMigrated,
00102         "HasMigrated[NumberPhotons]/I");
00103     CherenkovTree.Branch("ParticleLocation", &ParticleLocation, "ParticleLocation/I");
00104     CherenkovTree.Branch("PhotonStatus", &PhotonStatus, "PhotonStatus[NumberPhotons]/I");
00105     CherenkovTree.Branch("Entrance_x", &Entrance_x);
00106     CherenkovTree.Branch("Entrance_y", &Entrance_y);
00107     CherenkovTree.Branch("Entrance_z", &Entrance_z);
00108     CherenkovTree.Branch("MirrorHit_x", &MirrorHit_x);
00109     CherenkovTree.Branch("MirrorHit_y", &MirrorHit_y);
00110     CherenkovTree.Branch("MirrorHit_z", &MirrorHit_z);
00111     CherenkovTree.Branch("PhotonHit_x",
00112         &PhotonHit_x,
00113         "PhotonHit_x[NumberPhotons]/D");
00114     CherenkovTree.Branch("PhotonHit_y",
00115         &PhotonHit_y,
00116         "PhotonHit_y[NumberPhotons]/D");
00117     CherenkovTree.Branch("PhotonHit_z",
00118         &PhotonHit_z,
00119         "PhotonHit_z[NumberPhotons]/D");
00120     CherenkovTree.Branch("Momentum", &Momentum);
00121     CherenkovTree.Branch("CosTheta", &CosTheta);
00122     CherenkovTree.Branch("Phi", &Phi);
00123     CherenkovTree.Branch("SinglePhotonResolution", &SinglePhotonResolution);
00124     CherenkovTree.Branch("TotalResolution", &TotalResolution);
00125     CherenkovTree.Branch("Significance", &Significance);
00126 }
00130 void Fill() {
00131     CherenkovTree.Fill();
00132 }
00136 void Close() {
00137     CherenkovTree.Write();
00138     File.Close();
00139 }
00140 };
00141
00142 int main(int argc, char *argv[]) {
00143     if(argc%2 != 0) {
00144         return 0;
00145     }
00146     std::cout << "Welcome to the ARC simulation and reconstruction\n";
00147     for(int i = 2; i < argc; i += 2) {
00148         const std::string SettingsName = argv[i];
00149         const std::string SettingsFilename = argv[i + 1];
00150         Settings::AddSettings(SettingsName, SettingsFilename);
00151         std::cout << "Added " << SettingsName << " settings from " << SettingsFilename << "\n";
00152     }
00153     const std::string RunMode(argv[1]);
00154     if(Settings::Exists("General/Seed")) {
00155         gRandom->SetSeed(Settings::GetSizeT("General/Seed"));
00156         Utilities::Random().SetSeed(Settings::GetSizeT("General/Seed"));
00157     }
00158     EventDisplay eventDisplay;
00159     const TrackingVolume InnerTracker;
00160     eventDisplay.AddObject(InnerTracker.DrawARCGeometry());
00161     std::unique_ptr<RadiatorArray> radiatorArray;
00162     const std::string BarrelOrEndcap = Settings::GetString("General/BarrelOrEndcap");
00163     if(BarrelOrEndcap == "Barrel") {
00164         radiatorArray = std::make_unique<BarrelRadiatorArray>();
00165     } else if(BarrelOrEndcap == "EndCap") {
00166         radiatorArray = std::make_unique<EndCapRadiatorArray>();
00167     } else {
00168         return 0;
00169     }
00170     eventDisplay.AddObject(radiatorArray->DrawRadiatorArray());
00171     if(RunMode == "SingleTrack") {
00172         std::cout << "Run mode: Single track\n";
00173         const Vector Momentum = Utilities::VectorFromSpherical(
00174             Settings::GetDouble("Particle/Momentum"),
00175             Settings::GetDouble("Particle/CosTheta"),

```

```

00176     Settings::GetDouble("Particle/Phi"));
00177     const int ParticleID = Settings::GetInt("Particle/ID");
00178     ParticleTrack particleTrack(ParticleID, Momentum,
00179     0, InnerTracker.GetFieldStrength());
00180     if(!particleTrack.TrackThroughTracker(InnerTracker)) {
00181         std::cout << "Could not track through tracker...\n";
00182         return 0;
00183     }
00184     if(!particleTrack.FindRadiator(*radiatorArray)) {
00185         std::cout << "Track does not enter any radiator cell...\n";
00186         return 0;
00187     }
00188     if(!particleTrack.TrackThroughAerogel()) {
00189         std::cout << "Could not track through aerogel...\n";
00190         return 0;
00191     }
00192     // Aerogel photons are generated before the track continues to the mirror
00193     auto PhotonsAerogel = particleTrack.GeneratePhotonsFromAerogel();
00194     particleTrack.TrackThroughGasToMirror();
00195     if(particleTrack.GetParticleLocation() != ParticleTrack::Location::Mirror &&
00196     !particleTrack.TrackToNextCell(*radiatorArray)) {
00197         std::cout << "Track left the radiator array before reaching a mirror...\n";
00198         return 0;
00199     }
00200     auto PhotonsGas =
00201     particleTrack.GetParticleLocation() == ParticleTrack::Location::Mirror
00202     ? particleTrack.GeneratePhotonsFromGas() : std::vector<Photon>();
00203     std::vector<PhotonHit> photonHits;
00204     for(auto &photon : PhotonsAerogel) {
00205         auto photonHit = PhotonMapper::TracePhoton(photon, *radiatorArray);
00206         if(photonHit) {
00207             photonHits.push_back(*photonHit);
00208         }
00209     }
00210     for(auto &photon : PhotonsGas) {
00211         auto photonHit = PhotonMapper::TracePhoton(photon, *radiatorArray);
00212         if(photonHit) {
00213             photonHits.push_back(*photonHit);
00214         }
00215     }
00216     particleTrack.GetRadiatorCell()->GetDetector().PlotHits("PhotonHits.pdf", photonHits);
00217 } else if(RunMode == "CherenkovAngleResolution") {
00218     std::cout << "Run mode: Cherenkov angle resolution\n";
00219     CherenkovFile File("CherenkovFile.root");
00220     const std::size_t NumberTracks = Settings::GetSizeT("General/NumberTracks");
00221     const bool Aerogel = Settings::GetString("General/GasOrAerogel") == "Aerogel";
00222     const Photon::Radiator Radiator = Aerogel ?
00223     Photon::Radiator::Aerogel :
00224     Photon::Radiator::Gas;
00225     const int Hypothesis1 = Settings::GetInt("General/MassHypothesis1");
00226     const int Hypothesis2 = Settings::GetInt("General/MassHypothesis2");
00227     for(std::size_t i = 0; i < NumberTracks; i++) {
00228         if(i%(NumberTracks/10) == 0) {
00229             std::cout << 100*i/NumberTracks;
00230             std::cout << "%\n";
00231         }
00232         File.NumberPhotons = 0;
00233         File.NumberGoodPhotons = 0.0;
00234         File.TrackNumber = i;
00235         File.RadiatorRowNumber = static_cast<std::size_t>(-1);
00236         File.RadiatorColumnNumber = static_cast<std::size_t>(-1);
00237         auto GetMomentum = [&] () {
00238             if(BarrelOrEndcap == "Barrel") {
00239                 return Utilities::GenerateRandomBarrelTrack(File.CosTheta, File.Phi);
00240             } else if(BarrelOrEndcap == "EndCap") {
00241                 return Utilities::GenerateRandomEndCapTrack();
00242             } else {
00243                 return Vector(0.0, 0.0, 0.0);
00244             }
00245         };
00246         const Vector Momentum = GetMomentum();
00247         File.Momentum = TMath::Sqrt(Momentum.Mag2());
00248         if(BarrelOrEndcap == "EndCap") {
00249             File.CosTheta = TMath::Cos(Momentum.Theta());
00250         }
00251         const Vector Position(0.0, 0.0, 0.0);
00252         const int ParticleID = Settings::GetInt("Particle/ID");
00253         ParticleTrack particleTrack(ParticleID, Momentum,
00254         1, InnerTracker.GetFieldStrength());
00255         const double MomentumMag = TMath::Sqrt(Momentum.Mag2());
00256         const double CherenkovAngleDifference =
00257         Utilities::GetCherenkovAngleDifference(MomentumMag,
00258         Hypothesis1,
00259         Hypothesis2,
00260         Radiator);
00261         if(!particleTrack.TrackThroughTracker(InnerTracker)) {
00262             continue;

```

```

00263     }
00264     if(!particleTrack.FindRadiator(*radiatorArray)) {
00265     continue;
00266     }
00267     File.Phi = particleTrack.GetPosition().GlobalVector().Phi();
00268     File.RadiatorRowNumber = particleTrack.GetRadiatorRowNumber();
00269     File.RadiatorColumnNumber = particleTrack.GetRadiatorColumnNumber();
00270     auto EntranceWindowPosition = particleTrack.GetEntranceWindowPosition();
00271     File.Entrance_x = EntranceWindowPosition.X();
00272     File.Entrance_y = EntranceWindowPosition.Y();
00273     File.Entrance_z = EntranceWindowPosition.Z();
00274     if(particleTrack.GetParticleLocation() != ParticleTrack::Location::EntranceWindow) {
00275     File.ParticleLocation = particleTrack.GetParticleLocation();
00276     File.Fill();
00277     continue;
00278     }
00279     if(!particleTrack.TrackThroughAerogel()) {
00280     continue;
00281     }
00282     std::vector<Photon> Photons;
00283     if(Aerogel) {
00284     Photons = particleTrack.GeneratePhotonsFromAerogel();
00285     }
00286     particleTrack.TrackThroughGasToMirror();
00287     if(particleTrack.GetParticleLocation() != ParticleTrack::Location::Mirror) {
00288     const bool HitMirror = particleTrack.TrackToNextCell(*radiatorArray);
00289     if(!HitMirror) {
00290     File.ParticleLocation = particleTrack.GetParticleLocation();
00291     File.FinalRadiatorRowNumber = static_cast<std::size_t>(-1);
00292     File.FinalRadiatorColumnNumber = static_cast<std::size_t>(-1);
00293     File.Fill();
00294     continue;
00295     }
00296     }
00297     if(!Aerogel) {
00298     Photons = particleTrack.GeneratePhotonsFromGas();
00299     }
00300     File.ParticleLocation = particleTrack.GetParticleLocation();
00301     File.Phi = particleTrack.GetPosition().GlobalVector().Phi();
00302     auto MirrorHitPosition = particleTrack.GetPosition().GlobalVector();
00303     File.MirrorHit_x = MirrorHitPosition.X();
00304     File.MirrorHit_y = MirrorHitPosition.Y();
00305     File.MirrorHit_z = MirrorHitPosition.Z();
00306     File.FinalRadiatorRowNumber = particleTrack.GetRadiatorRowNumber();
00307     File.FinalRadiatorColumnNumber = particleTrack.GetRadiatorColumnNumber();
00308     eventDisplay.AddObject(particleTrack.DrawParticleTrack());
00309     std::vector<double> GoodAngles;
00310     if(Photons.size() > static_cast<std::size_t>(CherenkovFile::MaxPhotons)) {
00311     std::cout << "Warning! Track " << i << " has " << Photons.size()
00312     << " photons, only the first " << CherenkovFile::MaxPhotons
00313     << " are stored\n";
00314     }
00315     for(auto &Photon : Photons) {
00316     if(File.NumberPhotons >= CherenkovFile::MaxPhotons) {
00317     break;
00318     }
00319     File.CherenkovAngle_True[File.NumberPhotons] =
00320     TMath::ACos(Photon.GetCosCherenkovAngle());
00321     auto photonHit = PhotonMapper::TracePhoton(Photon, *radiatorArray);
00322     eventDisplay.AddObject(Photon.DrawPhotonPath());
00323     if(!Photon.GetMirrorHitPosition()) {
00324     File.CherenkovAngle_Reco_TrueEmissionPoint[File.NumberPhotons] = -1.0;
00325     File.CherenkovAngle_Reco[File.NumberPhotons] = -1.0;
00326     } else {
00327     auto reconstructedPhoton =
00328     PhotonReconstructor::ReconstructPhoton(*photonHit);
00329     // Failed reconstructions are stored as -1.0, like photons without a mirror hit
00330     auto GetAngle = [] (double CosAngle) {
00331     return PhotonReconstructor::IsReconstructed(CosAngle) ? TMath::ACos(CosAngle) : -1.0;
00332     };
00333     File.CherenkovAngle_Reco_TrueEmissionPoint[File.NumberPhotons] =
00334     GetAngle(reconstructedPhoton.m_CosCherenkovAngle_TrueEmissionPoint);
00335     File.CherenkovAngle_Reco[File.NumberPhotons] =
00336     GetAngle(reconstructedPhoton.m_CosCherenkovAngle);
00337     File.HasMigrated[File.NumberPhotons] = Photon.HasPhotonMigrated() ? 1 : 0;
00338     if(Photon.GetStatus() == Photon::Status::DetectorHit &&
00339     File.CherenkovAngle_Reco[File.NumberPhotons] >= 0.0) {
00340     File.NumberGoodPhotons += Photon.GetWeight();
00341     GoodAngles.push_back(File.CherenkovAngle_Reco[File.NumberPhotons]);
00342     }
00343     }
00344     File.PhotonEnergy[File.NumberPhotons] = Photon.GetEnergy();
00345     File.PhotonStatus[File.NumberPhotons] = Photon.GetStatus();
00346     const auto PhotonPosition = Photon.GetPosition().GlobalVector();
00347     File.PhotonHit_x[File.NumberPhotons] = PhotonPosition.X();
00348     File.PhotonHit_y[File.NumberPhotons] = PhotonPosition.Y();
00349     File.PhotonHit_z[File.NumberPhotons] = PhotonPosition.Z();

```

```

00350     File.NumberPhotons++;
00351 }
00352     const double RMS = TMath::RMS(GoodAngles.begin(), GoodAngles.end());
00353     File.SinglePhotonResolution = RMS;
00354     File.TotalResolution = RMS/TMath::Sqrt(File.NumberGoodPhotons);
00355     File.Significance = CherenkovAngleDifference/File.TotalResolution;
00356     File.Fill();
00357 }
00358     eventDisplay.DrawEventDisplay();
00359     File.Close();
00360 }
00361     return 0;
00362 }

```

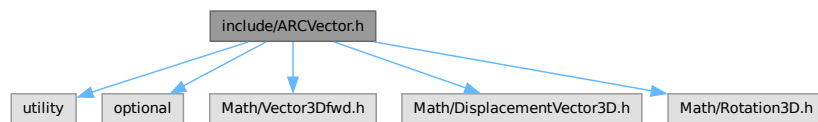
7.5 include/ARCVector.h File Reference

```

#include <utility>
#include <optional>
#include "Math/Vector3Dfwd.h"
#include "Math/DisplacementVector3D.h"
#include "Math/Rotation3D.h"

```

Include dependency graph for ARCVector.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [ARCVector](#)

Typedefs

- using [Vector](#) = ROOT::Math::XYZVector
- using [Rotation3D](#) = ROOT::Math::Rotation3D

7.5.1 Typedef Documentation

7.5.1.1 Rotation3D

```
using Rotation3D = ROOT::Math::Rotation3D
```

Definition at line 20 of file [ARCVector.h](#).

7.5.1.2 Vector

```
using Vector = ROOT::Math::XYZVector
```

[ARCVector](#) is a vector that can optionally hold a local coordinate system. The local coordinate system is displaced and rotated from the global one. [Vector](#) operations are performed on the global coordinate system if no optional coordinate system is held, otherwise vector operations are relative to the local coordinate system.

[BarrelRadiatorArray](#) contains the ARC barrel geometry of radiators.

[EndCapRadiatorArray](#) contains the ARC end cap geometry of radiators.

[HelixFunctor](#) is an abstract class for functors to solve the particle position.

[HelixPath](#) is a class that stores the helical trajectory of a charged particle in a magnetic field.

[Particle](#) is an abstract class that stores the position and radiator cell that the particle is in.

[ParticleTrack](#) represents a charged particle with position, direction and momentum inside the detector volume.

A [Photon](#) has a position, direction and energy.

PhotonReconstructor is a namespace with all the tools for reconstructing photons by solving a quartic equation.

[RadiatorCell](#) describes the geometry of a single radiator cell of the ARC. The radiator cell has a local coordinate system with its origin in the middle of the detector plane. All lengths are in meter.

[SiPM](#) represents a single Silicon Photo-Multiplier, which is located on the detector plane of each radiator cell. All positions are in the local radiator coordinate system.

Various utility functions.

OptimiseARC is an application for optimising an ARC cell. Run with `RunARC ColumnNumber RowNumber <settings name> <settings filename> ...`. There can be an arbitrary number of settings files added.

RunARC is an application for running ARC simulations and reconstructions. Run with `RunARC <run mode> <settings name> <settings filename> ...`. There can be an arbitrary number of settings files added. Possible run modes: "SingleTrack" Send a single charged track and generate a photon hit map. "CherenkovAngleResolution" Generate photons from a single track, reconstruct the photons and study the Cherenkov angle resolution.

Definition at line 19 of file [ARCVector.h](#).

7.6 ARCVector.h

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 16th October 2022
00009
00010 #ifndef ARCVECTOR
00011 #define ARCVECTOR
00012
00013 #include<utility>
00014 #include<optional>
00015 #include"Math/Vector3Dfwd.h"
00016 #include"Math/DisplacementVector3D.h"
00017 #include"Math/Rotation3D.h"
00018
00019 using Vector = ROOT::Math::XYZVector;
00020 using Rotation3D = ROOT::Math::Rotation3D;
00021
00022 class ARCVector {
00023 public:
00027   ARCVector() = default;
00032   explicit ARCVector(const Vector &Vec);
00039   ARCVector(const Vector &Vec,
00040             const Vector &RadiatorPosition,
00041             const Rotation3D &RadiatorRotation);
00045   ARCVector(const ARCVector &Vec) = default;
00051   void AssignLocalCoordinates(const Vector &Position, const Rotation3D &Rotation);
00055   bool IsLocal() const;
00059   void ConvertToGlobal();
00063   const Vector& LocalVector() const;
00067   Vector GlobalVector() const;
00071   ARCVector& operator +=(const Vector &Vec);
00075   void MapPhi(double DeltaPhi);
00079   void ReflectZ();
00083   void ReflectY();
00087   void SetX(double x);
00091   void SetZ(double z);
00095   void SetGlobalVector(const Vector &Vec);
00096 private:
00102   Vector m_Vector;
00106   std::optional<std::pair<Vector, Rotation3D>> m_CoordinateTransform;
00111   void GlobalToLocal(Vector &Vec) const;
00116   void LocalToGlobal(Vector &Vec) const;
00117
00118 };
00119
00120 #endif

```

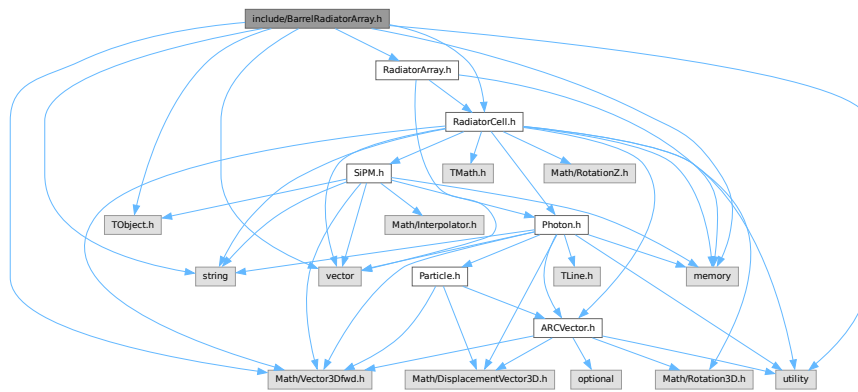
7.7 include/BarrelRadiatorArray.h File Reference

```

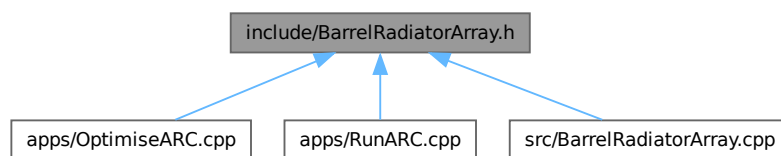
#include <vector>
#include <memory>
#include <utility>
#include <string>
#include "TObject.h"
#include "Math/Vector3Dfwd.h"
#include "RadiatorCell.h"
#include "RadiatorArray.h"

```

Include dependency graph for BarrelRadiatorArray.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [BarrelRadiatorArray](#)

7.8 BarrelRadiatorArray.h

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 23rd June 2022
00005
00006 #ifndef BARRELRADIATORARRAY
00007 #define BARRELRADIATORARRAY
00008
00009 #include<vector>
00010 #include<memory>
00011 #include<utility>
00012 #include<string>
00013 #include"TOObject.h"
00014 #include"Math/Vector3Dfwd.h"
00015 #include"RadiatorCell.h"
00016 #include"RadiatorArray.h"
00017
00018 using Vector = ROOT::Math::XYZVector;
00019
00020 class Particle;
00021
00022 class BarrelRadiatorArray: public RadiatorArray {
00023 public:
00027   BarrelRadiatorArray();
00031   BarrelRadiatorArray(const BarrelRadiatorArray &radiatorCell) = delete;
  
```

```

00035     std::vector<std::pair<std::unique_ptr<TObject>, std::string>>
00036     DrawRadiatorArray() const override;
00041     virtual const RadiatorCell* FindRadiator(Particle &particle) const override;
00047     bool IsBelowMainRow(double x, double y) const;
00054     bool IsAboveUpperRow(double x, double y, double Radius) const;
00060     bool IsAboveMainRow(double x, double y) const;
00061 private:
00065     const bool m_FullArray;
00069     const double m_BarrelRadius;
00073     const double m_DeltaPhi;
00077     virtual int FindRadiatorIndex(std::size_t i, std::size_t j) const override;
00078 };
00079
00080 #endif

```

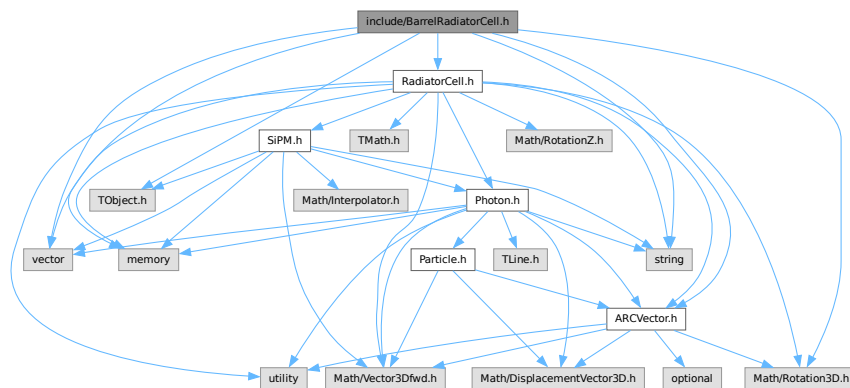
7.9 include/BarrelRadiatorCell.h File Reference

```

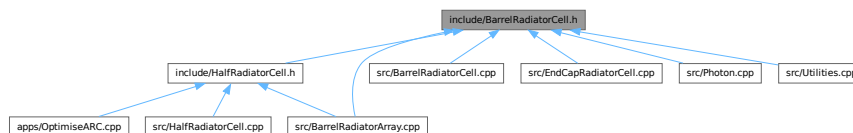
#include <vector>
#include <memory>
#include <string>
#include "TObject.h"
#include "Math/Rotation3D.h"
#include "RadiatorCell.h"
#include "ARCVector.h"

```

Include dependency graph for BarrelRadiatorCell.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [BarrelRadiatorCell](#)

7.10 BarrelRadiatorCell.h

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 6th September 2022
00005
00006 #ifndef BARRELRADIATORCELL
00007 #define BARRELRADIATORCELL
00008
00009 #include<vector>
00010 #include<memory>
00011 #include<string>
00012 #include"TObject.h"
00013 #include"Math/Rotation3D.h"
00014 #include"RadiatorCell.h"
00015 #include"ARCVector.h"
00016
00017 class BarrelRadiatorCell: public RadiatorCell {
00018 public:
00025     BarrelRadiatorCell(std::size_t CellColumnNumber,
00026                       std::size_t CellRowNumber,
00027                       double HexagonSize);
00031     BarrelRadiatorCell(const BarrelRadiatorCell &radiatorCell) = delete;
00035     virtual bool IsInsideCell(const ARCVector &Position) const override;
00039     virtual std::vector<std::pair<std::unique_ptr<TObject>, std::string>>
00040     DrawRadiatorGeometry() const override;
00044     virtual RotationZ ReversePhiRotation() const override;
00048     virtual bool IsDetectorInsideCell() const override;
00052     virtual bool IsEdgeCell() const override;
00053 protected:
00057     const double m_BarrelRadius;
00058 private:
00062     Vector GetCellPosition(std::size_t CellColumnNumber,
00063                           std::size_t CellRowNumber,
00064                           double HexagonSize);
00068     Rotation3D GetCellOrientation(double HexagonSize,
00069                                  std::size_t CellRowNumber);
00070 };
00071
00072 #endif

```

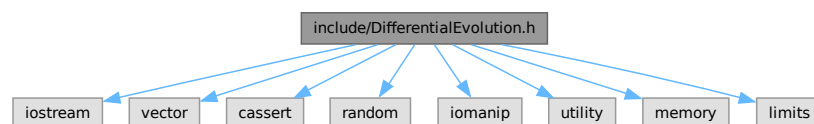
7.11 include/DifferentialEvolution.h File Reference

```

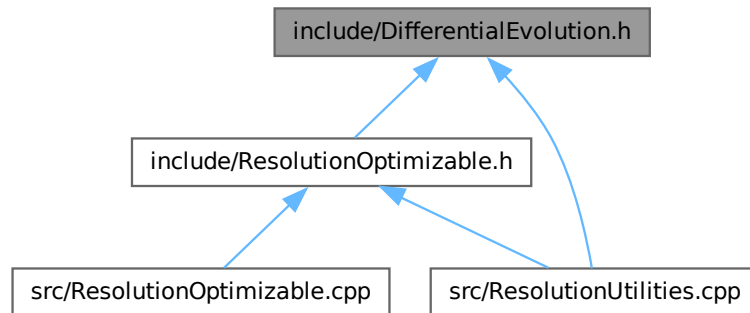
#include <iostream>
#include <vector>
#include <cassert>
#include <random>
#include <iomanip>
#include <utility>
#include <memory>
#include <limits>

```

Include dependency graph for DifferentialEvolution.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [de::IOptimizable](#)
- struct [de::IOptimizable::Constraints](#)
- class [de::DifferentialEvolution](#)

Namespaces

- namespace [de](#)

7.11.1 Detailed Description

Author

Milos Stojanovic Stojke (milsto)

Implementation of Differential evolution algorithm.

Definition in file [DifferentialEvolution.h](#).

7.12 DifferentialEvolution.h

[Go to the documentation of this file.](#)

```

00001
00007
00008 #pragma once
00009
00010 #include <iostream>
00011 #include <vector>
00012 #include <cassert>
00013 #include <random>
00014 #include <iomanip>
00015 #include <utility>
00016 #include <memory>
00017 #include <limits>
00018

```

```

00019 namespace de
00020 {
00021     class IOptimizable
00022     {
00023     public:
00024         struct Constraints
00025         {
00026             Constraints(double lower = 0.0, double upper = 1.0, bool isConstrained = false) :
00027                 lower(lower),
00028                 upper(upper),
00029                 isConstrained(isConstrained)
00030             {
00031             }
00032         }
00033
00034         bool Check(double candidate)
00035         {
00036             if (isConstrained)
00037             {
00038                 if (candidate <= upper && candidate >= lower)
00039                 {
00040                     return true;
00041                 }
00042                 else
00043                 {
00044                     return false;
00045                 }
00046             }
00047             else
00048             {
00049                 return true;
00050             }
00051         }
00052
00053         double lower;
00054         double upper;
00055         bool isConstrained;
00056     };
00057
00058     virtual double EvaluateCost(std::vector<double> inputs) const = 0;
00059     virtual std::size_t NumberOfParameters() const = 0;
00060     virtual std::vector<Constraints> GetConstraints() const = 0;
00061     virtual ~IOptimizable() {}
00062 };
00063
00064 class DifferentialEvolution
00065 {
00066 public:
00067     DifferentialEvolution( const IOptimizable& costFunction,
00068                           std::size_t populationSize,
00069                           std::size_t randomSeed = 123,
00070                           bool shouldCheckConstraints = true,
00071                           std::function<void(const DifferentialEvolution&)> callback = nullptr,
00072                           std::function<bool(const DifferentialEvolution&)> terminationCondition
00073                           = nullptr) :
00074         m_cost(costFunction),
00075         m_populationSize(populationSize),
00076         m_F(0.8),
00077         m_CR(0.9),
00078         m_bestAgentIndex(0),
00079         m_minCost(-std::numeric_limits<double>::infinity()),
00080         m_shouldCheckConstraints(shouldCheckConstraints),
00081         m_callback(callback),
00082         m_terminationCondition(terminationCondition)
00083     {
00084         m_generator.seed(randomSeed);
00085         assert(m_populationSize >= 4);
00086
00087         m_numberOfParameters = m_cost.NumberOfParameters();
00088
00089         m_population.resize(populationSize);
00090         for (auto& agent : m_population)
00091         {
00092             agent.resize(m_numberOfParameters);
00093         }
00094
00095         m_minCostPerAgent.resize(m_populationSize);
00096
00097         m_constraints = costFunction.GetConstraints();
00098     }
00099
00100     void InitPopulation()
00101     {
00102         // Init population based on random sampling of the cost function
00103         std::shared_ptr<std::uniform_real_distribution<double>> distribution;
00104
00105         for (auto& agent : m_population)

```

```

00117         {
00118             for (std::size_t i = 0; i < m_numberOfParameters; i++)
00119             {
00120                 if (m_constraints[i].isConstrained)
00121                 {
00122                     distribution =
00123                     std::make_shared<std::uniform_real_distribution<double>>(std::uniform_real_distribution<double>(m_constraints[i].lower,
00124                     m_constraints[i].upper));
00125                 }
00126                 else
00127                 {
00128                     distribution =
00129                     std::make_shared<std::uniform_real_distribution<double>>(std::uniform_real_distribution<double>(g_defaultLowerConstraint,
00130                     g_defaultUpperConstraint));
00131                 }
00132             }
00133             agent[i] = (*distribution)(m_generator);
00134         }
00135     }
00136     // Initialize minimum cost, best agent and best agent index
00137     for (std::size_t i = 0; i < m_populationSize; i++)
00138     {
00139         m_minCostPerAgent[i] = m_cost.EvaluateCost(m_population[i]);
00140         if (m_minCostPerAgent[i] < m_minCost)
00141         {
00142             m_minCost = m_minCostPerAgent[i];
00143             m_bestAgentIndex = i;
00144         }
00145     }
00146 }
00147 void SelectionAndCorssing()
00148 {
00149     std::uniform_real_distribution<double>
00150     distribution(0, static_cast<double>(m_populationSize));
00151     double minCost = m_minCostPerAgent[0];
00152     std::size_t bestAgentIndex = 0;
00153     for (std::size_t x = 0; x < m_populationSize; x++)
00154     {
00155         // For x in population select 3 random agents (a, b, c) different from x
00156         std::size_t a = x;
00157         std::size_t b = x;
00158         std::size_t c = x;
00159         // Agents must be different from each other and from x
00160         while (a == x || b == x || c == x || a == b || a == c || b == c)
00161         {
00162             a = static_cast<std::size_t>(distribution(m_generator));
00163             b = static_cast<std::size_t>(distribution(m_generator));
00164             c = static_cast<std::size_t>(distribution(m_generator));
00165         }
00166         // Form intermediate solution z
00167         std::vector<double> z(m_numberOfParameters);
00168         for (std::size_t i = 0; i < m_numberOfParameters; i++)
00169         {
00170             z[i] = m_population[a][i] + m_F * (m_population[b][i] - m_population[c][i]);
00171         }
00172         // Chose random R
00173         std::uniform_real_distribution<double>
00174         distributionParam(0, static_cast<double>(m_numberOfParameters));
00175         std::size_t R = static_cast<std::size_t>(distributionParam(m_generator));
00176         // Chose random r for each dimension
00177         std::vector<double> r(m_numberOfParameters);
00178         std::uniform_real_distribution<double> distributionPerX(0, 1);
00179         for (auto& var : r)
00180         {
00181             var = distributionPerX(m_generator);
00182         }
00183         std::vector<double> newX(m_numberOfParameters);
00184         // Execute crossing
00185         for (std::size_t i = 0; i < m_numberOfParameters; i++)
00186         {
00187             if (r[i] < m_CR || i == R)
00188             {
00189                 newX[i] = z[i];
00190             }
00191             else
00192             {
00193                 //
00194             }
00195         }
00196     }

```

```

00200         newX[i] = m_population[x][i];
00201     }
00202 }
00203
00204     // Check if newX candidate satisfies constraints and skip it if not.
00205     // If agent is skipped loop iteration x is decreased so that it is ensured
00206     // that the population has constant size (equal to m_populationSize).
00207     if (m_shouldCheckConstraints && !CheckConstraints(newX))
00208     {
00209         x--;
00210         continue;
00211     }
00212
00213     // Calculate new cost and decide should the newX be kept.
00214     double newCost = m_cost.EvaluateCost(newX);
00215     if (newCost < m_minCostPerAgent[x])
00216     {
00217         m_population[x] = newX;
00218         m_minCostPerAgent[x] = newCost;
00219     }
00220
00221     // Track the global best agent.
00222     if (m_minCostPerAgent[x] < minCost)
00223     {
00224         minCost = m_minCostPerAgent[x];
00225         bestAgentIndex = x;
00226     }
00227 }
00228
00229 m_minCost = minCost;
00230 m_bestAgentIndex = bestAgentIndex;
00231 }
00232
00233 std::vector<double> GetBestAgent() const
00234 {
00235     return m_population[m_bestAgentIndex];
00236 }
00237
00238 double GetBestCost() const
00239 {
00240     return m_minCostPerAgent[m_bestAgentIndex];
00241 }
00242
00243 std::vector<std::pair<std::vector<double>, double>> GetPopulationWithCosts() const
00244 {
00245     std::vector<std::pair<std::vector<double>, double>> toRet;
00246     for (std::size_t i = 0; i < m_populationSize; i++)
00247     {
00248         toRet.push_back(std::make_pair(m_population[i], m_minCostPerAgent[i]));
00249     }
00250
00251     return toRet;
00252 }
00253
00254 void PrintPopulation() const
00255 {
00256     for (auto agent : m_population)
00257     {
00258         for (auto& var : agent)
00259         {
00260             std::cout << var << " ";
00261         }
00262         std::cout << std::endl;
00263     }
00264 }
00265
00266 void Optimize(int iterations, bool verbose = true)
00267 {
00268     InitPopulation();
00269
00270     // Optimization loop
00271     for (int i = 0; i < iterations; i++)
00272     {
00273         // Optimization step
00274         SelectionAndCorssing();
00275
00276         if (verbose)
00277         {
00278             std::cout << "Iteration " << i + 1 << " of " << iterations << "\n";
00279             std::cout << std::fixed << std::setprecision(5);
00280             std::cout << "Current minimal cost: " << m_minCost*1000.0;
00281             std::cout << "e-3" << "\t\t";
00282             std::cout << "Best agent: ";
00283             for (std::size_t i = 0; i < m_numberOfParameters; i++)
00284             {
00285                 std::cout << m_population[m_bestAgentIndex][i] << " ";
00286             }

```

```

00287         std::cout << std::endl;
00288     }
00289
00290     if (m_callback)
00291     {
00292         m_callback(*this);
00293     }
00294
00295     if (m_terminationCondition)
00296     {
00297         if (m_terminationCondition(*this))
00298         {
00299             if (verbose)
00300             {
00301                 std::cout << "Terminated due to positive evaluation of the termination
condition." << std::endl;
00302             }
00303             return;
00304         }
00305     }
00306 }
00307
00308 if (verbose)
00309 {
00310     std::cout << "Terminated due to exceeding total number of generations." << std::endl;
00311 }
00312 }
00313
00314 private:
00315 bool CheckConstraints(std::vector<double> agent)
00316 {
00317     for (std::size_t i = 0; i < agent.size(); i++)
00318     {
00319         if (!m_constraints[i].Check(agent[i]))
00320         {
00321             return false;
00322         }
00323     }
00324
00325     return true;
00326 }
00327
00328 const IOptimizable& m_cost;
00329 std::size_t m_populationSize;
00330 double m_F;
00331 double m_CR;
00332
00333 std::size_t m_bestAgentIndex;
00334 double m_minCost;
00335
00336 std::size_t m_numberOfParameters;
00337
00338 bool m_shouldCheckConstraints;
00339
00340 std::function<void(const DifferentialEvolution&)> m_callback;
00341 std::function<bool(const DifferentialEvolution&)> m_terminationCondition;
00342
00343 std::default_random_engine m_generator;
00344 std::vector<std::vector<double>> m_population;
00345
00346 std::vector<double> m_minCostPerAgent;
00347
00348 std::vector<IOptimizable::Constraints> m_constraints;
00349
00350 static constexpr double g_defaultLowerConstraint = -std::numeric_limits<double>::infinity();
00351 static constexpr double g_defaultUpperConstraint = std::numeric_limits<double>::infinity();
00352 };
00353 }

```

7.13 include/EndCapRadiatorArray.h File Reference

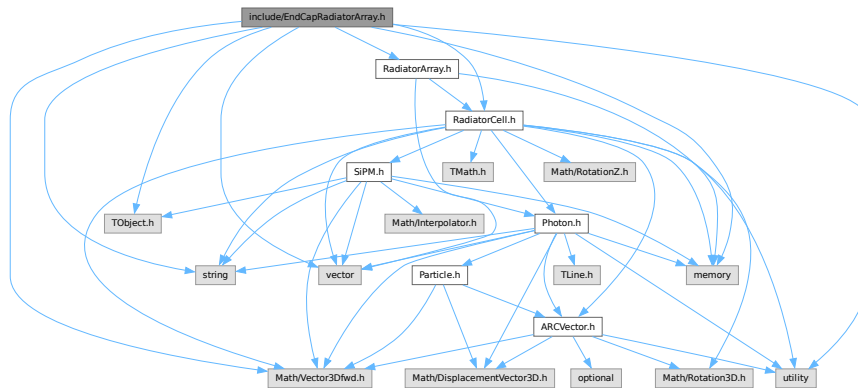
```

#include <vector>
#include <memory>
#include <utility>
#include <string>
#include "TObject.h"
#include "Math/Vector3Dfwd.h"
#include "RadiatorCell.h"

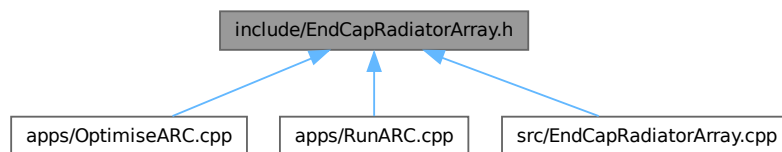
```

```
#include "RadiatorArray.h"
```

Include dependency graph for EndCapRadiatorArray.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [EndCapRadiatorArray](#)

7.14 EndCapRadiatorArray.h

[Go to the documentation of this file.](#)

```
00001 // Martin Duy Tat 23rd June 2022
00005
00006 #ifndef ENDCAPRADIATORARRAY
00007 #define ENDCAPRADIATORARRAY
00008
00009 #include<vector>
00010 #include<memory>
00011 #include<utility>
00012 #include<string>
00013 #include"TOObject.h"
00014 #include"Math/Vector3Dfwd.h"
00015 #include"RadiatorCell.h"
00016 #include"RadiatorArray.h"
00017
00018 using Vector = ROOT::Math::XYZVector;
00019
00020 class Particle;
00021
00022 class EndCapRadiatorArray: public RadiatorArray {
00023 public:
00027     EndCapRadiatorArray();
```

```

00031 EndCapRadiatorArray(const EndCapRadiatorArray &radiatorCell) = delete;
00035 std::vector<std::pair<std::unique_ptr<TObject>, std::string>>
00036 DrawRadiatorArray() const override;
00041 virtual const RadiatorCell* FindRadiator(Particle &particle) const override;
00048 bool IsBelowThisRow(std::size_t Row, double x, double y) const;
00055 bool IsBelowOddRow(double x, double y) const;
00062 bool IsBelowEvenRow(double x, double y) const;
00063 private:
00067     const double m_InnerRadius;
00071     const double m_OuterRadius;
00075     virtual int FindRadiatorIndex(std::size_t i, std::size_t j) const override;
00076 };
00077
00078 #endif

```

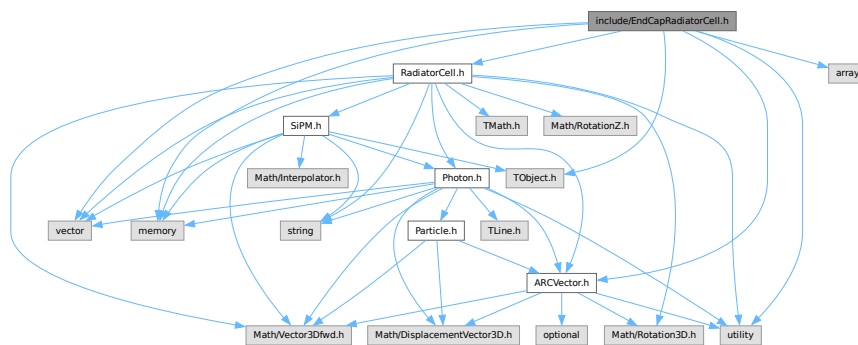
7.15 include/EndCapRadiatorCell.h File Reference

```

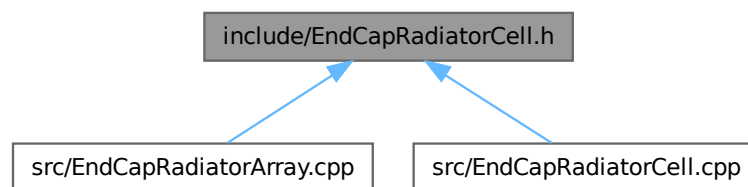
#include <vector>
#include <utility>
#include <memory>
#include <array>
#include "TObject.h"
#include "RadiatorCell.h"
#include "ARCVector.h"

```

Include dependency graph for EndCapRadiatorCell.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [EndCapRadiatorCell](#)

7.16 EndCapRadiatorCell.h

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 1st September 2022
00006
00007 #ifndef ENDCAPRADIATORCELL
00008 #define ENDCAPRADIATORCELL
00009
00010 #include<vector>
00011 #include<utility>
00012 #include<memory>
00013 #include<array>
00014 #include"TObject.h"
00015 #include"RadiatorCell.h"
00016 #include"ARCVector.h"
00017
00018 class EndCapRadiatorCell: public RadiatorCell {
00019 public:
00023     EndCapRadiatorCell(std::size_t CellColumnNumber,
00024                       std::size_t CellRowNumber,
00025                       double HexagonSize);
00029     EndCapRadiatorCell(const EndCapRadiatorCell &radiatorCell) = delete;
00033     virtual bool IsInsideCell(const ARCVector &Position) const override;
00037     bool IsInsideCell(const Vector &Position) const;
00041     virtual std::vector<std::pair<std::unique_ptr<TObject>, std::string>
00042     DrawRadiatorGeometry() const override;
00046     virtual bool IsDetectorInsideCell() const override;
00050     virtual bool IsEdgeCell() const override;
00054     static constexpr std::array<std::pair<std::size_t, std::size_t>, 23>
00055     m_ValidCells{{
00056         {1, 1}, {2, 1}, {3, 1}, {4, 1}, {5, 1}, {6, 1}, {7, 1},
00057         {2, 2}, {3, 2}, {4, 2}, {5, 2}, {6, 2}, {7, 2}, {8, 2},
00058         {3, 3}, {4, 3}, {5, 3}, {6, 3}, {7, 3},
00059         {5, 4}, {6, 4}, {7, 4},
00060         {6, 5}
00061     }};
00065     static constexpr std::array<std::pair<std::size_t, std::size_t>, 1>
00066     m_NotValidCells{{
00067         /*{8, 1}, {9, 2}, {8, 3}, {8, 4}, {7, 5},
00068         {0, 1}, {1, 2}*/
00069         {7, 5}
00070     }};
00071 private:
00075     const double m_InnerRadius;
00079     const double m_OuterRadius;
00083     static Vector GetCellPosition(std::size_t CellColumnNumber,
00084                                   std::size_t CellRowNumber,
00085                                   double HexagonSize);
00089     static Rotation3D GetCellOrientation(std::size_t CellColumnNumber,
00090                                          std::size_t CellRowNumber,
00091                                          double HexagonSize);
00096     static double GetCoordinateRotation(std::size_t CellColumnNumber,
00097                                         std::size_t CellRowNumber,
00098                                         double HexagonSize);
00103     bool IsInsideRadialAcceptance(const Vector &Position) const;
00104 };
00105
00106 #endif

```

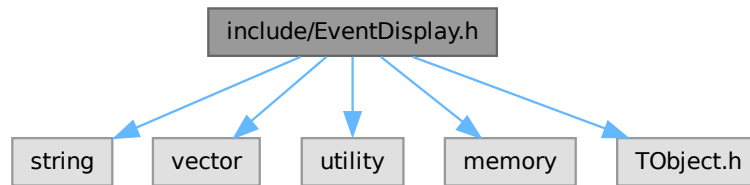
7.17 include/EventDisplay.h File Reference

```

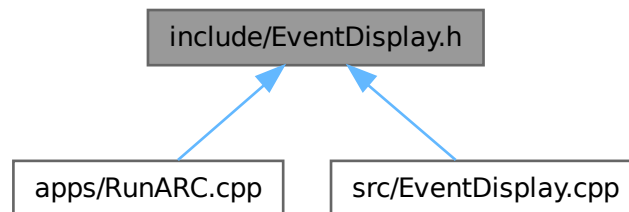
#include <string>
#include <vector>
#include <utility>
#include <memory>
#include "TObject.h"

```

Include dependency graph for EventDisplay.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [EventDisplay](#)

7.18 EventDisplay.h

[Go to the documentation of this file.](#)

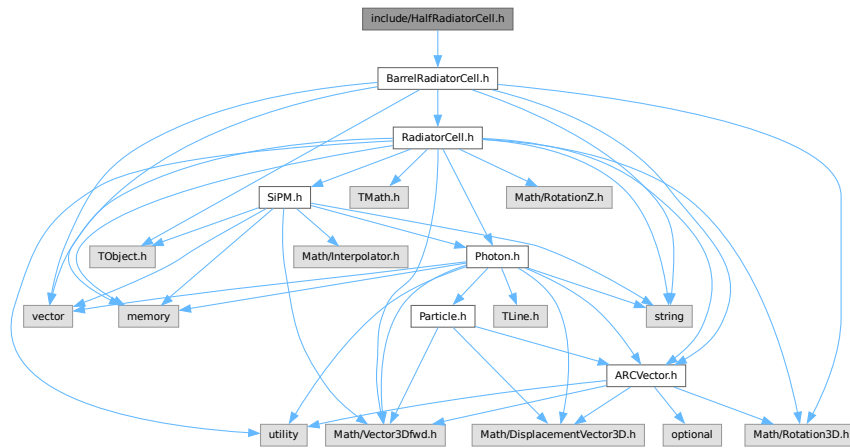
```

00001 // Martin Duy Tat 27th May 2022
00005
00006 #ifndef EVENTDISPLAY
00007 #define EVENTDISPLAY
00008
00009 #include<string>
00010 #include<vector>
00011 #include<utility>
00012 #include<utility>
00013 #include<memory>
00014 #include"TObject.h"
00015
00016 class EventDisplay {
00017 public:
00021   EventDisplay() = default;
00025   void DrawEventDisplay();
00029   void AddObject(std::unique_ptr<TObject> Object, const std::string &Option = "");
00033   void AddObject(std::vector<std::pair<std::unique_ptr<TObject>, std::string> Objects);
00034 private:
00038   std::vector<std::pair<std::unique_ptr<TObject>, std::string> m_EventObjects;
00039 };
00040
00041 #endif
  
```

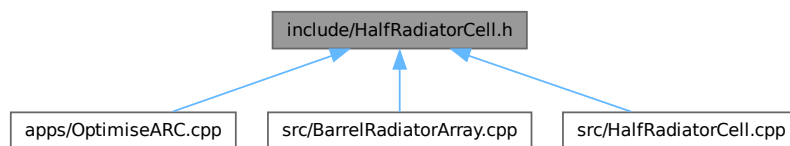
7.19 include/HalfRadiatorCell.h File Reference

```
#include "BarrelRadiatorCell.h"
```

Include dependency graph for HalfRadiatorCell.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [HalfRadiatorCell](#)

Macros

- `#define` [HAIFRADIATORCELL](#)

7.19.1 Macro Definition Documentation

7.19.1.1 HAIFRADIATORCELL

```
#define HAIFRADIATORCELL
```

[HalfRadiatorCell](#) inherits from [RadiatorCell](#) and is more or less identical, but the cell is only the left half

Definition at line 7 of file [HalfRadiatorCell.h](#).

7.20 HalfRadiatorCell.h

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 27th August 2022
00005
00006 #ifndef HALFRADIATORCELL
00007 #define HALFRADIATORCELL
00008
00009 #include "BarrelRadiatorCell.h"
00010
00011 class HalfRadiatorCell: public BarrelRadiatorCell {
00012 public:
00016   HalfRadiatorCell(std::size_t CellColumnNumber, std::size_t CellRowNumber, double HexagonSize);
00020   HalfRadiatorCell(const HalfRadiatorCell &radiatorCell) = delete;
00024   virtual bool IsInsideCell(const ARCVector &Position) const override;
00028   virtual std::vector<std::pair<std::unique_ptr<TObject>, std::string>
00029   DrawRadiatorGeometry() const override;
00030 };
00031
00032 #endif

```

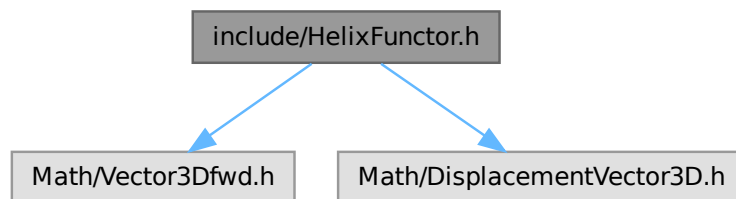
7.21 include/HelixFunctor.h File Reference

```

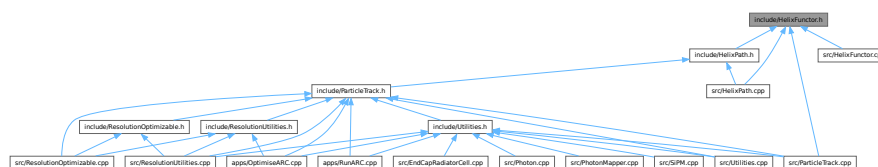
#include "Math/Vector3Dfwd.h"
#include "Math/DisplacementVector3D.h"

```

Include dependency graph for HelixFunctor.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [HelixFunctor](#)
- class [BarrelHelixFunctor](#)
- class [EndCapHelixFunctor](#)
- class [ZPlaneHelixFunctor](#)
- class [MirrorHelixFunctor](#)

7.22 HelixFunctor.h

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 28th October 2022
00005
00006 #ifndef HELIXFUNCTOR
00007 #define HELIXFUNCTOR
00008
00009 #include "Math/Vector3Dfwd.h"
00010 #include "Math/DisplacementVector3D.h"
00011
00012 using Vector = ROOT::Math::XYZVector;
00013
00014 class HelixFunctor {
00015 public:
00019     HelixFunctor() = default;
00024     virtual double operator() (const Vector &Vect) const = 0;
00025 };
00026
00030 class BarrelHelixFunctor: public HelixFunctor {
00031 public:
00036     BarrelHelixFunctor(double Radius);
00040     BarrelHelixFunctor(const BarrelHelixFunctor &Functor) = delete;
00045     virtual double operator() (const Vector &Vect) const override;
00046 private:
00050     const double m_Radius;
00051 };
00052
00056 class EndCapHelixFunctor: public HelixFunctor {
00057 public:
00062     EndCapHelixFunctor(double z);
00066     EndCapHelixFunctor(const BarrelHelixFunctor &Functor) = delete;
00071     virtual double operator() (const Vector &Vect) const override;
00072 private:
00076     const double m_z;
00077 };
00078
00083
00084 class ZPlaneHelixFunctor: public HelixFunctor {
00085 public:
00093     ZPlaneHelixFunctor(double z, const Vector &CellPosition, bool Barrel);
00097     ZPlaneHelixFunctor(const ZPlaneHelixFunctor &Functor) = delete;
00102     virtual double operator() (const Vector &Vect) const override;
00103 private:
00107     const double m_z;
00111     const Vector m_UnitZ;
00115     const double m_CellPositionZ;
00121     static Vector GetUnitZ(bool Barrel, const Vector &Position);
00122 };
00123
00127 class MirrorHelixFunctor: public HelixFunctor {
00128 public:
00134     MirrorHelixFunctor(const Vector &MirrorPosition, double Curvature);
00138     MirrorHelixFunctor(const MirrorHelixFunctor &Functor) = delete;
00143     virtual double operator() (const Vector &Vect) const override;
00144 private:
00148     const Vector m_MirrorPosition;
00152     const double m_Curvature;
00153 };
00154
00155 #endif

```

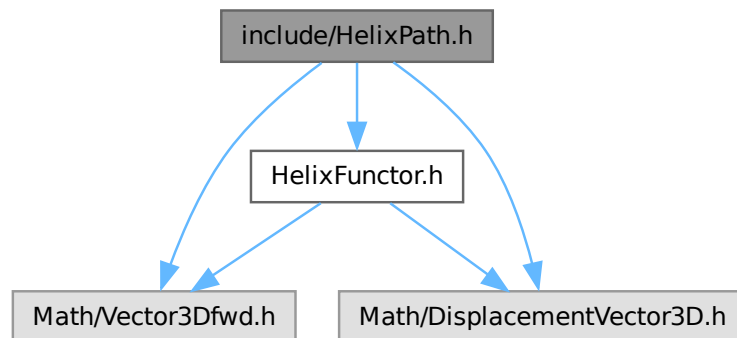
7.23 include/HelixPath.h File Reference

```

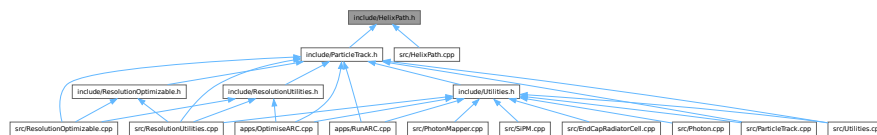
#include "Math/Vector3Dfwd.h"
#include "Math/DisplacementVector3D.h"
#include "HelixFunctor.h"

```

Include dependency graph for HelixPath.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [HelixPath](#)

Typedefs

- using [RotationZ](#) = ROOT::Math::RotationZ

7.23.1 Typedef Documentation

7.23.1.1 RotationZ

using [RotationZ](#) = ROOT::Math::RotationZ

Definition at line 15 of file [HelixPath.h](#).

7.24 HelixPath.h

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 28th October 2022
00006
00007 #ifndef HELIXPATH
00008 #define HELIXPATH
00009
00010 #include "Math/Vector3Dfwd.h"
00011 #include "Math/DisplacementVector3D.h"
00012 #include "HelixFunctor.h"
00013
00014 using Vector = ROOT::Math::XYZVector;
00015 using RotationZ = ROOT::Math::RotationZ;
00016
00017 class HelixPath {
00018 public:
00025   HelixPath(const Vector &Momentum, int Q, double B);
00029   HelixPath(const HelixPath &Helix) = default;
00034   Vector GetPosition(double s) const;
00039   Vector GetDirection(double s) const;
00044   double SolvePathLength(const HelixFunctor &Functor,
00045                           double Min, double Max) const;
00050   void ResetOrigin(double NewOrigin_s);
00056   void MapPhi(double DeltaPhi);
00060   void ReflectZ();
00065   void ReflectY();
00066 private:
00070   Vector m_n;
00074   double m_Radius;
00078   double m_InvRadius;
00082   const bool m_BFieldOn;
00086   double m_Origin_s;
00090   Vector m-Origin;
00091 };
00092
00093 #endif

```

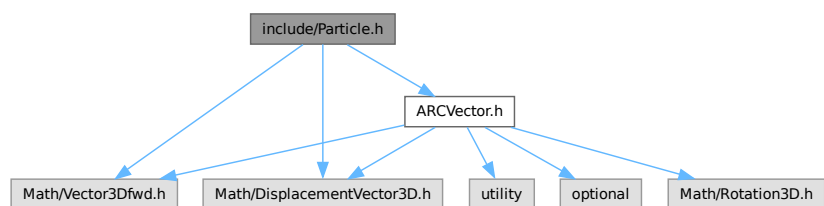
7.25 include/Particle.h File Reference

```

#include "Math/Vector3Dfwd.h"
#include "Math/DisplacementVector3D.h"
#include "ARCVector.h"

```

Include dependency graph for Particle.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Particle](#)

7.26 Particle.h

[Go to the documentation of this file.](#)

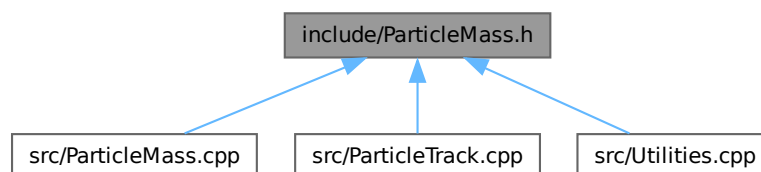
```

00001 // Martin Duy Tat 29th September 2022
00005
00006 #ifndef PARTICLE
00007 #define PARTICLE
00008
00009 #include "Math/Vector3Dfwd.h"
00010 #include "Math/DisplacementVector3D.h"
00011 #include "ARCVector.h"
00012
00013 using Vector = ROOT::Math::XYZVector;
00014
00015 class RadiatorCell;
00016 class RadiatorArray;
00017
00018 class Particle {
00019 public:
00025     Particle(const Vector &Position,
00026             const RadiatorCell *radiatorCell = nullptr);
00030     ~Particle() = default;
00034     bool FindRadiator(const RadiatorArray &radiatorArray);
00039     virtual void ConvertToRadiatorCoordinates();
00043     virtual void ConvertBackToGlobalCoordinates();
00047     const ARCVector& GetPosition() const;
00052     virtual void MapPhi(double DeltaPhi);
00056     virtual void ReflectZ();
00060     virtual void ReflectY();
00064     const RadiatorCell* GetRadiatorCell() const;
00068     virtual bool IsAtRadiator() const = 0;
00072     std::size_t GetRadiatorColumnNumber() const;
00076     std::size_t GetRadiatorRowNumber() const;
00077 protected:
00081     ARCVector m_Position;
00085     const RadiatorCell *m_RadiatorCell;
00086 };
00087
00088 #endif

```

7.27 include/ParticleMass.h File Reference

This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [ParticleMass](#)

Functions

- double `ParticleMass::GetMass` (int PID)

7.28 ParticleMass.h

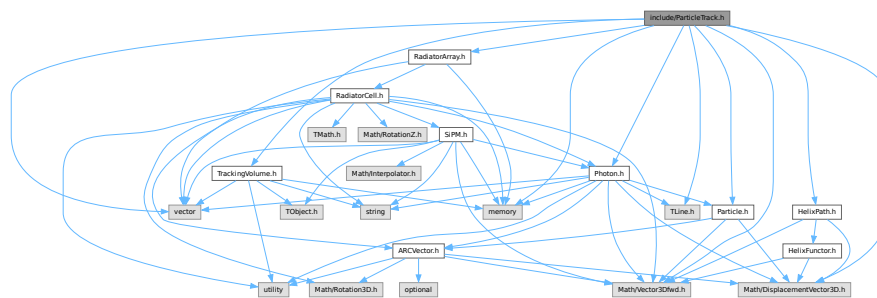
[Go to the documentation of this file.](#)

```
00001 // Martin Duy Tat 1st May 2022
00005
00006 #ifndef PARTICLEMASS
00007 #define PARTICLEMASS
00008
00009 namespace ParticleMass {
00014     double GetMass(int PID);
00015 }
00016
00017 #endif
```

7.29 include/ParticleTrack.h File Reference

```
#include <vector>
#include <memory>
#include "TLine.h"
#include "Math/Vector3Dfwd.h"
#include "Math/DisplacementVector3D.h"
#include "Photon.h"
#include "TrackingVolume.h"
#include "RadiatorArray.h"
#include "Particle.h"
#include "HelixPath.h"
#include "HelixFunction.h"
```

Include dependency graph for ParticleTrack.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [ParticleTrack](#)

7.30 ParticleTrack.h

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 29th April 2022
00005
00006 #ifndef PARTICLETRACK
00007 #define PARTICLETRACK
00008
00009 #include<vector>
00010 #include<memory>
00011 #include"TLines.h"
00012 #include"Math/Vector3Dfwd.h"
00013 #include"Math/DisplacementVector3D.h"
00014 #include"Photon.h"
00015 #include"TrackingVolume.h"
00016 #include"RadiatorArray.h"
00017 #include"Particle.h"
00018 #include"HelixPath.h"
00019
00020 using Vector = ROOT::Math::XYZVector;
00021
00022 class ParticleTrack: public Particle {
00023 public:
00031 ParticleTrack(int ParticleID,
00032               const Vector &Momentum,
00033               std::size_t TrackNumber,
00034               double BField);
00038 ~ParticleTrack() = default;
00049 enum class Location{
00050     TrackerVolume,
00051     EntranceWindow,
00052     MissedEntranceWindow,
00053     Radiator,
00054     Mirror,
00055     MissedMirror};
00061 bool TrackThroughTracker(const TrackingVolume &InnerTracker);
00065 void SetRadiator(const RadiatorCell *radiatorCell);
00069 virtual void ConvertToRadiatorCoordinates() override;
00073 virtual void ConvertBackToGlobalCoordinates() override;
00078 bool TrackThroughAerogel();
00082 void TrackThroughGasToMirror();
00086 bool TrackToNextCell(const RadiatorArray &radiatorArray);
00090 Photon GeneratePhotonFromAerogel() const;
00094 Photon GeneratePhotonFromGas() const;
00098 std::vector<Photon> GeneratePhotonsFromAerogel() const;
00102 std::vector<Photon> GeneratePhotonsFromGas() const;
00109 Photon GeneratePhoton(double Entry,
00110                      double Exit,
00111                      Photon::Radiator Radiator) const;
00115 double Beta() const;
00119 std::unique_ptr<TLines> DrawParticleTrack() const;
00123 Location GetParticleLocation() const;
00127 Vector GetEntranceWindowPosition() const;
00132 virtual void MapPhi(double DeltaPhi) override;
00136 virtual void ReflectZ() override;
00140 virtual void ReflectY() override;
00144 virtual bool IsAtRadiator() const override;
00145 private:
00149 ARCVector m_Momentum;
00153 ARCVector m_InitialPosition;
00157 double m_AerogelEntry_s;
00161 double m_AerogelExit_s;
00165 double m_GasEntry_s;
00169 double m_GasExit_s;
00173 ARCVector m_EntranceWindowPosition;
00177 int m_ParticleID;
00181 Location m_Location;
00185 bool m_RandomEmissionPoint;
00189 bool m_ChromaticDispersion;
00193 const double m_Mass;
00197 double GetPhotonYield(double x, double Beta, double n) const;
00201 const double m_PhotonMultiplier;
00205 std::size_t m_TrackNumber;
00209 std::vector<std::size_t> m_TracksToDraw;
00213 HelixPath m_Helix;
00217 double m_PathLength;

```

```

00221     bool IsTrackDrawn() const;
00225     static double GetPhotonMultiplier(const Vector &Momentum);
00226 };
00227
00228 #endif

```

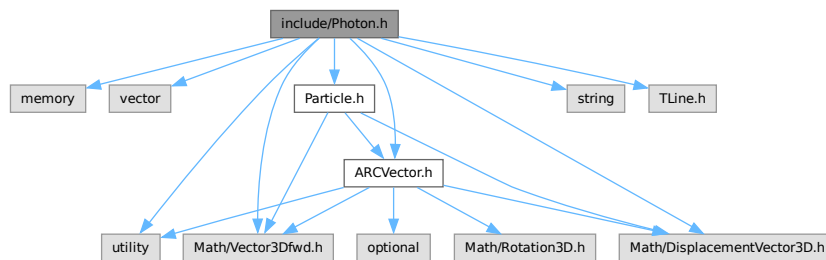
7.31 include/Photon.h File Reference

```

#include <memory>
#include <vector>
#include <utility>
#include <string>
#include "TLine.h"
#include "Math/Vector3Dfwd.h"
#include "Math/DisplacementVector3D.h"
#include "Particle.h"
#include "ARCVector.h"

```

Include dependency graph for Photon.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Photon](#)

7.32 Photon.h

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 29th April 2022
00005
00006 #ifndef PHOTON
00007 #define PHOTON
00008
00009 #include<memory>
00010 #include<vector>

```

```

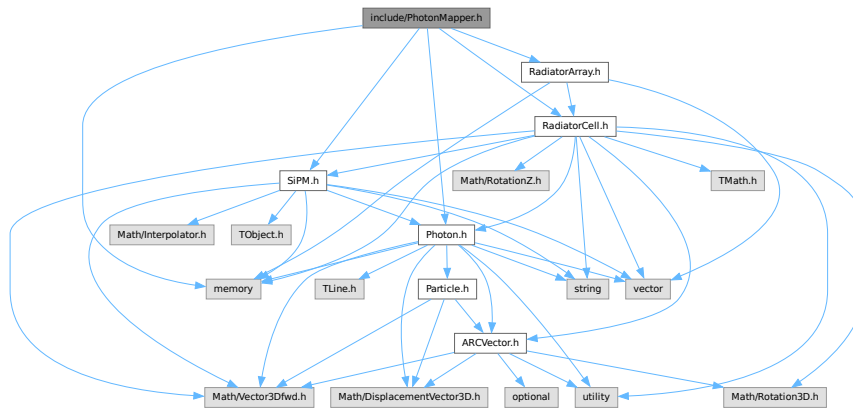
00011 #include<utility>
00012 #include<string>
00013 #include"TLine.h"
00014 #include"Math/Vector3Dfwd.h"
00015 #include"Math/DisplacementVector3D.h"
00016 #include"Particle.h"
00017 #include"ARCVector.h"
00018
00019 using Vector = ROOT::Math::XYZVector;
00020
00021 class RadiatorCell;
00022
00023 class Photon: public Particle {
00024 public:
00028     enum class Radiator{Aerogel, Gas};
00041     enum class Status {
00042         Emitted,
00043         MirrorHit,
00044         MirrorMiss,
00045         EfficiencyMiss,
00046         DetectorHit,
00047         DetectorMiss,
00048         AerogelScattered,
00049         WallMiss,
00050         Backwards,
00051         OutsideCell,
00052         OutsideMirrorRadius
00053     };
00070     Photon(const Vector &Position,
00071           const Vector &AssumedPosition,
00072           const Vector &Direction,
00073           const Vector &ParticleDirection,
00074           const Vector &AssumedParticleDirection,
00075           double Energy,
00076           double CosCherenkovAngle,
00077           Radiator radiator,
00078           const RadiatorCell *radiatorCell,
00079           bool IsTrackDrawn,
00080           double Weight);
00084     ~Photon() = default;
00088     Photon(const Photon &photon) = delete;
00092     Photon(Photon &&photon) = default;
00096     std::vector<std::pair<std::unique_ptr<TObject>, std::string>> DrawPhotonPath() const;
00101     void PropagatePhoton(const Vector &Displacement);
00105     const Vector& GetDirection() const;
00110     void KickPhoton(const Vector &Kick);
00114     Status GetStatus() const;
00118     const Vector& GetEmissionPoint(bool TrueEmissionPoint) const;
00122     void UpdatePhotonStatus(Status status);
00126     double GetEnergy() const;
00130     void AddAerogelTravelDistance(double Distance);
00134     double GetAerogelTravelDistance() const;
00138     const ARCVector* GetMirrorHitPosition() const;
00142     void RegisterMirrorHitPosition(const Vector &MirrorHitPosition);
00146     Radiator GetRadiator() const;
00150     double GetCosCherenkovAngle() const;
00154     virtual void ConvertToRadiatorCoordinates() override;
00158     virtual void ConvertBackToGlobalCoordinates() override;
00163     virtual void MapPhi(double DeltaPhi) override;
00167     virtual void ReflectZ() override;
00171     virtual void ReflectY() override;
00175     virtual bool IsAtRadiator() const override;
00179     void PutPhotonToEmissionPoint();
00183     void PhotonHasMigrated();
00187     bool HasPhotonMigrated() const;
00191     const Vector& GetParticleDirection(bool TrueEmissionPoint) const;
00195     double GetWeight() const;
00196 private:
00200     ARCVector m_AssumedEmissionPoint;
00204     ARCVector m_EmissionPoint;
00208     ARCVector m_Direction;
00212     ARCVector m_ParticleDirection;
00216     ARCVector m_AssumedParticleDirection;
00220     const double m_Energy;
00224     Radiator m_Radiator;
00228     const double m_CosCherenkovAngle;
00232     Status m_Status;
00236     double m_AerogelTravelDistance;
00240     std::unique_ptr<ARCVector> m_MirrorHitPosition;
00244     double m_Weight;
00248     bool m_HasMigrated;
00252     bool m_IsTrackDrawn;
00253 };
00254
00255 #endif

```

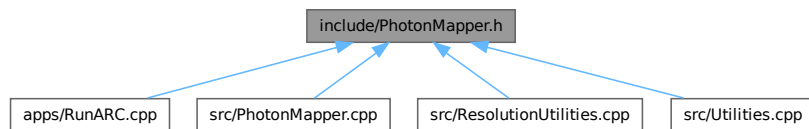
7.33 include/PhotonMapper.h File Reference

```
#include <memory>
#include "RadiatorCell.h"
#include "Photon.h"
#include "SiPM.h"
#include "RadiatorArray.h"
```

Include dependency graph for PhotonMapper.h:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace `PhotonMapper`

Functions

- double `PhotonMapper::PhotonMirrorDistance` (const `Photon` &photon)
- void `PhotonMapper::TracePhotonToMirror` (`Photon` &photon)
- `PhotonHit` `PhotonMapper::TracePhotonToDetector` (`Photon` &photon)
- `std::unique_ptr< PhotonHit >` `PhotonMapper::TracePhoton` (`Photon` &photon, const `RadiatorArray` &radiatorArray)
- bool `PhotonMapper::IsScatteredInAerogel` (const `Photon` &photon)

7.34 PhotonMapper.h

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 1st May 2022
00005
00006 #ifndef PHOTONMAPPER
00007 #define PHOTONMAPPER
00008
00009 #include<memory>
00010 #include"RadiatorCell.h"
00011 #include"Photon.h"
00012 #include"SiPM.h"
00013 #include"RadiatorArray.h"
00014
00015 namespace PhotonMapper {
00019     double PhotonMirrorDistance(const Photon &photon);
00023     void TracePhotonToMirror(Photon &photon);
00027     PhotonHit TracePhotonToDetector(Photon &photon);
00031     std::unique_ptr<PhotonHit> TracePhoton(Photon &photon,
00032                                         const RadiatorArray &radiatorArray);
00036     bool IsScatteredInAerogel(const Photon &photon);
00037 }
00038
00039 #endif

```

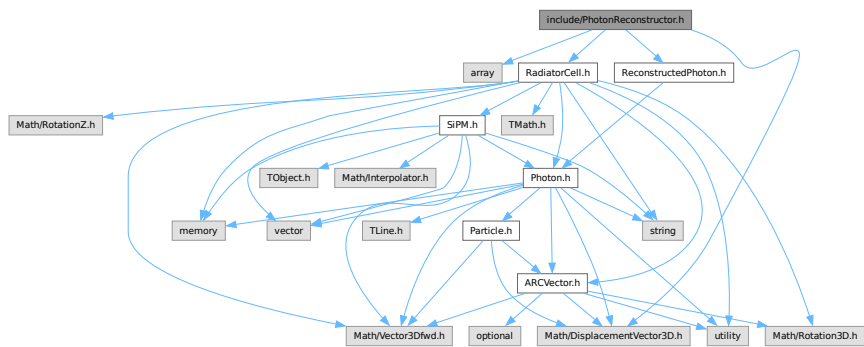
7.35 include/PhotonReconstructor.h File Reference

```

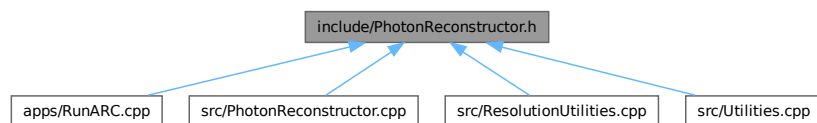
#include <array>
#include "Math/DisplacementVector3D.h"
#include "RadiatorCell.h"
#include "ReconstructedPhoton.h"

```

Include dependency graph for PhotonReconstructor.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [PhotonReconstructor::QuarticSolution](#)

Namespaces

- namespace [PhotonReconstructor](#)

Functions

- double [PhotonReconstructor::ReconstructCosCherenkovAngle](#) (const [PhotonHit](#) &photonHit, bool TrueEmissionPoint)
- [ReconstructedPhoton](#) [PhotonReconstructor::ReconstructPhoton](#) (const [PhotonHit](#) &photonHit)
- bool [PhotonReconstructor::IsReconstructed](#) (double CosAngle)
- [QuarticSolution](#) [PhotonReconstructor::SolveQuartic](#) (double EmissionMirrorDist, double DetectionMirrorParaDist, double DetectionMirrorPerpDist)
- [Vector](#) [PhotonReconstructor::GetReflectionPoint](#) (const [Vector](#) &EmissionPoint, const [Vector](#) &DetectionPoint, double DetectionMirrorParaDist, double DetectionMirrorPerpDist, double EmissionMirrorDist, const [QuarticSolution](#) &quarticSolution, std::size_t SolutionNumber)

Variables

- constexpr double [PhotonReconstructor::FailedReconstruction](#) = -2.0

7.36 PhotonReconstructor.h

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 3rd May 2022
00005
00006 #ifndef PHOTONRECONSTRUCTOR
00007 #define PHOTONRECONSTRUCTOR
00008
00009 #include<array>
00010 #include"Math/DisplacementVector3D.h"
00011 #include"RadiatorCell.h"
00012 #include"ReconstructedPhoton.h"
00013
00014 using Vector = ROOT::Math::XYZVector;
00015
00016 namespace PhotonReconstructor {
00017
00023     double ReconstructCosCherenkovAngle(const PhotonHit &photonHit,
00024                                         bool TrueEmissionPoint);
00030     ReconstructedPhoton ReconstructPhoton(const PhotonHit &photonHit);
00034     constexpr double FailedReconstruction = -2.0;
00038     inline bool IsReconstructed(double CosAngle) {
00039         return CosAngle >= -1.0 && CosAngle <= 1.0;
00040     }
00044     struct QuarticSolution {
00048         std::array<double, 2> m_SinBeta{};
00049         std::array<double, 2> m_CosBeta{};
00053         bool m_DegenerateSolution = false;
00054     };
00061     QuarticSolution SolveQuartic(double EmissionMirrorDist,
00062                                   double DetectionMirrorParaDist,
00063                                   double DetectionMirrorPerpDist);
00067     Vector GetReflectionPoint(const Vector &EmissionPoint,
00068                               const Vector &DetectionPoint,
00069                               double DetectionMirrorParaDist,
00070                               double DetectionMirrorPerpDist,
00071                               double EmissionMirrorDist,
00072                               const QuarticSolution &quarticSolution,
00073                               std::size_t SolutionNumber);
00074 }
00075
00076 #endif

```


7.40 RadiatorCell.h

[Go to the documentation of this file.](#)

```

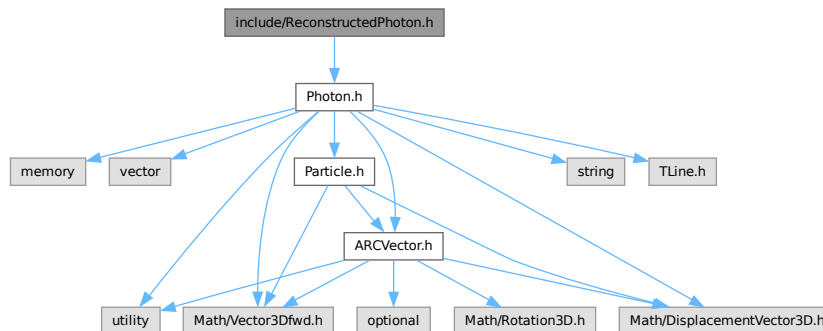
00001 // Martin Duy Tat 29th April 2022
00008
00009 #ifndef RADIATORCELL
00010 #define RADIATORCELL
00011
00012 #include<string>
00013 #include<vector>
00014 #include<utility>
00015 #include<memory>
00016 #include"TMath.h"
00017 #include"Math/Vector3Dfwd.h"
00018 #include"Math/Rotation3D.h"
00019 #include"Math/RotationZ.h"
00020 #include"SiPM.h"
00021 #include"Photon.h"
00022 #include"ARCVector.h"
00023
00024 using Vector = ROOT::Math::XYZVector;
00025 using Rotation3D = ROOT::Math::Rotation3D;
00026 using RotationZ = ROOT::Math::RotationZ;
00027
00028 class RadiatorCell {
00029 public:
00039     RadiatorCell(std::size_t CellColumnNumber,
00040                 std::size_t CellRowNumber,
00041                 double HexagonSize,
00042                 const Vector &Position,
00043                 const Rotation3D &Rotation,
00044                 const std::string &Prefix = "");
00048     RadiatorCell(const RadiatorCell &radiatorCell) = delete;
00052     virtual ~RadiatorCell() = default;
00056     double GetRadiatorThickness() const;
00060     double GetVesselThickness() const;
00064     double GetCoolingThickness() const;
00068     double GetAerogelThickness() const;
00072     const Vector& GetMirrorCentre() const;
00076     Vector GetGlobalMirrorCentre() const;
00080     double GetMirrorCurvature() const;
00084     const Vector& GetRadiatorPosition() const;
00088     const Rotation3D& GetRadiatorRotation() const;
00092     virtual bool IsInsideCell(const ARCVector &Position) const = 0;
00096     bool IsInsideCell(const Photon &photon) const;
00100     virtual std::vector<std::pair<std::unique_ptr<TObject>, std::string>
00101     DrawRadiatorGeometry() const = 0;
00105     double GetHexagonSize() const;
00109     std::pair<std::size_t, std::size_t> GetCellNumber() const;
00113     const SiPM& GetDetector() const;
00118     void SetMirrorCurvature(double Curvature);
00122     void SetMirrorXPosition(double x);
00126     void SetMirrorZPosition(double z);
00130     void SetDetectorPosition(double x);
00134     void SetDetectorTilt(double Angle);
00138     virtual bool IsDetectorInsideCell() const = 0;
00142     virtual bool IsEdgeCell() const = 0;
00146     virtual RotationZ ReversePhiRotation() const;
00147 protected:
00151     const Vector m_Position;
00155     const Rotation3D m_Rotation;
00159     double GetMirrorCurvatureCentreZ() const;
00163     const double m_RadiatorThickness;
00167     const double m_VesselThickness;
00171     const double m_CoolingThickness;
00175     const double m_AerogelThickness;
00179     const double m_HexagonSize;
00186     const std::pair<std::size_t, std::size_t> m_CellNumber;
00190     SiPM m_Detector;
00194     double m_MirrorCurvature;
00198     const Vector m_DefaultMirrorCentre;
00202     ARCVector m_MirrorCentre;
00203 };
00204
00205 #endif

```

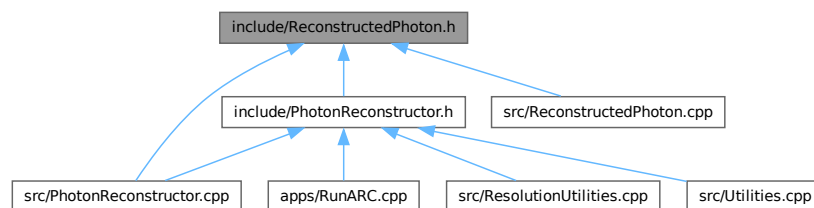
7.41 include/ReconstructedPhoton.h File Reference

```
#include "Photon.h"
```

Include dependency graph for ReconstructedPhoton.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [ReconstructedPhoton](#)

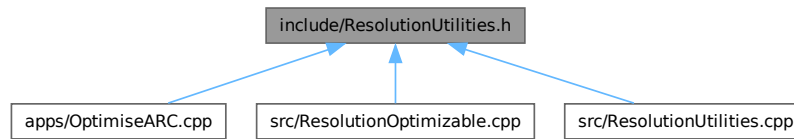
7.42 ReconstructedPhoton.h

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 3rd May 2022
00005
00006 #ifndef RECONSTRUCTEDPHOTON
00007 #define RECONSTRUCTEDPHOTON
00008
00009 #include "Photon.h"
00010
00011 struct ReconstructedPhoton {
00016     ReconstructedPhoton(const Photon &photon);
00020     const Photon *m_Photon;
00024     double m_CosCherenkovAngle_TrueEmissionPoint;
00028     double m_CosCherenkovAngle;
00029 };
00030
00031 #endif
  
```


This graph shows which files directly or indirectly include this file:



Namespaces

- namespace [ResolutionUtilities](#)

Typedefs

- using [Tracks](#) = `std::vector<ParticleTrack>`

Functions

- double [ResolutionUtilities::CalculateResolution](#) (const [Tracks](#) &Particles, const [RadiatorCell](#) *radiatorCell, const [RadiatorArray](#) &radiatorArray, std::size_t Seed, bool IncludeCentrePenalty)
- double [ResolutionUtilities::fcn](#) (double MirrorCurvature, double MirrorXPosition, double MirrorZPosition, double DetectorPosition, double DetectorTilt, [RadiatorCell](#) &radiatorCell, const [RadiatorArray](#) &radiatorArray, const [Tracks](#) &Particles, std::size_t Seed, bool IncludeCentrePenalty)
- void [ResolutionUtilities::PlotProjections](#) ([RadiatorCell](#) &radiatorCell, const [RadiatorArray](#) &radiatorArray, const [Tracks](#) &Particles)
- void [ResolutionUtilities::DoFit](#) ([RadiatorCell](#) &radiatorCell, const [RadiatorArray](#) &radiatorArray, const [Tracks](#) &Particles, std::size_t Column, std::size_t Row)

7.45.1 Typedef Documentation

7.45.1.1 Tracks

```
using Tracks = std::vector<ParticleTrack>
```

The ResolutionUtilites namespace contains the function to calculate the resolution from some generated tracks and their photons OpenMP is used to parallelise the calculation

Definition at line 17 of file [ResolutionUtilities.h](#).

7.46 ResolutionUtilities.h

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 28th July 2022
00006
00007 #ifndef RESOLUTIONUTILITIES
00008 #define RESOLUTIONUTILITIES
00009
00010 #include<vector>
00011 #include<utility>
00012 #include"Photon.h"
00013 #include"RadiatorCell.h"
00014 #include"ParticleTrack.h"
00015 #include"RadiatorArray.h"
00016
00017 using Tracks = std::vector<ParticleTrack>;
00018
00019 namespace ResolutionUtilities {
00023     double CalculateResolution(const Tracks &Particles,
00024                               const RadiatorCell *radiatorCell,
00025                               const RadiatorArray &radiatorArray,
00026                               std::size_t Seed,
00027                               bool IncludeCentrePenalty);
00031     double fcn(double MirrorCurvature,
00032               double MirrorXPosition,
00033               double MirrorZPosition,
00034               double DetectorPosition,
00035               double DetectorTilt,
00036               RadiatorCell &radiatorCell,
00037               const RadiatorArray &radiatorArray,
00038               const Tracks &Particles,
00039               std::size_t Seed,
00040               bool IncludeCentrePenalty);
00044     void PlotProjections(RadiatorCell &radiatorCell,
00045                          const RadiatorArray &radiatorArray,
00046                          const Tracks &Particles);
00050     void DoFit(RadiatorCell &radiatorCell,
00051               const RadiatorArray &radiatorArray,
00052               const Tracks &Particles,
00053               std::size_t Column,
00054               std::size_t Row);
00055
00056 }
00057
00058 #endif

```

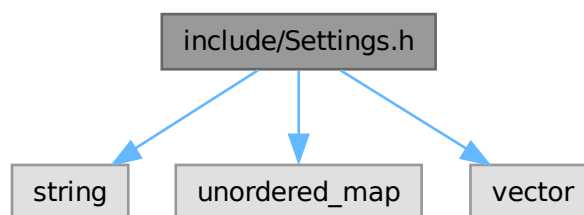
7.47 include/Settings.h File Reference

```

#include <string>
#include <unordered_map>
#include <vector>

```

Include dependency graph for Settings.h:



This graph shows which files directly or indirectly include this file:



Classes

- class **Settings**

Typedefs

- using `ssMap = std::unordered_map<std::string, std::string>`

7.47.1 Typedef Documentation

7.47.1.1 ssMap

```
using ssMap = std::unordered_map<std::string, std::string>
```

Settings is a class with only static member functions and the class contains all the settings and configurations. Each setting will belong to a set of settings with a given name. To obtain a setting, pass a string of the form "Name/Key".

Definition at line 15 of file Settings.h.

7.48 Settings.h

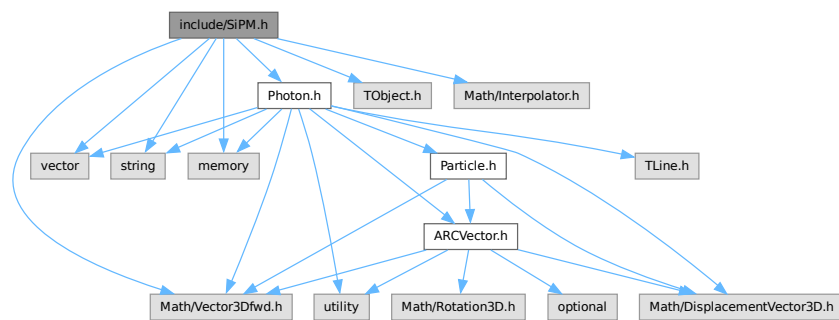
[Go to the documentation of this file.](#)

```
00001 // Martin Duy Tat 5th May 2022
00002
00003 #ifndef SETTINGS
00004 #define SETTINGS
00005
00006 #include<string>
00007 #include<unordered_map>
00008 #include<vector>
00009
00010 using ssMap = std::unordered_map<std::string, std::string>;
00011
00012 class Settings {
00013 public:
00014     Settings() = delete;
00015     static void AddSettings(const std::string &Name, const std::string &Filename);
00016     static std::string GetString(const std::string &Setting);
00017     static double GetDouble(const std::string &Setting);
00018     static int GetInt(const std::string &Setting);
00019     static std::size_t GetSizeT(const std::string &Setting);
00020     static bool GetBool(const std::string &Setting);
00021     static std::vector<int> GetIntVector(const std::string &Setting);
00022     static std::vector<std::size_t> GetSizeTVector(const std::string &Setting);
00023     static bool Exists(const std::string &Setting);
00024 private:
00025     static std::unordered_map<std::string, ssMap> m_Settings;
00026 };
00027
00028 #endif
```

7.49 include/SiPM.h File Reference

```
#include <vector>
#include <string>
#include <memory>
#include "TObject.h"
#include "Math/Interpolator.h"
#include "Math/Vector3Dfwd.h"
#include "Photon.h"
```

Include dependency graph for SiPM.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [PhotonHit](#)
- class [SiPM](#)

7.50 SiPM.h

[Go to the documentation of this file.](#)

```
00001 // Martin Duy Tat 29th April 2022
00006
00007 #ifndef SIPM
00008 #define SIPM
00009
00010 #include<vector>
00011 #include<string>
00012 #include<memory>
00013 #include"TObject.h"
00014 #include"Math/Interpolator.h"
00015 #include"Math/Vector3Dfwd.h"
00016 #include"Photon.h"
00017
00018 using Vector = ROOT::Math::XYZVector;
00019 using ROOT::Math::Interpolator;
00020 namespace InterpolationType = ROOT::Math::Interpolation;
```

```

00021
00022 struct PhotonHit {
00026     const Vector m_HitPosition;
00030     const Photon *m_Photon;
00034     const Vector m_CentreHitDistance;
00035 };
00036
00037 class SiPM {
00038 public:
00042     SiPM();
00046     PhotonHit AddPhotonHit(Photon &photon) const;
00050     void PlotHits(const std::string &Filename,
00051                 const std::vector<PhotonHit> &photonHits) const;
00055     const std::vector<PhotonHit> & GetPhotonHits() const;
00060     std::unique_ptr<TObject> DrawSiPM(const Vector &RadiatorPosition) const;
00064     bool IsDetectorHit(const Photon &photon) const;
00068     void SetDetectorPosition(double x);
00072     void SetDetectorTilt(double Angle);
00076     double GetDetectorXPosition() const;
00080     double GetDetectorZPosition() const;
00084     double GetDetectorTilt() const;
00088     double GetDetectorSizeX() const;
00092     double GetDetectorSizeY() const;
00093 private:
00097     const double m_DetectorSizeX;
00101     const double m_DetectorSizeY;
00105     double m_DetectorPositionX;
00109     const double m_DetectorPositionY;
00113     double m_DetectorPositionZ;
00118     double m_DetectorTilt;
00122     double m_CoolingThickness;
00126     const double m_MaxPDE;
00130     const bool m_EndCap;
00134     static constexpr std::array<double, 16> m_Lambda{
00135         287.0, 299.0, 322.0, 340.0,
00136         367.0, 392.0, 402.0, 413.0,
00137         422.0, 437.0, 452.0, 467.0,
00138         503.0, 590.0, 700.0, 800.0};
00142     static constexpr std::array<double, 16> m_PDE{
00143         0.20, 0.41, 0.46, 0.47,
00144         0.52, 0.59, 0.60, 0.60,
00145         0.59, 0.57, 0.56, 0.53,
00146         0.51, 0.40, 0.26, 0.13};
00151     static thread_local const Interpolator m_Interpolator;
00152 };
00153
00154 #endif

```

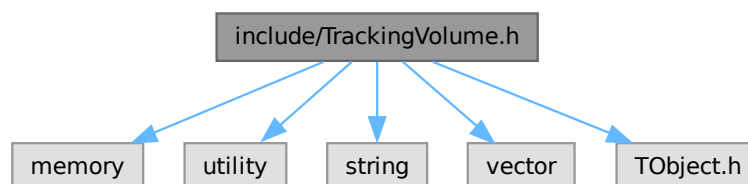
7.51 include/TrackingVolume.h File Reference

```

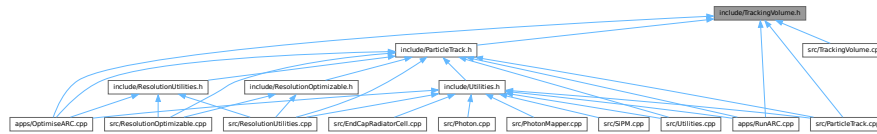
#include <memory>
#include <utility>
#include <string>
#include <vector>
#include "TObject.h"

```

Include dependency graph for TrackingVolume.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [TrackingVolume](#)

7.52 TrackingVolume.h

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 29th April 2022
00005
00006 #ifndef TRACKINGVOLUME
00007 #define TRACKINGVOLUME
00008
00009 #include<memory>
00010 #include<utility>
00011 #include<string>
00012 #include<vector>
00013 #include"TObject.h"
00014
00015 class TrackingVolume {
00016 public:
00020     TrackingVolume();
00024     double GetRadius() const;
00028     double GetFieldStrength() const;
00032     std::vector<std::pair<std::unique_ptr<TObject>, std::string>> DrawARCGeometry() const;
00033 private:
00037     const double m_Radius;
00041     const double m_Length;
00045     const int m_CellsPerRow;
00049     const double m_FieldStrength;
00050 };
00051
00052 #endif

```

7.53 include/Utilities.h File Reference

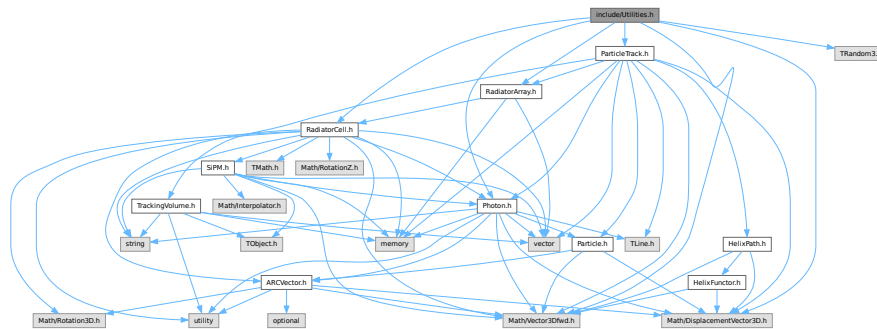
```

#include "Math/Vector3Dfwd.h"
#include "Math/DisplacementVector3D.h"
#include "TRandom3.h"
#include "RadiatorCell.h"
#include "RadiatorArray.h"
#include "ParticleTrack.h"

```

```
#include "Photon.h"
```

Include dependency graph for Utilities.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [Utilities::ResolutionStruct](#)

Namespaces

- namespace [Utilities](#)

Functions

- [Vector Utilities::VectorFromSpherical](#) (double R, double CosTheta, double Phi)
- [Vector Utilities::GenerateRandomBarrelTrack](#) (double &CosTheta, double &Phi)
- [Vector Utilities::GenerateRandomBarrelTrackZRange](#) (double z_min, double z_max)
- [Vector Utilities::GenerateRandomEndCapTrack](#) ()
- double [Utilities::GetMomentumMag](#) ()
- TRandom3 & [Utilities::Random](#) ()
- ULong_t [Utilities::TrackSeed](#) (std::size_t Seed, std::size_t TrackNumber)
- [ResolutionStruct Utilities::TrackPhotons](#) ([ParticleTrack](#) particleTrack, const [RadiatorCell](#) &radiatorCell, const [RadiatorArray](#) &radiatorArray)
- double [Utilities::GetIndexRefraction](#) ([Photon::Radiator](#) Radiator, bool ChromaticDispersion, double Energy=0.0)
- double [Utilities::GetPredictedCherenkovAngle](#) (double Momentum, int ID, [Photon::Radiator](#) Radiator)
- double [Utilities::GetCherenkovAngleDifference](#) (double Momentum, int ID1, int ID2, [Photon::Radiator](#) Radiator)

7.54 Utilities.h

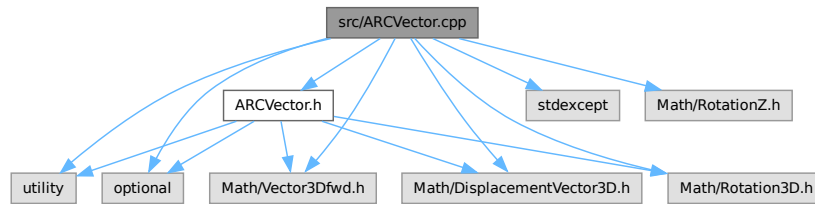
[Go to the documentation of this file.](#)

```
00001 // Martin Duy Tat 6th September 2022
00005
00006 #ifndef UTILITIES
00007 #define UTILITIES
00008
00009 #include "Math/Vector3Dfwd.h"
00010 #include "Math/DisplacementVector3D.h"
00011 #include "TRandom3.h"
00012 #include "RadiatorCell.h"
00013 #include "RadiatorArray.h"
00014 #include "ParticleTrack.h"
00015 #include "Photon.h"
00016
00017 using Vector = ROOT::Math::XYZVector;
00018
00019 namespace Utilities {
00023   Vector VectorFromSpherical(double R, double CosTheta, double Phi);
00027   Vector GenerateRandomBarrelTrack(double &CosTheta, double &Phi);
00031   Vector GenerateRandomBarrelTrackZRange(double z_min, double z_max);
00035   Vector GenerateRandomEndCapTrack();
00039   double GetMomentumMag();
00044   struct ResolutionStruct {
00048       double x = 0.0;
00052       int N = 0;
00056       bool HitTopWall = false;
00060       bool HitCorrectCell = false;
00064       Vector CentreHitDistance{0.0, 0.0, 0.0};
00069       double SumWeights = 0.0;
00070   };
00075   TRandom3 &Random();
00080   ULong_t TrackSeed(std::size_t Seed, std::size_t TrackNumber);
00084   ResolutionStruct TrackPhotons(ParticleTrack particleTrack,
00085                               const RadiatorCell &radiatorCell,
00086                               const RadiatorArray &radiatorArray);
00093   double GetIndexRefraction(Photon::Radiator Radiator,
00094                             bool ChromaticDispersion,
00095                             double Energy = 0.0);
00102   double GetPredictedCherenkovAngle(double Momentum,
00103                                     int ID,
00104                                     Photon::Radiator Radiator);
00112   double GetCherenkovAngleDifference(double Momentum,
00113                                     int ID1,
00114                                     int ID2,
00115                                     Photon::Radiator Radiator);
00116 }
00117
00118 #endif
```

7.55 src/ARCVector.cpp File Reference

```
#include <utility>
#include <optional>
#include <stdexcept>
#include "Math/Vector3Dfwd.h"
#include "Math/DisplacementVector3D.h"
#include "Math/Rotation3D.h"
#include "Math/RotationZ.h"
#include "ARCVector.h"
```

Include dependency graph for ARCVector.cpp:



7.56 ARCVector.cpp

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 16th October 2022
00002
00003 #include<utility>
00004 #include<optional>
00005 #include<stdexcept>
00006 #include"Math/Vector3Dfwd.h"
00007 #include"Math/DisplacementVector3D.h"
00008 #include"Math/Rotation3D.h"
00009 #include"Math/RotationZ.h"
00010 #include"ARCVector.h"
00011
00012 using RotationZ = ROOT::Math::RotationZ;
00013
00014 ARCVector::ARCVector(const Vector &Vec): m_Vector(Vec) {
00015 }
00016
00017 ARCVector::ARCVector(const Vector &Vec,
00018                     const Vector &RadiatorPosition,
00019                     const Rotation3D &RadiatorRotation):
00020     m_Vector(Vec),
00021     m_CoordinateTransform(std::make_pair(RadiatorPosition, RadiatorRotation)) {
00022 }
00023
00024 void ARCVector::AssignLocalCoordinates(const Vector &Position,
00025                                       const Rotation3D &Rotation) {
00026     if(m_CoordinateTransform.has_value()) {
00027         throw std::runtime_error("Cannot assign local coordinates twice");
00028     }
00029     m_CoordinateTransform = std::make_pair(Position, Rotation);
00030     GlobalToLocal(m_Vector);
00031 }
00032
00033 bool ARCVector::IsLocal() const {
00034     return m_CoordinateTransform.has_value();
00035 }
00036
00037 void ARCVector::ConvertToGlobal() {
00038     LocalToGlobal(m_Vector);
00039     m_CoordinateTransform.reset();
00040 }
00041
00042 const Vector& ARCVector::LocalVector() const {
00043     if(m_CoordinateTransform.has_value()) {
00044         return m_Vector;
00045     } else {
00046         throw std::logic_error("Local coordinates don't exist, no local vector exists");
00047     }
00048 }
00049
00050 Vector ARCVector::GlobalVector() const {
00051     if(m_CoordinateTransform.has_value()) {
00052         Vector Vec = m_Vector;
00053         LocalToGlobal(Vec);
00054         return Vec;
00055     } else {
00056         return m_Vector;
00057     }
00058 }

```

```

00059
00060 ARCVector& ARCVector::operator +=(const Vector &Vec) {
00061     m_Vector += Vec;
00062     return *this;
00063 }
00064
00065 void ARCVector::MapPhi(double DeltaPhi) {
00066     const RotationZ PhiRotation(DeltaPhi);
00067     if(m_CoordinateTransform.has_value()) {
00068         LocalToGlobal(m_Vector);
00069         m_Vector = PhiRotation(m_Vector);
00070         GlobalToLocal(m_Vector);
00071     } else {
00072         m_Vector = PhiRotation(m_Vector);
00073     }
00074 }
00075
00076 void ARCVector::ReflectZ() {
00077     if(m_CoordinateTransform.has_value()) {
00078         LocalToGlobal(m_Vector);
00079         m_Vector.SetZ(-m_Vector.Z());
00080         GlobalToLocal(m_Vector);
00081     } else {
00082         m_Vector.SetZ(-m_Vector.Z());
00083     }
00084 }
00085
00086 void ARCVector::ReflectY() {
00087     if(m_CoordinateTransform.has_value()) {
00088         LocalToGlobal(m_Vector);
00089         m_Vector.SetY(-m_Vector.Y());
00090         GlobalToLocal(m_Vector);
00091     } else {
00092         m_Vector.SetY(-m_Vector.Y());
00093     }
00094 }
00095
00096 void ARCVector::SetX(double x) {
00097     m_Vector.SetX(x);
00098 }
00099
00100 void ARCVector::SetZ(double z) {
00101     m_Vector.SetZ(z);
00102 }
00103
00104 void ARCVector::SetGlobalVector(const Vector &Vec) {
00105     m_Vector = Vec;
00106     if(m_CoordinateTransform) {
00107         GlobalToLocal(m_Vector);
00108     }
00109 }
00110
00111 void ARCVector::GlobalToLocal(Vector &Vec) const {
00112     Vec -= m_CoordinateTransform->first;
00113     Vec = m_CoordinateTransform->second(Vec);
00114 }
00115
00116 void ARCVector::LocalToGlobal(Vector &Vec) const {
00117     const auto InverseRotation = m_CoordinateTransform->second.Inverse();
00118     Vec = InverseRotation(Vec);
00119     Vec += m_CoordinateTransform->first;
00120 }

```

7.57 src/BarrelRadiatorArray.cpp File Reference

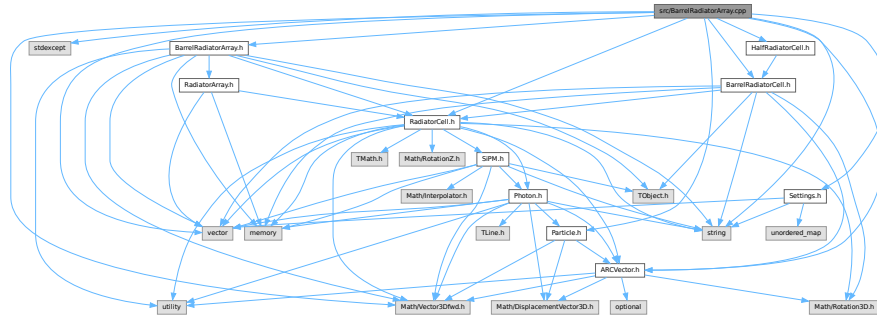
```

#include <vector>
#include <stdexcept>
#include <string>
#include "Math/Vector3Dfwd.h"
#include "BarrelRadiatorArray.h"
#include "RadiatorCell.h"
#include "HalfRadiatorCell.h"
#include "Settings.h"
#include "Particle.h"
#include "BarrelRadiatorCell.h"

```

```
#include "ARCVector.h"
```

Include dependency graph for BarrelRadiatorArray.cpp:



7.58 BarrelRadiatorArray.cpp

[Go to the documentation of this file.](#)

```
00001 // Martin Duy Tat 23rd June 2022
00002
00003 #include<vector>
00004 #include<stdexcept>
00005 #include<string>
00006 #include"Math/Vector3Dfwd.h"
00007 #include"BarrelRadiatorArray.h"
00008 #include"RadiatorCell.h"
00009 #include"HalfRadiatorCell.h"
00010 #include"Settings.h"
00011 #include"Particle.h"
00012 #include"BarrelRadiatorCell.h"
00013 #include"ARCVector.h"
00014
00015 BarrelRadiatorArray::BarrelRadiatorArray():
00016     RadiatorArray::RadiatorArray(),
00017     m_FullArray(Settings::GetBool("General/FullArray")),
00018     m_BarrelRadius(Settings::GetDouble("ARCGeometry/Radius") +
00019         Settings::GetDouble("RadiatorCell/CoolingThickness")),
00020     m_DeltaPhi(2.0*m_yHexDist/m_BarrelRadius) {
00021     if(m_FullArray) {
00022         m_Cells.reserve(2*m_NumberMainRowCells);
00023         for(std::size_t i = 0; i < m_NumberMainRowCells; i++) {
00024             // Main row
00025             m_Cells.emplace_back(std::make_unique<BarrelRadiatorCell>(i, 1, m_xHexDist));
00026         }
00027         for(std::size_t i = 0; i < m_NumberMainRowCells - 1; i++) {
00028             // Upper row
00029             m_Cells.emplace_back(std::make_unique<BarrelRadiatorCell>(i + 1, 2, m_xHexDist));
00030         }
00031         m_Cells.emplace_back(std::make_unique<HalfRadiatorCell>(m_NumberMainRowCells,
00032             2, m_xHexDist));
00033     } else {
00034         m_Cells.emplace_back(std::make_unique<BarrelRadiatorCell>(0, 0, m_xHexDist));
00035     }
00036 }
00037
00038 std::vector<std::pair<std::unique_ptr<TObject>, std::string>>
00039 BarrelRadiatorArray::DrawRadiatorArray() const {
00040     std::vector<std::pair<std::unique_ptr<TObject>, std::string>> RadiatorArrayObjects;
00041     for(const auto &Cell : m_Cells) {
00042         if(Cell->GetCellNumber().second == Settings::GetSizeT("EventDisplay/RowToDraw")) {
00043             auto RadiatorObjects = Cell->DrawRadiatorGeometry();
00044             RadiatorArrayObjects.insert(RadiatorArrayObjects.end(),
00045                 std::make_move_iterator(RadiatorObjects.begin()),
00046                 std::make_move_iterator(RadiatorObjects.end()));
00047         }
00048     }
00049     return RadiatorArrayObjects;
00050 }
00051
00052 const RadiatorCell* BarrelRadiatorArray::FindRadiator(Particle &particle) const {
00053     if(!particle.IsAtRadiator()) {
00054         throw std::runtime_error("Cannot find radiator since track is not at radiator");
00055     }
00056 }
```

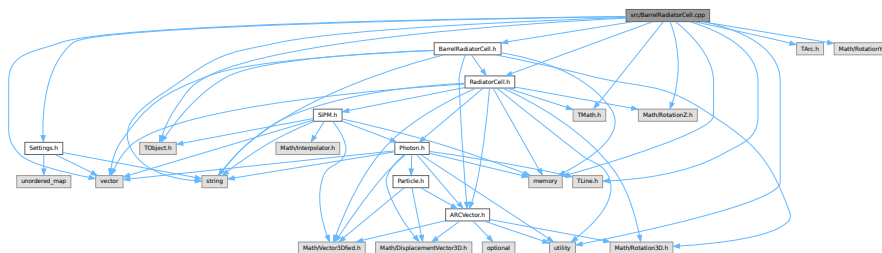
```

00055     }
00056     if(m_FullArray) {
00057         auto Position = particle.GetPosition().GlobalVector();
00058         if(TMath::Abs(Position.Phi()) > TMath::Pi()/2.0) {
00059             particle.MapPhi(TMath::Pi());
00060         }
00061         Position = particle.GetPosition().GlobalVector();
00062         double x = Position.Z();
00063         const double Radius = TMath::Sqrt(Position.X()*Position.X() +
00064             Position.Y()*Position.Y());
00065         // Check if particle hits outside half cell
00066         if(x > 8.5*m_xHexDist) {
00067             return nullptr;
00068         }
00069         // Account for curvature when calculating y coordinate (azimuthal)
00070         double ProjectedYAngle = TMath::ASin(Position.Y()/Radius);
00071         double y = m_BarrelRadius*ProjectedYAngle;
00072         // First check if track is on the correct side and not hitting endcap
00073         bool IsReflected = false;
00074         if(x < 0.0) {
00075             particle.ReflectZ();
00076             IsReflected = true;
00077             x = particle.GetPosition().GlobalVector().Z();
00078         }
00079         // Make sure track is not below the main row
00080         while(IsBelowMainRow(x, y)) {
00081             particle.MapPhi(+m_DeltaPhi);
00082             Position = particle.GetPosition().GlobalVector();
00083             ProjectedYAngle = TMath::ASin(Position.Y()/Radius);
00084             y = m_BarrelRadius*ProjectedYAngle;
00085         }
00086         // Make sure track is not above the upper row
00087         while(IsAboveUpperRow(x, y, m_BarrelRadius)) {
00088             particle.MapPhi(-m_DeltaPhi);
00089             Position = particle.GetPosition().GlobalVector();
00090             ProjectedYAngle = TMath::ASin(Position.Y()/Radius);
00091             y = m_BarrelRadius*ProjectedYAngle;
00092         }
00093         if(IsAboveMainRow(x, y)) {
00094             // Particle hits the upper row
00095             const std::size_t xIndex = static_cast<std::size_t>(x/m_xHexDist) + 1;
00096             const std::size_t yIndex = 2;
00097             return (*this)(xIndex, yIndex);
00098         } else {
00099             // Particle hits the main row
00100             const std::size_t xIndex = static_cast<std::size_t>((x + m_xHexDist/2.0)/m_xHexDist);
00101             const std::size_t yIndex = 1;
00102             if(xIndex == 0 && yIndex == 1 && IsReflected) {
00103                 particle.ReflectZ();
00104             }
00105             return (*this)(xIndex, yIndex);
00106         }
00107     } else {
00108         return m_Cells.begin()->get();
00109     }
00110 }
00111
00112 bool BarrelRadiatorArray::IsBelowMainRow(double x, double y) const {
00113     // Count half-hexagon lengths along x
00114     const std::size_t xUnits = static_cast<std::size_t>(x/(m_xHexDist/2.0));
00115     // Shift x coordinate
00116     const double x_shift = x - static_cast<double>(xUnits)*m_xHexDist/2.0;
00117     if(xUnits%2 == 0) {
00118         // If hexagon slope is upwards
00119         return x_shift > TMath::Sqrt(3)*y + m_xHexDist;
00120     } else {
00121         // If hexagon slope is downwards
00122         return x_shift < -m_xHexDist/2.0 - TMath::Sqrt(3)*y;
00123     }
00124 }
00125
00126 bool BarrelRadiatorArray::IsAboveUpperRow(double x, double y, double Radius) const {
00127     // The boundary on the top is just the boundary below, but shifted
00128     const double y_shift = y - m_DeltaPhi*Radius;
00129     return !IsBelowMainRow(x, y_shift);
00130 }
00131
00132 bool BarrelRadiatorArray::IsAboveMainRow(double x, double y) const {
00133     // Count half-hexagon lengths along x
00134     const std::size_t xUnits = static_cast<std::size_t>(x/(m_xHexDist/2.0));
00135     // Shift x coordinate
00136     const double x_shift = x - static_cast<double>(xUnits)*m_xHexDist/2.0;
00137     if(xUnits%2 == 0) {
00138         // If hexagon slope is downwards
00139         return x_shift > m_xHexDist - TMath::Sqrt(3)*y;
00140     } else {
00141         // If hexagon slope is upwards

```

7.59 src/BarrelRadiatorCell.cpp File Reference

Include dependency graph for BarrelRadiatorCell.cpp:



Typedefs

- using [RotationY](#) = ROOT::Math::RotationY

7.59.1 Typedef Documentation

7.59.1.1 RotationY

using [RotationY](#) = ROOT::Math::RotationY

Definition at line 17 of file [BarrelRadiatorCell.cpp](#).

7.60 BarrelRadiatorCell.cpp

[Go to the documentation of this file.](#)

```
00001 // Martin Duy Tat 6th September 2022
00002
00003 #include<vector>
00004 #include<memory>
00005 #include<utility>
00006 #include<string>
00007 #include"TArc.h"
00008 #include"TLLine.h"
00009 #include"TMATH.h"
00010 #include"Math/RotationY.h"
00011 #include"Math/RotationZ.h"
00012 #include"TOBJECT.h"
00013 #include"BarrelRadiatorCell.h"
00014 #include"RadiatorCell.h"
00015 #include"Settings.h"
00016
00017 using RotationY = ROOT::Math::RotationY;
00018 using RotationZ = ROOT::Math::RotationZ;
00019
00020 BarrelRadiatorCell::BarrelRadiatorCell(std::size_t CellColumnNumber,
00021                                         std::size_t CellRowNumber,
00022                                         double HexagonSize):
00023     RadiatorCell(CellColumnNumber,
00024                  CellRowNumber,
00025                  HexagonSize,
00026                  GetCellPosition(CellColumnNumber, CellRowNumber, HexagonSize),
00027                  GetCellOrientation(HexagonSize, CellRowNumber)),
00028     m_BarrelRadius(Settings::GetDouble("ARCGeometry/Radius")
00029                   + Settings::GetDouble("RadiatorCell/CoolingThickness")) {
00030 }
00031
00032 bool BarrelRadiatorCell::IsInsideCell(const ARCVector &Position) const {
00033     // Use global position to determine the radius
00034     const auto GlobalPosition = Position.GlobalVector();
00035     const double Radius = TMath::Sqrt(GlobalPosition.X()*GlobalPosition.X()
00036                                       + GlobalPosition.Y()*GlobalPosition.Y());
00037     // Get x and y coordinates after mapping everything to first quadrant
00038     const double x = TMath::Abs(Position.LocalVector().X());
00039     // Need to project y coordinate down to the barrel arc length
00040     const double ProjectedYAngle = TMath::ASin(Position.LocalVector().Y()/Radius);
00041     const double y = TMath::Abs(m_BarrelRadius*ProjectedYAngle);
00042     // First part is checking the sloped part, the other is the vertical part
00043     return x < std::min(m_HexagonSize - y*TMath::Sqrt(3.0), m_HexagonSize*0.5);
00044 }
00045
00046 std::vector<std::pair<std::unique_ptr<TOBJECT>, std::string>>
00047 BarrelRadiatorCell::DrawRadiatorGeometry() const {
00048     // First find the mirror intersections with the walls in local coordinates
00049     // by solving a quadratic
00050     const auto MirrorCentre = GetMirrorCentre();
00051     const double b = MirrorCentre.Z();
00052     const double c_left = TMath::Power(m_HexagonSize/2.0, 2)
00053                          + MirrorCentre.Mag2()
00054                          - m_MirrorCurvature*m_MirrorCurvature
00055                          + MirrorCentre.X()*m_HexagonSize;
00056     const double c_right = TMath::Power(m_HexagonSize/2.0, 2)
00057                           + MirrorCentre.Mag2()
```

```

00058         - m_MirrorCurvature*m_MirrorCurvature
00059         - MirrorCentre.X()*m_HexagonSize;
00060 const double s_left = TMath::Sqrt(b*b - c_left);
00061 const double s_right = TMath::Sqrt(b*b - c_right);
00062 const double SinLeft = s_left/m_MirrorCurvature;
00063 const double SinRight = s_right/m_MirrorCurvature;
00064 double LeftAngle = MirrorCentre.X() < -m_HexagonSize/2.0 ?
00065     TMath::ASin(SinLeft)*180.0/TMath::Pi() :
00066     180.0 - TMath::ASin(SinLeft)*180.0/TMath::Pi();
00067 double RightAngle = MirrorCentre.X() > m_HexagonSize/2.0 ?
00068     180.0 - TMath::ASin(SinRight)*180.0/TMath::Pi() :
00069     TMath::ASin(SinRight)*180.0/TMath::Pi();
00070 // Check if mirror intersects with the top wall
00071 const double bb = MirrorCentre.X();
00072 const double h = m_RadiatorThickness - 2.0*m_VesselThickness - m_CoolingThickness;
00073 const double cc = h*h + MirrorCentre.Mag2() - 2.0*MirrorCentre.Z()*h
00074     - m_MirrorCurvature*m_MirrorCurvature;
00075 const double s1 = bb + TMath::Sqrt(bb*bb - cc);
00076 const double s2 = bb - TMath::Sqrt(bb*bb - cc);
00077 if(bb*bb > cc &&
00078     (TMath::Abs(s1) < m_HexagonSize/2.0 || TMath::Abs(s2) < m_HexagonSize/2.0)) {
00079     const double s = TMath::Abs(s1) < m_HexagonSize/2.0 ? s1 : s2;
00080     const double MidAngle = TMath::ATan2(h - MirrorCentre.Z(),
00081         s - MirrorCentre.X())*180.0/TMath::Pi();
00082     if(MirrorCentre.X() < 0.0) {
00083         LeftAngle = MidAngle;
00084     } else {
00085         RightAngle = MidAngle;
00086     }
00087 }
00088 // Finally draw everything
00089 const auto ReversePhi = ReversePhiRotation();
00090 const auto MirrorCentreGlobal = ReversePhi(m_MirrorCentre.GlobalVector());
00091 TArc MirrorArc(MirrorCentreGlobal.Z(),
00092     MirrorCentreGlobal.X(),
00093     m_MirrorCurvature,
00094     RightAngle,
00095     LeftAngle);
00096 MirrorArc.SetLineColor(kBlack);
00097 MirrorArc.SetLineWidth(2);
00098 MirrorArc.SetFillColorAlpha(kWhite, 0.0);
00099 std::vector<std::pair<std::unique_ptr<TObject>, std::string>> Objects;
00100 Objects.push_back(std::make_pair(std::make_unique<TArc>(MirrorArc), "ONLY"));
00101 // Draw the walls
00102 TLine LeftLine(-m_HexagonSize/2.0 + m_Position.Z(),
00103     Settings::GetDouble("ARCGeometry/Radius") - m_VesselThickness,
00104     -m_HexagonSize/2.0 + m_Position.Z(),
00105     Settings::GetDouble("ARCGeometry/Radius")
00106     + m_RadiatorThickness - m_VesselThickness);
00107 LeftLine.SetLineColor(kBlack);
00108 Objects.push_back(std::make_pair(std::make_unique<TLine>(LeftLine), ""));
00109 TLine RightLine(m_HexagonSize/2.0 + m_Position.Z(),
00110     Settings::GetDouble("ARCGeometry/Radius") - m_VesselThickness,
00111     m_HexagonSize/2.0 + m_Position.Z(),
00112     Settings::GetDouble("ARCGeometry/Radius")
00113     + m_RadiatorThickness - m_VesselThickness);
00114 RightLine.SetLineColor(kBlack);
00115 Objects.push_back(std::make_pair(std::make_unique<TLine>(RightLine), ""));
00116 // Draw the detector plane
00117 TLine DetectorLine(-m_HexagonSize/2.0 + m_Position.Z(),
00118     Settings::GetDouble("ARCGeometry/Radius") + m_CoolingThickness,
00119     m_HexagonSize/2.0 + m_Position.Z(),
00120     Settings::GetDouble("ARCGeometry/Radius") + m_CoolingThickness);
00121 DetectorLine.SetLineColor(kBlack);
00122 Objects.push_back(std::make_pair(std::make_unique<TLine>(DetectorLine), ""));
00123 // Draw the aerogel plane
00124 TLine AerogelLine(-m_HexagonSize/2.0 + m_Position.Z(),
00125     Settings::GetDouble("ARCGeometry/Radius")
00126     + m_CoolingThickness + m_AerogelThickness,
00127     m_HexagonSize/2.0 + m_Position.Z(),
00128     Settings::GetDouble("ARCGeometry/Radius")
00129     + m_CoolingThickness + m_AerogelThickness);
00130 AerogelLine.SetLineColor(kBlack);
00131 Objects.push_back(std::make_pair(std::make_unique<TLine>(AerogelLine), ""));
00132 // Draw the bottom cooling plane
00133 TLine CoolingLine(-m_HexagonSize/2.0 + m_Position.Z(),
00134     Settings::GetDouble("ARCGeometry/Radius"),
00135     m_HexagonSize/2.0 + m_Position.Z(),
00136     Settings::GetDouble("ARCGeometry/Radius"));
00137 CoolingLine.SetLineColor(kBlack);
00138 Objects.push_back(std::make_pair(std::make_unique<TLine>(CoolingLine), ""));
00139 // Draw SiPM
00140 const auto RadiatorPosition = ReversePhi(GetRadiatorPosition());
00141 Objects.push_back(std::make_pair(m_Detector.DrawSiPM(RadiatorPosition), ""));
00142 return Objects;
00143 }
00144

```

```

00145 RotationZ BarrelRadiatorCell::ReversePhiRotation() const {
00146     if(m_CellNumber.second == 1) {
00147         return RotationZ(0.0);
00148     } else {
00149         const double DeltaPhi = 0.5*TMath::Sqrt(3)*m_HexagonSize/m_BarrelRadius;
00150         return RotationZ(-DeltaPhi);
00151     }
00152 }
00153
00154 bool BarrelRadiatorCell::IsDetectorInsideCell() const {
00155     const double Position = m_Detector.GetDetectorXPosition();
00156     const double Tilt = m_Detector.GetDetectorTilt();
00157     const double DetSize = m_Detector.GetDetectorSizeX();
00158     if(Position + DetSize*TMath::Cos(Tilt)/2.0 > m_HexagonSize/2.0) {
00159         return false;
00160     } else if(Position - DetSize*TMath::Cos(Tilt)/2.0 < -m_HexagonSize/2.0) {
00161         return false;
00162     } else {
00163         return true;
00164     }
00165 }
00166
00167 Vector BarrelRadiatorCell::GetCellPosition(std::size_t CellColumnNumber,
00168     std::size_t CellRowNumber,
00169     double HexagonSize) {
00170     if(CellColumnNumber > 9) {
00171         throw std::invalid_argument("Invalid cell column number: "
00172             + std::to_string(CellColumnNumber));
00173     }
00174     const double XPosition = Settings::GetDouble("ARCGeometry/Radius")
00175         + Settings::GetDouble("RadiatorCell/CoolingThickness");
00176     if(CellRowNumber == 0 && CellColumnNumber == 0) {
00177         return Vector(XPosition, 0.0, 0.0);
00178     }
00179     if(CellRowNumber == 1) {
00180         const double ZPosition = HexagonSize*static_cast<double>(CellColumnNumber);
00181         return Vector(XPosition, 0.0, ZPosition);
00182     } else if(CellRowNumber == 2) {
00183         const double ZPosition = HexagonSize*(static_cast<double>(CellColumnNumber) - 0.5);
00184         const double BarrelRadius = Settings::GetDouble("ARCGeometry/Radius")
00185             + Settings::GetDouble("RadiatorCell/CoolingThickness");
00186         const double DeltaPhi = 0.5*TMath::Sqrt(3)*HexagonSize/BarrelRadius;
00187         const ROOT::Math::RotationZ PhiRotation(DeltaPhi);
00188         return PhiRotation(Vector(XPosition, 0.0, ZPosition));
00189     } else {
00190         throw std::invalid_argument("Invalid cell row number: "
00191             + std::to_string(CellRowNumber));
00192     }
00193 }
00194
00195 Rotation3D BarrelRadiatorCell::GetCellOrientation(double HexagonSize,
00196     std::size_t CellRowNumber) {
00197     const RotationY Rotation1(-TMath::Pi()/2.0);
00198     const RotationZ Rotation2(-TMath::Pi());
00199     if(CellRowNumber == 1) {
00200         return Rotation2*Rotation1;
00201     } else {
00202         const double BarrelRadius = Settings::GetDouble("ARCGeometry/Radius")
00203             + Settings::GetDouble("RadiatorCell/CoolingThickness");
00204         const double DeltaPhi = 0.5*TMath::Sqrt(3)*HexagonSize/BarrelRadius;
00205         const ROOT::Math::RotationZ Rotation0(-DeltaPhi);
00206         return Rotation2*Rotation1*Rotation0;
00207     }
00208 }
00209
00210 bool BarrelRadiatorCell::IsEdgeCell() const {
00211     return (m_CellNumber == std::make_pair(std::size_t{8}, std::size_t{1}) ||
00212         m_CellNumber == std::make_pair(std::size_t{9}, std::size_t{2}));
00213 }

```

7.61 apps/CMakeLists.txt File Reference

7.62 src/CMakeLists.txt File Reference

7.63 src/EndCapRadiatorArray.cpp File Reference

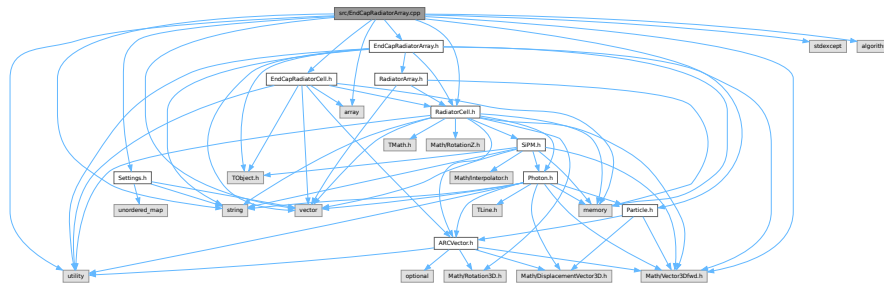
```

#include <vector>
#include <stdexcept>

```

```
#include <string>
#include <utility>
#include <algorithm>
#include <array>
#include "Math/Vector3Dfwd.h"
#include "EndCapRadiatorArray.h"
#include "RadiatorCell.h"
#include "EndCapRadiatorCell.h"
#include "Settings.h"
#include "Particle.h"
```

Include dependency graph for EndCapRadiatorArray.cpp:



7.64 EndCapRadiatorArray.cpp

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 23rd June 2022
00002
00003 #include<vector>
00004 #include<stdexcept>
00005 #include<string>
00006 #include<utility>
00007 #include<algorithm>
00008 #include<array>
00009 #include"Math/Vector3Dfwd.h"
00010 #include"EndCapRadiatorArray.h"
00011 #include"RadiatorCell.h"
00012 #include"EndCapRadiatorCell.h"
00013 #include"Settings.h"
00014 #include"Particle.h"
00015
00016 EndCapRadiatorArray::EndCapRadiatorArray():
00017     RadiatorArray::RadiatorArray(),
00018     m_InnerRadius(Settings::GetDouble("ARCGeometry/EndCapInnerRadius")),
00019     m_OuterRadius(Settings::GetDouble("ARCGeometry/EndCapOuterRadius")) {
00020     constexpr std::size_t NumberCells = EndCapRadiatorCell::m_ValidCells.size();
00021     m_Cells.reserve(NumberCells);
00022     for(std::size_t i = 0; i < NumberCells; i++) {
00023         const std::size_t CellColumnNumber = EndCapRadiatorCell::m_ValidCells[i].first;
00024         const std::size_t CellRowNumber = EndCapRadiatorCell::m_ValidCells[i].second;
00025         m_Cells.emplace_back(std::make_unique<EndCapRadiatorCell>(CellColumnNumber,
00026             CellRowNumber,
00027             m_xHexDist));
00028     }
00029 }
00030
00031 std::vector<std::pair<std::unique_ptr<TObject>, std::string>>
00032 EndCapRadiatorArray::DrawRadiatorArray() const {
00033     std::vector<std::pair<std::unique_ptr<TObject>, std::string>> RadiatorArrayObjects;
00034     for(const auto &Cell : m_Cells) {
00035         if(Cell->GetCellNumber().second == Settings::GetSizeT("EventDisplay/RowToDraw")) {
00036             auto RadiatorObjects = Cell->DrawRadiatorGeometry();
00037             RadiatorArrayObjects.insert(RadiatorArrayObjects.end(),
00038                 std::make_move_iterator(RadiatorObjects.begin()),
00039                 std::make_move_iterator(RadiatorObjects.end()));
00040         }
00041     }
00042     return RadiatorArrayObjects;

```

```

00043 }
00044
00045 const RadiatorCell* EndCapRadiatorArray::FindRadiator(Particle &particle) const {
00046     if(!particle.IsAtRadiator()) {
00047         throw std::runtime_error("Cannot find radiator since track is not at entrance window");
00048     }
00049     const auto GlobalPosition = particle.GetPosition().GlobalVector();
00050     const double Radius = TMath::Sqrt(GlobalPosition.X()*GlobalPosition.X() +
00051         GlobalPosition.Y()*GlobalPosition.Y());
00052     if(Radius < m_InnerRadius || Radius > m_OuterRadius) {
00053         return nullptr;
00054     }
00055     // If track hits the other end cap, reflect before the polar angle cut
00056     if(particle.GetPosition().GlobalVector().Z() < 0.0) {
00057         particle.ReflectZ();
00058     }
00059     const double CosTheta_min = Settings::GetDouble("ARCGeometry/CosTheta_boundary");
00060     const double Theta = particle.GetPosition().GlobalVector().Theta();
00061     if(TMath::Cos(Theta) < CosTheta_min) {
00062         return nullptr;
00063     }
00064     // If track hits the lower half, rotate by 180 degrees
00065     if(particle.GetPosition().GlobalVector().Phi() < 0.0) {
00066         particle.MapPhi(TMath::Pi());
00067     }
00068     // Rotate by 60 degrees until the track is in the interval [0, 60] degrees
00069     while(particle.GetPosition().GlobalVector().Phi() > TMath::Pi()/3.0) {
00070         particle.MapPhi(-TMath::Pi()/3.0);
00071     }
00072     bool IsReflected = false;
00073     // Reflect if particle is on the other side of the symmetry line
00074     if(particle.GetPosition().GlobalVector().Phi() > TMath::Pi()/6.0) {
00075         particle.MapPhi(-TMath::Pi()/6.0);
00076         particle.ReflectY();
00077         particle.MapPhi(TMath::Pi()/6.0);
00078         IsReflected = true;
00079     }
00080     // Loop from the bottom row until we find the correct row
00081     const double x = particle.GetPosition().GlobalVector().X();
00082     const double y = particle.GetPosition().GlobalVector().Y();
00083     std::size_t CellRowNumber = 1;
00084     while(true) {
00085         if(IsBelowThisRow(CellRowNumber + 1, x, y)) {
00086             break;
00087         } else {
00088             CellRowNumber++;
00089         }
00090     }
00091     std::size_t CellColumnNumber;
00092     if(CellRowNumber%2 == 1) {
00093         CellColumnNumber = static_cast<std::size_t>((x + m_xHexDist/2.0)/m_xHexDist);
00094     } else {
00095         CellColumnNumber = static_cast<std::size_t>(x/m_xHexDist) + 1;
00096     }
00097     if(IsReflected) {
00098         constexpr std::array<std::pair<std::size_t, std::size_t>, 4> ReflectCells{{
00099             {2, 2}, {3, 3}, {5, 4}, {6, 5}}};
00100         if(CellRowNumber == 1) {
00101             particle.ReflectY();
00102         } else if(std::find(ReflectCells.begin(),
00103             ReflectCells.end(),
00104             std::make_pair(CellColumnNumber, CellRowNumber))
00105             != ReflectCells.end()) {
00106             particle.MapPhi(-TMath::Pi()/6.0);
00107             particle.ReflectY();
00108             particle.MapPhi(TMath::Pi()/6.0);
00109         }
00110     }
00111     return (*this)(CellColumnNumber, CellRowNumber);
00112 }
00113
00114 bool EndCapRadiatorArray::IsBelowThisRow(std::size_t Row, double x, double y) const {
00115     // Shift y coordinate so that we're comparing with the main row
00116     const double y_shift = y - static_cast<double>(Row - 1)*m_yHexDist;
00117     if(Row%2 == 1) {
00118         return IsBelowOddRow(x, y_shift);
00119     } else {
00120         return IsBelowEvenRow(x, y_shift);
00121     }
00122 }
00123
00124 bool EndCapRadiatorArray::IsBelowOddRow(double x, double y) const {
00125     // Count half-hexagon lengths along x
00126     const int xUnits = static_cast<int>(x/(m_xHexDist/2.0));
00127     // Shift x coordinate
00128     const double x_shift = x - static_cast<double>(xUnits)*m_xHexDist/2.0;
00129     if(xUnits%2 == 0) {

```

```

00130     // If hexagon slope is upwards
00131     return x_shift > TMath::Sqrt(3)*y + m_xHexDist;
00132   } else {
00133     // If hexagon slope is downwards
00134     return x_shift < -m_xHexDist/2.0 - TMath::Sqrt(3)*y;
00135   }
00136 }
00137
00138 bool EndCapRadiatorArray::IsBelowEvenRow(double x, double y) const {
00139   // Count half-hexagon lengths along x
00140   const int xUnits = static_cast<int>(x/(m_xHexDist/2.0));
00141   // Shift x coordinate
00142   const double x_shift = x - static_cast<double>(xUnits)*m_xHexDist/2.0;
00143   if(xUnits%2 == 0) {
00144     // If hexagon slope is downwards
00145     return x_shift < -m_xHexDist/2.0 - TMath::Sqrt(3)*y;
00146   } else {
00147     // If hexagon slope is upwards
00148     return x_shift > TMath::Sqrt(3)*y + m_xHexDist;
00149   }
00150 }
00151
00152 int EndCapRadiatorArray::FindRadiatorIndex(std::size_t i, std::size_t j) const {
00153   const auto iter = std::find(EndCapRadiatorCell::m_ValidCells.begin(),
00154                               EndCapRadiatorCell::m_ValidCells.end(),
00155                               std::make_pair(i, j));
00156   if(iter == EndCapRadiatorCell::m_ValidCells.end()) {
00157     const auto iter2 = std::find(EndCapRadiatorCell::m_NotValidCells.begin(),
00158                                 EndCapRadiatorCell::m_NotValidCells.end(),
00159                                 std::make_pair(i, j));
00160     if(iter2 != EndCapRadiatorCell::m_NotValidCells.end()) {
00161       return -1;
00162     } else {
00163       throw std::invalid_argument("Radiator ( "
00164                                   + std::to_string(i)
00165                                   + ", "
00166                                   + std::to_string(j)
00167                                   + ") does not exist");
00168     }
00169   } else {
00170     const auto Index = iter - EndCapRadiatorCell::m_ValidCells.begin();
00171     return static_cast<int>(Index);
00172   }
00173 }

```

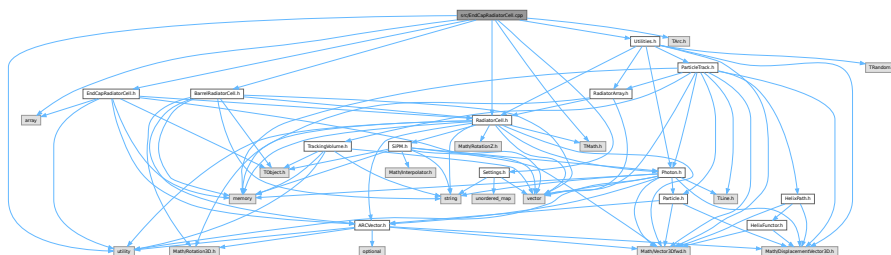
7.65 src/EndCapRadiatorCell.cpp File Reference

```

#include <array>
#include <utility>
#include "TMath.h"
#include "TArc.h"
#include "EndCapRadiatorCell.h"
#include "RadiatorCell.h"
#include "Settings.h"
#include "BarrelRadiatorCell.h"
#include "Utilities.h"

```

Include dependency graph for EndCapRadiatorCell.cpp:



7.66 EndCapRadiatorCell.cpp

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 1st September 2022
00002
00003 #include<array>
00004 #include<utility>
00005 #include"TMath.h"
00006 #include"TArc.h"
00007 #include"EndCapRadiatorCell.h"
00008 #include"RadiatorCell.h"
00009 #include"Settings.h"
00010 #include"BarrelRadiatorCell.h"
00011 #include"Utilities.h"
00012
00013 EndCapRadiatorCell::EndCapRadiatorCell(std::size_t CellColumnNumber,
00014                                         std::size_t CellRowNumber,
00015                                         double HexagonSize):
00016     RadiatorCell(CellColumnNumber,
00017                 CellRowNumber,
00018                 HexagonSize,
00019                 GetCellPosition(CellColumnNumber, CellRowNumber, HexagonSize),
00020                 GetCellOrientation(CellColumnNumber, CellRowNumber, HexagonSize),
00021                 "EndCap"),
00022     m_InnerRadius(Settings::GetDouble("ARCGeometry/EndCapInnerRadius")),
00023     m_OuterRadius(Settings::GetDouble("ARCGeometry/EndCapOuterRadius")) {
00024 }
00025
00026 bool EndCapRadiatorCell::IsInsideCell(const ARCVector &Position) const {
00027     // First check if particle is within the radial acceptance
00028     if(!IsInsideRadialAcceptance(Position.GlobalVector())) {
00029         return false;
00030     }
00031     // Rotate coordinate system to align with hexagon
00032     const double Angle = GetCoordinateRotation(m_CellNumber.first,
00033                                             m_CellNumber.second,
00034                                             m_HexagonSize);
00035     const RotationZ BackRotation(Angle);
00036     const Vector PositionVec = BackRotation(Position.LocalVector());
00037     return IsInsideCell(PositionVec);
00038 }
00039
00040 bool EndCapRadiatorCell::IsInsideCell(const Vector &Position) const {
00041     // Get x and y coordinates after mapping everything to first quadrant
00042     const double x = TMath::Abs(Position.X());
00043     const double y = TMath::Abs(Position.Y());
00044     // First part is checking the sloped part, the other is the vertical part
00045     return x < std::min(m_HexagonSize - y*TMath::Sqrt(3.0), m_HexagonSize*0.5);
00046 }
00047
00048 std::vector<std::pair<std::unique_ptr<TObject>, std::string>>
00049 EndCapRadiatorCell::DrawRadiatorGeometry() const {
00050     // First find the mirror intersections with the walls in local coordinates
00051     // by solving a quadratic
00052     const auto MirrorCentre = GetMirrorCentre();
00053     const double b = MirrorCentre.Z();
00054     const double c_left = TMath::Power(m_HexagonSize/2.0, 2)
00055                         + MirrorCentre.Mag2()
00056                         - m_MirrorCurvature*m_MirrorCurvature
00057                         - MirrorCentre.X()*m_HexagonSize;
00058     const double c_right = TMath::Power(m_HexagonSize/2.0, 2)
00059                          + MirrorCentre.Mag2()
00060                          - m_MirrorCurvature*m_MirrorCurvature
00061                          + MirrorCentre.X()*m_HexagonSize;
00062     const double s_left = TMath::Sqrt(b*b - c_left);
00063     const double s_right = TMath::Sqrt(b*b - c_right);
00064     const double SinLeft = s_left/m_MirrorCurvature;
00065     const double SinRight = s_right/m_MirrorCurvature;
00066     double LeftAngle = -MirrorCentre.X() < -m_HexagonSize/2.0 ?
00067         TMath::ASin(SinLeft)*180.0/TMath::Pi() :
00068         180.0 - TMath::ASin(SinLeft)*180.0/TMath::Pi();
00069     double RightAngle = -MirrorCentre.X() > m_HexagonSize/2.0 ?
00070         180.0 - TMath::ASin(SinRight)*180.0/TMath::Pi() :
00071         TMath::ASin(SinRight)*180.0/TMath::Pi();
00072     // Check if mirror intersects with the top wall
00073     const double bb = -MirrorCentre.X();
00074     const double h = m_RadiatorThickness - 2.0*m_VesselThickness - m_CoolingThickness;
00075     const double cc = h*h + MirrorCentre.Mag2() - 2.0*MirrorCentre.Z()*h
00076                     - m_MirrorCurvature*m_MirrorCurvature;
00077     if(bb*bb > cc) {
00078         const double s1 = bb + TMath::Sqrt(bb*bb - cc);
00079         const double s2 = bb - TMath::Sqrt(bb*bb - cc);
00080         if(TMath::Abs(s1) < m_HexagonSize/2.0 || TMath::Abs(s2) < m_HexagonSize/2.0) {
00081             const double s = TMath::Abs(s1) < m_HexagonSize/2.0 ? s1 : s2;
00082             const double MidAngle = TMath::ATan2(h - MirrorCentre.Z(),

```

```

00083         s + MirrorCentre.X())*180.0/TMath::Pi();
00084     if(MirrorCentre.X() > 0.0) {
00085         LeftAngle = MidAngle;
00086     } else {
00087         RightAngle = MidAngle;
00088     }
00089 }
00090 }
00091 // Finally draw everything
00092 const auto MirrorCentreGlobal = m_MirrorCentre.GlobalVector();
00093 TArc MirrorArc(MirrorCentreGlobal.Z(),
00094               MirrorCentreGlobal.X(),
00095               m_MirrorCurvature,
00096               RightAngle - 90.0,
00097               LeftAngle - 90.0);
00098 MirrorArc.SetLineColor(kBlack);
00099 MirrorArc.SetLineWidth(2);
00100 MirrorArc.SetFillColorAlpha(kWhite, 0.0);
00101 std::vector<std::pair<std::unique_ptr<TObject>, std::string>> Objects;
00102 Objects.push_back(std::make_pair(std::make_unique<TArc>(MirrorArc), "ONLY"));
00103 // Draw the walls
00104 TLine LeftLine(Settings::GetDouble("ARCGeometry/BarrelZ") - m_VesselThickness,
00105               -m_HexagonSize/2.0 + m_Position.X(),
00106               Settings::GetDouble("ARCGeometry/BarrelZ")
00107               + m_RadiatorThickness - m_VesselThickness,
00108               -m_HexagonSize/2.0 + m_Position.X());
00109 LeftLine.SetLineColor(kBlack);
00110 Objects.push_back(std::make_pair(std::make_unique<TLine>(LeftLine), ""));
00111 TLine RightLine(Settings::GetDouble("ARCGeometry/BarrelZ") - m_VesselThickness,
00112                m_HexagonSize/2.0 + m_Position.X(),
00113                Settings::GetDouble("ARCGeometry/BarrelZ")
00114                + m_RadiatorThickness - m_VesselThickness,
00115                m_HexagonSize/2.0 + m_Position.X());
00116 RightLine.SetLineColor(kBlack);
00117 Objects.push_back(std::make_pair(std::make_unique<TLine>(RightLine), ""));
00118 // Draw the detector plane
00119 TLine DetectorLine(Settings::GetDouble("ARCGeometry/BarrelZ") + m_CoolingThickness,
00120                   -m_HexagonSize/2.0 + m_Position.X(),
00121                   Settings::GetDouble("ARCGeometry/BarrelZ") + m_CoolingThickness,
00122                   m_HexagonSize/2.0 + m_Position.X());
00123 DetectorLine.SetLineColor(kBlack);
00124 Objects.push_back(std::make_pair(std::make_unique<TLine>(DetectorLine), ""));
00125 // Draw the aerogel plane
00126 TLine AerogelLine(Settings::GetDouble("ARCGeometry/BarrelZ")
00127                  + m_CoolingThickness + m_AerogelThickness,
00128                  -m_HexagonSize/2.0 + m_Position.X(),
00129                  Settings::GetDouble("ARCGeometry/BarrelZ")
00130                  + m_CoolingThickness + m_AerogelThickness,
00131                  m_HexagonSize/2.0 + m_Position.X());
00132 AerogelLine.SetLineColor(kBlack);
00133 Objects.push_back(std::make_pair(std::make_unique<TLine>(AerogelLine), ""));
00134 // Draw the bottom cooling plane
00135 TLine CoolingLine(Settings::GetDouble("ARCGeometry/BarrelZ"),
00136                  -m_HexagonSize/2.0 + m_Position.X(),
00137                  Settings::GetDouble("ARCGeometry/BarrelZ"),
00138                  m_HexagonSize/2.0 + m_Position.X());
00139 CoolingLine.SetLineColor(kBlack);
00140 Objects.push_back(std::make_pair(std::make_unique<TLine>(CoolingLine), ""));
00141 // Draw SiPM
00142 auto DrawnDetector = m_Detector.DrawSiPM(GetRadiatorPosition());
00143 Objects.push_back(std::make_pair(std::move(DrawnDetector), ""));
00144 return Objects;
00145 }
00146
00147 bool EndCapRadiatorCell::IsDetectorInsideCell() const {
00148     // Position of detector centre in local (radially oriented) coordinates
00149     const double x = m_Detector.GetDetectorXPosition();
00150     const Vector Position(x, 0.0, 0.0);
00151     // Rotation matrix back to lab frame orientation
00152     const RotationZ BackRotation(-GetCoordinateRotation(m_CellNumber.first,
00153                                                         m_CellNumber.second,
00154                                                         m_HexagonSize));
00155     // Detector tilt and size
00156     const double Tilt = m_Detector.GetDetectorTilt();
00157     const double DetSizeX = m_Detector.GetDetectorSizeX();
00158     const double DetSizeY = m_Detector.GetDetectorSizeY();
00159     // Function for checking if point is inside hexagon
00160     auto IsInsideHexagon = [this] (double x, double y) {
00161         x = TMath::Abs(x);
00162         y = TMath::Abs(y);
00163         return x < std::min(m_HexagonSize - y*TMath::Sqrt(3.0), m_HexagonSize*0.5);
00164     };
00165     // Top right corner
00166     const auto Corner1_Pos = Position +
00167                               Vector(DetSizeX*TMath::Cos(Tilt)/2.0, DetSizeY/2.0, 0.0);
00168     const auto RotatedCorner1 = BackRotation(Corner1_Pos);
00169     if(!IsInsideRadialAcceptance(m_Position + RotatedCorner1)) {

```

```

00170     return false;
00171 }
00172 if(!IsInsideHexagon(RotatedCorner1.X(), RotatedCorner1.Y())) {
00173     return false;
00174 }
00175 // Top left corner
00176 const auto Corner2_Pos = Position +
00177     Vector(-DetSizeX*TMath::Cos(Tilt)/2.0, DetSizeY/2.0, 0.0);
00178 const auto RotatedCorner2 = BackRotation(Corner2_Pos);
00179 if(!IsInsideRadialAcceptance(m_Position + RotatedCorner2)) {
00180     return false;
00181 }
00182 if(!IsInsideHexagon(RotatedCorner2.X(), RotatedCorner2.Y())) {
00183     return false;
00184 }
00185 // Bottom left corner
00186 const auto Corner3_Pos = Position +
00187     Vector(-DetSizeX*TMath::Cos(Tilt)/2.0, -DetSizeY/2.0, 0.0);
00188 const auto RotatedCorner3 = BackRotation(Corner3_Pos);
00189 if(!IsInsideRadialAcceptance(m_Position + RotatedCorner3)) {
00190     return false;
00191 }
00192 if(!IsInsideHexagon(RotatedCorner3.X(), RotatedCorner3.Y())) {
00193     return false;
00194 }
00195 // Bottom right corner
00196 const auto Corner4_Pos = Position +
00197     Vector(DetSizeX*TMath::Cos(Tilt)/2.0, -DetSizeY/2.0, 0.0);
00198 const auto RotatedCorner4 = BackRotation(Corner4_Pos);
00199 if(!IsInsideRadialAcceptance(m_Position + RotatedCorner4)) {
00200     return false;
00201 }
00202 if(!IsInsideHexagon(RotatedCorner4.X(), RotatedCorner4.Y())) {
00203     return false;
00204 }
00205 return true;
00206 }
00207
00208 Vector EndCapRadiatorCell::GetCellPosition(std::size_t CellColumnNumber,
00209     std::size_t CellRowNumber,
00210     double HexagonSize) {
00211     if(std::find(m_ValidCells.begin(),
00212         m_ValidCells.end(),
00213         std::make_pair(CellColumnNumber, CellRowNumber)) ==
00214         m_ValidCells.end()) {
00215         throw std::invalid_argument("Invalid cell column number: "
00216             + std::to_string(CellColumnNumber));
00217     }
00218     const double ZPosition = Settings::GetDouble("ARCGeometry/BarrelZ")
00219         + Settings::GetDouble("RadiatorCell/CoolingThickness");
00220     const double YPosition = HexagonSize*(TMath::Sqrt(3)/2.0)*
00221         static_cast<double>(CellRowNumber - 1);
00222     if(CellRowNumber%2 == 1) {
00223         const double XPosition = HexagonSize*static_cast<double>(CellColumnNumber);
00224         return Vector(XPosition, YPosition, ZPosition);
00225     } else {
00226         const double XPosition = HexagonSize*(static_cast<double>(CellColumnNumber) - 0.5);
00227         return Vector(XPosition, YPosition, ZPosition);
00228     }
00229 }
00230
00231 Rotation3D EndCapRadiatorCell::GetCellOrientation(std::size_t CellColumnNumber,
00232     std::size_t CellRowNumber,
00233     double HexagonSize) {
00234     const double Angle = GetCoordinateRotation(CellColumnNumber,
00235         CellRowNumber,
00236         HexagonSize);
00237     return Rotation3D(RotationZ(-Angle));
00238 }
00239
00240 double EndCapRadiatorCell::GetCoordinateRotation(std::size_t CellColumnNumber,
00241     std::size_t CellRowNumber,
00242     double HexagonSize) {
00243     const double YPosition = HexagonSize*(TMath::Sqrt(3)/2.0)*
00244         static_cast<double>(CellRowNumber - 1);
00245     if(CellRowNumber%2 == 1) {
00246         const double XPosition = HexagonSize*static_cast<double>(CellColumnNumber);
00247         return TMath::ATan2(YPosition, XPosition);
00248     } else {
00249         const double XPosition = HexagonSize*(static_cast<double>(CellColumnNumber) - 0.5);
00250         return TMath::ATan2(YPosition, XPosition);
00251     }
00252 }
00253
00254 bool EndCapRadiatorCell::IsInsideRadialAcceptance(const Vector &Position) const {
00255     const double Radius = TMath::Sqrt(Position.X()*Position.X() +
00256         Position.Y()*Position.Y());

```

```

00257     return Radius >= m_InnerRadius && Radius <= m_OuterRadius;
00258 }
00259
00260 bool EndCapRadiatorCell::IsEdgeCell() const {
00261     return false;
00262 }

```

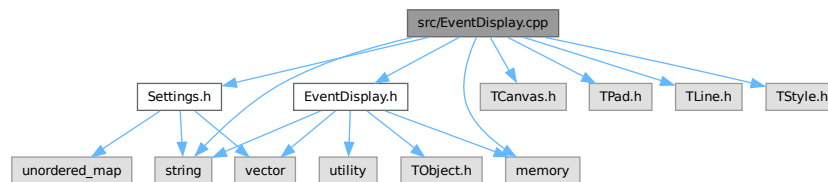
7.67 src/EventDisplay.cpp File Reference

```

#include <string>
#include <memory>
#include "TCanvas.h"
#include "TPad.h"
#include "TLine.h"
#include "TStyle.h"
#include "EventDisplay.h"
#include "Settings.h"

```

Include dependency graph for EventDisplay.cpp:



7.68 EventDisplay.cpp

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 27th May 2022
00002
00003 #include<string>
00004 #include<memory>
00005 #include "TCanvas.h"
00006 #include "TPad.h"
00007 #include "TLine.h"
00008 #include "TStyle.h"
00009 #include "EventDisplay.h"
00010 #include "Settings.h"
00011
00012 void EventDisplay::DrawEventDisplay() {
00013     gStyle->SetLineScalePS(0.01f);
00014     TCanvas c("c", "", Settings::GetInt("EventDisplay/CanvasWidth"),
00015             Settings::GetInt("EventDisplay/CanvasHeight"));
00016     gPad->DrawFrame(-Settings::GetDouble("ARCGeometry/Length")/2.0,
00017             0.0,
00018             Settings::GetDouble("ARCGeometry/Length")/2.0,
00019             Settings::GetDouble("ARCGeometry/Radius")
00020             + Settings::GetDouble("RadiatorCell/RadiatorThickness")
00021             - Settings::GetDouble("RadiatorCell/VesselThickness"));
00022     TLine InnerARC(-Settings::GetDouble("ARCGeometry/Length")/2.0,
00023             Settings::GetDouble("ARCGeometry/Radius")
00024             - Settings::GetDouble("RadiatorCell/VesselThickness"),
00025             Settings::GetDouble("ARCGeometry/Length")/2.0,
00026             Settings::GetDouble("ARCGeometry/Radius")
00027             - Settings::GetDouble("RadiatorCell/VesselThickness"));
00028     TLine OuterWall(-Settings::GetDouble("ARCGeometry/Length")/2.0,
00029             Settings::GetDouble("ARCGeometry/Radius")
00030             + Settings::GetDouble("RadiatorCell/RadiatorThickness")
00031             - 2*Settings::GetDouble("RadiatorCell/VesselThickness"),
00032             Settings::GetDouble("ARCGeometry/Length")/2.0,

```

```

00033         Settings::GetDouble("ARCGeometry/Radius")
00034         + Settings::GetDouble("RadiatorCell/RadiatorThickness")
00035         - 2*Settings::GetDouble("RadiatorCell/VesselThickness"));
00036 TLine OuterARC(-Settings::GetDouble("ARCGeometry/Length")/2.0,
00037 Settings::GetDouble("ARCGeometry/Radius")
00038 + Settings::GetDouble("RadiatorCell/RadiatorThickness")
00039 - Settings::GetDouble("RadiatorCell/VesselThickness"),
00040 Settings::GetDouble("ARCGeometry/Length")/2.0,
00041 Settings::GetDouble("ARCGeometry/Radius")
00042 + Settings::GetDouble("RadiatorCell/RadiatorThickness")
00043 - Settings::GetDouble("RadiatorCell/VesselThickness"));
00044 InnerARC.SetLineColor(kBlack);
00045 OuterWall.SetLineColor(kBlack);
00046 OuterARC.SetLineColor(kBlack);
00047 TLine InnerEndCap(Settings::GetDouble("ARCGeometry/BarrelZ")
00048 - Settings::GetDouble("RadiatorCell/VesselThickness"),
00049 0.0,
00050 Settings::GetDouble("ARCGeometry/BarrelZ")
00051 - Settings::GetDouble("RadiatorCell/VesselThickness"),
00052 Settings::GetDouble("ARCGeometry/Radius"));
00053 TLine OuterWallEndCap(Settings::GetDouble("ARCGeometry/BarrelZ")
00054 + Settings::GetDouble("RadiatorCell/RadiatorThickness")
00055 - 2*Settings::GetDouble("RadiatorCell/VesselThickness"),
00056 0.0,
00057 Settings::GetDouble("ARCGeometry/BarrelZ")
00058 + Settings::GetDouble("RadiatorCell/RadiatorThickness")
00059 - 2*Settings::GetDouble("RadiatorCell/VesselThickness"),
00060 Settings::GetDouble("ARCGeometry/Radius"));
00061 TLine OuterEndCap(Settings::GetDouble("ARCGeometry/BarrelZ")
00062 + Settings::GetDouble("RadiatorCell/RadiatorThickness")
00063 - Settings::GetDouble("RadiatorCell/VesselThickness"),
00064 0.0,
00065 Settings::GetDouble("ARCGeometry/BarrelZ")
00066 + Settings::GetDouble("RadiatorCell/RadiatorThickness")
00067 - Settings::GetDouble("RadiatorCell/VesselThickness"),
00068 Settings::GetDouble("ARCGeometry/Radius"));
00069 InnerEndCap.SetLineColor(kBlack);
00070 OuterWallEndCap.SetLineColor(kBlack);
00071 OuterEndCap.SetLineColor(kBlack);
00072 m_EventObjects.push_back(std::make_pair(std::make_unique<TLine>(InnerARC), ""));
00073 m_EventObjects.push_back(std::make_pair(std::make_unique<TLine>(OuterWall), ""));
00074 m_EventObjects.push_back(std::make_pair(std::make_unique<TLine>(OuterARC), ""));
00075 m_EventObjects.push_back(std::make_pair(std::make_unique<TLine>(InnerEndCap), ""));
00076 m_EventObjects.push_back(std::make_pair(std::make_unique<TLine>(OuterWallEndCap), ""));
00077 m_EventObjects.push_back(std::make_pair(std::make_unique<TLine>(OuterEndCap), ""));
00078 for(auto &Object : m_EventObjects) {
00079     Object.first->Draw(Object.second.c_str());
00080 }
00081 c.Draw();
00082 std::string EventDisplayFilename("EventDisplay");
00083 if(Settings::GetString("General/BarrelOrEndcap") != "Barrel") {
00084     EventDisplayFilename += "_EndCap";
00085 } else {
00086     if(Settings::GetInt("EventDisplay/RowToDraw") == 1) {
00087         EventDisplayFilename += "_MainRow";
00088     } else if(Settings::GetInt("EventDisplay/RowToDraw") == 2) {
00089         EventDisplayFilename += "_UpperRow";
00090     }
00091 }
00092 EventDisplayFilename += ".pdf";
00093 c.SaveAs(EventDisplayFilename.c_str());
00094 }
00095
00096 void EventDisplay::AddObject(std::unique_ptr<TObject> Object, const std::string &Option) {
00097     if(Object) {
00098         m_EventObjects.push_back(std::make_pair(std::move(Object), Option));
00099     }
00100 }
00101
00102 void EventDisplay::AddObject(std::vector<std::pair<std::unique_ptr<TObject>,
00103 std::string>> Objects) {
00104     for(auto &Object : Objects) {
00105         AddObject(std::move(Object.first), Object.second);
00106     }
00107 }

```

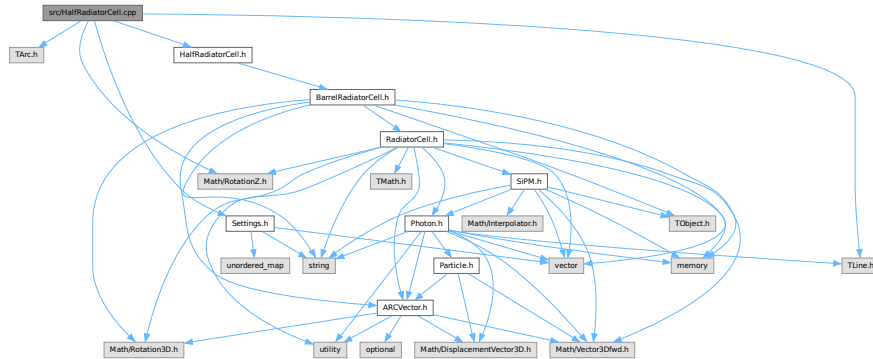
7.69 src/HalfRadiatorCell.cpp File Reference

```

#include "TArc.h"
#include "Math/RotationZ.h"

```

```
#include "TLine.h"
#include "HalfRadiatorCell.h"
#include "Settings.h"
Include dependency graph for HalfRadiatorCell.cpp:
```



7.70 HalfRadiatorCell.cpp

[Go to the documentation of this file.](#)

```
00001 // Martin Duy Tat 27th August 2022
00002
00003 #include "TArch.h"
00004 #include "Math/RotationZ.h"
00005 #include "TLine.h"
00006 #include "HalfRadiatorCell.h"
00007 #include "Settings.h"
00008
00009 using RotationZ = ROOT::Math::RotationZ;
00010
00011 HalfRadiatorCell::HalfRadiatorCell(std::size_t CellColumnNumber,
00012                                     std::size_t CellRowNumber,
00013                                     double HexagonSize):
00014     BarrelRadiatorCell(CellColumnNumber, CellRowNumber, HexagonSize) {
00015 }
00016
00017 bool HalfRadiatorCell::IsInsideCell(const ARCVector &Position) const {
00018     if(Position.LocalVector().X() > 0.0) {
00019         return false;
00020     } else {
00021         return BarrelRadiatorCell::IsInsideCell(Position);
00022     }
00023 }
00024
00025 std::vector<std::pair<std::unique_ptr<TObject>, std::string>>
00026 HalfRadiatorCell::DrawRadiatorGeometry() const {
00027     // First find the mirror intersections with the walls in local coordinates
00028     // by solving a quadratic
00029     const auto MirrorCentre = GetMirrorCentre();
00030     const double b = MirrorCentre.Z();
00031     const double c_left = TMath::Power(m_HexagonSize/2.0, 2)
00032         + MirrorCentre.Mag2()
00033         - m_MirrorCurvature*m_MirrorCurvature
00034         + MirrorCentre.X()*m_HexagonSize;
00035     const double c_right = MirrorCentre.Mag2()
00036         - m_MirrorCurvature*m_MirrorCurvature;
00037     const double s_left = TMath::Sqrt(b*b - c_left);
00038     const double s_right = TMath::Sqrt(b*b - c_right);
00039     const double SinLeft = s_left/m_MirrorCurvature;
00040     const double SinRight = s_right/m_MirrorCurvature;
00041     double LeftAngle = MirrorCentre.X() < -m_HexagonSize/2.0 ?
00042         TMath::ASin(SinLeft)*180.0/TMath::Pi() :
00043         180.0 - TMath::ASin(SinLeft)*180.0/TMath::Pi();
00044     double RightAngle = MirrorCentre.X() > m_HexagonSize/2.0 ?
00045         180.0 - TMath::ASin(SinRight)*180.0/TMath::Pi() :
00046         TMath::ASin(SinRight)*180.0/TMath::Pi();
00047     // Check if mirror intersects with the top wall
00048     const double bb = MirrorCentre.X();
00049     const double h = m_RadiatorThickness - 2.0*m_VesselThickness - m_CoolingThickness;
```

```

00050     const double cc = h*h + MirrorCentre.Mag2() - 2.0*MirrorCentre.Z()*h
00051         - m_MirrorCurvature*m_MirrorCurvature;
00052     const double s1 = bb + TMath::Sqrt(bb*bb - cc);
00053     const double s2 = bb + TMath::Sqrt(bb*bb - cc);
00054     if(bb*bb > cc &&
00055        (TMath::Abs(s1) < m_HexagonSize/2.0 || TMath::Abs(s2) < m_HexagonSize/2.0)) {
00056         const double s = TMath::Abs(s1) < m_HexagonSize/2.0 ? s1 : s2;
00057         const double MidAngle = TMath::ATan2(h - MirrorCentre.Z(),
00058            s - MirrorCentre.X())*180.0/TMath::Pi();
00059         if(MirrorCentre.X() < 0.0) {
00060             LeftAngle = MidAngle;
00061         } else {
00062             RightAngle = MidAngle;
00063         }
00064     }
00065     // Finally draw everything
00066     const auto ReversePhi = ReversePhiRotation();
00067     const auto MirrorCentreGlobal = ReversePhi(m_MirrorCentre.GlobalVector());
00068     TArc MirrorArc(MirrorCentreGlobal.Z(),
00069        MirrorCentreGlobal.X(),
00070        m_MirrorCurvature,
00071        RightAngle,
00072        LeftAngle);
00073     MirrorArc.SetLineColor(kBlack);
00074     MirrorArc.SetLineWidth(2);
00075     MirrorArc.SetFillColorAlpha(kWhite, 0.0);
00076     std::vector<std::pair<std::unique_ptr<TObject>, std::string>> Objects;
00077     Objects.push_back(std::make_pair(std::make_unique<TArc>(MirrorArc), "ONLY"));
00078     // Draw the walls
00079     TLine LeftLine(-m_HexagonSize/2.0 + m_Position.Z(),
00080        Settings::GetDouble("ARCGeometry/Radius") - m_VesselThickness,
00081        -m_HexagonSize/2.0 + m_Position.Z(),
00082        Settings::GetDouble("ARCGeometry/Radius")
00083        + m_RadiatorThickness - m_VesselThickness);
00084     LeftLine.SetLineColor(kBlack);
00085     Objects.push_back(std::make_pair(std::make_unique<TLine>(LeftLine), ""));
00086     TLine RightLine(m_Position.Z(),
00087        Settings::GetDouble("ARCGeometry/Radius") - m_VesselThickness,
00088        m_Position.Z(),
00089        Settings::GetDouble("ARCGeometry/Radius")
00090        + m_RadiatorThickness - m_VesselThickness);
00091     RightLine.SetLineColor(kBlack);
00092     Objects.push_back(std::make_pair(std::make_unique<TLine>(RightLine), ""));
00093     // Draw the detector plane
00094     TLine DetectorLine(-m_HexagonSize/2.0 + m_Position.Z(),
00095        Settings::GetDouble("ARCGeometry/Radius") + m_CoolingThickness,
00096        m_Position.Z(),
00097        Settings::GetDouble("ARCGeometry/Radius") + m_CoolingThickness);
00098     DetectorLine.SetLineColor(kBlack);
00099     Objects.push_back(std::make_pair(std::make_unique<TLine>(DetectorLine), ""));
00100     // Draw the aerogel plane
00101     TLine AerogelLine(-m_HexagonSize/2.0 + m_Position.Z(),
00102        Settings::GetDouble("ARCGeometry/Radius")
00103        + m_CoolingThickness + m_AerogelThickness,
00104        m_Position.Z(),
00105        Settings::GetDouble("ARCGeometry/Radius")
00106        + m_CoolingThickness + m_AerogelThickness);
00107     AerogelLine.SetLineColor(kBlack);
00108     Objects.push_back(std::make_pair(std::make_unique<TLine>(AerogelLine), ""));
00109     // Draw the bottom cooling plane
00110     TLine CoolingLine(-m_HexagonSize/2.0 + m_Position.Z(),
00111        Settings::GetDouble("ARCGeometry/Radius"),
00112        m_Position.Z(),
00113        Settings::GetDouble("ARCGeometry/Radius"));
00114     CoolingLine.SetLineColor(kBlack);
00115     Objects.push_back(std::make_pair(std::make_unique<TLine>(CoolingLine), ""));
00116     // Draw SiPM
00117     const auto RadiatorPosition = ReversePhi(GetRadiatorPosition());
00118     Objects.push_back(std::make_pair(m_Detector.DrawSiPM(RadiatorPosition), ""));
00119     return Objects;
00120 }

```

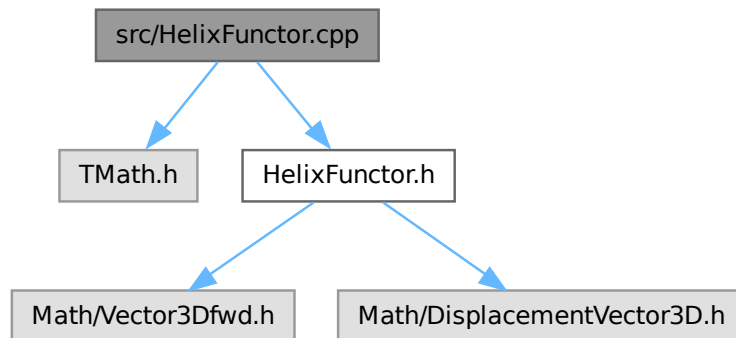
7.71 src/HelixFunctor.cpp File Reference

```

#include "TMath.h"
#include "HelixFunctor.h"

```

Include dependency graph for HelixFuncutor.cpp:



7.72 HelixFuncutor.cpp

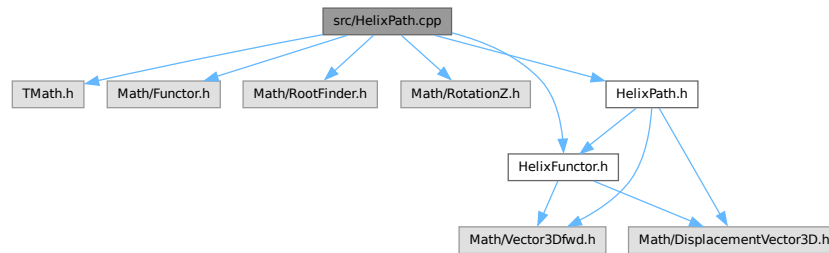
[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 28th October 2022
00002
00003 #include "TMath.h"
00004 #include "HelixFuncutor.h"
00005
00006 BarrelHelixFuncutor::BarrelHelixFuncutor(double Radius): m_Radius(Radius) {
00007 }
00008
00009 double BarrelHelixFuncutor::operator()(const Vector &Vect) const {
00010     return Vect.X()*Vect.X() + Vect.Y()*Vect.Y() - m_Radius*m_Radius;
00011 }
00012
00013 EndCapHelixFuncutor::EndCapHelixFuncutor(double z): m_z(z) {
00014 }
00015
00016 double EndCapHelixFuncutor::operator()(const Vector &Vect) const {
00017     return Vect.Z() - m_z;
00018 }
00019
00020 ZPlaneHelixFuncutor::ZPlaneHelixFuncutor(double z,
00021     const Vector &CellPosition,
00022     bool Barrel):
00023     m_z(z),
00024     m_UnitZ(GetUnitZ(Barrel, CellPosition)),
00025     m_CellPositionZ(m_UnitZ.Dot(CellPosition)) {
00026 }
00027
00028 double ZPlaneHelixFuncutor::operator()(const Vector &Vect) const {
00029     return Vect.Dot(m_UnitZ) - m_CellPositionZ - m_z;
00030 }
00031
00032 Vector ZPlaneHelixFuncutor::GetUnitZ(bool Barrel, const Vector &Position) {
00033     if(Barrel) {
00034         const Vector PositionProjection(Position.X(), Position.Y(), 0.0);
00035         return PositionProjection.Unit();
00036     } else {
00037         return Vector(0.0, 0.0, 1.0);
00038     }
00039 }
00040
00041 MirrorHelixFuncutor::MirrorHelixFuncutor(const Vector &MirrorPosition,
00042     double Curvature):
00043     m_MirrorPosition(MirrorPosition),
00044     m_Curvature(Curvature) {
00045 }
00046
00047 double MirrorHelixFuncutor::operator()(const Vector &Vect) const {
00048     return (Vect - m_MirrorPosition).Mag2() - m_Curvature*m_Curvature;
00049 }
  
```

7.73 src/HelixPath.cpp File Reference

```
#include "TMath.h"
#include "Math/Functor.h"
#include "Math/RootFinder.h"
#include "Math/RotationZ.h"
#include "HelixPath.h"
#include "HelixFunctor.h"
Include dependency graph for HelixPath.cpp:
```



Typedefs

- using [Functor1D](#) = ROOT::Math::Functor1D

7.73.1 Typedef Documentation

7.73.1.1 Functor1D

```
using Functor1D = ROOT::Math::Functor1D
```

Definition at line 10 of file [HelixPath.cpp](#).

7.74 HelixPath.cpp

[Go to the documentation of this file.](#)

```
00001 // Martin Duy Tat 28th October 2022
00002
00003 #include "TMath.h"
00004 #include "Math/Functor.h"
00005 #include "Math/RootFinder.h"
00006 #include "Math/RotationZ.h"
00007 #include "HelixPath.h"
00008 #include "HelixFunctor.h"
00009
00010 using Functor1D = ROOT::Math::Functor1D;
00011
00012 HelixPath::HelixPath(const Vector &Momentum, int Q, double B):
00013     m_n(Momentum.Unit()),
00014     m_Radius(3.335640952*TMath::Sqrt(Momentum.Mag2()) / (Q*B)),
00015     m_InvRadius(1.0/m_Radius),
00016     m_BFieldOn(B != 0.0),
00017     m-Origin_s(0.0) {
00018     gErrorIgnoreLevel = kFatal;
00019 }
```

```

00020
00021 Vector HelixPath::GetPosition(double s) const {
00022     s -= m_Origin_s;
00023     const double Sin = TMath::Sin(s*m_InvRadius);
00024     const double Cos = TMath::Cos(s*m_InvRadius);
00025     const double x = m_BFieldOn ?
00026         m_Radius*(m_n.X()*Sin + m_n.Y()*(1 - Cos)) :
00027         m_n.X()*s;
00028     const double y = m_BFieldOn ?
00029         m_Radius*(m_n.Y()*Sin - m_n.X()*(1 - Cos)) :
00030         m_n.Y()*s;
00031     const double z = m_n.Z()*s;
00032     return m_Origin + Vector(x, y, z);
00033 }
00034
00035 Vector HelixPath::GetDirection(double s) const {
00036     s -= m_Origin_s;
00037     const double Sin = TMath::Sin(s*m_InvRadius);
00038     const double Cos = TMath::Cos(s*m_InvRadius);
00039     const double x = m_BFieldOn ?
00040         m_n.X()*Cos + m_n.Y()*Sin :
00041         m_n.X();
00042     const double y = m_BFieldOn ?
00043         m_n.Y()*Cos - m_n.X()*Sin :
00044         m_n.Y();
00045     const double z = m_n.Z();
00046     return Vector(x, y, z);
00047 }
00048
00049 double HelixPath::SolvePathLength(const HelixFunctor &Functor,
00050     double Min, double Max) const {
00051     auto fcn = [&](double s) {
00052         return Functor(GetPosition(s));
00053     };
00054     ROOT::Math::RootFinder Solver;
00055     if(!Solver.Solve(fcn, Min, Max)) {
00056         return -999.0;
00057     }
00058     return Solver.Root();
00059 }
00060
00061 void HelixPath::ResetOrigin(double NewOrigin_s) {
00062     m_Origin = GetPosition(NewOrigin_s);
00063     m_n = GetDirection(NewOrigin_s).Unit();
00064     m_Origin_s = NewOrigin_s;
00065 }
00066
00067 void HelixPath::MapPhi(double DeltaPhi) {
00068     const RotationZ PhiRotation(DeltaPhi);
00069     m_n = PhiRotation(m_n);
00070     m_Origin = PhiRotation(m_Origin);
00071 }
00072
00073 void HelixPath::ReflectZ() {
00074     m_n.SetZ(-m_n.Z());
00075     m_Origin.SetZ(-m_Origin.Z());
00076 }
00077
00078 void HelixPath::ReflectY() {
00079     m_n.SetY(-m_n.Y());
00080     m_Origin.SetY(-m_Origin.Y());
00081     // The mirror image of a helix bends the other way, like the opposite charge
00082     m_Radius = -m_Radius;
00083     m_InvRadius = -m_InvRadius;
00084 }

```

7.75 src/Particle.cpp File Reference

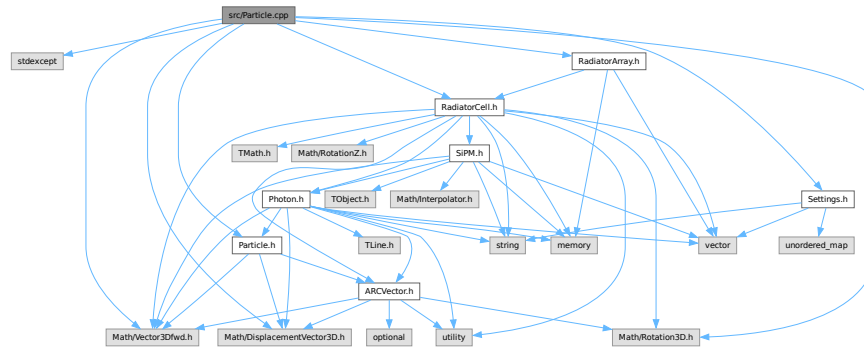
```

#include <stdexcept>
#include "Math/Rotation3D.h"
#include "Math/Vector3Dfwd.h"
#include "Math/DisplacementVector3D.h"
#include "Particle.h"
#include "RadiatorCell.h"
#include "RadiatorArray.h"

```

```
#include "Settings.h"
```

Include dependency graph for Particle.cpp:



7.76 Particle.cpp

[Go to the documentation of this file.](#)

```
00001 // Martin Duy Tat 29th September 2022
00002
00003 #include <stdexcept>
00004 #include "Math/Rotation3D.h"
00005 #include "Math/Vector3Dfwd.h"
00006 #include "Math/DisplacementVector3D.h"
00007 #include "Particle.h"
00008 #include "RadiatorCell.h"
00009 #include "RadiatorArray.h"
00010 #include "Settings.h"
00011
00012 Particle::Particle(const Vector &Position,
00013                  const RadiatorCell *radiatorCell):
00014     m_Position(Position),
00015     m_RadiatorCell(radiatorCell) {
00016 }
00017
00018 const ARCVector& Particle::GetPosition() const {
00019     return m_Position;
00020 }
00021
00022 const RadiatorCell* Particle::GetRadiatorCell() const {
00023     return m_RadiatorCell;
00024 }
00025
00026 bool Particle::FindRadiator(const RadiatorArray &radiatorArray) {
00027     m_RadiatorCell = radiatorArray.FindRadiator(*this);
00028     if(m_RadiatorCell) {
00029         ConvertToRadiatorCoordinates();
00030         return true;
00031     } else {
00032         return false;
00033     }
00034 }
00035
00036 void Particle::ConvertToRadiatorCoordinates() {
00037     // Check if radiator cell exists
00038     if(!m_RadiatorCell) {
00039         throw std::runtime_error("Cannot switch to non-existent local coordinates");
00040     }
00041     // Assign the local coordinate system
00042     const auto RadiatorPosition = m_RadiatorCell->GetRadiatorPosition();
00043     const auto RadiatorRotation = m_RadiatorCell->GetRadiatorRotation();
00044     m_Position.AssignLocalCoordinates(RadiatorPosition, RadiatorRotation);
00045 }
00046
00047 void Particle::ConvertBackToGlobalCoordinates() {
00048     // Remove the local coordinate system
00049     m_Position.ConvertToGlobal();
00050     // Remove the radiator cell
00051     m_RadiatorCell = nullptr;
00052 }
```

```

00053
00054 void Particle::MapPhi(double DeltaPhi) {
00055     m_Position.MapPhi(DeltaPhi);
00056 }
00057
00058 void Particle::ReflectZ() {
00059     m_Position.ReflectZ();
00060 }
00061
00062 void Particle::ReflectY() {
00063     m_Position.ReflectY();
00064 }
00065
00066 std::size_t Particle::GetRadiatorColumnNumber() const {
00067     if(m_RadiatorCell) {
00068         return m_RadiatorCell->GetCellNumber().first;
00069     } else {
00070         return static_cast<std::size_t>(-1);
00071     }
00072 }
00073
00074 std::size_t Particle::GetRadiatorRowNumber() const {
00075     if(m_RadiatorCell) {
00076         return m_RadiatorCell->GetCellNumber().second;
00077     } else {
00078         return static_cast<std::size_t>(-1);
00079     }
00080 }

```

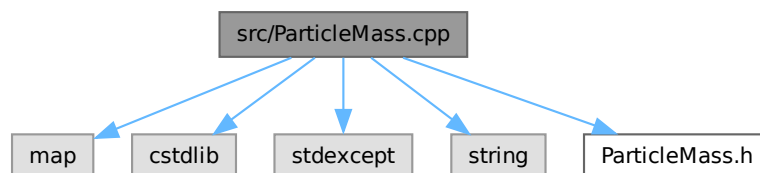
7.77 src/ParticleMass.cpp File Reference

```

#include <map>
#include <cstdlib>
#include <stdexcept>
#include <string>
#include "ParticleMass.h"

```

Include dependency graph for ParticleMass.cpp:



Namespaces

- namespace [ParticleMass](#)

Functions

- double [ParticleMass::GetMass](#) (int PID)

7.80 ParticleTrack.cpp

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 29th April 2022
00002
00003 #include<stdexcept>
00004 #include<memory>
00005 #include<utility>
00006 #include"TMath.h"
00007 #include"TRandom.h"
00008 #include"Math/RotationY.h"
00009 #include"Math/RotationZ.h"
00010 #include"ParticleTrack.h"
00011 #include"Photon.h"
00012 #include"TrackingVolume.h"
00013 #include"RadiatorCell.h"
00014 #include"ParticleMass.h"
00015 #include"Settings.h"
00016 #include"SiPM.h"
00017 #include"RadiatorArray.h"
00018 #include"Utilities.h"
00019 #include"HelixFunctor.h"
00020
00021 ParticleTrack::ParticleTrack(int ParticleID,
00022                             const Vector &Momentum,
00023                             std::size_t TrackNumber,
00024                             double BField):
00025     Particle::Particle({}),
00026     m_Momentum(Momentum),
00027     m_InitialPosition(Vector(0.0, 0.0, 0.0)),
00028     m_ParticleID(ParticleID),
00029     m_Location(Location::TrackerVolume),
00030     m_RandomEmissionPoint(Settings::GetBool("General/RandomEmissionPoint")),
00031     m_ChromaticDispersion(Settings::GetBool("General/ChromaticDispersion")),
00032     m_Mass(ParticleMass::GetMass(m_ParticleID)),
00033     m_PhotonMultiplier(GetPhotonMultiplier(m_Momentum.GlobalVector())),
00034     m_TrackNumber(TrackNumber),
00035     m_TracksToDraw(Settings::GetSizeTVector("General/TrackToDraw")),
00036     m_Helix(Momentum, m_ParticleID > 0 ? +1 : -1, BField),
00037     m_PathLength(0.0) {
00038 }
00039
00040 bool ParticleTrack::TrackThroughTracker(const TrackingVolume &InnerTracker) {
00041     if(m_Location != Location::TrackerVolume) {
00042         throw std::runtime_error("Cannot track particle through inner tracker again");
00043     }
00044     if(Settings::GetString("General/BarrelOrEndcap") == "Barrel") {
00045         const double TrackerRadius = InnerTracker.GetRadius();
00046         const BarrelHelixFunctor Functor(TrackerRadius);
00047         const double Tracker_s = m_Helix.SolvePathLength(Functor,
00048                 0.9*TrackerRadius,
00049                 5.0*TrackerRadius);
00050         if(Tracker_s == -999.0) {
00051             // Most likely the track didn't reach the barrel
00052             return false;
00053         }
00054         const auto NewPosition = m_Helix.GetPosition(Tracker_s);
00055         m_Position.SetGlobalVector(NewPosition);
00056         m_PathLength = Tracker_s;
00057     } else {
00058         const double BarrelZ = Settings::GetDouble("ARCGeometry/BarrelZ");
00059         const int Sign = m_Momentum.GlobalVector().Z() > 0 ? +1 : -1;
00060         const EndCapHelixFunctor Functor(BarrelZ*Sign);
00061         const double Tracker_s = m_Helix.SolvePathLength(Functor,
00062                 0.95*BarrelZ,
00063                 3.0*BarrelZ);
00064         if(Tracker_s == -999.0) {
00065             // Most likely the track didn't reach the end cap
00066             return false;
00067         }
00068         const auto NewPosition = m_Helix.GetPosition(Tracker_s);
00069         m_Position.SetGlobalVector(NewPosition);
00070         m_PathLength = Tracker_s;
00071     }
00072     m_Location = Location::EntranceWindow;
00073     return true;
00074 }
00075
00076 void ParticleTrack::SetRadiator(const RadiatorCell *radiatorCell) {
00077     m_RadiatorCell = radiatorCell;
00078 }
00079
00080
00081 void ParticleTrack::ConvertToRadiatorCoordinates() {
00082     Particle::ConvertToRadiatorCoordinates();

```

```

00083 // Assign local coordinates
00084 const auto RadiatorPosition = m_RadiatorCell->GetRadiatorPosition();
00085 const auto RadiatorRotation = m_RadiatorCell->GetRadiatorRotation();
00086 m_Momentum.AssignLocalCoordinates({}, RadiatorRotation);
00087 m_InitialPosition.AssignLocalCoordinates(RadiatorPosition, RadiatorRotation);
00088 m_EntranceWindowPosition.AssignLocalCoordinates(RadiatorPosition,
00089 RadiatorRotation);
00090 // Check if particle is within acceptance
00091 if(!m_RadiatorCell->IsInsideCell(m_Position) ||
00092 m_Momentum.GlobalVector().Dot(m_Position.GlobalVector()) < 0.0) {
00093 m_Location = Location::MissedEntranceWindow;
00094 }
00095 // Save the entrance window position
00096 m_EntranceWindowPosition = m_Position;
00097 }
00098
00099 void ParticleTrack::ConvertBackToGlobalCoordinates() {
00100 Particle::ConvertBackToGlobalCoordinates();
00101 // Remove the local coordinate system
00102 m_Momentum.ConvertToGlobal();
00103 m_InitialPosition.ConvertToGlobal();
00104 m_EntranceWindowPosition.ConvertToGlobal();
00105 }
00106
00107 bool ParticleTrack::TrackThroughAerogel() {
00108 if(!m_RadiatorCell) {
00109 throw std::runtime_error("No radiator cell to track through");
00110 }
00111 if(m_Location != Location::EntranceWindow) {
00112 throw std::runtime_error("Particle not at entrance window");
00113 }
00114 const bool InBarrel = Settings::GetString("General/BarrelOrEndcap") == "Barrel";
00115 const ZPlaneHelixFunctor CoolingFunctor(0.0,
00116 m_RadiatorCell->GetRadiatorPosition(),
00117 InBarrel);
00118 const double Cooling_s = m_Helix.SolvePathLength(CoolingFunctor,
00119 m_PathLength,
00120 1.2*m_PathLength);
00121 if(Cooling_s == -999.0) {
00122 return false;
00123 }
00124 const auto CoolingLayerPosition = m_Helix.GetPosition(Cooling_s);
00125 m_Position.SetGlobalVector(CoolingLayerPosition);
00126 m_PathLength = Cooling_s;
00127 m_AerogelEntry_s = Cooling_s;
00128 const double AerogelThickness = m_RadiatorCell->GetAerogelThickness();
00129 const ZPlaneHelixFunctor AerogelFunctor(AerogelThickness,
00130 m_RadiatorCell->GetRadiatorPosition(),
00131 InBarrel);
00132 const double Aerogel_s = m_Helix.SolvePathLength(AerogelFunctor,
00133 m_PathLength,
00134 1.2*m_PathLength);
00135 if(Aerogel_s == -999.0) {
00136 return false;
00137 }
00138 const auto AerogelPosition = m_Helix.GetPosition(Aerogel_s);
00139 m_Position.SetGlobalVector(AerogelPosition);
00140 m_PathLength = Aerogel_s;
00141 m_AerogelExit_s = m_PathLength;
00142 m_GasEntry_s = m_AerogelExit_s;
00143 m_Location = Location::Radiator;
00144 return true;
00145 }
00146
00147 void ParticleTrack::TrackThroughGasToMirror() {
00148 if(m_Location != Location::Radiator &&
00149 m_Location != Location::MissedEntranceWindow) {
00150 throw std::runtime_error("Cannot track through gas if not in radiator");
00151 }
00152 m_Position.SetGlobalVector(m_Helix.GetPosition(m_GasEntry_s));
00153 m_PathLength = m_GasEntry_s;
00154 const auto MirrorCentre = m_RadiatorCell->GetGlobalMirrorCentre();
00155 MirrorHelixFunctor MirrorFunctor(MirrorCentre,
00156 m_RadiatorCell->GetMirrorCurvature());
00157 const double Mirror_s = m_Helix.SolvePathLength(MirrorFunctor,
00158 m_PathLength,
00159 1.5*m_PathLength);
00160 if(Mirror_s == -999.0) {
00161 return;
00162 }
00163 const auto MirrorPosition = m_Helix.GetPosition(Mirror_s);
00164 m_Position.SetGlobalVector(MirrorPosition);
00165 m_PathLength = Mirror_s;
00166 m_GasExit_s = Mirror_s;
00167 if(m_RadiatorCell->IsInsideCell(m_Position)) {
00168 m_Location = Location::Mirror;
00169 }

```

```

00170 }
00171
00172 bool ParticleTrack::TrackToNextCell(const RadiatorArray &radiatorArray) {
00173     std::size_t Counter = 0;
00174     while(m_Location != Location::Mirror) {
00175         ConvertBackToGlobalCoordinates();
00176         m_Location = Location::Radiator;
00177         if(!FindRadiator(radiatorArray) || Counter > 10) {
00178             return false;
00179         }
00180         TrackThroughGasToMirror();
00181         Counter++;
00182     }
00183     return true;
00184 }
00185
00186 Photon ParticleTrack::GeneratePhotonFromAerogel() const {
00187     if(m_Location != Location::Radiator) {
00188         throw std::runtime_error(
00189             "Cannot generate photons before aerogel or after gas");
00190     }
00191     Photon photon = GeneratePhoton(m_AerogelEntry_s,
00192                                     m_AerogelExit_s,
00193                                     Photon::Radiator::Aerogel);
00194     return photon;
00195 }
00196
00197 Photon ParticleTrack::GeneratePhotonFromGas() const {
00198     if(m_Location != Location::Mirror) {
00199         throw std::runtime_error(
00200             "Cannot generate photons from tracks not at the mirror");
00201     }
00202     Photon photon = GeneratePhoton(m_GasEntry_s,
00203                                     m_GasExit_s,
00204                                     Photon::Radiator::Gas);
00205     return photon;
00206 }
00207
00208 std::vector<Photon> ParticleTrack::GeneratePhotonsFromAerogel() const {
00209     if(m_Location != Location::Radiator) {
00210         throw std::runtime_error(
00211             "Cannot generate photons before aerogel or after gas");
00212     }
00213     const double RadiatorDistance = m_AerogelExit_s - m_AerogelEntry_s;
00214     const double IndexRefraction =
00215         Utilities::GetIndexRefraction(Photon::Radiator::Aerogel, false);
00216     const std::size_t PhotonYield = static_cast<std::size_t>(
00217         std::round(GetPhotonYield(RadiatorDistance, Beta(), IndexRefraction)));
00218     std::vector<Photon> Photons;
00219     for(std::size_t i = 0; i < PhotonYield; i++) {
00220         Photons.push_back(GeneratePhoton(m_AerogelEntry_s,
00221                                           m_AerogelExit_s,
00222                                           Photon::Radiator::Aerogel));
00223     }
00224     return Photons;
00225 }
00226
00227 std::vector<Photon> ParticleTrack::GeneratePhotonsFromGas() const {
00228     if(m_Location != Location::Mirror) {
00229         throw std::runtime_error(
00230             "Cannot generate photons from tracks not at the mirror");
00231     }
00232     const double RadiatorDistance = m_GasExit_s - m_GasEntry_s;
00233     const double IndexRefraction =
00234         Utilities::GetIndexRefraction(Photon::Radiator::Gas, false);
00235     const std::size_t PhotonYield = static_cast<std::size_t>(
00236         std::round(GetPhotonYield(RadiatorDistance, Beta(), IndexRefraction)));
00237     std::vector<Photon> Photons;
00238     Photons.reserve(PhotonYield);
00239     for(std::size_t i = 0; i < PhotonYield; i++) {
00240         Photons.emplace_back(GeneratePhoton(m_GasEntry_s,
00241                                           m_GasExit_s,
00242                                           Photon::Radiator::Gas));
00243     }
00244     return Photons;
00245 }
00246
00247 Photon ParticleTrack::GeneratePhoton(double Entry_s,
00248                                     double Exit_s,
00249                                     Photon::Radiator Radiator) const {
00250     // Random emission point
00251     const double RandomFraction = m_RandomEmissionPoint
00252         ? Utilities::Random().Uniform(0.005, 0.995) : 0.5;
00253     const double EmissionPoint_s = Entry_s + (Exit_s - Entry_s)*RandomFraction;
00254     const auto EmissionPoint = m_Helix.GetPosition(EmissionPoint_s);
00255     // Assumed emission point
00256     const double AssumedEmissionPoint_s = 0.5*(Entry_s + Exit_s);

```

```

00257     const auto AssumedEmissionPoint = m_Helix.GetPosition(AssumedEmissionPoint_s);
00258     // Energy and index of refraction
00259     const double Energy = Utilities::Random().Uniform(1.55, 4.31);
00260     const double n_phase = Utilities::GetIndexRefraction(Radiator,
00261                                                         m_ChromaticDispersion,
00262                                                         Energy);
00263     // Random azimuthal angle
00264     const double phi = Utilities::Random().Uniform(0.0, 2*TMath::Pi());
00265     // Cherenkov angle
00266     auto GetCosTheta = [&] () {
00267         double CosTheta = 1.0/(Beta()*n_phase);
00268         if(CosTheta > 1.0) {
00269             return 1.0;
00270         } else {
00271             return CosTheta;
00272         }
00273     };
00274     // Trigonometry calculations
00275     const double CosTheta = GetCosTheta();
00276     const double SinTheta = TMath::Sqrt(1.0 - CosTheta*CosTheta);
00277     const double CosPhi = TMath::Cos(phi);
00278     const double SinPhi = (phi > TMath::Pi() ? -1.0 : +1.0)
00279                          *TMath::Sqrt(1.0 - CosPhi*CosPhi);
00280     // Photon direction in particle rest frame
00281     Vector Direction(SinTheta*CosPhi, SinTheta*SinPhi, CosTheta);
00282     // Rotate to lab frame
00283     const auto ParticleDirection = m_Helix.GetDirection(EmissionPoint_s);
00284     const ROOT::Math::RotationY RotateY(ParticleDirection.Theta());
00285     Direction = RotateY(Direction);
00286     const ROOT::Math::RotationZ RotateZ(ParticleDirection.Phi());
00287     Direction = RotateZ(Direction);
00288     // Finally create the photon
00289     return Photon(EmissionPoint,
00290                  AssumedEmissionPoint,
00291                  Direction,
00292                  ParticleDirection,
00293                  m_Helix.GetDirection(AssumedEmissionPoint_s),
00294                  Energy,
00295                  CosTheta,
00296                  Radiator,
00297                  &(*m_RadiatorCell),
00298                  IsTrackDrawn(),
00299                  1.0/m_PhotonMultiplier);
00300 }
00301
00302 double ParticleTrack::Beta() const {
00303     const double Momentum = TMath::Sqrt(m_Momentum.LocalVector().Mag2());
00304     return Momentum/TMath::Sqrt(m_Mass*m_Mass + Momentum*Momentum);
00305 }
00306
00307 double ParticleTrack::GetPhotonYield(double x, double Beta, double n) const {
00308     // TODO: Move this to separate class
00309     if(Beta*n < 1.0) {
00310         return 0.0;
00311     } else {
00312         const double Efficiency = 0.60*0.90*0.80;
00313         const double DeltaE = 2.55;
00314         const double SinThetaC2 = (1.0 - 1.0/TMath::Power(Beta*n, 2));
00315         return x*DeltaE*37000.0*SinThetaC2*Efficiency*m_PhotonMultiplier;
00316     }
00317 }
00318
00319 void ParticleTrack::MapPhi(double DeltaPhi) {
00320     Particle::MapPhi(DeltaPhi);
00321     m_InitialPosition.MapPhi(DeltaPhi);
00322     m_Momentum.MapPhi(DeltaPhi);
00323     m_Helix.ResetOrigin(m_PathLength);
00324     m_Helix.MapPhi(DeltaPhi);
00325 }
00326
00327 void ParticleTrack::ReflectZ() {
00328     Particle::ReflectZ();
00329     m_InitialPosition.ReflectZ();
00330     m_Momentum.ReflectZ();
00331     m_Helix.ReflectZ();
00332 }
00333
00334 void ParticleTrack::ReflectY() {
00335     Particle::ReflectY();
00336     m_InitialPosition.ReflectY();
00337     m_Momentum.ReflectY();
00338     m_Helix.ResetOrigin(m_PathLength);
00339     m_Helix.ReflectY();
00340 }
00341
00342 std::unique_ptr<TLine> ParticleTrack::DrawParticleTrack() const {
00343     if(IsTrackDrawn()) {

```

```

00344     const auto ReversePhi = m_RadiatorCell->ReversePhiRotation();
00345     const auto InitialPosition = ReversePhi(m_InitialPosition.GlobalVector());
00346     const auto Position = ReversePhi(m_Position.GlobalVector());
00347     TLine Track(InitialPosition.Z(),
00348                 InitialPosition.X(),
00349                 Position.Z(),
00350                 Position.X());
00351     Track.SetLineColor(kRed);
00352     return std::make_unique<TLine>(Track);
00353 } else {
00354     return nullptr;
00355 }
00356 }
00357
00358 ParticleTrack::Location ParticleTrack::GetParticleLocation() const {
00359     return m_Location;
00360 }
00361
00362 Vector ParticleTrack::GetEntranceWindowPosition() const {
00363     return m_EntranceWindowPosition.GlobalVector();
00364 }
00365
00366 bool ParticleTrack::IsAtRadiator() const {
00367     if(m_Location != Location::EntranceWindow &&
00368        m_Location != Location::MissedEntranceWindow &&
00369        m_Location != Location::Radiator) {
00370         return false;
00371     } else {
00372         return true;
00373     }
00374 }
00375 bool ParticleTrack::IsTrackDrawn() const {
00376     if(!Settings::Exists("EventDisplay/RowToDraw")) {
00377         return false;
00378     }
00379     const std::size_t RowToDraw = Settings::GetSizeT("EventDisplay/RowToDraw");
00380     if(RowToDraw == GetRadiatorRowNumber()) {
00381         const bool DrawAllTracks = Settings::GetBool("General/DrawAllTracks");
00382         if(DrawAllTracks) {
00383             return true;
00384         }
00385         const auto iter = std::find(m_TracksToDraw.begin(),
00386                                     m_TracksToDraw.end(),
00387                                     m_TrackNumber);
00388         if(iter != m_TracksToDraw.end()) {
00389             return true;
00390         }
00391     }
00392     return false;
00393 }
00394
00395 double ParticleTrack::GetPhotonMultiplier(const Vector &Momentum) {
00396     double LowMomentumLimit = Settings::GetDouble("General/LowMomentumLimit");
00397     double PhotonMultiplier = Settings::GetDouble("General/PhotonMultiplier");
00398     const std::string GasOrAerogel = Settings::GetString("General/GasOrAerogel");
00399     if(GasOrAerogel == "Gas" && TMath::Sqrt(Momentum.Mag2()) < LowMomentumLimit) {
00400         return PhotonMultiplier*20.0;
00401     } else {
00402         return PhotonMultiplier;
00403     }
00404 }

```

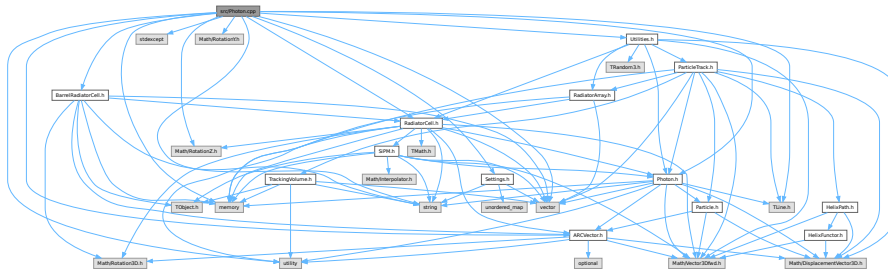
7.81 src/Photon.cpp File Reference

```

#include <memory>
#include <vector>
#include <utility>
#include <string>
#include <stdexcept>
#include "TLine.h"
#include "Math/RotationY.h"
#include "Math/RotationZ.h"
#include "Photon.h"
#include "RadiatorCell.h"
#include "BarrelRadiatorCell.h"
#include "Utilities.h"

```

```
#include "ARCVector.h"
#include "Settings.h"
Include dependency graph for Photon.cpp:
```



7.82 Photon.cpp

[Go to the documentation of this file.](#)

```
00001 // Martin Duy Tat 29th April 2022
00002
00003 #include<memory>
00004 #include<vector>
00005 #include<utility>
00006 #include<string>
00007 #include<stdexcept>
00008 #include"TLLine.h"
00009 #include"Math/RotationY.h"
00010 #include"Math/RotationZ.h"
00011 #include"Photon.h"
00012 #include"RadiatorCell.h"
00013 #include"BarrelRadiatorCell.h"
00014 #include"Utilities.h"
00015 #include"ARCVector.h"
00016 #include"Settings.h"
00017
00018 Photon::Photon(const Vector &Position,
00019               const Vector &AssumedPosition,
00020               const Vector &Direction,
00021               const Vector &ParticleDirection,
00022               const Vector &AssumedParticleDirection,
00023               double Energy,
00024               double CosCherenkovAngle,
00025               Radiator radiator,
00026               const RadiatorCell *radiatorCell,
00027               bool IsTrackDrawn,
00028               double Weight):
00029 Particle(Position, radiatorCell),
00030 m_AssumedEmissionPoint(AssumedPosition),
00031 m_EmissionPoint(Position),
00032 m_Direction(Direction),
00033 m_ParticleDirection(ParticleDirection),
00034 m_AssumedParticleDirection(AssumedParticleDirection),
00035 m_Energy(Energy),
00036 m_Radiator(radiator),
00037 m_CosCherenkovAngle(CosCherenkovAngle),
00038 m_Status(Status::Emitted),
00039 m_AerogelTravelDistance(0.0),
00040 m_MirrorHitPosition(nullptr),
00041 m_Weight(Weight),
00042 m_HasMigrated(false),
00043 m_IsTrackDrawn(IsTrackDrawn) {
00044     const auto RadiatorPosition = radiatorCell->GetRadiatorPosition();
00045     const auto RadiatorRotation = radiatorCell->GetRadiatorRotation();
00046     m_Position.AssignLocalCoordinates(RadiatorPosition, RadiatorRotation);
00047     m_AssumedEmissionPoint.AssignLocalCoordinates(RadiatorPosition,
00048                                                  RadiatorRotation);
00049     m_EmissionPoint.AssignLocalCoordinates(RadiatorPosition, RadiatorRotation);
00050     m_Direction.AssignLocalCoordinates({}, RadiatorRotation);
00051     m_ParticleDirection.AssignLocalCoordinates({}, RadiatorRotation);
00052     m_AssumedParticleDirection.AssignLocalCoordinates({}, RadiatorRotation);
00053 }
00054
00055 std::vector<std::pair<std::unique_ptr<TObject>, std::string>
```

```

00056 Photon::DrawPhotonPath() const {
00057     std::vector<std::pair<std::unique_ptr<TObject>, std::string>> PhotonLine;
00058     if(!m_IsTrackDrawn) {
00059         return PhotonLine;
00060     }
00061     const std::size_t RowToDraw = Settings::GetSizeT("EventDisplay/RowToDraw");
00062     if(!m_RadiatorCell || GetRadiatorRowNumber() != RowToDraw) {
00063         return PhotonLine;
00064     }
00065     const auto ReversePhi = m_RadiatorCell->ReversePhiRotation();
00066     const auto EmissionPoint = ReversePhi(m_EmissionPoint.GlobalVector());
00067     const auto Position = ReversePhi(m_Position.GlobalVector());
00068     if(!m_MirrorHitPosition) {
00069         TLine PhotonLine1(EmissionPoint.Z(),
00070             EmissionPoint.X(),
00071             Position.Z(),
00072             Position.X());
00073         if(m_Status == Status::MirrorMiss) {
00074             PhotonLine1.SetLineColor(7);
00075         } else {
00076             PhotonLine1.SetLineColor(kGreen);
00077         }
00078         PhotonLine.push_back(std::make_pair(std::make_unique<TLine>(PhotonLine1), ""));
00079     } else {
00080         const auto MirrorHitPosition = ReversePhi(m_MirrorHitPosition->GlobalVector());
00081         TLine PhotonLine1(EmissionPoint.Z(),
00082             EmissionPoint.X(),
00083             MirrorHitPosition.Z(),
00084             MirrorHitPosition.X());
00085         TLine PhotonLine2(MirrorHitPosition.Z(),
00086             MirrorHitPosition.X(),
00087             Position.Z(),
00088             Position.X());
00089         if(m_Status != Status::DetectorHit) {
00090             PhotonLine1.SetLineColor(6);
00091             PhotonLine2.SetLineColor(6);
00092         } else if(m_Status == Status::DetectorMiss) {
00093             PhotonLine1.SetLineColor(8);
00094             PhotonLine2.SetLineColor(8);
00095         } else {
00096             PhotonLine1.SetLineColor(kBlue);
00097             PhotonLine2.SetLineColor(kBlue);
00098         }
00099         PhotonLine.push_back(std::make_pair(std::make_unique<TLine>(PhotonLine1), ""));
00100         PhotonLine.push_back(std::make_pair(std::make_unique<TLine>(PhotonLine2), ""));
00101     }
00102     return PhotonLine;
00103 }
00104
00105 void Photon::PropagatePhoton(const Vector &Displacement) {
00106     m_Position += Displacement;
00107 }
00108
00109 const Vector& Photon::GetDirection() const {
00110     return m_Direction.LocalVector();
00111 }
00112
00113 void Photon::KickPhoton(const Vector &Kick) {
00114     m_Direction += Kick;
00115 }
00116
00117 Photon::Status Photon::GetStatus() const {
00118     return m_Status;
00119 }
00120
00121 const Vector& Photon::GetEmissionPoint(bool TrueEmissionPoint) const {
00122     if(TrueEmissionPoint) {
00123         return m_EmissionPoint.LocalVector();
00124     } else {
00125         return m_AssumedEmissionPoint.LocalVector();
00126     }
00127 }
00128
00129 void Photon::UpdatePhotonStatus(Photon::Status status) {
00130     if(m_Status == status) {
00131         throw std::runtime_error("New photon status must be different");
00132     }
00133     m_Status = status;
00134 }
00135
00136 double Photon::GetEnergy() const {
00137     return m_Energy;
00138 }
00139
00140 void Photon::AddAerogelTravelDistance(double Distance) {
00141     if(Distance < 0.0) {
00142         throw std::runtime_error("Aerogel travel distance cannot be negative");

```

```

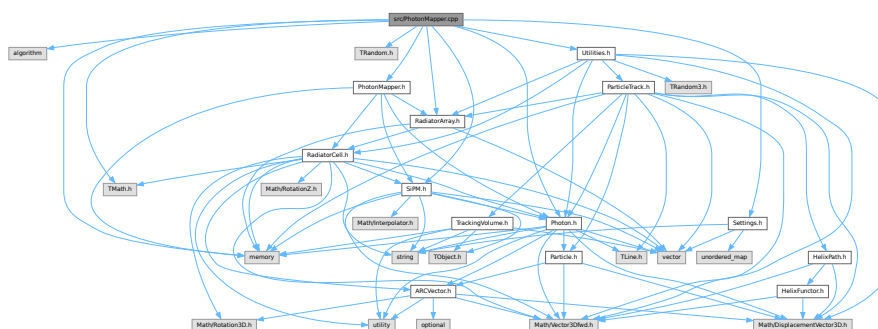
00143     }
00144     m_AerogelTravelDistance += Distance;
00145 }
00146
00147 double Photon::GetAerogelTravelDistance() const {
00148     return m_AerogelTravelDistance;
00149 }
00150
00151 const ARCVector* Photon::GetMirrorHitPosition() const {
00152     return m_MirrorHitPosition.get();
00153 }
00154
00155 void Photon::RegisterMirrorHitPosition(const Vector &MirrorHitPosition) {
00156     if(m_MirrorHitPosition) {
00157         throw std::runtime_error("Mirror hit position already exists");
00158     } else {
00159         m_MirrorHitPosition = std::make_unique<ARCVector>(MirrorHitPosition);
00160     }
00161 }
00162
00163 Photon::Radiator Photon::GetRadiator() const {
00164     return m_Radiator;
00165 }
00166
00167 double Photon::GetCosCherenkovAngle() const {
00168     return m_CosCherenkovAngle;
00169 }
00170
00171 void Photon::ConvertToRadiatorCoordinates() {
00172     Particle::ConvertToRadiatorCoordinates();
00173     const auto RadiatorPosition = m_RadiatorCell->GetRadiatorPosition();
00174     const auto RadiatorRotation = m_RadiatorCell->GetRadiatorRotation();
00175     m_AssumedEmissionPoint.AssignLocalCoordinates(RadiatorPosition, RadiatorRotation);
00176     m_EmissionPoint.AssignLocalCoordinates(RadiatorPosition, RadiatorRotation);
00177     m_Direction.AssignLocalCoordinates({}, RadiatorRotation);
00178     m_ParticleDirection.AssignLocalCoordinates({}, RadiatorRotation);
00179     m_AssumedParticleDirection.AssignLocalCoordinates({}, RadiatorRotation);
00180     // Check if particle is within acceptance
00181     if(!m_RadiatorCell->IsInsideCell(m_Position)) {
00182         m_Status = Status::OutsideCell;
00183     }
00184 }
00185
00186 void Photon::ConvertBackToGlobalCoordinates() {
00187     Particle::ConvertBackToGlobalCoordinates();
00188     m_AssumedEmissionPoint.ConvertToGlobal();
00189     m_EmissionPoint.ConvertToGlobal();
00190     m_Direction.ConvertToGlobal();
00191     m_ParticleDirection.ConvertToGlobal();
00192     m_AssumedParticleDirection.ConvertToGlobal();
00193     // Photon doesn't hit any mirror
00194     m_MirrorHitPosition = nullptr;
00195 }
00196
00197 void Photon::MapPhi(double DeltaPhi) {
00198     Particle::MapPhi(DeltaPhi);
00199     m_AssumedEmissionPoint.MapPhi(DeltaPhi);
00200     m_EmissionPoint.MapPhi(DeltaPhi);
00201     m_Direction.MapPhi(DeltaPhi);
00202     m_ParticleDirection.MapPhi(DeltaPhi);
00203     m_AssumedParticleDirection.MapPhi(DeltaPhi);
00204 }
00205
00206 void Photon::ReflectZ() {
00207     Particle::ReflectZ();
00208     m_AssumedEmissionPoint.ReflectZ();
00209     m_EmissionPoint.ReflectZ();
00210     m_Direction.ReflectZ();
00211     m_ParticleDirection.ReflectZ();
00212     m_AssumedParticleDirection.ReflectZ();
00213 }
00214
00215 void Photon::ReflectY() {
00216     Particle::ReflectY();
00217     m_AssumedEmissionPoint.ReflectY();
00218     m_EmissionPoint.ReflectY();
00219     m_Direction.ReflectY();
00220     m_ParticleDirection.ReflectY();
00221     m_AssumedParticleDirection.ReflectY();
00222 }
00223
00224 bool Photon::IsAtRadiator() const {
00225     return m_Status == Status::Emitted;
00226 }
00227
00228 void Photon::PutPhotonToEmissionPoint() {

```

```
00230     m_Position = m_EmissionPoint;
00231 }
00232
00233 void Photon::PhotonHasMigrated() {
00234     m_HasMigrated = true;
00235 }
00236
00237 bool Photon::HasPhotonMigrated() const {
00238     return m_HasMigrated;
00239 }
00240
00241 const Vector& Photon::GetParticleDirection(bool TrueEmissionPoint) const {
00242     if(TrueEmissionPoint) {
00243         return m_ParticleDirection.LocalVector();
00244     } else {
00245         return m_AssumedParticleDirection.LocalVector();
00246     }
00247 }
00248
00249 double Photon::GetWeight() const {
00250     return m_Weight;
00251 }
```

```
#include <algorithm>
#include <memory>
#include "TMath.h"
#include "TRandom.h"
#include "PhotonMapper.h"
#include "Photon.h"
#include "Settings.h"
#include "SiPM.h"
#include "RadiatorArray.h"
#include "Utilities.h"
```

Include dependency graph for PhotonMapper.cpp:



- namespace **PhotonMapper**

- double `PhotonMapper::PhotonMirrorDistance` (const `Photon` &photon)
- void `PhotonMapper::TracePhotonToMirror` (`Photon` &photon)
- `PhotonHit` `PhotonMapper::TracePhotonToDetector` (`Photon` &photon)
- `std::unique_ptr< PhotonHit >` `PhotonMapper::TracePhoton` (`Photon` &photon, const `RadiatorArray` &radiatorArray)
- bool `PhotonMapper::IsScatteredInAerogel` (const `Photon` &photon)

7.84 PhotonMapper.cpp

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 1st May 2022
00002
00003 #include<algorithm>
00004 #include<memory>
00005 #include"TMath.h"
00006 #include"TRandom.h"
00007 #include"PhotonMapper.h"
00008 #include"Photon.h"
00009 #include"Settings.h"
00010 #include"SiPM.h"
00011 #include"RadiatorArray.h"
00012 #include"Utilities.h"
00013
00014 namespace PhotonMapper {
00015
00016     double PhotonMirrorDistance(const Photon &photon) {
00017         auto MirrorCentre = photon.GetRadiatorCell()->GetMirrorCentre();
00018         const double R = photon.GetRadiatorCell()->GetMirrorCurvature();
00019         const Vector MirrorCentreMinusPosition = MirrorCentre
00020             - photon.GetPosition().LocalVector();
00021         const double b = -photon.GetDirection().Dot(MirrorCentreMinusPosition);
00022         const double c = (MirrorCentreMinusPosition).Mag2() - R*R;
00023         if(b*b - c < 0.0) {
00024             return -100.0;
00025         }
00026         const double Discriminant = TMath::Sqrt(b*b - c);
00027         const double s1 = -b + Discriminant;
00028         const double s2 = -b - Discriminant;
00029         return std::max(s1, s2);
00030     }
00031
00032     void TracePhotonToMirror(Photon &photon) {
00033         const double s = PhotonMirrorDistance(photon);
00034         photon.PropagatePhoton(s*photon.GetDirection());
00035         const double MaxHeight = photon.GetRadiatorCell()->GetRadiatorThickness()
00036             - 2*photon.GetRadiatorCell()->GetVesselThickness()
00037             - photon.GetRadiatorCell()->GetCoolingThickness();
00038         auto MirrorCentre = photon.GetRadiatorCell()->GetMirrorCentre();
00039         const auto Curvature = photon.GetRadiatorCell()->GetMirrorCurvature();
00040         if(photon.GetPosition().LocalVector().Z() > MaxHeight) {
00041             photon.UpdatePhotonStatus(Photon::Status::WallMiss);
00042         } else if(s < 0.0) {
00043             photon.UpdatePhotonStatus(Photon::Status::Backwards);
00044         } else if((photon.GetPosition().LocalVector() - MirrorCentre).Mag2()
00045             - Curvature*Curvature > 1e-6) {
00046             photon.UpdatePhotonStatus(Photon::Status::OutsideMirrorRadius);
00047         } else if(photon.GetRadiatorCell()->IsInsideCell(photon)) {
00048             photon.UpdatePhotonStatus(Photon::Status::MirrorHit);
00049             photon.RegisterMirrorHitPosition(photon.GetPosition().GlobalVector());
00050         } else {
00051             photon.UpdatePhotonStatus(Photon::Status::MirrorMiss);
00052         }
00053     }
00054
00055     PhotonHit TracePhotonToDetector(Photon &photon) {
00056         const Vector Mirror = photon.GetPosition().LocalVector();
00057         auto MirrorCentre = photon.GetRadiatorCell()->GetMirrorCentre();
00058         const auto Curvature = photon.GetRadiatorCell()->GetMirrorCurvature();
00059         const Vector Normal = (Mirror - MirrorCentre)/Curvature;
00060         photon.KickPhoton(-2*photon.GetDirection().Dot(Normal)*Normal);
00061         const auto Position = photon.GetPosition().LocalVector();
00062         const auto Direction = photon.GetDirection();
00063         auto Detector = photon.GetRadiatorCell()->GetDetector();
00064         const double Theta = Detector.GetDetectorTilt();
00065         const double CosTheta = TMath::Cos(Theta);
00066         const double SinTheta = (Theta < 0.0 ? -1.0 : +1.0)
00067             *TMath::Sqrt(1.0 - CosTheta*CosTheta);
00068         // Find intersection between straight photon trajectory and tilted detector plane
00069         const double Det = SinTheta*Direction.X() - CosTheta*Direction.Z();
00070         const double s = ((Position.Z() - Detector.GetDetectorZPosition())*CosTheta
00071             - (Position.X() - Detector.GetDetectorXPosition())*SinTheta)/Det;
00072         photon.PropagatePhoton(s*Direction);
00073         // Check if hit position is inside aerogel
00074         const double AerogelThickness = photon.GetRadiatorCell()->GetAerogelThickness();
00075         if(photon.GetPosition().LocalVector().Z() < AerogelThickness) {
00076             const double Slope = TMath::Abs(Direction.Z());
00077             photon.AddAerogelTravelDistance(AerogelThickness/Slope);
00078         }
00079         // TODO: Account for photons generated in aerogel
00080         if(IsScatteredInAerogel(photon)) {
00081             photon.UpdatePhotonStatus(Photon::Status::AerogelScattered);
00082         }
00083     }

```

```

00083     const PhotonHit photonHit = photon.GetRadiatorCell()->GetDetector().AddPhotonHit(photon);
00084     return photonHit;
00085 }
00086
00087 std::unique_ptr<PhotonHit> TracePhoton(Photon &photon,
00088                                     const RadiatorArray &radiatorArray) {
00089     TracePhotonToMirror(photon);
00090     std::size_t Counter = 0;
00091     while(photon.GetStatus() == Photon::Status::MirrorMiss && Counter < 10) {
00092         photon.UpdatePhotonStatus(Photon::Status::Emitted);
00093         photon.ConvertBackToGlobalCoordinates();
00094         if(photon.FindRadiator(radiatorArray)) {
00095             photon.PutPhotonToEmissionPoint();
00096             TracePhotonToMirror(photon);
00097             photon.PhotonHasMigrated();
00098         } else {
00099             photon.UpdatePhotonStatus(Photon::Status::MirrorMiss);
00100             break;
00101         }
00102         Counter++;
00103     }
00104     if(photon.GetMirrorHitPosition()) {
00105         PhotonHit photonHit = TracePhotonToDetector(photon);
00106         return std::make_unique<PhotonHit>(photonHit);
00107     } else {
00108         return nullptr;
00109     }
00110 }
00111
00112 bool IsScatteredInAerogel(const Photon &photon) {
00113     constexpr double T0 = 0.9679;
00114     constexpr double Clarity = 5.087e10;
00115     const double Lambda = 1239.8/photon.GetEnergy();
00116     const double Lambda2 = Lambda*Lambda;
00117     const double Lambda4 = Lambda2*Lambda2;
00118     const double Exponent = -Clarity*(photon.GetAerogelTravelDistance())/Lambda4;
00119     const double Transmission = T0*TMath::Exp(Exponent);
00120     return Utilities::Random().Uniform(0.0, 1.0) > Transmission;
00121 }
00122
00123 }

```

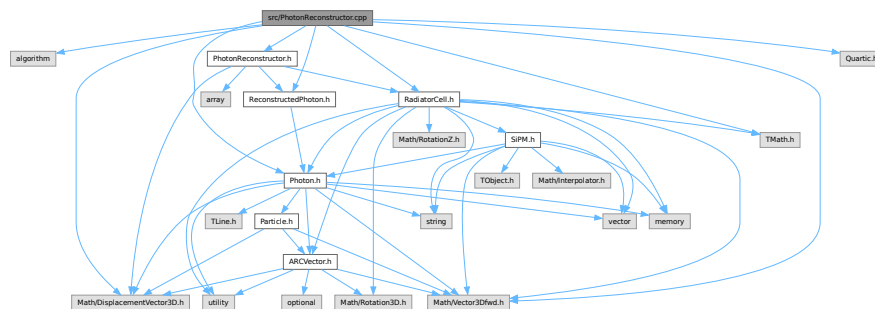
7.85 src/PhotonReconstructor.cpp File Reference

```

#include <algorithm>
#include "PhotonReconstructor.h"
#include "Photon.h"
#include "RadiatorCell.h"
#include "ReconstructedPhoton.h"
#include "TMath.h"
#include "Math/Vector3Dfwd.h"
#include "Math/DisplacementVector3D.h"
#include "Quartic.h"

```

Include dependency graph for PhotonReconstructor.cpp:



Namespaces

- namespace [PhotonReconstructor](#)

Functions

- double [PhotonReconstructor::ReconstructCosCherenkovAngle](#) (const [PhotonHit](#) &photonHit, bool TrueEmissionPoint)
- [ReconstructedPhoton](#) [PhotonReconstructor::ReconstructPhoton](#) (const [PhotonHit](#) &photonHit)
- [QuarticSolution](#) [PhotonReconstructor::SolveQuartic](#) (double EmissionMirrorDist, double DetectionMirrorParaDist, double DetectionMirrorPerpDist)
- [Vector](#) [PhotonReconstructor::GetReflectionPoint](#) (const [Vector](#) &EmissionPoint, const [Vector](#) &DetectionPoint, double DetectionMirrorParaDist, double DetectionMirrorPerpDist, double EmissionMirrorDist, const [QuarticSolution](#) &quarticSolution, std::size_t SolutionNumber)

7.86 PhotonReconstructor.cpp

[Go to the documentation of this file.](#)

```
00001 // Martin Duy Tat 3rd March 2022
00002
00003 #include<algorithm>
00004 #include"PhotonReconstructor.h"
00005 #include"Photon.h"
00006 #include"RadiatorCell.h"
00007 #include"ReconstructedPhoton.h"
00008 #include"TMath.h"
00009 #include"Math/Vector3Dfwd.h"
00010 #include"Math/DisplacementVector3D.h"
00011 #include"Quartic.h"
00012
00013 using Vector = ROOT::Math::XYZVector;
00014
00015 namespace PhotonReconstructor {
00016
00017     double ReconstructCosCherenkovAngle(const PhotonHit &photonHit,
00018                                         bool TrueEmissionPoint) {
00019         const RadiatorCell *radiatorCell = photonHit.m_Photon->GetRadiatorCell();
00020         auto MirrorCentre = radiatorCell->GetMirrorCentre();
00021         const auto Photon = photonHit.m_Photon;
00022         const Vector EmissionPoint = Photon->GetEmissionPoint(TrueEmissionPoint)
00023             - MirrorCentre;
00024         const Vector DetectionPoint = photonHit.m_HitPosition - MirrorCentre;
00025         const double Curvature = radiatorCell->GetMirrorCurvature();
00026         // Distance between emission point and mirror centre,
00027         // in units of mirror curvature
00028         const double EmissionMirrorDist = TMath::Sqrt(EmissionPoint.Mag2())/Curvature;
00029         // Parallel component of distance between detection point and mirror centre,
00030         // in units of mirror curvature
00031         const double DetectionMirrorParaDist = EmissionPoint.Dot(DetectionPoint)
00032             / (EmissionMirrorDist*Curvature*Curvature);
00033         const double InsideSqrt = DetectionPoint.Mag2() / (Curvature*Curvature)
00034             - TMath::Power(DetectionMirrorParaDist, 2);
00035         if(InsideSqrt < 0.0) {
00036             return FailedReconstruction;
00037         }
00038         const double DetectionMirrorPerpDist = TMath::Sqrt(InsideSqrt);
00039         auto quarticSolution = SolveQuartic(EmissionMirrorDist,
00040             DetectionMirrorParaDist,
00041             DetectionMirrorPerpDist);
00042         if(quarticSolution.m_DegenerateSolution) {
00043             return FailedReconstruction;
00044         }
00045         Vector ReflectionPoint = GetReflectionPoint(EmissionPoint,
00046             DetectionPoint,
00047             DetectionMirrorParaDist,
00048             DetectionMirrorPerpDist,
00049             EmissionMirrorDist,
00050             quarticSolution, 0);
00051         Vector OtherReflectionPoint = GetReflectionPoint(EmissionPoint,
00052             DetectionPoint,
00053             DetectionMirrorParaDist,
00054             DetectionMirrorPerpDist,
```

```

00055             EmissionMirrorDist,
00056             quarticSolution, 1);
00057     if(OtherReflectionPoint.Z() > ReflectionPoint.Z()) {
00058         std::swap(ReflectionPoint, OtherReflectionPoint);
00059     }
00060     const Vector ReflectionEmissionPoint = ReflectionPoint - EmissionPoint;
00061     const Vector Direction = Photon->GetParticleDirection(TrueEmissionPoint);
00062     const double CosTheta = ReflectionEmissionPoint.Unit().Dot(Direction);
00063     return CosTheta;
00064 }
00065
00066 ReconstructedPhoton ReconstructPhoton(const PhotonHit &photonHit) {
00067     ReconstructedPhoton reconstructedPhoton(*photonHit.m_Photon);
00068     reconstructedPhoton.m_CosCherenkovAngle_TrueEmissionPoint =
00069         ReconstructCosCherenkovAngle(photonHit, true);
00070     reconstructedPhoton.m_CosCherenkovAngle =
00071         ReconstructCosCherenkovAngle(photonHit, false);
00072     return reconstructedPhoton;
00073 }
00074
00075 QuarticSolution SolveQuartic(double EmMirrorDist,
00076                             double DetMirrorParaDist,
00077                             double DetMirrorPerpDist) {
00078     // Em = Emission
00079     // Det = Detection
00080     const double DetMirrorDist2 = DetMirrorParaDist*DetMirrorParaDist
00081         + DetMirrorPerpDist*DetMirrorPerpDist;
00082     const double Denominator = 4.0*EmMirrorDist*EmMirrorDist*DetMirrorDist2;
00083     const double a = -4.0*EmMirrorDist*EmMirrorDist*DetMirrorPerpDist/Denominator;
00084     const double b = (DetMirrorPerpDist*DetMirrorPerpDist
00085         + TMath::Power(EmMirrorDist + DetMirrorParaDist, 2))/Denominator
00086         - 1.0;
00087     const double c = 2.0*EmMirrorDist*DetMirrorPerpDist
00088         *(EmMirrorDist - DetMirrorParaDist)/Denominator;
00089     const double d = DetMirrorPerpDist*DetMirrorPerpDist
00090         *(EmMirrorDist*EmMirrorDist - 1.0)/Denominator;
00091     auto polySolutions = Quartic::solve_quartic(a, b, c, d);
00092     QuarticSolution quarticSolution;
00093     std::size_t i = 0;
00094     for(const auto &polySolution : polySolutions) {
00095         if(polySolution.imag() == 0.0) {
00096             // Only two real solutions can be stored; any other count is degenerate
00097             if(i < quarticSolution.m_SinBeta.size()) {
00098                 double RealPart = polySolution.real();
00099                 quarticSolution.m_SinBeta[i] = RealPart;
00100                 quarticSolution.m_CosBeta[i] = (EmMirrorDist + DetMirrorParaDist)*RealPart;
00101                 quarticSolution.m_CosBeta[i] += EmMirrorDist*DetMirrorPerpDist
00102                     *(1.0 - 2.0*RealPart*RealPart);
00103                 quarticSolution.m_CosBeta[i] /= DetMirrorPerpDist
00104                     + 2.0*EmMirrorDist*DetMirrorParaDist*RealPart;
00105             }
00106             i++;
00107         }
00108     }
00109     if(i != quarticSolution.m_SinBeta.size()) {
00110         quarticSolution.m_DegenerateSolution = true;
00111     }
00112     return quarticSolution;
00113 }
00114
00115 Vector GetReflectionPoint(const Vector &EmPoint,
00116                         const Vector &DetPoint,
00117                         double DetMirrorParaDist,
00118                         double DetMirrorPerpDist,
00119                         double EmMirrorDist,
00120                         const QuarticSolution &quarticSolution,
00121                         std::size_t SolutionNumber) {
00122     Vector Reflection = EmPoint*(quarticSolution.m_CosBeta[SolutionNumber]/EmMirrorDist);
00123     Reflection += (quarticSolution.m_SinBeta[SolutionNumber]/DetMirrorPerpDist)
00124         *(DetPoint - EmPoint*(DetMirrorParaDist/EmMirrorDist));
00125     return Reflection;
00126 }
00127
00128 }

```

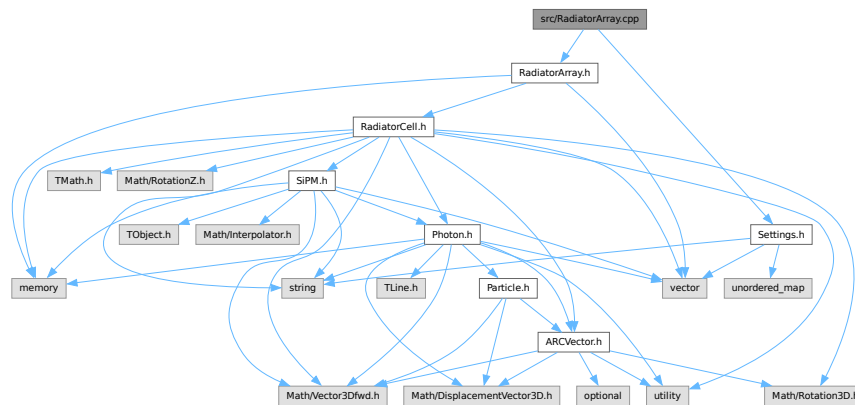
7.87 src/RadiatorArray.cpp File Reference

```

#include "RadiatorArray.h"
#include "Settings.h"

```

Include dependency graph for RadiatorArray.cpp:



7.88 RadiatorArray.cpp

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 2nd September 2022
00002
00003 #include "RadiatorArray.h"
00004 #include "Settings.h"
00005
00006 RadiatorArray::RadiatorArray() :
00007     m_NumberMainRowCells(Settings::GetSizeT("ARCGeometry/CellsPerRow")),
00008     m_xHexDist(Settings::GetDouble("ARCGeometry/Length") /
00009         static_cast<double>(2 * m_NumberMainRowCells - 1)),
00010     m_yHexDist(m_xHexDist * TMath::Sqrt(3) / 2.0) {
00011 }
00012
00013 RadiatorCell* RadiatorArray::GetRadiatorCell(std::size_t i, std::size_t j) {
00014     int Index = FindRadiatorIndex(i, j);
00015     if (Index < 0) {
00016         return nullptr;
00017     } else {
00018         return m_Cells[static_cast<std::size_t>(Index)].get();
00019     }
00020 }
00021
00022 const RadiatorCell* RadiatorArray::operator()(std::size_t i, std::size_t j) const {
00023     int Index = FindRadiatorIndex(i, j);
00024     if (Index < 0) {
00025         return nullptr;
00026     } else {
00027         return m_Cells[static_cast<std::size_t>(Index)].get();
00028     }
00029 }

```

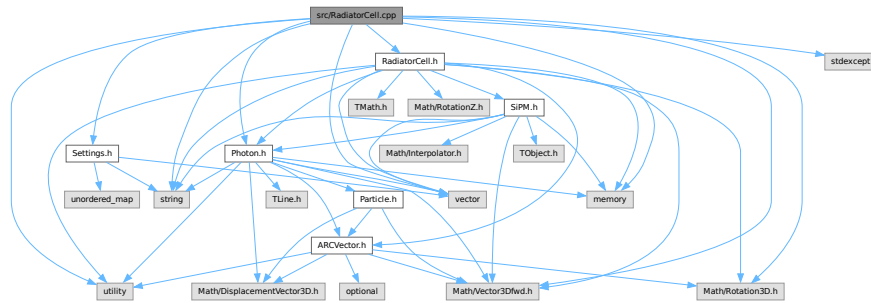
7.89 src/RadiatorCell.cpp File Reference

```

#include <vector>
#include <utility>
#include <string>
#include <memory>
#include <stdexcept>
#include "Math/Vector3Dfwd.h"
#include "Math/Rotation3D.h"
#include "RadiatorCell.h"

```

```
#include "Settings.h"
#include "Photon.h"
Include dependency graph for RadiatorCell.cpp:
```



7.90 RadiatorCell.cpp

[Go to the documentation of this file.](#)

```
00001 // Martin Duy Tat 29th April 2022
00002
00003 #include<vector>
00004 #include<utility>
00005 #include<string>
00006 #include<memory>
00007 #include<stdexcept>
00008 #include"Math/Vector3Dfwd.h"
00009 #include"Math/Rotation3D.h"
00010 #include"RadiatorCell.h"
00011 #include"Settings.h"
00012 #include"Photon.h"
00013
00014 RadiatorCell::RadiatorCell(std::size_t CellColumnNumber,
00015                             std::size_t CellRowNumber,
00016                             double HexagonSize,
00017                             const Vector &Position,
00018                             const Rotation3D &Rotation,
00019                             const std::string &Prefix):
00020     m_Position(Position),
00021     m_Rotation(Rotation),
00022     m_RadiatorThickness(Settings::GetDouble("RadiatorCell/RadiatorThickness")),
00023     m_VesselThickness(Settings::GetDouble("RadiatorCell/VesselThickness")),
00024     m_CoolingThickness(Settings::GetDouble("RadiatorCell/CoolingThickness")),
00025     m_AerogelThickness(Settings::GetDouble("RadiatorCell/AerogelThickness")),
00026     m_HexagonSize(HexagonSize),
00027     m_CellNumber(std::make_pair(CellColumnNumber, CellRowNumber)),
00028     m_MirrorCurvature(Settings::GetDouble("RadiatorCell/MirrorCurvature")),
00029     m_DefaultMirrorCentre(0.0, 0.0, GetMirrorCurvatureCentreZ()),
00030     m_MirrorCentre(m_DefaultMirrorCentre, m_Position, m_Rotation) {
00031     const std::string RadiatorName = Prefix + "Radiator_c"
00032                                     + std::to_string(m_CellNumber.first)
00033                                     + "_r"
00034                                     + std::to_string(m_CellNumber.second)
00035                                     + "_";
00036     const std::string CurvatureName = "RadiatorCell/" + RadiatorName + "Curvature";
00037     if(Settings::Exists(CurvatureName)) {
00038         SetMirrorCurvature(Settings::GetDouble(CurvatureName));
00039     }
00040     const std::string XPositionName = "RadiatorCell/" + RadiatorName + "XPosition";
00041     if(Settings::Exists(XPositionName)) {
00042         SetMirrorXPosition(Settings::GetDouble(XPositionName));
00043     }
00044     const std::string ZPositionName = "RadiatorCell/" + RadiatorName + "ZPosition";
00045     if(Settings::Exists(ZPositionName)) {
00046         SetMirrorZPosition(Settings::GetDouble(ZPositionName));
00047     }
00048     const std::string DetPositionName = "RadiatorCell/" + RadiatorName + "DetPosition";
00049     if(Settings::Exists(DetPositionName)) {
00050         SetDetectorPosition(Settings::GetDouble(DetPositionName));
00051     }
00052     const std::string DetTiltName = "RadiatorCell/" + RadiatorName + "DetTilt";
00053     if(Settings::Exists(DetTiltName)) {
```

```

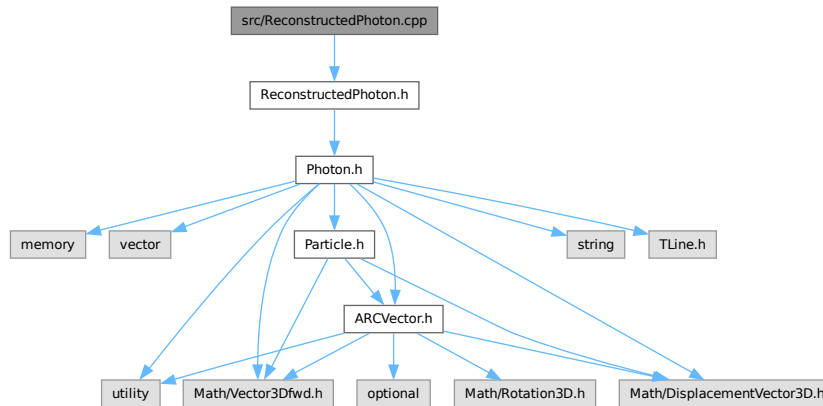
00054     SetDetectorTilt (Settings::GetDouble (DetTiltName));
00055 }
00056 }
00057
00058 double RadiatorCell::GetRadiatorThickness() const {
00059     return m_RadiatorThickness;
00060 }
00061
00062 double RadiatorCell::GetVesselThickness() const {
00063     return m_VesselThickness;
00064 }
00065
00066 double RadiatorCell::GetCoolingThickness() const {
00067     return m_CoolingThickness;
00068 }
00069
00070 double RadiatorCell::GetAerogelThickness() const {
00071     return m_AerogelThickness;
00072 }
00073
00074 const Vector& RadiatorCell::GetMirrorCentre() const {
00075     return m_MirrorCentre.LocalVector();
00076 }
00077
00078 Vector RadiatorCell::GetGlobalMirrorCentre() const {
00079     return m_MirrorCentre.GlobalVector();
00080 }
00081
00082 double RadiatorCell::GetMirrorCurvatureCentreZ() const {
00083     return m_RadiatorThickness
00084         - m_MirrorCurvature
00085         - 2*m_VesselThickness
00086         - m_CoolingThickness;
00087 }
00088
00089 double RadiatorCell::GetMirrorCurvature() const {
00090     return m_MirrorCurvature;
00091 }
00092
00093 bool RadiatorCell::IsInsideCell(const Photon &photon) const {
00094     return IsInsideCell(photon.GetPosition());
00095 }
00096
00097 const Vector& RadiatorCell::GetRadiatorPosition() const {
00098     return m_Position;
00099 }
00100
00101 const Rotation3D& RadiatorCell::GetRadiatorRotation() const {
00102     return m_Rotation;
00103 }
00104
00105 double RadiatorCell::GetHexagonSize() const {
00106     return m_HexagonSize;
00107 }
00108
00109 std::pair<std::size_t, std::size_t> RadiatorCell::GetCellNumber() const {
00110     return m_CellNumber;
00111 }
00112
00113 const SiPM& RadiatorCell::GetDetector() const {
00114     return m_Detector;
00115 }
00116
00117 void RadiatorCell::SetMirrorCurvature(double Curvature) {
00118     m_MirrorCurvature = Curvature;
00119 }
00120
00121 void RadiatorCell::SetMirrorXPosition(double x) {
00122     m_MirrorCentre.SetX(m_DefaultMirrorCentre.X() + x);
00123 }
00124
00125 void RadiatorCell::SetMirrorZPosition(double z) {
00126     m_MirrorCentre.SetZ(m_DefaultMirrorCentre.Z() + z);
00127 }
00128
00129 void RadiatorCell::SetDetectorPosition(double x) {
00130     m_Detector.SetDetectorPosition(x);
00131 }
00132
00133 void RadiatorCell::SetDetectorTilt(double Angle) {
00134     m_Detector.SetDetectorTilt (Angle);
00135 }
00136
00137 RotationZ RadiatorCell::ReversePhiRotation() const {
00138     return RotationZ(0.0);
00139 }

```

7.91 src/ReconstructedPhoton.cpp File Reference

```
#include "ReconstructedPhoton.h"
```

Include dependency graph for ReconstructedPhoton.cpp:



7.92 ReconstructedPhoton.cpp

[Go to the documentation of this file.](#)

```

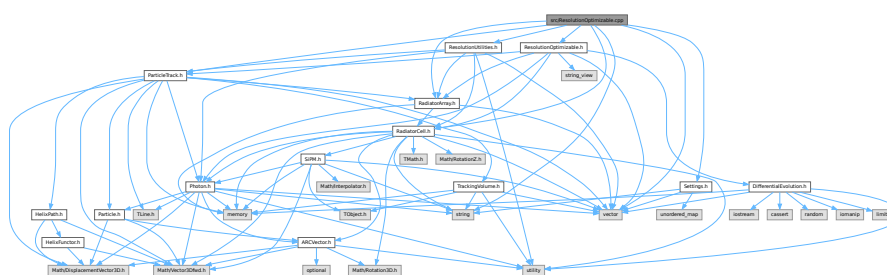
00001 // Martin Duy Tat 3rd May 2022
00002
00003 #include "ReconstructedPhoton.h"
00004
00005 ReconstructedPhoton::ReconstructedPhoton(const Photon &photon) :
00006     m_Photon(&photon),
00007     m_CosCherenkovAngle_TrueEmissionPoint(1.0),
00008     m_CosCherenkovAngle(1.0) {
00009 }
  
```

7.93 src/ResolutionOptimizable.cpp File Reference

```

#include <vector>
#include <string>
#include "ResolutionOptimizable.h"
#include "Settings.h"
#include "RadiatorCell.h"
#include "ParticleTrack.h"
#include "ResolutionUtilities.h"
#include "RadiatorArray.h"
  
```

Include dependency graph for ResolutionOptimizable.cpp:



Typedefs

- using `Constraints` = `de::IOptimizable::Constraints`

7.93.1 Typedef Documentation

7.93.1.1 Constraints

using `Constraints` = `de::IOptimizable::Constraints`

Definition at line 12 of file `ResolutionOptimizable.cpp`.

7.94 ResolutionOptimizable.cpp

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 28th July 2022
00002
00003 #include<vector>
00004 #include<string>
00005 #include"ResolutionOptimizable.h"
00006 #include"Settings.h"
00007 #include"RadiatorCell.h"
00008 #include"ParticleTrack.h"
00009 #include"ResolutionUtilities.h"
00010 #include"RadiatorArray.h"
00011
00012 using Constraints = de::IOptimizable::Constraints;
00013
00014 ResolutionOptimizable::ResolutionOptimizable(RadiatorCell &radiatorCell,
00015                                             const RadiatorArray &radiatorArray,
00016                                             const Tracks &Particles):
00017     IOptimizable(),
00018     m_RadiatorCell(&radiatorCell),
00019     m_RadiatorArray(&radiatorArray),
00020     m_Particles(&Particles),
00021     m_Seed(Settings::GetSizeT("General/Seed")) {
00022     for(std::size_t i = 0; i < m_Parameters.size(); i++) {
00023         std::string ParameterName = "Optimisation/" + std::string(m_Parameters[i]) + "_";
00024         if(Settings::GetBool(ParameterName + "IsFixed")) {
00025             double Value = Settings::GetDouble(ParameterName + "value");
00026             m_FixedParameters.insert({i, Value});
00027         } else {
00028             double Min = Settings::GetDouble(ParameterName + "min");
00029             double Max = Settings::GetDouble(ParameterName + "max");
00030             m_ParameterSpace.emplace_back(Min, Max, true);
00031         }
00032     }
00033 }
00034
00035 double ResolutionOptimizable::EvaluateCost(std::vector<double> x) const {
00036     std::array<double, 5> xx;
00037     std::size_t j = 0;
00038     for(std::size_t i = 0; i < m_Parameters.size(); i++) {
00039         const auto iter = m_FixedParameters.find(i);
00040         if(iter == m_FixedParameters.end()) {
00041             xx[i] = x[j];
00042             j++;
00043         } else {
00044             xx[i] = iter->second;
00045         }
00046     }
00047     const double Resolution =
00048         ResolutionUtilities::fcn(xx[0], xx[1], xx[2], xx[3], xx[4],
00049                                *m_RadiatorCell, *m_RadiatorArray,
00050                                *m_Particles, m_Seed, true);
00051     return Resolution;
00052 }
00053
00054 std::size_t ResolutionOptimizable::NumberOfParameters() const {
00055     return m_Parameters.size() - m_FixedParameters.size();
00056 }
00057

```

```

00058 std::vector<Constraints> ResolutionOptimizable::GetConstraints() const {
00059     return m_ParameterSpace;
00060 }
00061
00062 std::unordered_map<std::size_t, double>
00063 ResolutionOptimizable::GetFixedParameters() const {
00064     return m_FixedParameters;
00065 }

```

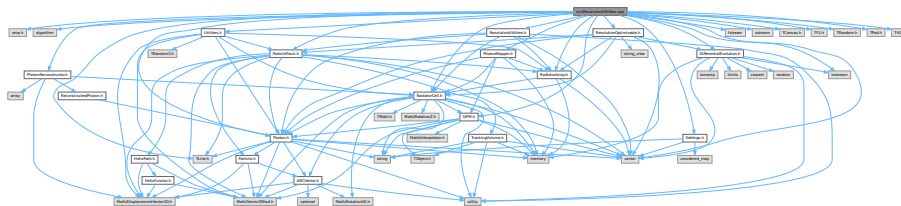
7.95 src/ResolutionUtilities.cpp File Reference

```

#include <omp.h>
#include <algorithm>
#include <vector>
#include <fstream>
#include <iostream>
#include <sstream>
#include "TCanvas.h"
#include "TF1.h"
#include "TLine.h"
#include "TRandom.h"
#include "TPad.h"
#include "TH1.h"
#include "ResolutionUtilities.h"
#include "RadiatorCell.h"
#include "ParticleTrack.h"
#include "Photon.h"
#include "PhotonMapper.h"
#include "PhotonReconstructor.h"
#include "SiPM.h"
#include "DifferentialEvolution.h"
#include "ResolutionOptimizable.h"
#include "Settings.h"
#include "RadiatorArray.h"
#include "Utilities.h"

```

Include dependency graph for ResolutionUtilities.cpp:



Classes

- struct [ResolutionUtilities::ResolutionStruct](#)

Namespaces

- namespace [ResolutionUtilities](#)

Functions

- double [ResolutionUtilities::CalculateResolution](#) (const [Tracks](#) &Particles, const [RadiatorCell](#) *radiatorCell, const [RadiatorArray](#) &radiatorArray, std::size_t Seed, bool IncludeCentrePenalty)
- double [ResolutionUtilities::fcn](#) (double MirrorCurvature, double MirrorXPosition, double MirrorZPosition, double DetectorPosition, double DetectorTilt, [RadiatorCell](#) &radiatorCell, const [RadiatorArray](#) &radiatorArray, const [Tracks](#) &Particles, std::size_t Seed, bool IncludeCentrePenalty)
- void [ResolutionUtilities::PlotProjections](#) ([RadiatorCell](#) &radiatorCell, const [RadiatorArray](#) &radiatorArray, const [Tracks](#) &Particles)
- void [ResolutionUtilities::DoFit](#) ([RadiatorCell](#) &radiatorCell, const [RadiatorArray](#) &radiatorArray, const [Tracks](#) &Particles, std::size_t Column, std::size_t Row)

7.96 ResolutionUtilities.cpp

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 28th July 2022
00002
00003 #include<omp.h>
00004 #include<algorithm>
00005 #include<vector>
00006 #include<fstream>
00007 #include<iostream>
00008 #include<sstream>
00009 #include"TCanvas.h"
00010 #include"TF1.h"
00011 #include"TLLine.h"
00012 #include"TRandom.h"
00013 #include"TPad.h"
00014 #include"TH1.h"
00015 #include"ResolutionUtilities.h"
00016 #include"RadiatorCell.h"
00017 #include"ParticleTrack.h"
00018 #include"Photon.h"
00019 #include"PhotonMapper.h"
00020 #include"PhotonReconstructor.h"
00021 #include"SiPM.h"
00022 #include"DifferentialEvolution.h"
00023 #include"ResolutionOptimizable.h"
00024 #include"Settings.h"
00025 #include"RadiatorArray.h"
00026 #include"Utilities.h"
00027
00028 namespace ResolutionUtilities {
00029     using Utilities::ResolutionStruct;
00030
00031     namespace {
00032         constexpr int DefaultNumberThreads = 8;
00033         int GetNumberThreads() {
00034             const int NumberThreads = Settings::Exists("Optimisation/NumberThreads")
00035                 ? Settings::GetInt("Optimisation/NumberThreads")
00036                 : DefaultNumberThreads;
00037             // num_threads(0) is undefined in OpenMP
00038             return std::max(1, NumberThreads);
00039         }
00040     }
00041
00042     double CalculateResolution(const Tracks &Particles,
00043                               const RadiatorCell *radiatorCell,
00044                               const RadiatorArray &radiatorArray,
00045                               std::size_t Seed,
00046                               bool IncludeCentrePenalty) {
00047         // Trace the photons of each track in parallel, one result per track
00048         std::vector<ResolutionStruct> TrackResults(Particles.size());
00049         const int NumberThreads = GetNumberThreads();
00050         #pragma omp parallel for num_threads(NumberThreads)
00051         for(std::size_t i = 0; i < Particles.size(); i++) {
00052             // Each track gets its own photon sequence, independent of the thread schedule
00053             Utilities::Random() .SetSeed(Utilities::TrackSeed(Seed, i));
00054             TrackResults[i] =
00055                 Utilities::TrackPhotons(Particles[i], *radiatorCell, radiatorArray);
00056         }
00057         // Put it all together, in track order so that the sums do not depend on the thread schedule
00058         ResolutionStruct Total{};
00059         std::size_t PhotonsHitWallTracks = 0;
00060         for(const auto &resolutionStruct : TrackResults) {

```

```

00064         if(!resolutionStruct.HitCorrectCell) {
00065             continue;
00066         }
00067         if(resolutionStruct.HitTopWall) {
00068             PhotonsHitWallTracks++;
00069         } else if(resolutionStruct.N <= 1) {
00070             PhotonsHitWallTracks++;
00071         } else {
00072             Total.x += resolutionStruct.x;
00073             Total.N++;
00074             Total.CentreHitDistance += resolutionStruct.CentreHitDistance;
00075         }
00076     }
00077     if(Total.N == 0) {
00078         return 1000.0;
00079     }
00080     // 0.0003 is the pixel size angular resolution
00081     const double Resolution = Total.x/static_cast<double>(Total.N) + 0.0003*0.0003;
00082     const auto NumberParticles = Particles.size();
00083     auto GetFinalResolution = [&] (bool IncludeCentrePenalty) {
00084         // Penalty when hitting the upper wall
00085         const double WallPenalty = (10.0*static_cast<double>(PhotonsHitWallTracks))/
00086                                     static_cast<double>(NumberParticles);
00087         const double ResolutionWithPenalty = Resolution + WallPenalty;
00088         if(IncludeCentrePenalty) {
00089             // Penalty when far from the mirror centre
00090             const Vector AverageRingPosition = Total.CentreHitDistance/Total.N;
00091             const double CentrePenalty = 0.01*TMath::Sqrt(AverageRingPosition.Mag2());
00092             return ResolutionWithPenalty + CentrePenalty;
00093         } else {
00094             return ResolutionWithPenalty;
00095         }
00096     };
00097     const double FinalResolution = GetFinalResolution(IncludeCentrePenalty);
00098     return FinalResolution;
00099 }
00100
00101 double fcn(double MirrorCurvature,
00102            double MirrorXPosition,
00103            double MirrorZPosition,
00104            double DetectorPosition,
00105            double DetectorTilt,
00106            RadiatorCell &radiatorCell,
00107            const RadiatorArray &radiatorArray,
00108            const Tracks &Particles,
00109            std::size_t Seed,
00110            bool IncludeCentrePenalty) {
00111     radiatorCell.SetMirrorCurvature(MirrorCurvature);
00112     radiatorCell.SetMirrorXPosition(MirrorXPosition);
00113     radiatorCell.SetMirrorZPosition(MirrorZPosition);
00114     radiatorCell.SetDetectorPosition(DetectorPosition);
00115     radiatorCell.SetDetectorTilt(DetectorTilt);
00116     if(!radiatorCell.IsDetectorInsideCell()) {
00117         return 1000.0;
00118     }
00119     const double Resolution = CalculateResolution(Particles,
00120                                                  &radiatorCell,
00121                                                  radiatorArray,
00122                                                  Seed,
00123                                                  IncludeCentrePenalty);
00124     return Resolution;
00125 }
00126
00127 void PlotProjections(RadiatorCell &radiatorCell,
00128                    const RadiatorArray &radiatorArray,
00129                    const Tracks &Particles) {
00130     std::string ResultFilename = Settings::GetString("Optimisation/Filename");
00131     std::ifstream File(ResultFilename);
00132     std::vector<double> Result;
00133     std::string Line;
00134     while(std::getline(File, Line)) {
00135         std::string dummy;
00136         double Value;
00137         std::stringstream ss(Line);
00138         ss >> dummy >> Value;
00139         Result.push_back(Value);
00140     }
00141     File.close();
00142     const std::size_t Seed = Settings::GetSizeT("General/Seed");
00143     auto MinimiseFunctionMirrorCurvature = [&] (double *x, double*) {
00144         return fcn(x[0], Result[1], Result[2], Result[3], Result[4],
00145                  radiatorCell, radiatorArray, Particles, Seed, false);
00146     };
00147     auto MinimiseFunctionXPosition = [&] (double *x, double*) {
00148         return fcn(Result[0], x[0], Result[2], Result[3], Result[4],
00149                  radiatorCell, radiatorArray, Particles, Seed, false);
00150     };

```

```

00151     auto MinimiseFunctionZPosition = [&] (double *x, double*) {
00152         return fcn(Result[0], Result[1], x[0], Result[3], Result[4],
00153             radiatorCell, radiatorArray, Particles, Seed, false);
00154     };
00155     auto MinimiseFunctionDetPosition = [&] (double *x, double*) {
00156         return fcn(Result[0], Result[1], Result[2], x[0], Result[4],
00157             radiatorCell, radiatorArray, Particles, Seed, false);
00158     };
00159     auto MinimiseFunctionDetTilt = [&] (double *x, double*) {
00160         return fcn(Result[0], Result[1], Result[2], Result[3], x[0],
00161             radiatorCell, radiatorArray, Particles, Seed, false);
00162     };
00163     std::string Name1("Optimisation/MirrorCurvaturePlot_");
00164     double Curvature_min = Settings::GetDouble(Name1 + "min");
00165     double Curvature_max = Settings::GetDouble(Name1 + "max");
00166     TFl f1("MirrorCurvature", MinimiseFunctionMirrorCurvature,
00167         Curvature_min, Curvature_max, 0);
00168     std::string Name2("Optimisation/MirrorXPositionPlot_");
00169     double XPosition_min = Settings::GetDouble(Name2 + "min");
00170     double XPosition_max = Settings::GetDouble(Name2 + "max");
00171     TFl f2("XPosition", MinimiseFunctionXPosition,
00172         XPosition_min, XPosition_max, 0);
00173     std::string Name3("Optimisation/MirrorZPositionPlot_");
00174     double ZPosition_min = Settings::GetDouble(Name3 + "min");
00175     double ZPosition_max = Settings::GetDouble(Name3 + "max");
00176     TFl f3("ZPosition", MinimiseFunctionZPosition,
00177         ZPosition_min, ZPosition_max, 0);
00178
00179     std::string Name4("Optimisation/DetectorPositionPlot_");
00180     double DetPosition_min = Settings::GetDouble(Name4 + "min");
00181     double DetPosition_max = Settings::GetDouble(Name4 + "max");
00182     TFl f4("DetectorPosition", MinimiseFunctionDetPosition,
00183         DetPosition_min, DetPosition_max, 0);
00184     std::string Name5("Optimisation/DetectorTiltPlot_");
00185     double DetTilt_min = Settings::GetDouble(Name5 + "min");
00186     double DetTilt_max = Settings::GetDouble(Name5 + "max");
00187     TFl f5("DetectorTilt", MinimiseFunctionDetTilt,
00188         DetTilt_min, DetTilt_max, 0);
00189     auto GetSolutionLine = [] (double Result, double ymax = 0.0) {
00190         TLine Solution(Result, 0.0, Result, 0.9*ymax);
00191         Solution.SetLineWidth(3);
00192         return Solution;
00193     };
00194     TCanvas c1("c1", "", 1200, 900);
00195     f1.SetTitle("Mirror curvature;Curvature (m);Resolution (rad)");
00196     f1.Draw();
00197     auto Solution1 = GetSolutionLine(Result[0], f1.GetHistogram()->GetMaximum());
00198     Solution1.Draw("SAME");
00199     c1.SaveAs("MirrorCurvatureOptimisation.pdf");
00200     TCanvas c2("c2", "", 1200, 900);
00201     f2.SetTitle("Horizontal mirror position;x (m);Resolution (rad)");
00202     f2.Draw();
00203     auto Solution2 = GetSolutionLine(Result[1], f2.GetHistogram()->GetMaximum());
00204     Solution2.Draw("SAME");
00205     c2.SaveAs("XPositionOptimisation.pdf");
00206     TCanvas c3("c3", "", 1200, 900);
00207     f3.SetTitle("Vertical mirror position;z (m);Resolution (rad)");
00208     f3.Draw();
00209     auto Solution3 = GetSolutionLine(Result[2], f3.GetHistogram()->GetMaximum());
00210     Solution3.Draw("SAME");
00211     c3.SaveAs("ZPositionOptimisation.pdf");
00212     TCanvas c4("c4", "", 1200, 900);
00213     f4.SetTitle("Horizontal detector position;x (m);Resolution (rad)");
00214     f4.Draw();
00215     auto Solution4 = GetSolutionLine(Result[3], f4.GetHistogram()->GetMaximum());
00216     Solution4.Draw("SAME");
00217     c4.SaveAs("DetectorPositionOptimisation.pdf");
00218     TCanvas c5("c4", "", 1200, 900);
00219     f5.SetTitle("Detector tilt angle;#theta (^o);Resolution (rad)");
00220     f5.Draw();
00221     auto Solution5 = GetSolutionLine(Result[4], f5.GetHistogram()->GetMaximum());
00222     Solution5.Draw("SAME");
00223     c5.SaveAs("DetectorTiltOptimisation.pdf");
00224 }
00225
00226 void DoFit(RadiatorCell &radiatorCell,
00227     const RadiatorArray &radiatorArray,
00228     const Tracks &Particles,
00229     std::size_t Column,
00230     std::size_t Row) {
00231     ResolutionOptimizable resolutionOptimisable(radiatorCell, radiatorArray, Particles);
00232     const std::size_t NumberAgents = Settings::GetSizeT("Optimisation/NumberAgents");
00233     // Seed of the differential evolution search, separate from the photon seed
00234     const std::size_t DESeed = Settings::Exists("Optimisation/Seed")
00235         ? Settings::GetSizeT("Optimisation/Seed")
00236         : Settings::GetSizeT("General/Seed");
00237     de::DifferentialEvolution de(resolutionOptimisable, NumberAgents, DESeed);

```

```

00238     const int Iterations = Settings::GetInt("Optimisation/Iterations");
00239     de.Optimize(Iterations, true);
00240     auto Result = de.GetBestAgent();
00241     auto FixedParameters = resolutionOptimisable.GetFixedParameters();
00242     std::size_t TotalParameters = Result.size() + FixedParameters.size();
00243     std::vector<double> AllParameters;
00244     constexpr std::array<std::string_view, 5> ParameterNames{
00245         "Curvature",
00246         "XPosition",
00247         "ZPosition",
00248         "DetPosition",
00249         "DetTilt"};
00250     std::size_t j = 0;
00251     for(std::size_t i = 0; i < TotalParameters; i++) {
00252         auto iter = FixedParameters.find(i);
00253         if(iter != FixedParameters.end()) {
00254             AllParameters.push_back(iter->second);
00255         } else {
00256             AllParameters.push_back(Result[j]);
00257             j++;
00258         }
00259     }
00260     std::string ResultFilename = Settings::GetString("Optimisation/Filename");
00261     std::ofstream File(ResultFilename);
00262     const std::string BarrelOrEndcap = Settings::GetString("General/BarrelOrEndcap");
00263     const std::string Prefix = BarrelOrEndcap == "Barrel" ? "" : "EndCap";
00264     for(std::size_t i = 0; i < TotalParameters; i++) {
00265         File << Prefix;
00266         File << "Radiator_c" << Column << "_r" << Row << "_" << ParameterNames[i] << " ";
00267         File << AllParameters[i] << "\n";
00268     }
00269     //File << "\n" << "OptimalResolution: " << de.GetBestCost()*1000.0 << " mrad" << "\n";
00270     File.close();
00271 }
00272
00273 }

```

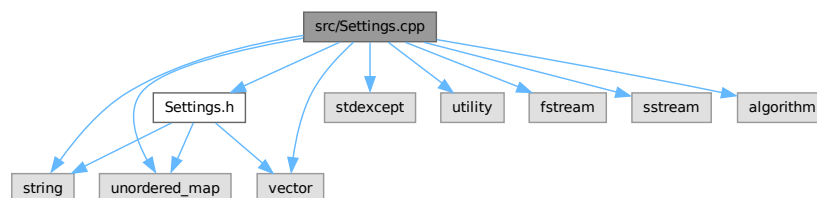
7.97 src/Settings.cpp File Reference

```

#include <string>
#include <unordered_map>
#include <stdexcept>
#include <utility>
#include <fstream>
#include <sstream>
#include <vector>
#include <algorithm>
#include "Settings.h"

```

Include dependency graph for Settings.cpp:



7.98 Settings.cpp

[Go to the documentation of this file.](#)

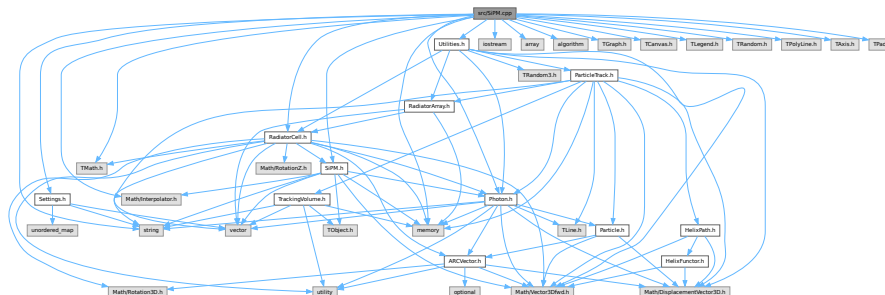
```

00001 // Martin Duy Tat 5th May 2022
00002
00003 #include<string>
00004 #include<unordered_map>
00005 #include<stdexcept>
00006 #include<utility>
00007 #include<fstream>
00008 #include<sstream>
00009 #include<vector>
00010 #include<sstream>
00011 #include<algorithm>
00012 #include"Settings.h"
00013
00014 std::unordered_map<std::string, ssMap> Settings::m_Settings;
00015
00016 void Settings::AddSettings(const std::string &Name, const std::string &Filename) {
00017     if(m_Settings.find(Name) != m_Settings.end()) {
00018         throw std::runtime_error("Settings " + Name + " already exists");
00019     }
00020     ssMap NewSettings;
00021     std::ifstream File(Filename);
00022     std::string Line;
00023     while(std::getline(File, Line)) {
00024         Line = Line.substr(0, Line.find('#'));
00025         if(Line.empty()) {
00026             continue;
00027         }
00028         std::stringstream ss(Line);
00029         std::string Key, Value;
00030         ss >> Key >> Value;
00031         if(NewSettings.find(Key) != NewSettings.end()) {
00032             throw std::runtime_error("Key " + Key + " in settings " + Name + " already exists");
00033         }
00034         NewSettings.insert({Key, Value});
00035     }
00036     File.close();
00037     m_Settings.insert({Name, std::move(NewSettings)});
00038 }
00039
00040 std::string Settings::GetString(const std::string &Setting) {
00041     const std::size_t SlashPos = Setting.find('/');
00042     std::string Name = Setting.substr(0, SlashPos);
00043     auto iter1 = m_Settings.find(Name);
00044     if(iter1 == m_Settings.end()) {
00045         std::string Known;
00046         for(const auto &[KnownName, KnownSettings] : m_Settings) {
00047             Known += " " + KnownName;
00048         }
00049         throw std::runtime_error("Cannot find settings name " + Name
00050             + ", the settings added are:" + Known);
00051     }
00052     std::string Key = Setting.substr(SlashPos + 1);
00053     auto iter2 = iter1->second.find(Key);
00054     if(iter2 == iter1->second.end()) {
00055         throw std::runtime_error("Cannot find key " + Key + " in settings " + Name);
00056     }
00057     return iter2->second;
00058 }
00059
00060 double Settings::GetDouble(const std::string &Setting) {
00061     return std::stod(GetString(Setting));
00062 }
00063
00064 int Settings::GetInt(const std::string &Setting) {
00065     return std::stoi(GetString(Setting));
00066 }
00067
00068 std::size_t Settings::GetSizeT(const std::string &Setting) {
00069     const int Integer = std::stoi(GetString(Setting));
00070     if(Integer < 0) {
00071         throw std::runtime_error("Cannot load negative integer into std::size_t");
00072     } else {
00073         return static_cast<std::size_t>(Integer);
00074     }
00075 }
00076
00077 bool Settings::GetBool(const std::string &Setting) {
00078     return GetString(Setting) == "true";
00079 }
00080
00081 std::vector<int> Settings::GetIntVector(const std::string &Setting) {
00082     std::string CommaSeparatedList = GetString(Setting);
00083     std::replace(CommaSeparatedList.begin(), CommaSeparatedList.end(), ',', ' ');
00084     std::stringstream ss(CommaSeparatedList);
00085     std::vector<int> List;
00086     int Number;
00087     while(ss >> Number) {

```

7.99 src/SiPM.cpp File Reference

Include dependency graph for SiPM.cpp:



7.100 SiPM.cpp

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 29th April 2022
00002
00003 #include<string>
00004 #include<iostream>
00005 #include<memory>
00006 #include<array>
00007 #include<algorithm>
00008 #include"TGraph.h"
00009 #include"TCanvas.h"
00010 #include"TLegend.h"
00011 #include"TRandom.h"
00012 #include"TPolyLine.h"
00013 #include"TAxis.h"
00014 #include"TPad.h"
00015 #include"TMath.h"
00016 #include"Math/Interpolator.h"
00017 #include"SiPM.h"
00018 #include"Photon.h"
00019 #include"Settings.h"
00020 #include"RadiatorCell.h"
00021 #include"Utilities.h"
00022
00023 SiPM::SiPM():
00024     m_DetectorSizeX(Settings::GetDouble("RadiatorCell/DetectorSize")),
00025     m_DetectorSizeY(m_DetectorSizeX),
00026     m_DetectorPositionX(0.0),
00027     m_DetectorPositionY(0.0),
00028     m_DetectorPositionZ(0.0),
00029     m_DetectorTilt(0.0),
00030     m_CoolingThickness(Settings::GetDouble("RadiatorCell/CoolingThickness")),
00031     m_MaxPDE(0.432),
00032     m_EndCap(Settings::GetString("General/BarrelOrEndcap") == "EndCap") {
00033 }
00034
00035 PhotonHit SiPM::AddPhotonHit(Photon &photon) const {
00036     const Vector DetectorCentre(m_DetectorPositionX,
00037                                 0.0,
00038                                 m_DetectorPositionZ);
00039     const auto PhotonPosition = photon.GetPosition().LocalVector();
00040     const Vector PhotonXZPosition(PhotonPosition.X(),
00041                                   0.0,
00042                                   PhotonPosition.Z());
00043     PhotonHit photonHit(PhotonPosition,
00044                        &photon,
00045                        PhotonXZPosition - DetectorCentre);
00046     if(photon.GetStatus() != Photon::Status::MirrorHit) {
00047         return photonHit;
00048     }
00049     const double PhotonLambda = 1239.8/photon.GetEnergy();
00050     if(PhotonLambda < m_Lambda[0] || PhotonLambda > m_Lambda[15]) {
00051         std::cout << "Warning! Generated photon outside energy range of SiPM!\n";
00052     }
00053     if(!IsDetectorHit(photon)) {
00054         photon.UpdatePhotonStatus(Photon::Status::DetectorMiss);
00055         return photonHit;
00056     }
00057     const double RandomNumber = Utilities::Random().Uniform(0.0, m_MaxPDE);
00058     const double Efficiency = m_Interpolator.Eval(PhotonLambda)*0.90*0.80;
00059     if(RandomNumber <= Efficiency) {
00060         photon.UpdatePhotonStatus(Photon::Status::DetectorHit);
00061     } else {
00062         photon.UpdatePhotonStatus(Photon::Status::EfficiencyMiss);
00063     }
00064     return photonHit;
00065 }
00066
00067 void SiPM::PlotHits(const std::string &Filename,
00068                    const std::vector<PhotonHit> &photonHits) const {
00069     std::vector<double> xHits_Aerogel, yHits_Aerogel, xHits_Gas, yHits_Gas;
00070     for(const auto &PhotonHit : photonHits) {
00071         if(PhotonHit.m_Photon->GetRadiator() == Photon::Radiators::Aerogel) {
00072             xHits_Aerogel.push_back(PhotonHit.m_HitPosition.X()*100.0);
00073             yHits_Aerogel.push_back(PhotonHit.m_HitPosition.Y()*100.0);
00074         } else if(PhotonHit.m_Photon->GetRadiator() == Photon::Radiators::Gas) {
00075             xHits_Gas.push_back(PhotonHit.m_HitPosition.X()*100.0);
00076             yHits_Gas.push_back(PhotonHit.m_HitPosition.Y()*100.0);
00077         } else {
00078             std::cout << "Warning! Detector hit with unknown origin\n";
00079         }
00080     }
00081     TGraph Graph_Aerogel(static_cast<int>(xHits_Aerogel.size()),
00082                          xHits_Aerogel.data(),

```

```

00083         yHits_Aerogel.data());
00084     TGraph Graph_Gas(static_cast<int>(xHits_Gas.size()),
00085         xHits_Gas.data(),
00086         yHits_Gas.data());
00087     TCanvas c("c", "", 1200, 1200);
00088     Graph_Aerogel.SetMarkerStyle(kFullDotLarge);
00089     Graph_Gas.SetMarkerStyle(kFullDotLarge);
00090     Graph_Aerogel.SetMarkerColor(kBlue);
00091     Graph_Gas.SetMarkerColor(kRed);
00092     Graph_Aerogel.SetTitle("Photon hits;x (cm); y (cm)");
00093     Graph_Aerogel.Draw("AP");
00094     gPad->SetLeftMargin(0.15f);
00095     gPad->SetBottomMargin(0.15f);
00096     Graph_Aerogel.GetXaxis()->SetLimits((m_DetectorPositionX
00097         - m_DetectorSizeX/2.0)*100.0,
00098         (m_DetectorPositionX
00099             + m_DetectorSizeX/2.0)*100.0);
00100     Graph_Aerogel.SetMinimum((m_DetectorPositionY - m_DetectorSizeY/2.0)*100.0);
00101     Graph_Aerogel.SetMaximum((m_DetectorPositionY + m_DetectorSizeY/2.0)*100.0);
00102     Graph_Aerogel.Draw("AP");
00103     Graph_Gas.Draw("P SAME");
00104     TLegend Legend(0.7, 0.8, 0.9, 0.9);
00105     const int Momentum = Settings::GetInt("Particle/Momentum");
00106     const std::string AerogelLabel = "Aerogel " + std::to_string(Momentum) + " GeV";
00107     Legend.AddEntry(&Graph_Aerogel, AerogelLabel.c_str(), "P");
00108     const std::string GasLabel = "Gas " + std::to_string(Momentum) + " GeV";
00109     Legend.AddEntry(&Graph_Gas, GasLabel.c_str(), "P");
00110     Legend.Draw();
00111     c.Draw();
00112     c.SaveAs(Filename.c_str());
00113 }
00114
00115 std::unique_ptr<TObject> SiPM::DrawSiPM(const Vector &RadiatorPosition) const {
00116     const double PositionZ = RadiatorPosition.X()
00117         + (m_EndCap ? m_DetectorPositionX : m_DetectorPositionZ);
00118     const double PositionX = RadiatorPosition.Z()
00119         + (m_EndCap ? m_DetectorPositionZ : m_DetectorPositionX);
00120     std::array<double, 5> xPoints, zPoints;
00121     const double DetectorTilt = m_EndCap ?
00122         -m_DetectorTilt + TMath::Pi()/2.0 :
00123         m_DetectorTilt;
00124     const double CosTheta = TMath::Cos(DetectorTilt);
00125     const double SinTheta = TMath::Sin(DetectorTilt);
00126     // Lower left corner
00127     xPoints[0] = PositionX - CosTheta*m_DetectorSizeX/2.0;
00128     zPoints[0] = PositionZ - 0.002 - SinTheta*m_DetectorSizeX/2.0;
00129     // Lower right corner
00130     xPoints[1] = PositionX + CosTheta*m_DetectorSizeX/2.0;
00131     zPoints[1] = PositionZ - 0.002 + SinTheta*m_DetectorSizeX/2.0;
00132     // Upper right corner
00133     xPoints[2] = PositionX + CosTheta*m_DetectorSizeX/2.0;
00134     zPoints[2] = PositionZ - 0.0 + SinTheta*m_DetectorSizeX/2.0;
00135     // Upper left corner
00136     xPoints[3] = PositionX - CosTheta*m_DetectorSizeX/2.0;
00137     zPoints[3] = PositionZ - 0.0 - SinTheta*m_DetectorSizeX/2.0;
00138     // Back to left corner
00139     xPoints[4] = PositionX - CosTheta*m_DetectorSizeX/2.0;
00140     zPoints[4] = PositionZ - 0.002 - SinTheta*m_DetectorSizeX/2.0;
00141     TPolyLine DetectorLine(5, xPoints.data(), zPoints.data());
00142     DetectorLine.SetFillColor(kBlack);
00143     return std::make_unique<TPolyLine>(DetectorLine);
00144 }
00145
00146 bool SiPM::IsDetectorHit(const Photon &photon) const {
00147     if(!photon.GetRadiatorCell()->IsInsideCell(photon)) {
00148         return false;
00149     }
00150     const auto PhotonPosition = photon.GetPosition().LocalVector();
00151     const double photon_x = PhotonPosition.X();
00152     const double photon_z = PhotonPosition.Z();
00153     const double XDist = TMath::Sqrt((photon_x - m_DetectorPositionX)*
00154         (photon_x - m_DetectorPositionX)
00155         + (photon_z - m_DetectorPositionZ)*
00156         (photon_z - m_DetectorPositionZ));
00157     if(XDist > m_DetectorSizeX/2.0) {
00158         return false;
00159     } else if(TMath::Abs(PhotonPosition.Y() - m_DetectorPositionY)
00160         > m_DetectorSizeY/2.0) {
00161         return false;
00162     } else {
00163         return true;
00164     }
00165 }
00166
00167 void SiPM::SetDetectorPosition(double x) {
00168     m_DetectorPositionX = x;
00169 }

```

```

00170
00171 void SiPM::SetDetectorTilt(double Angle) {
00172     m_DetectorTilt = Angle;
00173     const double AbsAngle = TMath::Abs(Angle);
00174     const double DetectorSizeXOver2 = m_DetectorSizeX*0.5;
00175     if(AbsAngle > TMath::ASin(m_CoolingThickness/DetectorSizeXOver2)) {
00176         m_DetectorPositionZ = TMath::Sin(AbsAngle)*DetectorSizeXOver2;
00177         m_DetectorPositionZ -= m_CoolingThickness;
00178     } else {
00179         m_DetectorPositionZ = 0.0;
00180     }
00181 }
00182
00183 thread_local const Interpolator SiPM::m_Interpolator{
00184     std::vector<double>(m_Lambda.begin(), m_Lambda.end()),
00185     std::vector<double>(m_PDE.begin(), m_PDE.end()),
00186     InterpolationType::kAKIMA
00187 };
00188
00189 double SiPM::GetDetectorXPosition() const {
00190     return m_DetectorPositionX;
00191 }
00192
00193 double SiPM::GetDetectorZPosition() const {
00194     return m_DetectorPositionZ;
00195 }
00196
00197 double SiPM::GetDetectorTilt() const {
00198     return m_DetectorTilt;
00199 }
00200
00201 double SiPM::GetDetectorSizeX() const {
00202     return m_DetectorSizeX;
00203 }
00204
00205 double SiPM::GetDetectorSizeY() const {
00206     return m_DetectorSizeY;
00207 }

```

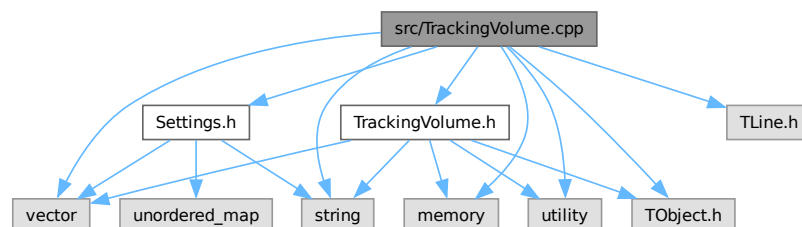
7.101 src/TrackingVolume.cpp File Reference

```

#include <vector>
#include <string>
#include <memory>
#include <utility>
#include "TObject.h"
#include "TLine.h"
#include "TrackingVolume.h"
#include "Settings.h"

```

Include dependency graph for TrackingVolume.cpp:



7.102 TrackingVolume.cpp

[Go to the documentation of this file.](#)

```

00001 // Martin Duy Tat 29th April 2022
00002
00003 #include<vector>
00004 #include<string>
00005 #include<memory>
00006 #include<utility>
00007 #include"TObject.h"
00008 #include"TLine.h"
00009 #include"TrackingVolume.h"
00010 #include"Settings.h"
00011
00012 TrackingVolume::TrackingVolume():
00013     m_Radius(Settings::GetDouble("ARCGeometry/Radius")),
00014     m_Length(Settings::GetDouble("ARCGeometry/Length")),
00015     m_CellsPerRow(Settings::GetInt("ARCGeometry/CellsPerRow")),
00016     m_FieldStrength(Settings::GetDouble("ARCGeometry/FieldStrength")) {
00017 }
00018
00019 double TrackingVolume::GetRadius() const {
00020     return m_Radius;
00021 }
00022
00023 double TrackingVolume::GetFieldStrength() const {
00024     return m_FieldStrength;
00025 }
00026
00027 std::vector<std::pair<std::unique_ptr<TObject>, std::string>
00028 TrackingVolume::DrawARCGeometry() const {
00029     /*std::vector<std::pair<std::unique_ptr<TObject>, std::string> Lines;
00030     const double ThetaLength = m_Length/(2*m_CellsPerRow - 1);
00031     const double VesselThickness = Settings::GetDouble("RadiatorCell/VesselThickness");
00032     if(Settings::GetBool("General/FullArray")) {
00033         for(int i = 0; i <= m_CellsPerRow; i++) {
00034             TLine Line1(0.0, 0.0, ThetaLength*(i - 0.5), m_Radius - VesselThickness);
00035             Line1.SetLineStyle(kDotted);
00036             Line1.SetLineWidth(1);
00037             Line1.SetLineColor(kBlack);
00038             Lines.push_back(std::make_pair(std::make_unique<TLine>(Line1), ""));
00039         }
00040     } else {
00041         TLine Line1(0.0, 0.0, ThetaLength/2.0, m_Radius - VesselThickness);
00042         Line1.SetLineStyle(kDashed);
00043         Line1.SetLineWidth(1);
00044         Line1.SetLineColor(kBlack);
00045         Lines.push_back(std::make_pair(std::make_unique<TLine>(Line1), ""));
00046         TLine Line2(0.0, 0.0, -ThetaLength/2.0, m_Radius - VesselThickness);
00047         Line2.SetLineStyle(kDashed);
00048         Line2.SetLineWidth(1);
00049         Line2.SetLineColor(kBlack);
00050         Lines.push_back(std::make_pair(std::make_unique<TLine>(Line2), ""));
00051     }
00052     return Lines;*/
00053     return {};
00054 }

```

7.103 src/Utilities.cpp File Reference

```

#include <cstdint>
#include "TRandom.h"
#include "TMath.h"
#include "Utilities.h"
#include "Settings.h"
#include "RadiatorCell.h"
#include "BarrelRadiatorCell.h"
#include "ParticleTrack.h"
#include "RadiatorArray.h"
#include "PhotonMapper.h"
#include "PhotonReconstructor.h"
#include "ParticleMass.h"

```



```

00022     return Vector{R*CosPhi*SinTheta, R*SinPhi*SinTheta, R*CosTheta};
00023 }
00024
00025 Vector GenerateRandomBarrelTrackZRange(double z_min, double z_max) {
00026     const double CosTheta_boundary =
00027         Settings::GetDouble("ARCGeometry/CosTheta_boundary");
00028     while(true) {
00029         const double Radius = Settings::GetDouble("ARCGeometry/Radius");
00030         const double z = gRandom->Uniform(z_min, z_max);
00031         const double CosTheta = z/TMath::Sqrt(z*z + Radius*Radius);
00032         if(TMath::Abs(CosTheta) > CosTheta_boundary) {
00033             continue;
00034         }
00035         const double Phi = Settings::GetBool("Particle/RandomPhi")
00036 ? gRandom->Uniform(-TMath::Pi(), TMath::Pi())
00037 : gRandom->Uniform(Settings::GetDouble("Particle/Phi_min"),
00038                 Settings::GetDouble("Particle/Phi_max"));
00039         const Vector Momentum = VectorFromSpherical(GetMomentumMag(), CosTheta, Phi);
00040         return Momentum;
00041     }
00042 }
00043
00044 Vector GenerateRandomBarrelTrack(double &CosTheta, double &Phi) {
00045     const double CosTheta_boundary =
00046         Settings::GetDouble("ARCGeometry/CosTheta_boundary");
00047     while(true) {
00048         const double Radius = Settings::GetDouble("ARCGeometry/Radius");
00049         const double z = gRandom->Uniform(Settings::GetDouble("Particle/z_min"),
00050                 Settings::GetDouble("Particle/z_max"));
00051         CosTheta = z/TMath::Sqrt(z*z + Radius*Radius);
00052         if(TMath::Abs(CosTheta) > CosTheta_boundary) {
00053             continue;
00054         }
00055         Phi = Settings::GetBool("Particle/RandomPhi")
00056 ? gRandom->Uniform(-TMath::Pi(), TMath::Pi())
00057 : gRandom->Uniform(Settings::GetDouble("Particle/Phi_min"),
00058                 Settings::GetDouble("Particle/Phi_max"));
00059         const Vector Momentum = VectorFromSpherical(GetMomentumMag(), CosTheta, Phi);
00060         return Momentum;
00061     }
00062 }
00063
00064 Vector GenerateRandomEndCapTrack() {
00065     const double CosTheta_boundary =
00066         Settings::GetDouble("ARCGeometry/CosTheta_boundary");
00067     const double SecTheta = 1.0/CosTheta_boundary;
00068     const double TanTheta = TMath::Sqrt(SecTheta*SecTheta - 1.0);
00069     const double z = Settings::GetDouble("ARCGeometry/BarrelZ");
00070     const double Radius_max = TanTheta*z;
00071     const double Radius_min = Settings::GetDouble("ARCGeometry/EndCapInnerRadius");
00072     auto GetUniformCircle = [=] () {
00073         double x = CosTheta_boundary, y = CosTheta_boundary;
00074         double r2, CosTheta;
00075         do {
00076             x = gRandom->Uniform(-Radius_max, Radius_max);
00077             y = gRandom->Uniform(-Radius_max, Radius_max);
00078             r2 = x*x + y*y;
00079             CosTheta = z/TMath::Sqrt(r2 + z*z);
00080         } while(CosTheta < CosTheta_boundary ||
00081                 r2 < Radius_min*Radius_min);
00082         return std::make_pair(x, y);
00083     };
00084     auto [x, y] = GetUniformCircle();
00085     const Vector Momentum = Vector(x, y, z).Unit()*GetMomentumMag();
00086     return Momentum;
00087 }
00088
00089 double GetMomentumMag() {
00090     if(Settings::GetBool("Particle/ConstantMomentum")) {
00091         return Settings::GetDouble("Particle/Momentum");
00092     } else {
00093         const double Momentum_min = Settings::GetDouble("Particle/Momentum_min");
00094         const double Momentum_max = Settings::GetDouble("Particle/Momentum_max");
00095         const double Momentum_log_min = TMath::Log(Momentum_min);
00096         const double Momentum_log_max = TMath::Log(Momentum_max);
00097         const double Momentum_log = gRandom->Uniform(Momentum_log_min,
00098                 Momentum_log_max);
00099         return TMath::Exp(Momentum_log);
00100     }
00101 }
00102
00103 ResolutionStruct TrackPhotons(ParticleTrack particleTrack,
00104                             const RadiatorCell &radiatorCell,
00105                             const RadiatorArray &radiatorArray) {
00106     // First find the correct cell
00107     if(!particleTrack.FindRadiator(radiatorArray)) {
00108         return ResolutionStruct{};

```

```

00109     }
00110     // Track through the cell, ignoring tracks that cannot be tracked (as in apps/RunARC.cpp)
00111     if(particleTrack.GetParticleLocation() != ParticleTrack::Location::EntranceWindow) {
00112         return ResolutionStruct{};
00113     }
00114     if(!particleTrack.TrackThroughAerogel()) {
00115         return ResolutionStruct{};
00116     }
00117     particleTrack.TrackThroughGasToMirror();
00118     if(particleTrack.GetParticleLocation() != ParticleTrack::Location::Mirror) {
00119         particleTrack.TrackToNextCell(radiatorArray);
00120     }
00121     // Check if track hit the cell we're optimising
00122     if(particleTrack.GetRadiatorCell() != &radiatorCell) {
00123         if(particleTrack.GetRadiatorCell()) {
00124             return ResolutionStruct{};
00125         } else {
00126             ResolutionStruct resolutionStruct;
00127             resolutionStruct.HitCorrectCell = true;
00128             resolutionStruct.HitTopWall = true;
00129             return resolutionStruct;
00130         }
00131     }
00132     // If we're in the cell we're optimising, make sure we hit the mirror
00133     const auto Location = particleTrack.GetParticleLocation();
00134     if(Location == ParticleTrack::Location::MissedMirror) {
00135         return ResolutionStruct{0.0, 0, true, true};
00136     }
00137     // If particle hit the mirror, generate photons from gas
00138     auto Photons = Location == ParticleTrack::Location::Mirror ?
00139         particleTrack.GeneratePhotonsFromGas() :
00140         std::vector<Photon>();
00141     // Loop over all photons
00142     ResolutionStruct resolutionStruct;
00143     resolutionStruct.HitCorrectCell = true;
00144     std::vector<double> CherenkovAngles;
00145     CherenkovAngles.reserve(Photons.size());
00146     for(auto &Photon : Photons) {
00147         // Trace photons through optics and obtain detector hits
00148         auto photonHit = PhotonMapper::TracePhoton(Photon, radiatorArray);
00149         if(Photon.GetRadiatorCell() != &radiatorCell) {
00150             // If photon hits a different cell, skip
00151             continue;
00152         } else if(Photon.GetStatus() == Photon::Status::DetectorHit) {
00153             // If photon is detected, reconstruct Cherenkov angle
00154             auto reconstructedPhoton =
00155                 PhotonReconstructor::ReconstructPhoton(*photonHit);
00156             const double CosCherenkovAngle = reconstructedPhoton.m_CosCherenkovAngle;
00157             if(!PhotonReconstructor::IsReconstructed(CosCherenkovAngle)) {
00158                 continue;
00159             }
00160             const double CherenkovAngle = TMath::ACos(CosCherenkovAngle);
00161             CherenkovAngles.push_back(CherenkovAngle);
00162             resolutionStruct.N++;
00163             resolutionStruct.SumWeights += Photon.GetWeight();
00164             resolutionStruct.CentreHitDistance += photonHit->m_CentreHitDistance;
00165             } else if(Photon.GetStatus() == Photon::Status::WallMiss ||
00166                 Photon.GetStatus() == Photon::Status::Backwards ||
00167                 Photon.GetStatus() == Photon::Status::DetectorMiss) {
00168                 // If photon doesn't hit detector, a penalty will be added later
00169                 resolutionStruct.HitTopWall = true;
00170             }
00171         }
00172         if(resolutionStruct.N > 0) {
00173             resolutionStruct.x = TMath::RMS(CherenkovAngles.begin(),
00174                 CherenkovAngles.end());
00175             // Divide by the physical photon count, as RunARC does, so that the
00176             // photon multiplier only improves the statistics of the estimate
00177             resolutionStruct.x /= TMath::Sqrt(resolutionStruct.SumWeights);
00178             resolutionStruct.CentreHitDistance /= resolutionStruct.N;
00179         }
00180         return resolutionStruct;
00181     }
00182
00183 double GetIndexRefraction(Photon::Radiator Radiator,
00184     bool ChromaticDispersion,
00185     double Energy) {
00186     switch(Radiator) {
00187     case Photon::Radiator::Aerogel:
00188     {
00189         auto GetIndex = [] (double eph) {
00190             const double InsideSqrt = 46.41/(113.8 - eph*eph) +
00191                 228.7/(328.5 - eph*eph);
00192             return 1.0 + 0.03*2.1467*(TMath::Sqrt(1.0 + InsideSqrt) - 1.0);
00193         };
00194         if(ChromaticDispersion) {
00195             // Equation from Roger Forty via email

```

```

00196         return GetIndex(Energy);
00197     } else {
00198         const double eph = 1239.841987427/400.0;
00199         return GetIndex(eph);
00200     }
00201 }
00202 case Photon::Radiator::Gas:
00203 {
00204     auto GetIndex = [] (double L) {
00205         return 1.0 + 0.25324*1e-6/((1.0/(73.7*73.7)) - (1.0/(L*L)));
00206     };
00207     if(ChromaticDispersion) {
00208         // Pressure at 1.0 bar
00209         // Sellmeier equation with coefficients from
00210         // https://twiki.cern.ch/twiki/bin/view/LHCb/C4F10
00211         // They are similar to A. Filippas, et al. Nucl. Instr. and Meth. B, 196 (2002),
00212         // p. 340 but now quite...?
00213         const double Lambda = 1239.841987427/Energy;
00214         return GetIndex(Lambda);
00215     } else {
00216         return GetIndex(400);
00217     }
00218 }
00219 default:
00220     return 1.0;
00221 }
00222 }
00223
00224 double GetPredictedCherenkovAngle(double Momentum,
00225     int ID,
00226     Photon::Radiator Radiator) {
00227     const double n = GetIndexRefraction(Radiator, false);
00228     const double Mass = ParticleMass::GetMass(ID);
00229     const double Energy = TMath::Sqrt(Mass*Mass + Momentum*Momentum);
00230     const double v = Momentum/Energy;
00231     return TMath::ACos(1.0/(v*n));
00232 }
00233
00234 double GetCherenkovAngleDifference(double Momentum,
00235     int ID1,
00236     int ID2,
00237     Photon::Radiator Radiator) {
00238     const double Angle1 = GetPredictedCherenkovAngle(Momentum, ID1, Radiator);
00239     const double Angle2 = GetPredictedCherenkovAngle(Momentum, ID2, Radiator);
00240     return TMath::Abs(Angle1 - Angle2);
00241 }
00242
00243 TRandom3 &Random() {
00244     static thread_local TRandom3 Generator;
00245     return Generator;
00246 }
00247
00248 ULONG_t TrackSeed(std::size_t Seed, std::size_t TrackNumber) {
00249     // Mix the two integers (64-bit finaliser of MurmurHash3)
00250     std::uint64_t x = (static_cast<std::uint64_t>(Seed) « 32) ^ TrackNumber;
00251     x ^= x » 33;
00252     x *= 0xff51afd7ed558ccdULL;
00253     x ^= x » 33;
00254     x *= 0xc4ceb9fela85ec53ULL;
00255     x ^= x » 33;
00256     // TRandom3 interprets seed 0 as "seed from the clock"
00257     const std::uint32_t Mixed = static_cast<std::uint32_t>(x);
00258     return Mixed == 0 ? 1 : Mixed;
00259 }
00260
00261 }

```


Index

- ~IOptimizable
 - de::IOptimizable, [62](#)
- ~Particle
 - Particle, [66](#)
- ~ParticleTrack
 - ParticleTrack, [71](#)
- ~Photon
 - Photon, [79](#)
- ~RadiatorArray
 - RadiatorArray, [88](#)
- ~RadiatorCell
 - RadiatorCell, [92](#)
- AddAerogelTravelDistance
 - Photon, [79](#)
- AddObject
 - EventDisplay, [53](#)
- AddPhotonHit
 - SiPM, [111](#)
- AddSettings
 - Settings, [109](#)
- Aerogel
 - Photon, [77](#)
- AerogelScattered
 - Photon, [78](#)
- apps/CMakeLists.txt, [176](#)
- apps/OptimiseARC.cpp, [119](#), [120](#)
- apps/RunARC.cpp, [122](#), [123](#)
- ARCVector, [19](#)
 - ARCVector, [19](#), [21](#)
 - AssignLocalCoordinates, [21](#)
 - ConvertToGlobal, [21](#)
 - GlobalVector, [22](#)
 - IsLocal, [22](#)
 - LocalVector, [22](#)
 - MapPhi, [22](#)
 - operator+=, [22](#)
 - ReflectY, [22](#)
 - ReflectZ, [23](#)
 - SetGlobalVector, [23](#)
 - SetX, [23](#)
 - SetZ, [23](#)
- ARCVector.h
 - Rotation3D, [127](#)
 - Vector, [127](#)
- AssignLocalCoordinates
 - ARCVector, [21](#)
- Backwards
 - Photon, [78](#)
- BarrelHelixFunctor, [24](#)
 - BarrelHelixFunctor, [25](#)
 - operator(), [25](#)
- BarrelRadiatorArray, [26](#)
 - BarrelRadiatorArray, [27](#)
 - DrawRadiatorArray, [27](#)
 - FindRadiator, [27](#)
 - IsAboveMainRow, [28](#)
 - IsAboveUpperRow, [29](#)
 - IsBelowMainRow, [29](#)
- BarrelRadiatorCell, [30](#)
 - BarrelRadiatorCell, [32](#)
 - DrawRadiatorGeometry, [32](#)
 - IsDetectorInsideCell, [32](#)
 - IsEdgeCell, [33](#)
 - IsInsideCell, [33](#)
 - m_BarrelRadius, [33](#)
 - ReversePhiRotation, [33](#)
- BarrelRadiatorCell.cpp
 - RotationY, [174](#)
- Beta
 - ParticleTrack, [71](#)
- CalculateResolution
 - ResolutionUtilities, [13](#)
- CentreHitDistance
 - ResolutionUtilities::ResolutionStruct, [105](#)
 - Utilities::ResolutionStruct, [107](#)
- Check
 - de::IOptimizable::Constraints, [40](#)
- CherenkovAngle_Reco
 - CherenkovFile, [35](#)
- CherenkovAngle_Reco_TrueEmissionPoint
 - CherenkovFile, [35](#)
- CherenkovAngle_True
 - CherenkovFile, [35](#)
- CherenkovFile, [34](#)
 - CherenkovAngle_Reco, [35](#)
 - CherenkovAngle_Reco_TrueEmissionPoint, [35](#)
 - CherenkovAngle_True, [35](#)
 - CherenkovFile, [35](#)
 - CherenkovTree, [35](#)
 - Close, [35](#)
 - CosTheta, [36](#)
 - Entrance_x, [36](#)
 - Entrance_y, [36](#)
 - Entrance_z, [36](#)
 - File, [36](#)
 - Fill, [35](#)
 - FinalRadiatorColumnNumber, [36](#)

- FinalRadiatorRowNumber, [36](#)
- HasMigrated, [37](#)
- MaxPhotons, [37](#)
- MirrorHit_x, [37](#)
- MirrorHit_y, [37](#)
- MirrorHit_z, [37](#)
- Momentum, [37](#)
- NumberGoodPhotons, [37](#)
- NumberPhotons, [38](#)
- ParticleLocation, [38](#)
- Phi, [38](#)
- PhotonEnergy, [38](#)
- PhotonHit_x, [38](#)
- PhotonHit_y, [38](#)
- PhotonHit_z, [38](#)
- PhotonStatus, [38](#)
- RadiatorColumnNumber, [39](#)
- RadiatorRowNumber, [39](#)
- Significance, [39](#)
- SinglePhotonResolution, [39](#)
- TotalResolution, [39](#)
- TrackNumber, [39](#)
- CherenkovTree
 - CherenkovFile, [35](#)
- Close
 - CherenkovFile, [35](#)
- Constraints
 - de::IOptimizable::Constraints, [40](#)
 - ResolutionOptimizable.cpp, [210](#)
- ConvertBackToGlobalCoordinates
 - Particle, [66](#)
 - ParticleTrack, [71](#)
 - Photon, [79](#)
- ConvertToGlobal
 - ARCVector, [21](#)
- ConvertToRadiatorCoordinates
 - Particle, [66](#)
 - ParticleTrack, [72](#)
 - Photon, [80](#)
- CosTheta
 - CherenkovFile, [36](#)
- de, [9](#)
- de::DifferentialEvolution, [41](#)
 - DifferentialEvolution, [41](#)
 - GetBestAgent, [42](#)
 - GetBestCost, [42](#)
 - GetPopulationWithCosts, [42](#)
 - InitPopulation, [42](#)
 - Optimize, [42](#)
 - PrintPopulation, [42](#)
 - SelectionAndCorssing, [43](#)
- de::IOptimizable, [61](#)
 - ~IOptimizable, [62](#)
 - EvaluateCost, [62](#)
 - GetConstraints, [62](#)
 - NumberOfParameters, [62](#)
- de::IOptimizable::Constraints, [40](#)
 - Check, [40](#)
 - Constraints, [40](#)
 - isConstrained, [40](#)
 - lower, [40](#)
 - upper, [41](#)
- DetectorHit
 - Photon, [78](#)
- DetectorMiss
 - Photon, [78](#)
- DifferentialEvolution
 - de::DifferentialEvolution, [41](#)
- DoFit
 - ResolutionUtilities, [13](#)
- DrawARCGeometry
 - TrackingVolume, [114](#)
- DrawEventDisplay
 - EventDisplay, [53](#)
- DrawParticleTrack
 - ParticleTrack, [72](#)
- DrawPhotonPath
 - Photon, [80](#)
- DrawRadiatorArray
 - BarrelRadiatorArray, [27](#)
 - EndCapRadiatorArray, [47](#)
 - RadiatorArray, [88](#)
- DrawRadiatorGeometry
 - BarrelRadiatorCell, [32](#)
 - EndCapRadiatorCell, [51](#)
 - HalfRadiatorCell, [57](#)
 - RadiatorCell, [93](#)
- DrawSiPM
 - SiPM, [111](#)
- EfficiencyMiss
 - Photon, [78](#)
- Emitted
 - Photon, [78](#)
- EndCapHelixFunctor, [43](#)
 - EndCapHelixFunctor, [44](#)
 - operator(), [44](#)
- EndCapRadiatorArray, [45](#)
 - DrawRadiatorArray, [47](#)
 - EndCapRadiatorArray, [46](#)
 - FindRadiator, [47](#)
 - IsBelowEvenRow, [47](#)
 - IsBelowOddRow, [47](#)
 - IsBelowThisRow, [48](#)
- EndCapRadiatorCell, [48](#)
 - DrawRadiatorGeometry, [51](#)
 - EndCapRadiatorCell, [50](#)
 - IsDetectorInsideCell, [51](#)
 - IsEdgeCell, [51](#)
 - IsInsideCell, [51](#)
 - m_NotValidCells, [52](#)
 - m_ValidCells, [52](#)
- Entrance_x
 - CherenkovFile, [36](#)
- Entrance_y
 - CherenkovFile, [36](#)
- Entrance_z

- CherenkovFile, [36](#)
- EntranceWindow
 - ParticleTrack, [71](#)
- EvaluateCost
 - de::IOptimizable, [62](#)
 - ResolutionOptimizable, [104](#)
- EventDisplay, [52](#)
 - AddObject, [53](#)
 - DrawEventDisplay, [53](#)
 - EventDisplay, [53](#)
- Exists
 - Settings, [109](#)
- FailedReconstruction
 - PhotonReconstructor, [13](#)
- fcn
 - ResolutionUtilities, [14](#)
- File
 - CherenkovFile, [36](#)
- Fill
 - CherenkovFile, [35](#)
- FinalRadiatorColumnNumber
 - CherenkovFile, [36](#)
- FinalRadiatorRowNumber
 - CherenkovFile, [36](#)
- FindRadiator
 - BarrelRadiatorArray, [27](#)
 - EndCapRadiatorArray, [47](#)
 - Particle, [67](#)
 - RadiatorArray, [88](#)
- Functor1D
 - HelixPath.cpp, [188](#)
- Gas
 - Photon, [77](#)
- GeneratePhoton
 - ParticleTrack, [72](#)
- GeneratePhotonFromAerogel
 - ParticleTrack, [72](#)
- GeneratePhotonFromGas
 - ParticleTrack, [73](#)
- GeneratePhotonsFromAerogel
 - ParticleTrack, [73](#)
- GeneratePhotonsFromGas
 - ParticleTrack, [73](#)
- GenerateRandomBarrelTrack
 - Utilities, [15](#)
- GenerateRandomBarrelTrackZRange
 - Utilities, [15](#)
- GenerateRandomEndCapTrack
 - Utilities, [15](#)
- GetAerogelThickness
 - RadiatorCell, [93](#)
- GetAerogelTravelDistance
 - Photon, [80](#)
- GetBestAgent
 - de::DifferentialEvolution, [42](#)
- GetBestCost
 - de::DifferentialEvolution, [42](#)
- GetBool
 - Settings, [109](#)
- GetCellNumber
 - RadiatorCell, [93](#)
- GetCherenkovAngleDifference
 - Utilities, [15](#)
- GetConstraints
 - de::IOptimizable, [62](#)
 - ResolutionOptimizable, [104](#)
- GetCoolingThickness
 - RadiatorCell, [93](#)
- GetCosCherenkovAngle
 - Photon, [80](#)
- GetDetector
 - RadiatorCell, [93](#)
- GetDetectorSizeX
 - SiPM, [112](#)
- GetDetectorSizeY
 - SiPM, [112](#)
- GetDetectorTilt
 - SiPM, [112](#)
- GetDetectorXPosition
 - SiPM, [112](#)
- GetDetectorZPosition
 - SiPM, [112](#)
- GetDirection
 - HelixPath, [59](#)
 - Photon, [80](#)
- GetDouble
 - Settings, [109](#)
- GetEmissionPoint
 - Photon, [80](#)
- GetEnergy
 - Photon, [81](#)
- GetEntranceWindowPosition
 - ParticleTrack, [73](#)
- GetFieldStrength
 - TrackingVolume, [114](#)
- GetFixedParameters
 - ResolutionOptimizable, [104](#)
- GetGlobalMirrorCentre
 - RadiatorCell, [93](#)
- GetHexagonSize
 - RadiatorCell, [94](#)
- GetIndexRefraction
 - Utilities, [16](#)
- GetInt
 - Settings, [109](#)
- GetIntVector
 - Settings, [110](#)
- GetMass
 - ParticleMass, [9](#)
- GetMirrorCentre
 - RadiatorCell, [94](#)
- GetMirrorCurvature
 - RadiatorCell, [94](#)
- GetMirrorCurvatureCentreZ
 - RadiatorCell, [94](#)

- GetMirrorHitPosition
 - Photon, [81](#)
- GetMomentumMag
 - Utilities, [16](#)
- GetParticleDirection
 - Photon, [81](#)
- GetParticleLocation
 - ParticleTrack, [73](#)
- GetPhotonHits
 - SiPM, [112](#)
- GetPopulationWithCosts
 - de::DifferentialEvolution, [42](#)
- GetPosition
 - HelixPath, [60](#)
 - Particle, [67](#)
- GetPredictedCherenkovAngle
 - Utilities, [16](#)
- GetRadiator
 - Photon, [81](#)
- GetRadiatorCell
 - Particle, [67](#)
 - RadiatorArray, [89](#)
- GetRadiatorColumnNumber
 - Particle, [67](#)
- GetRadiatorPosition
 - RadiatorCell, [94](#)
- GetRadiatorRotation
 - RadiatorCell, [94](#)
- GetRadiatorRowNumber
 - Particle, [67](#)
- GetRadiatorThickness
 - RadiatorCell, [95](#)
- GetRadius
 - TrackingVolume, [114](#)
- GetReflectionPoint
 - PhotonReconstructor, [11](#)
- GetSizeT
 - Settings, [110](#)
- GetSizeTVector
 - Settings, [110](#)
- GetStatus
 - Photon, [81](#)
- GetString
 - Settings, [110](#)
- GetVesselThickness
 - RadiatorCell, [95](#)
- GetWeight
 - Photon, [81](#)
- GlobalVector
 - ARCVector, [22](#)
- HAIFRADIATORCELL
 - HalfRadiatorCell.h, [142](#)
- HalfRadiatorCell, [54](#)
 - DrawRadiatorGeometry, [57](#)
 - HalfRadiatorCell, [57](#)
 - IsInsideCell, [57](#)
- HalfRadiatorCell.h
 - HAIFRADIATORCELL, [142](#)
- HasMigrated
 - CherenkovFile, [37](#)
- HasPhotonMigrated
 - Photon, [82](#)
- HelixFunctor, [58](#)
 - HelixFunctor, [58](#)
 - operator(), [58](#)
- HelixPath, [59](#)
 - GetDirection, [59](#)
 - GetPosition, [60](#)
 - HelixPath, [59](#)
 - MapPhi, [60](#)
 - ReflectY, [60](#)
 - ReflectZ, [60](#)
 - ResetOrigin, [60](#)
 - SolvePathLength, [61](#)
- HelixPath.cpp
 - Functor1D, [188](#)
- HelixPath.h
 - RotationZ, [145](#)
- HitCorrectCell
 - ResolutionUtilities::ResolutionStruct, [105](#)
 - Utilities::ResolutionStruct, [107](#)
- HitTopWall
 - ResolutionUtilities::ResolutionStruct, [106](#)
 - Utilities::ResolutionStruct, [107](#)
- include/ARCVector.h, [127](#), [129](#)
- include/BarrelRadiatorArray.h, [129](#), [130](#)
- include/BarrelRadiatorCell.h, [131](#), [132](#)
- include/DifferentialEvolution.h, [132](#), [133](#)
- include/EndCapRadiatorArray.h, [137](#), [138](#)
- include/EndCapRadiatorCell.h, [139](#), [140](#)
- include/EventDisplay.h, [140](#), [141](#)
- include/HalfRadiatorCell.h, [142](#), [143](#)
- include/HelixFunctor.h, [143](#), [144](#)
- include/HelixPath.h, [144](#), [146](#)
- include/Particle.h, [146](#), [147](#)
- include/ParticleMass.h, [147](#), [148](#)
- include/ParticleTrack.h, [148](#), [149](#)
- include/Photon.h, [150](#)
- include/PhotonMapper.h, [152](#), [153](#)
- include/PhotonReconstructor.h, [153](#), [154](#)
- include/RadiatorArray.h, [155](#)
- include/RadiatorCell.h, [156](#), [157](#)
- include/ReconstructedPhoton.h, [158](#)
- include/ResolutionOptimizable.h, [159](#), [160](#)
- include/ResolutionUtilities.h, [160](#), [162](#)
- include/Settings.h, [162](#), [163](#)
- include/SiPM.h, [164](#)
- include/TrackingVolume.h, [165](#), [166](#)
- include/Utilities.h, [166](#), [168](#)
- InitPopulation
 - de::DifferentialEvolution, [42](#)
- IsAboveMainRow
 - BarrelRadiatorArray, [28](#)
- IsAboveUpperRow
 - BarrelRadiatorArray, [29](#)
- IsAtRadiator

- Particle, [67](#)
- ParticleTrack, [73](#)
- Photon, [82](#)
- IsBelowEvenRow
 - EndCapRadiatorArray, [47](#)
- IsBelowMainRow
 - BarrelRadiatorArray, [29](#)
- IsBelowOddRow
 - EndCapRadiatorArray, [47](#)
- IsBelowThisRow
 - EndCapRadiatorArray, [48](#)
- isConstrained
 - de::IOptimizable::Constraints, [40](#)
- IsDetectorHit
 - SiPM, [113](#)
- IsDetectorInsideCell
 - BarrelRadiatorCell, [32](#)
 - EndCapRadiatorCell, [51](#)
 - RadiatorCell, [95](#)
- IsEdgeCell
 - BarrelRadiatorCell, [33](#)
 - EndCapRadiatorCell, [51](#)
 - RadiatorCell, [95](#)
- IsInsideCell
 - BarrelRadiatorCell, [33](#)
 - EndCapRadiatorCell, [51](#)
 - HalfRadiatorCell, [57](#)
 - RadiatorCell, [95](#)
- IsLocal
 - ARCVector, [22](#)
- IsReconstructed
 - PhotonReconstructor, [11](#)
- IsScatteredInAerogel
 - PhotonMapper, [10](#)
- KickPhoton
 - Photon, [82](#)
- LocalVector
 - ARCVector, [22](#)
- Location
 - ParticleTrack, [70](#)
- lower
 - de::IOptimizable::Constraints, [40](#)
- m_AerogelThickness
 - RadiatorCell, [97](#)
- m_BarrelRadius
 - BarrelRadiatorCell, [33](#)
- m_CellNumber
 - RadiatorCell, [97](#)
- m_Cells
 - RadiatorArray, [89](#)
- m_CentreHitDistance
 - PhotonHit, [85](#)
- m_CoolingThickness
 - RadiatorCell, [97](#)
- m_CosBeta
 - PhotonReconstructor::QuarticSolution, [86](#)
- m_CosCherenkovAngle
 - ReconstructedPhoton, [102](#)
- m_CosCherenkovAngle_TrueEmissionPoint
 - ReconstructedPhoton, [102](#)
- m_DefaultMirrorCentre
 - RadiatorCell, [97](#)
- m_DegenerateSolution
 - PhotonReconstructor::QuarticSolution, [86](#)
- m_Detector
 - RadiatorCell, [98](#)
- m_HexagonSize
 - RadiatorCell, [98](#)
- m_HitPosition
 - PhotonHit, [85](#)
- m_MirrorCentre
 - RadiatorCell, [98](#)
- m_MirrorCurvature
 - RadiatorCell, [98](#)
- m_NotValidCells
 - EndCapRadiatorCell, [52](#)
- m_NumberMainRowCells
 - RadiatorArray, [89](#)
- m_Photon
 - PhotonHit, [86](#)
 - ReconstructedPhoton, [102](#)
- m_Position
 - Particle, [68](#)
 - RadiatorCell, [98](#)
- m_RadiatorCell
 - Particle, [68](#)
- m_RadiatorThickness
 - RadiatorCell, [98](#)
- m_Rotation
 - RadiatorCell, [99](#)
- m_SinBeta
 - PhotonReconstructor::QuarticSolution, [87](#)
- m_ValidCells
 - EndCapRadiatorCell, [52](#)
- m_VesselThickness
 - RadiatorCell, [99](#)
- m_xHexDist
 - RadiatorArray, [89](#)
- m_yHexDist
 - RadiatorArray, [90](#)
- main
 - OptimiseARC.cpp, [120](#)
 - RunARC.cpp, [123](#)
- MapPhi
 - ARCVector, [22](#)
 - HelixPath, [60](#)
 - Particle, [67](#)
 - ParticleTrack, [73](#)
 - Photon, [82](#)
- MaxPhotons
 - CherenkovFile, [37](#)
- Mirror
 - ParticleTrack, [71](#)
- MirrorHelixFunctor, [63](#)

- MirrorHelixFunctor, 64
- operator(), 64
- MirrorHit
 - Photon, 78
- MirrorHit_x
 - CherenkovFile, 37
- MirrorHit_y
 - CherenkovFile, 37
- MirrorHit_z
 - CherenkovFile, 37
- MirrorMiss
 - Photon, 78
- MissedEntranceWindow
 - ParticleTrack, 71
- MissedMirror
 - ParticleTrack, 71
- Momentum
 - CherenkovFile, 37
- N
 - ResolutionUtilities::ResolutionStruct, 106
 - Utilities::ResolutionStruct, 107
- NumberGoodPhotons
 - CherenkovFile, 37
- NumberOfParameters
 - de::IOptimizable, 62
 - ResolutionOptimizable, 105
- NumberPhotons
 - CherenkovFile, 38
- operator()
 - BarrelHelixFunctor, 25
 - EndCapHelixFunctor, 44
 - HelixFunctor, 58
 - MirrorHelixFunctor, 64
 - RadiatorArray, 89
 - ZPlaneHelixFunctor, 116
- operator+=
 - ARCVector, 22
- OptimiseARC.cpp
 - main, 120
 - Tracks, 120
- Optimize
 - de::DifferentialEvolution, 42
- OutsideCell
 - Photon, 78
- OutsideMirrorRadius
 - Photon, 78
- Particle, 65
 - ~Particle, 66
 - ConvertBackToGlobalCoordinates, 66
 - ConvertToRadiatorCoordinates, 66
 - FindRadiator, 67
 - GetPosition, 67
 - GetRadiatorCell, 67
 - GetRadiatorColumnNumber, 67
 - GetRadiatorRowNumber, 67
 - IsAtRadiator, 67
 - m_Position, 68
 - m_RadiatorCell, 68
 - MapPhi, 67
 - Particle, 66
 - ReflectY, 68
 - ReflectZ, 68
- ParticleLocation
 - CherenkovFile, 38
- ParticleMass, 9
 - GetMass, 9
- ParticleTrack, 69
 - ~ParticleTrack, 71
 - Beta, 71
 - ConvertBackToGlobalCoordinates, 71
 - ConvertToRadiatorCoordinates, 72
 - DrawParticleTrack, 72
 - EntranceWindow, 71
 - GeneratePhoton, 72
 - GeneratePhotonFromAerogel, 72
 - GeneratePhotonFromGas, 73
 - GeneratePhotonsFromAerogel, 73
 - GeneratePhotonsFromGas, 73
 - GetEntranceWindowPosition, 73
 - GetParticleLocation, 73
 - IsAtRadiator, 73
 - Location, 70
 - MapPhi, 73
 - Mirror, 71
 - MissedEntranceWindow, 71
 - MissedMirror, 71
 - ParticleTrack, 71
 - Radiator, 71
 - ReflectY, 74
 - ReflectZ, 74
 - SetRadiator, 74
 - TrackerVolume, 71
 - TrackThroughAerogel, 74
 - TrackThroughGasToMirror, 74
 - TrackThroughTracker, 74
 - TrackToNextCell, 75
- Phi
 - CherenkovFile, 38
- Photon, 75
 - ~Photon, 79
 - AddAerogelTravelDistance, 79
 - Aerogel, 77
 - AerogelScattered, 78
 - Backwards, 78
 - ConvertBackToGlobalCoordinates, 79
 - ConvertToRadiatorCoordinates, 80
 - DetectorHit, 78
 - DetectorMiss, 78
 - DrawPhotonPath, 80
 - EfficiencyMiss, 78
 - Emitted, 78
 - Gas, 77
 - GetAerogelTravelDistance, 80
 - GetCosCherenkovAngle, 80

- GetDirection, 80
- GetEmissionPoint, 80
- GetEnergy, 81
- GetMirrorHitPosition, 81
- GetParticleDirection, 81
- GetRadiator, 81
- GetStatus, 81
- GetWeight, 81
- HasPhotonMigrated, 82
- IsAtRadiator, 82
- KickPhoton, 82
- MapPhi, 82
- MirrorHit, 78
- MirrorMiss, 78
- OutsideCell, 78
- OutsideMirrorRadius, 78
- Photon, 78, 79
- PhotonHasMigrated, 83
- PropagatePhoton, 83
- PutPhotonToEmissionPoint, 83
- Radiator, 77
- ReflectY, 83
- ReflectZ, 83
- RegisterMirrorHitPosition, 84
- Status, 77
- UpdatePhotonStatus, 84
- WallMiss, 78
- PhotonEnergy
 - CherenkovFile, 38
- PhotonHasMigrated
 - Photon, 83
- PhotonHit, 84
 - m_CentreHitDistance, 85
 - m_HitPosition, 85
 - m_Photon, 86
- PhotonHit_x
 - CherenkovFile, 38
- PhotonHit_y
 - CherenkovFile, 38
- PhotonHit_z
 - CherenkovFile, 38
- PhotonMapper, 10
 - IsScatteredInAerogel, 10
 - PhotonMirrorDistance, 10
 - TracePhoton, 10
 - TracePhotonToDetector, 10
 - TracePhotonToMirror, 11
- PhotonMirrorDistance
 - PhotonMapper, 10
- PhotonReconstructor, 11
 - FailedReconstruction, 13
 - GetReflectionPoint, 11
 - IsReconstructed, 11
 - ReconstructCosCherenkovAngle, 12
 - ReconstructPhoton, 12
 - SolveQuartic, 12
- PhotonReconstructor::QuarticSolution, 86
 - m_CosBeta, 86
 - m_DegenerateSolution, 86
 - m_SinBeta, 87
- PhotonStatus
 - CherenkovFile, 38
- PlotHits
 - SiPM, 113
- PlotProjections
 - ResolutionUtilities, 14
- PrintPopulation
 - de::DifferentialEvolution, 42
- PropagatePhoton
 - Photon, 83
- PutPhotonToEmissionPoint
 - Photon, 83
- Radiator
 - ParticleTrack, 71
 - Photon, 77
- RadiatorArray, 87
 - ~RadiatorArray, 88
 - DrawRadiatorArray, 88
 - FindRadiator, 88
 - GetRadiatorCell, 89
 - m_Cells, 89
 - m_NumberMainRowCells, 89
 - m_xHexDist, 89
 - m_yHexDist, 90
 - operator(), 89
 - RadiatorArray, 88
- RadiatorCell, 90
 - ~RadiatorCell, 92
 - DrawRadiatorGeometry, 93
 - GetAerogelThickness, 93
 - GetCellNumber, 93
 - GetCoolingThickness, 93
 - GetDetector, 93
 - GetGlobalMirrorCentre, 93
 - GetHexagonSize, 94
 - GetMirrorCentre, 94
 - GetMirrorCurvature, 94
 - GetMirrorCurvatureCentreZ, 94
 - GetRadiatorPosition, 94
 - GetRadiatorRotation, 94
 - GetRadiatorThickness, 95
 - GetVesselThickness, 95
 - IsDetectorInsideCell, 95
 - IsEdgeCell, 95
 - IsInsideCell, 95
 - m_AerogelThickness, 97
 - m_CellNumber, 97
 - m_CoolingThickness, 97
 - m_DefaultMirrorCentre, 97
 - m_Detector, 98
 - m_HexagonSize, 98
 - m_MirrorCentre, 98
 - m_MirrorCurvature, 98
 - m_Position, 98
 - m_RadiatorThickness, 98
 - m_Rotation, 99

- m_VesselThickness, 99
 - RadiatorCell, 92
 - ReversePhiRotation, 96
 - SetDetectorPosition, 96
 - SetDetectorTilt, 96
 - SetMirrorCurvature, 96
 - SetMirrorXPosition, 96
 - SetMirrorZPosition, 97
- RadiatorColumnNumber
 - CherenkovFile, 39
- RadiatorRowNumber
 - CherenkovFile, 39
- Random
 - Utilities, 16
- ReconstructCosCherenkovAngle
 - PhotonReconstructor, 12
- ReconstructedPhoton, 99
 - m_CosCherenkovAngle, 102
 - m_CosCherenkovAngle_TrueEmissionPoint, 102
 - m_Photon, 102
 - ReconstructedPhoton, 101
- ReconstructPhoton
 - PhotonReconstructor, 12
- ReflectY
 - ARCVector, 22
 - HelixPath, 60
 - Particle, 68
 - ParticleTrack, 74
 - Photon, 83
- ReflectZ
 - ARCVector, 23
 - HelixPath, 60
 - Particle, 68
 - ParticleTrack, 74
 - Photon, 83
- RegisterMirrorHitPosition
 - Photon, 84
- ResetOrigin
 - HelixPath, 60
- ResolutionOptimizable, 103
 - EvaluateCost, 104
 - GetConstraints, 104
 - GetFixedParameters, 104
 - NumberOfParameters, 105
 - ResolutionOptimizable, 104
- ResolutionOptimizable.cpp
 - Constraints, 210
- ResolutionOptimizable.h
 - Tracks, 159
- ResolutionUtilities, 13
 - CalculateResolution, 13
 - DoFit, 13
 - fcn, 14
 - PlotProjections, 14
- ResolutionUtilities.h
 - Tracks, 161
- ResolutionUtilities::ResolutionStruct, 105
 - CentreHitDistance, 105
 - HitCorrectCell, 105
 - HitTopWall, 106
 - N, 106
 - SumWeights, 106
 - x, 106
- ReversePhiRotation
 - BarrelRadiatorCell, 33
 - RadiatorCell, 96
- Rotation3D
 - ARCVector.h, 127
- RotationY
 - BarrelRadiatorCell.cpp, 174
- RotationZ
 - HelixPath.h, 145
- RunARC.cpp
 - main, 123
- SelectionAndCorssing
 - de::DifferentialEvolution, 43
- SetDetectorPosition
 - RadiatorCell, 96
 - SiPM, 113
- SetDetectorTilt
 - RadiatorCell, 96
 - SiPM, 113
- SetGlobalVector
 - ARCVector, 23
- SetMirrorCurvature
 - RadiatorCell, 96
- SetMirrorXPosition
 - RadiatorCell, 96
- SetMirrorZPosition
 - RadiatorCell, 97
- SetRadiator
 - ParticleTrack, 74
- Settings, 108
 - AddSettings, 109
 - Exists, 109
 - GetBool, 109
 - GetDouble, 109
 - GetInt, 109
 - GetIntVector, 110
 - GetSizeT, 110
 - GetSizeTVector, 110
 - GetString, 110
 - Settings, 109
- Settings.h
 - ssMap, 163
- SetX
 - ARCVector, 23
- SetZ
 - ARCVector, 23
- Significance
 - CherenkovFile, 39
- SinglePhotonResolution
 - CherenkovFile, 39
- SiPM, 111
 - AddPhotonHit, 111
 - DrawSiPM, 111

- GetDetectorSizeX, [112](#)
- GetDetectorSizeY, [112](#)
- GetDetectorTilt, [112](#)
- GetDetectorXPosition, [112](#)
- GetDetectorZPosition, [112](#)
- GetPhotonHits, [112](#)
- IsDetectorHit, [113](#)
- PlotHits, [113](#)
- SetDetectorPosition, [113](#)
- SetDetectorTilt, [113](#)
- SiPM, [111](#)
- SolvePathLength
 - HelixPath, [61](#)
- SolveQuartic
 - PhotonReconstructor, [12](#)
- src/ARCVector.cpp, [168](#), [169](#)
- src/BarrelRadiatorArray.cpp, [170](#), [171](#)
- src/BarrelRadiatorCell.cpp, [173](#), [174](#)
- src/CMakeLists.txt, [176](#)
- src/EndCapRadiatorArray.cpp, [176](#), [177](#)
- src/EndCapRadiatorCell.cpp, [179](#), [180](#)
- src/EventDisplay.cpp, [183](#)
- src/HalfRadiatorCell.cpp, [184](#), [185](#)
- src/HelixFunctor.cpp, [186](#), [187](#)
- src/HelixPath.cpp, [188](#)
- src/Particle.cpp, [189](#), [190](#)
- src/ParticleMass.cpp, [191](#), [192](#)
- src/ParticleTrack.cpp, [192](#), [193](#)
- src/Photon.cpp, [197](#), [198](#)
- src/PhotonMapper.cpp, [201](#), [202](#)
- src/PhotonReconstructor.cpp, [203](#), [204](#)
- src/RadiatorArray.cpp, [205](#), [206](#)
- src/RadiatorCell.cpp, [206](#), [207](#)
- src/ReconstructedPhoton.cpp, [209](#)
- src/ResolutionOptimizable.cpp, [209](#), [210](#)
- src/ResolutionUtilities.cpp, [211](#), [212](#)
- src/Settings.cpp, [215](#)
- src/SiPM.cpp, [217](#), [218](#)
- src/TrackingVolume.cpp, [220](#)
- src/Utilities.cpp, [221](#), [222](#)
- ssMap
 - Settings.h, [163](#)
- Status
 - Photon, [77](#)
- SumWeights
 - ResolutionUtilities::ResolutionStruct, [106](#)
 - Utilities::ResolutionStruct, [107](#)
- TotalResolution
 - CherenkovFile, [39](#)
- TracePhoton
 - PhotonMapper, [10](#)
- TracePhotonToDetector
 - PhotonMapper, [10](#)
- TracePhotonToMirror
 - PhotonMapper, [11](#)
- TrackerVolume
 - ParticleTrack, [71](#)
- TrackingVolume, [114](#)
- DrawARCGeometry, [114](#)
- GetFieldStrength, [114](#)
- GetRadius, [114](#)
- TrackingVolume, [114](#)
- TrackNumber
 - CherenkovFile, [39](#)
- TrackPhotons
 - Utilities, [17](#)
- Tracks
 - OptimiseARC.cpp, [120](#)
 - ResolutionOptimizable.h, [159](#)
 - ResolutionUtilities.h, [161](#)
- TrackSeed
 - Utilities, [17](#)
- TrackThroughAerogel
 - ParticleTrack, [74](#)
- TrackThroughGasToMirror
 - ParticleTrack, [74](#)
- TrackThroughTracker
 - ParticleTrack, [74](#)
- TrackToNextCell
 - ParticleTrack, [75](#)
- UpdatePhotonStatus
 - Photon, [84](#)
- upper
 - de::IOptimizable::Constraints, [41](#)
- Utilities, [14](#)
 - GenerateRandomBarrelTrack, [15](#)
 - GenerateRandomBarrelTrackZRange, [15](#)
 - GenerateRandomEndCapTrack, [15](#)
 - GetCherenkovAngleDifference, [15](#)
 - GetIndexRefraction, [16](#)
 - GetMomentumMag, [16](#)
 - GetPredictedCherenkovAngle, [16](#)
 - Random, [16](#)
 - TrackPhotons, [17](#)
 - TrackSeed, [17](#)
 - VectorFromSpherical, [17](#)
- Utilities::ResolutionStruct, [106](#)
 - CentreHitDistance, [107](#)
 - HitCorrectCell, [107](#)
 - HitTopWall, [107](#)
 - N, [107](#)
 - SumWeights, [107](#)
 - x, [108](#)
- Vector
 - ARCVector.h, [127](#)
- VectorFromSpherical
 - Utilities, [17](#)
- WallMiss
 - Photon, [78](#)
- x
 - ResolutionUtilities::ResolutionStruct, [106](#)
 - Utilities::ResolutionStruct, [108](#)
- ZPlaneHelixFunctor, [115](#)

`operator()`, [116](#)
`ZPlaneHelixFunctor`, [116](#)